

Python source lines 1-37

```
1. # sample_id: 001
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 1
6. # objective: 816.14
7. # test_objective: 793.642
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;nonlinear_pipeline_composition;order_up_to;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer
    rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Core parameters
14.     base_stock = 852.0694799766587 # OPT_PARAM: {"initial": 852.0694799766587, "min": 400, "max":
    900, "type": "float"}
15.
16.     # Demand anticipation parameters
17.     demand_anticipation_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.8, "type":
    "float"}
18.     recent_demand_weight = 0.6 # OPT_PARAM: {"initial": 0.6, "min": 0.3, "max": 0.9, "type":
    "float"}
19.
20.     # Pipeline urgency factor
21.     pipeline_urgency_factor = 0.5795120417177109 # OPT_PARAM: {"initial": 0.5795120417177109, "min":
    0.1, "max": 0.8, "type": "float"}
22.
23.     # Cost-balancing parameters
24.     holding_cost_weight = 0.8138365213306138 # OPT_PARAM: {"initial": 0.8138365213306138, "min":
    0.3, "max": 1.2, "type": "float"}
25.     lost_sales_cost_weight = 1.4091129587809088 # OPT_PARAM: {"initial": 1.4091129587809088, "min":
    0.8, "max": 2.0, "type": "float"}
26.
27.     # Order smoothing parameters
28.     smoothing_factor = 0.13063463854289023 # OPT_PARAM: {"initial": 0.13063463854289023, "min": 0.1,
    "max": 0.5, "type": "float"}
29.     min_order_size = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 30, "type": "float"}
30.
31.     # NEW: Safety stock parameters
32.     safety_stock = 220.0 # OPT_PARAM: {"initial": 220.0, "min": 150, "max": 300, "type": "float"}
33.     safety_adjustment_factor = 0.3038678010386268 # OPT_PARAM: {"initial": 0.3038678010386268,
    "min": 0.1, "max": 0.6, "type": "float"}
34.     lost_sales_multiplier = 1.5 # OPT_PARAM: {"initial": 1.5, "min": 1.2, "max": 2.0, "type":
    "float"}
35.
36.     # Calculate pipeline urgency
37.     L = len(pipeline_orders)
```

Compact mathematical formula

Extra close brace or missing open brace

Python source lines 38-81

```
38.     if L > 0:
39.         # Calculate weighted pipeline with exponential decay for urgency
40.         weighted_pipeline = 0.0
41.         total_weight = 0.0
42.
43.         for i, q in enumerate(pipeline_orders):
44.             # Exponential decay: more urgent for orders arriving soon
45.             weight = 1.0 / (1.0 + i) # Simple reciprocal weighting
46.             weighted_pipeline += q * weight
47.             total_weight += weight
48.
49.         effective_pipeline = weighted_pipeline / total_weight if total_weight > 0 else
sum(pipeline_orders)
50.
51.         # Calculate pipeline urgency based on distribution
52.         pipeline_urgency = 0.0
53.         if L >= 2:
54.             # Measure how concentrated pipeline is in near future
55.             near_pipeline = sum(pipeline_orders[:L//2])
56.             far_pipeline = sum(pipeline_orders[L//2:])
57.             total_pipeline = near_pipeline + far_pipeline
58.
59.             if total_pipeline > 0:
60.                 # Higher urgency if more pipeline is in distant future
61.                 pipeline_urgency = far_pipeline / total_pipeline
62.         else:
63.             effective_pipeline = 0.0
64.             pipeline_urgency = 0.0
65.
66.         # Calculate inventory position
67.         inventory_position = on_hand_inventory + effective_pipeline
68.
69.         # Dynamic safety stock based on pipeline urgency
70.         dynamic_safety_stock = base_stock * (1.0 + pipeline_urgency * pipeline_urgency_factor)
71.
72.         # NEW: Safety stock deficit adjustment
73.         safety_adjusted_level = dynamic_safety_stock
74.         if inventory_position < safety_stock:
75.             deficit_ratio = max(0, (safety_stock - inventory_position) / max(1, safety_stock))
76.             adjustment = safety_adjustment_factor * (1.0 + deficit_ratio * lost_sales_multiplier)
77.             safety_adjusted_level += adjustment
78.
79.         # Cost-balanced order-up-to level
80.         cost_ratio = lost_sales_cost_weight / holding_cost_weight
81.         cost_balanced_level = safety_adjusted_level * (1.0 + demand_anticipation_factor * (cost_ratio -
1.0))
```

Compact mathematical formula

The compact formula is repeated for this source continuation.

Extra close brace or missing open brace

Python source lines 82-97

```
82.
83.     # Calculate raw order amount
84.     raw_order = max(0.0, cost_balanced_level - inventory_position)
85.
86.     # Apply pipeline-urgency-adjusted smoothing
87.     urgency_adjusted_smoothing = smoothing_factor * (1.0 - pipeline_urgency * 0.5)
88.     smoothed_order = raw_order * urgency_adjusted_smoothing
89.
90.     # Apply minimum order threshold
91.     if smoothed_order < min_order_size:
92.         smoothed_order = 0.0
93.
94.     # Round to nearest integer
95.     order_amount = int(round(smoothed_order))
96.
97.     return order_amount
```

Compact mathematical formula

The compact formula is repeated for this source continuation.

Extra close brace or missing open brace

Python source lines 1-39

```
1. # sample_id: 002
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_m2_4_r6
3. # distribution: poisson_L4_c1_5
4. # generation: 6
5. # rank_in_population_file: 4
6. # objective: 2121.78
7. # test_objective: 2139.074
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;order_up_to;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 620.0 # OPT_PARAM: {"initial": 620.0, "min": 500, "max": 750, "type": "float"}
14.     safety_stock = 193.25820241595864 # OPT_PARAM: {"initial": 193.25820241595864, "min": 100, "max": 200, "type": "float"}
15.     demand_estimate = 110.0 # OPT_PARAM: {"initial": 110.0, "min": 90, "max": 110, "type": "float"}
16.     adjustment_factor = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.8, "max": 1.0, "type": "float"}
17.     lead_time = len(pipeline_orders)
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand during lead time
23.     expected_lead_time_demand = demand_estimate * lead_time
24.
25.     # Calculate target inventory position
26.     target_position = expected_lead_time_demand + safety_stock
27.
28.     # Calculate order amount with adjustment
29.     order_amount = max(0, target_position - inventory_position)
30.     order_amount = order_amount * adjustment_factor
31.
32.     # Apply base stock as upper bound
33.     max_order = max(0, base_stock - inventory_position)
34.     order_amount = min(order_amount, max_order)
35.
36.     # Round to nearest integer
37.     order_amount = int(round(order_amount))
38.
39.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\min \left(\left(\max \left(0, \left((110.0 \cdot L) + 193.3 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \cdot 0.8 \right), \max \left(0, \left(620.0 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right) \right) \right).$$

Python source lines 1-33

```

1. # sample_id: 003
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_10_r3
3. # distribution: poisson_L6_c1_5
4. # generation: 2
5. # rank_in_population_file: 6
6. # objective: 3336.88008
7. # test_objective: 3369.77227
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 697.9997255235969 # OPT_PARAM: {"initial": 697.9997255235969, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 79.30000000000116 # OPT_PARAM: {"initial": 79.30000000000116, "min": 0, "max": 200,
16.     "type": "float"}
17.     demand_forecast = 149.9 # OPT_PARAM: {"initial": 149.9, "min": 50, "max": 150, "type": "float"}
18.     lead_time = len(pipeline_orders)
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.     # Calculate expected demand during lead time
22.     expected_lead_time_demand = demand_forecast * lead_time
23.     # Calculate target inventory position
24.     target_inventory = expected_lead_time_demand + safety_stock
25.     # Calculate order amount
26.     order_amount = max(0, target_inventory - inventory_position)
27.     # Apply base stock as upper bound
28.     order_amount = min(order_amount, max(0, base_stock - inventory_position))
29.     return order_amount

```

Compact mathematical formula

Inputs: h = on-hand inventory, $\mathbf{q} = (q_1, \dots, q_L)$ = pipeline orders.

$$I(h, \mathbf{q}) = h + \sum_{i=1}^L q_i \quad (\text{inventory position}).$$

$$T(L) = 149.9L + 79.3 \quad (\text{lead-time demand plus safety stock}).$$

$$[x]^+ = \max(0, x).$$

$$Q(h, \mathbf{q}) = [\min(698.0, T(L)) - I(h, \mathbf{q})]^+.$$

Python source lines 1-41

```

1. # sample_id: 004
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: poisson_L6_c1_5
4. # generation: 8
5. # rank_in_population_file: 2
6. # objective: 1238.22
7. # test_objective: 1272.82
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer
    _rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 583.5806013968356 # OPT_PARAM: {"initial": 583.5806013968356, "min": 400, "max":
    800, "type": "float"}
15.
16.     # Safety stock adjustment
17.     safety_stock = 122.3200350131986 # OPT_PARAM: {"initial": 122.3200350131986, "min": 20, "max":
    150, "type": "float"}
18.
19.     # Demand anticipation factor
20.     anticipation_factor = 1.5 # OPT_PARAM: {"initial": 1.5, "min": 0.5, "max": 1.5, "type": "float"}
21.
22.     # Order smoothing
23.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.8, "type": "float"}
24.
25.     # Order bounds
26.     min_order = 96.64831031968099 # OPT_PARAM: {"initial": 96.64831031968099, "min": 20, "max": 100,
    "type": "float"}
27.     max_order = 200.0 # OPT_PARAM: {"initial": 200.0, "min": 100, "max": 300, "type": "float"}
28.
29.     # Pipeline urgency adjustment
30.     urgency_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.5, "type": "float"}
31.     urgency_threshold = 50.0 # OPT_PARAM: {"initial": 50.0, "min": 10, "max": 150, "type": "float"}
32.
33.     # Lead time demand coverage
34.     coverage_periods = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 2.0, "type": "float"}
35.
36.     # NEW: Cost-ratio adjustment factor
37.     cost_ratio_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.5, "type": "float"}
38.
39.     # NEW: Predictive ordering based on pipeline pattern recognition
40.     pattern_weight = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.1, "max": 0.8, "type": "float"}
41.

```

Compact mathematical formula

Inputs: h = on-hand inventory, $\mathbf{q} = (q_1, \dots, q_L)$ = pipeline orders.

$$I = h + \sum_{i=1}^L q_i \quad (\text{inventory position}).$$

$$\bar{d} = \begin{cases} \frac{2}{L(L+1)} \sum_{i=1}^L (L-i+1)q_i, & L > 0, \\ 100, & L = 0 \end{cases} \quad (\text{weighted pipeline-demand estimate}).$$

$$s = \sum_{i=1}^{L-1} \text{sgn}(q_{i+1} - q_i) \quad (\text{pipeline trend score}).$$

$$d = \begin{cases} \bar{d} \left(1 + \frac{0.3}{L} s \right), & L \geq 3, \\ \bar{d}, & L < 3 \end{cases} \quad (\text{trend-adjusted demand estimate}).$$

$$n = \sum_{i=1}^{\min(2,L)} q_i \quad (\text{near-term pipeline}).$$

$$u(n) = \begin{cases} 1 + \frac{50.0 - n}{50.0} (0.5 - 1), & n < 50.0, \\ 1, & n \geq 50.0 \end{cases} \quad (\text{urgency multiplier}).$$

$$B = u(n) [583.6 + 1.5(d - 100)].$$

$$G = \max(B, 0.5Ld).$$

$$R = G \left[1 + 0.1 \left(1 - \min \left(1, \frac{I}{G} \right) \right) \right].$$

$$T = 0.5R + 122.3 \quad (\text{target inventory position}).$$

$$\text{clip}_{[96.6, 200.0]}(x) = \max(96.6, \min(200.0, x)).$$

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\text{clip}_{[96.6, 200.0]} \left(0.1 \max(0, T - I) + (1 - 0.1)96.6 \right) \right).$$

Python source lines 42-86

```

42. # NEW: Dynamic risk adjustment based on inventory position
43. risk_sensitivity = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 1.5, "type": "float"}
44.
45. # Calculate inventory position
46. inventory_position = on_hand_inventory + sum(pipeline_orders)
47.
48. # Estimate near-term demand using weighted average of recent pipeline arrivals
49. weighted_demand_estimate = 0
50. total_weight = 0
51. L = len(pipeline_orders)
52.
53. for i, q in enumerate(pipeline_orders):
54.     # Weight decreases for older pipeline orders
55.     weight = (L - i) / L
56.     weighted_demand_estimate += q * weight
57.     total_weight += weight
58.
59. if total_weight > 0:
60.     avg_weighted_demand = weighted_demand_estimate / total_weight
61. else:
62.     avg_weighted_demand = 100.0 # Default estimate
63.
64. # NEW: Pattern recognition - detect if pipeline shows increasing/decreasing trend
65. if L >= 3:
66.     recent_trend = 0
67.     for i in range(L-1):
68.         if pipeline_orders[i+1] > pipeline_orders[i]:
69.             recent_trend += 1
70.         elif pipeline_orders[i+1] < pipeline_orders[i]:
71.             recent_trend -= 1
72.
73.     # Adjust demand estimate based on trend
74.     trend_factor = 1.0 + (recent_trend / L) * pattern_weight
75.     pattern_adjusted_demand = avg_weighted_demand * trend_factor
76. else:
77.     pattern_adjusted_demand = avg_weighted_demand
78.
79. # Adjust base stock based on demand anticipation
80. adjusted_base_stock = base_stock + anticipation_factor * (pattern_adjusted_demand - 100.0)
81.
82. # Apply pipeline urgency adjustment
83. near_pipeline = sum(pipeline_orders[:min(2, L)])
84. if near_pipeline < urgency_threshold:
85.     urgency_adjustment = 1.0 + (urgency_threshold - near_pipeline) / urgency_threshold *
(urgency_factor - 1.0)
86.     adjusted_base_stock += urgency_adjustment

```

Compact mathematical formula

The compact formula is repeated for this source continuation.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_1, \dots, q_L)$ = pipeline orders.

$$I = h + \sum_{i=1}^L q_i \quad (\text{inventory position}).$$

$$\bar{d} = \begin{cases} \frac{2}{L(L+1)} \sum_{i=1}^L (L-i+1)q_i, & L > 0, \\ 100, & L = 0 \end{cases} \quad (\text{weighted pipeline-demand estimate}).$$

$$s = \sum_{i=1}^{L-1} \text{sgn}(q_{i+1} - q_i) \quad (\text{pipeline trend score}).$$

$$d = \begin{cases} \bar{d} \left(1 + \frac{0.3}{L} s \right), & L \geq 3, \\ \bar{d}, & L < 3 \end{cases} \quad (\text{trend-adjusted demand estimate}).$$

$$n = \sum_{i=1}^{\min(2, L)} q_i \quad (\text{near-term pipeline}).$$

$$u(n) = \begin{cases} 1 + \frac{50.0 - n}{50.0} (0.5 - 1), & n < 50.0, \\ 1, & n \geq 50.0 \end{cases} \quad (\text{urgency multiplier}).$$

$$B = u(n) [583.6 + 1.5(d - 100)].$$

$$G = \max(B, 0.5Ld).$$

$$R = G \left[1 + 0.1 \left(1 - \min \left(1, \frac{I}{G} \right) \right) \right].$$

$$T = 0.5R + 122.3 \quad (\text{target inventory position}).$$

$$\text{clip}_{[96.6, 200.0]}(x) = \max(96.6, \min(200.0, x)).$$

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\text{clip}_{[96.6, 200.0]} \left(0.1 \max(0, T - I) + (1 - 0.1)96.6 \right) \right).$$

Python source lines 87-117

```

87.
88.     # Ensure lead time demand coverage
89.     lead_time_demand = pattern_adjusted_demand * L
90.     coverage_adjustment = lead_time_demand * coverage_periods
91.     final_base_stock = max(adjusted_base_stock, coverage_adjustment)
92.
93.     # NEW: Dynamic risk adjustment based on current inventory position
94.     # Higher inventory reduces risk sensitivity, lower inventory increases it
95.     risk_adjustment = 1.0 + (risk_sensitivity * (1.0 - min(1.0, inventory_position /
final_base_stock)))
96.     final_base_stock *= risk_adjustment
97.
98.     # Adjust for cost ratio (p/h = 5/1 = 5)
99.     # Higher p/h ratio favors higher inventory to avoid stockouts
100.    cost_adjusted_base_stock = final_base_stock * cost_ratio_factor
101.
102.    # Add safety stock
103.    target_inventory_position = cost_adjusted_base_stock + safety_stock
104.
105.    # Calculate raw order
106.    raw_order = max(0, target_inventory_position - inventory_position)
107.
108.    # Apply smoothing
109.    smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * min_order
110.
111.    # Apply bounds
112.    bounded_order = max(min_order, min(max_order, smoothed_order))
113.
114.    # Round to nearest integer
115.    order_amount = int(round(bounded_order))
116.
117.    return order_amount

```

Compact mathematical formula

The compact formula is repeated for this source continuation.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_1, \dots, q_L)$ = pipeline orders.

$$I = h + \sum_{i=1}^L q_i \quad (\text{inventory position}).$$

$$\bar{d} = \begin{cases} \frac{2}{L(L+1)} \sum_{i=1}^L (L-i+1)q_i, & L > 0, \\ 100, & L = 0 \end{cases} \quad (\text{weighted pipeline-demand estimate}).$$

$$s = \sum_{i=1}^{L-1} \text{sgn}(q_{i+1} - q_i) \quad (\text{pipeline trend score}).$$

$$d = \begin{cases} \bar{d} \left(1 + \frac{0.3}{L}s\right), & L \geq 3, \\ \bar{d}, & L < 3 \end{cases} \quad (\text{trend-adjusted demand estimate}).$$

$$n = \sum_{i=1}^{\min(2,L)} q_i \quad (\text{near-term pipeline}).$$

$$u(n) = \begin{cases} 1 + \frac{50.0 - n}{50.0} (0.5 - 1), & n < 50.0, \\ 1, & n \geq 50.0 \end{cases} \quad (\text{urgency multiplier}).$$

$$B = u(n) [583.6 + 1.5(d - 100)].$$

$$G = \max(B, 0.5Ld).$$

$$R = G \left[1 + 0.1 \left(1 - \min\left(1, \frac{I}{G}\right)\right)\right].$$

$$T = 0.5R + 122.3 \quad (\text{target inventory position}).$$

$$\text{clip}_{[96.6, 200.0]}(x) = \max(96.6, \min(200.0, x)).$$

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\text{clip}_{[96.6, 200.0]} \left(0.1 \max(0, T - I) + (1 - 0.1)96.6 \right) \right).$$

Python source lines 1-37

```
1. # sample_id: 005
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-e2_6_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 2
5. # rank_in_population_file: 2
6. # objective: 2372.08
7. # test_objective: 2367.379
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;nonlinear_pipeline_composition;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding;nonlinear_gap_tra
    nsform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level for normal operation
14.     base_stock = 602.7471676191776 # OPT_PARAM: {"initial": 602.7471676191776, "min": 100, "max":
    1000, "type": "float"}
15.
16.     # Safety stock buffer
17.     safety_stock = 57.947167619179545 # OPT_PARAM: {"initial": 57.947167619179545, "min": 0, "max":
    200, "type": "float"}
18.
19.     # Weight for pipeline orders (higher = more conservative)
20.     pipeline_weight = 0.6918604694768504 # OPT_PARAM: {"initial": 0.6918604694768504, "min": 0.1,
    "max": 1.5, "type": "float"}
21.
22.     # Weight for on-hand inventory (higher = more aggressive)
23.     on_hand_weight = 0.9081860064437058 # OPT_PARAM: {"initial": 0.9081860064437058, "min": 0.5,
    "max": 2.0, "type": "float"}
24.
25.     # Smoothing factor for order adjustments
26.     smoothing_factor = 0.14628116825983467 # OPT_PARAM: {"initial": 0.14628116825983467, "min": 0.1,
    "max": 1.0, "type": "float"}
27.
28.     # Exponential weighting for recent pipeline orders
29.     recent_pipeline_weight = 1.66131668224612 # OPT_PARAM: {"initial": 1.66131668224612, "min": 0.5,
    "max": 3.0, "type": "float"}
30.
31.     # Minimum order threshold
32.     min_order_threshold = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 50, "type": "float"}
33.
34.     # NEW: Pipeline variability adjustment factor
35.     variability_factor = 0.443389619707202 # OPT_PARAM: {"initial": 0.443389619707202, "min": 0.0,
    "max": 2.0, "type": "float"}
36.
37.     # NEW: Recent demand anticipation factor
```

Declarative mathematical formula rows 1-17

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline_sum} = \sum_{i=0}^{L-1} \left((q_i \cdot (1.7)^{(L-i)-1}) \right)$$

$$\text{total_weight} = \sum_{i=0}^{L-1} \left((1.7)^{(L-i)-1} \right)$$

$$\text{effective_pipeline} = \begin{cases} \frac{\text{weighted_pipeline_sum}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean})^2)}{L} \\ \text{pipeline_std} &= (\text{variance})^{0.5} \end{aligned} \right\} (L > 1)$$

$$\text{pipeline_std} = 0 \quad (\neg (L > 1))$$

$$\text{variability_adjustment} = (\text{pipeline_std} \cdot 0.4)$$

$$\text{adjusted_safety_stock} = (57.9 + \text{variability_adjustment})$$

$$\text{anticipated_demand} = \begin{cases} (\text{effective_pipeline} \cdot 0.5), & \text{effective_pipeline} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{effective_on_hand} = (h \cdot 0.9)$$

$$\text{inventory_position} = (\text{effective_on_hand} + (\text{effective_pipeline} \cdot 0.7))$$

$$\text{target_order} = (((602.7 + \text{adjusted_safety_stock}) + \text{anticipated_demand}) - \text{inventory_position})$$

$$\text{order_amount} = \begin{cases} (\text{target_order} \cdot 0.1), & \text{target_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \begin{cases} 0, & (\text{order_amount} > 0 \wedge \text{order_amount} < 10.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$\text{order_amount} = \max(0, \text{order_amount})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 38-82

```

38.     demand_anticipation = 0.479255125254089 # OPT_PARAM: {"initial": 0.479255125254089, "min": 0.0,
    "max": 1.0, "type": "float"}
39.
40.     # Calculate weighted pipeline sum (recent orders weighted more heavily)
41.     weighted_pipeline_sum = 0.0
42.     total_weight = 0.0
43.     for i, order in enumerate(pipeline_orders):
44.         weight = recent_pipeline_weight * (len(pipeline_orders) - i - 1)
45.         weighted_pipeline_sum += order * weight
46.         total_weight += weight
47.
48.     # Calculate effective pipeline with exponential weighting
49.     effective_pipeline = weighted_pipeline_sum / total_weight if total_weight > 0 else 0
50.
51.     # NEW: Calculate pipeline variability (standard deviation)
52.     if len(pipeline_orders) > 1:
53.         pipeline_mean = sum(pipeline_orders) / len(pipeline_orders)
54.         variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / len(pipeline_orders)
55.         pipeline_std = variance ** 0.5
56.     else:
57.         pipeline_std = 0
58.
59.     # NEW: Adjust safety stock based on pipeline variability
60.     variability_adjustment = pipeline_std * variability_factor
61.     adjusted_safety_stock = safety_stock + variability_adjustment
62.
63.     # NEW: Anticipate demand from recent pipeline pattern
64.     if effective_pipeline > 0:
65.         anticipated_demand = effective_pipeline * demand_anticipation
66.     else:
67.         anticipated_demand = 0
68.
69.     # Calculate effective inventory position
70.     effective_on_hand = on_hand_inventory * on_hand_weight
71.
72.     # Calculate inventory position
73.     inventory_position = effective_on_hand + effective_pipeline * pipeline_weight
74.
75.     # Calculate target order using smoothed adjustment with NEW adjustments
76.     target_order = base_stock + adjusted_safety_stock + anticipated_demand - inventory_position
77.
78.     # Apply smoothing to avoid drastic changes
79.     if target_order > 0:
80.         order_amount = target_order * smoothing_factor
81.     else:
82.         order_amount = 0

```

Declarative mathematical formula rows 1-17

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline_sum} = \sum_{i=0}^{L-1} ((q_i \cdot (1.7)^{((L-i)-1)}))$$

$$\text{total_weight} = \sum_{i=0}^{L-1} ((1.7)^{((L-i)-1)})$$

$$\text{effective_pipeline} = \begin{cases} \frac{\text{weighted_pipeline_sum}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean})^2)}{L} \\ \text{pipeline_std} &= (\text{variance})^{0.5} \end{aligned} \right\} (L > 1)$$

$$\text{pipeline_std} = 0 \quad (\neg (L > 1))$$

$$\text{variability_adjustment} = (\text{pipeline_std} \cdot 0.4)$$

$$\text{adjusted_safety_stock} = (57.9 + \text{variability_adjustment})$$

$$\text{anticipated_demand} = \begin{cases} (\text{effective_pipeline} \cdot 0.5), & \text{effective_pipeline} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{effective_on_hand} = (h \cdot 0.9)$$

$$\text{inventory_position} = (\text{effective_on_hand} + (\text{effective_pipeline} \cdot 0.7))$$

$$\text{target_order} = (((602.7 + \text{adjusted_safety_stock}) + \text{anticipated_demand}) - \text{inventory_position})$$

$$\text{order_amount} = \begin{cases} (\text{target_order} \cdot 0.1), & \text{target_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \begin{cases} 0, & (\text{order_amount} > 0 \wedge \text{order_amount} < 10.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$\text{order_amount} = \max(0, \text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 83-94

```

83.
84.     # Apply minimum order threshold
85.     if order_amount > 0 and order_amount < min_order_threshold:
86.         order_amount = 0
87.
88.     # Round to nearest integer (orders must be integer quantities)
89.     order_amount = int(round(order_amount))
90.
91.     # Ensure non-negative
92.     order_amount = max(0, order_amount)
93.
94.     return order_amount

```

Declarative mathematical formula rows 1-17

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline_sum} = \sum_{i=0}^{L-1} ((q_i \cdot (1.7)^{((L-i)-1)}))$$

$$\text{total_weight} = \sum_{i=0}^{L-1} ((1.7)^{((L-i)-1)})$$

$$\text{effective_pipeline} = \begin{cases} \frac{\text{weighted_pipeline_sum}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean})^2)}{L} \\ \text{pipeline_std} &= (\text{variance})^{0.5} \end{aligned} \right\} (L > 1)$$

$$\text{pipeline_std} = 0 \quad (\neg(L > 1))$$

$$\text{variability_adjustment} = (\text{pipeline_std} \cdot 0.4)$$

$$\text{adjusted_safety_stock} = (57.9 + \text{variability_adjustment})$$

$$\text{anticipated_demand} = \begin{cases} (\text{effective_pipeline} \cdot 0.5), & \text{effective_pipeline} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{effective_on_hand} = (h \cdot 0.9)$$

$$\text{inventory_position} = (\text{effective_on_hand} + (\text{effective_pipeline} \cdot 0.7))$$

$$\text{target_order} = (((602.7 + \text{adjusted_safety_stock}) + \text{anticipated_demand}) - \text{inventory_position})$$

$$\text{order_amount} = \begin{cases} (\text{target_order} \cdot 0.1), & \text{target_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \begin{cases} 0, & (\text{order_amount} > 0 \wedge \text{order_amount} < 10.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$\text{order_amount} = \max(0, \text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-36

```

1. # sample_id: 006
2. # folder: deepseek-chat_poisson_L4_c1_2_50_plain_processed_scipy_15_default_m2_10_r2
3. # distribution: poisson_L4_c1_2
4. # generation: 2
5. # rank_in_population_file: 9
6. # objective: 1727.94079
7. # test_objective: 1745.73919
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 430.7383762019312 # OPT_PARAM: {"initial": 430.7383762019312, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 0.033629772620031445 # OPT_PARAM: {"initial": 0.033629772620031445, "min": 0,
16.     "max": 200, "type": "float"}
17.     demand_forecast = 51.26511638307087 # OPT_PARAM: {"initial": 51.26511638307087, "min": 50,
18.     "max": 150, "type": "float"}
19.     pipeline_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.0, "max": 1.0, "type": "float"}
20.
21.     # Calculate effective inventory position
22.     effective_pipeline = sum(pipeline_orders) * pipeline_weight
23.     inventory_position = on_hand_inventory + effective_pipeline
24.
25.     # Adjust base stock based on recent demand pattern
26.     recent_arrivals = pipeline_orders[0] if pipeline_orders else 0
27.     if recent_arrivals > 0:
28.         demand_adjustment = min(demand_forecast, recent_arrivals * 1.2)
29.     else:
30.         demand_adjustment = demand_forecast
31.
32.     # Dynamic target inventory level
33.     target_inventory = base_stock + safety_stock + demand_adjustment
34.
35.     # Calculate order amount with smoothing
36.     order_amount = max(0, target_inventory - inventory_position)
37.
38.     # Round to nearest integer (as required by output type)
39.     return order_amount

```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{effective_pipeline} = \left(\sum_{i=0}^{L-1} q_i \cdot 1.0 \right)$$

$$\text{inventory_position} = (h + \text{effective_pipeline})$$

$$\text{recent_arrivals} = \begin{cases} q_0, & \mathbf{q} \\ 0, & \text{otherwise} \end{cases}$$

$$\text{demand_adjustment} = \begin{cases} \min(51.3, (\text{recent_arrivals} \cdot 1.2)), & \text{recent_arrivals} > 0 \\ 51.3, & \text{otherwise} \end{cases}$$

$$\text{target_inventory} = ((430.7 + 3.4e - 02) + \text{demand_adjustment})$$

$$\text{order_amount} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-30

```
1. # sample_id: 007
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: normal_std30_L6_c1_2
4. # generation: 2
5. # rank_in_population_file: 9
6. # objective: 4301.9
7. # test_objective: 4257.044
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to
11. # extra_motifs: safety_stock_buffer;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 594.9887769151144 # OPT_PARAM: {"initial": 594.9887769151144, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 50.00013241732881 # OPT_PARAM: {"initial": 50.00013241732881, "min": 0, "max":
16.     200, "type": "float"}
17.     pipeline_factor = 1.1195224967701995 # OPT_PARAM: {"initial": 1.1195224967701995, "min": 0.5,
18.     "max": 1.2, "type": "float"}
19.
20.     # Calculate effective inventory position
21.     effective_pipeline = sum(pipeline_orders) * pipeline_factor
22.     inventory_position = on_hand_inventory + effective_pipeline
23.
24.     # Adjust base stock based on safety stock
25.     adjusted_base_stock = base_stock + safety_stock
26.
27.     # Calculate order amount
28.     order_amount = max(0, adjusted_base_stock - inventory_position)
29.
30.     # Round to nearest integer since order amount should be integer
31.     order_amount = int(round(order_amount))
32.
33.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\max \left(0, \left((595.0 + 50.0) - \left(h + \left(\sum_{i=0}^{L-1} q_i \cdot 1.1 \right) \right) \right) \right) \right).$$

Python source lines 1-43

```
1. # sample_id: 008
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_4_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 5
5. # rank_in_population_file: 8
6. # objective: 974.63576
7. # test_objective: 939.0302
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 510.93467142866245 # OPT_PARAM: {"initial": 510.93467142866245, "min": 400, "max": 700, "type": "float"}
14.     safety_stock = 45.0 # OPT_PARAM: {"initial": 45.0, "min": 20, "max": 80, "type": "float"}
15.     demand_estimate = 97.75006872229582 # OPT_PARAM: {"initial": 97.75006872229582, "min": 90, "max": 110, "type": "float"}
16.     smoothing_factor = 0.05 # OPT_PARAM: {"initial": 0.05, "min": 0.05, "max": 0.3, "type": "float"}
17.     pipeline_weight = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.1, "max": 0.5, "type": "float"}
18.
19.     # Calculate net inventory position
20.     net_inventory = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand during lead time
23.     expected_lead_time_demand = demand_estimate * len(pipeline_orders)
24.
25.     # Calculate target inventory position
26.     target_position = expected_lead_time_demand + safety_stock
27.
28.     # Calculate base order amount using (s,S) logic
29.     base_order = max(0, target_position - net_inventory)
30.
31.     # Apply base stock ceiling
32.     max_order = max(0, base_stock - net_inventory)
33.
34.     # Adjust for pipeline coverage (reduce order if pipeline is already high)
35.     pipeline_coverage = sum(pipeline_orders) / max(1, expected_lead_time_demand)
36.     pipeline_adjustment = 1.0 - pipeline_weight * min(1.0, pipeline_coverage)
37.
38.     # Calculate adjusted order
39.     adjusted_order = min(base_order, max_order) * pipeline_adjustment
40.
41.     # Smooth with demand estimate to prevent extreme orders
42.     order_amount = smoothing_factor * adjusted_order + (1 - smoothing_factor) * demand_estimate
43.
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, q) = \left(\left(0.1 \cdot \left(\min \left(\max \left(0, \left((97.8 \cdot L) + 45.0 \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right), \max \left(0, \left(510.9 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \cdot \left(1.0 - \left(0.5 \cdot \min \left(1.0, \frac{\sum_{i=0}^{L-1} q_i}{\max(1, (97.8 \cdot L))} \right) \right) \right) \right) \right) + ((1 - 0.1) \cdot 97.8) \right).$$

Python source lines 44-45

```
44.     # Round to nearest integer
45.     return order_amount
```

Declarative mathematical formula rows 1-2

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \left(\left(0.1 \cdot \left(\min \left(\max \left(0, \left((97.8 \cdot L) + 45.0 \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right), \max \left(0, \left(510.9 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right) \cdot \left(1.0 - \left(0.5 \cdot \min \left(1.0, \frac{\sum_{i=0}^{L-1} q_i}{\max(1, (97.8 \cdot L))} \right) \right) \right) \right) + ((1 - 0.1) \cdot 97.8) \right).$$

Python source lines 1-42

```

1. # sample_id: 009
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_e1-e1-e1_4_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 2
6. # objective: 6027.72
7. # test_objective: 6152.459
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 303.5203530187662 # OPT_PARAM: {"initial": 303.5203530187662, "min": 100, "max": 800, "type": "float"}
15.
16.     # Critical ratio for newsvendor-like adjustment
17.     critical_ratio = 0.9 # OPT_PARAM: {"initial": 0.9, "min": 0.5, "max": 0.9, "type": "float"}
18.
19.     # Pipeline pattern recognition sensitivity
20.     pattern_sensitivity = 0.20257569496074482 # OPT_PARAM: {"initial": 0.20257569496074482, "min": 0.0, "max": 2.0, "type": "float"}
21.
22.     # Order smoothing factor
23.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.0, "max": 1.0, "type": "float"}
24.
25.     # Minimum order quantity
26.     min_order = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 50, "type": "float"}
27.
28.     # Maximum order quantity
29.     max_order = 800.0 # OPT_PARAM: {"initial": 800.0, "min": 200, "max": 1500, "type": "float"}
30.
31.     # NEW STRUCTURAL ELEMENT: Pattern-based demand forecasting
32.     # Instead of just averaging pipeline, we detect specific patterns:
33.     # 1. Increasing trend
34.     # 2. Decreasing trend
35.     # 3. Spike pattern
36.     # 4. Stable pattern
37.
38.     L = len(pipeline_orders)
39.
40.     if L >= 3:
41.         # Split pipeline into thirds for pattern analysis
42.         third = L // 3

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 & \text{third} = \left\lfloor \frac{L}{3} \right\rfloor \\
 & \text{third} = \begin{cases} 1, & \text{third} = 0 \\ \text{third}, & \text{otherwise} \end{cases} \\
 & \text{first_avg} = \frac{\sum_{i=0}^{(\text{third})-1} q_i}{\text{third}} \\
 & \text{second_avg} = \begin{cases} \frac{\sum_{i=\text{third}}^{(2 \cdot \text{third})-1} q_i}{\text{third}}, & (2 \cdot \text{third}) \leq L \\ \frac{\sum_{i=\text{third}}^{L-1} q_i}{(L - \text{third})}, & \text{otherwise} \end{cases} \\
 & \text{third_avg} = \begin{cases} \frac{\sum_{i=(2 \cdot \text{third})}^{L-1} q_i}{(L - (2 \cdot \text{third}))}, & (2 \cdot \text{third}) < L \\ 0, & \text{otherwise} \end{cases} \\
 & \text{trend_ratio} = \begin{cases} \frac{(\text{third_avg} - \text{first_avg})}{\text{third_avg}}, & \text{third_avg} > 0 \\ 0, & \text{otherwise} \end{cases} \quad (L \geq 3) \\
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \quad ((\text{pipeline_mean} > 0 \wedge L > 1)) \\
 & \text{volatility} = 0.0 \quad (\neg ((\text{pipeline_mean} > 0 \wedge L > 1))) \\
 & \text{pattern_adjustment} = \begin{cases} ((0.2 \cdot \text{trend_ratio}) \cdot 303.5), & \text{trend_ratio} > 0 \\ (((0.2 \cdot \text{trend_ratio}) \cdot 303.5) \cdot 0.5), & \text{otherwise} \end{cases} \quad (|\text{trend_ratio}| > 0.3) \\
 & \text{pattern_adjustment} = \begin{cases} (((0.2 \cdot \text{volatility}) \cdot 303.5) \cdot 0.3), & \text{volatility} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (\neg (|\text{trend_ratio}| > 0.3)) \\
 & \text{pattern_adjustment} = 0 \quad (\neg (L \geq 3)) \\
 & \text{volatility} = 0.0 \\
 & \text{newsvendor_factor} = ((0.9 \cdot 2) - 1) \\
 & \text{safety_stock} = ((303.5 \cdot 0.2) \cdot ((1 + \text{newsvendor_factor}) + \text{volatility})) \\
 & \text{adjusted_base_stock} = ((303.5 + \text{pattern_adjustment}) + \text{safety_stock}) \\
 & \text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 & \text{raw_order} = \max(0, (\text{adjusted_base_stock} - \text{net_inventory})) \\
 & \text{weighted_ref} = 0 \\
 & \text{total_weight} = 0 \\
 & \text{weighted_ref} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right) \quad (L > 0) \\
 & \text{reference_order} = \begin{cases} \frac{\text{weighted_ref}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \text{raw_order}, & \text{otherwise} \end{cases} \\
 & \text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{reference_order})) \\
 & \text{smoothed_order} = \text{raw_order} \quad (\neg (L > 0)) \\
 & \text{order_amount} = \text{round}_{\text{even}}(\max(10.0, \min(800.0, \text{smoothed_order}))) \\
 & Q(h, \mathbf{q}) = \text{order_amount}
 \end{aligned}$$

Python source lines 43-87

```

43.         if third == 0:
44.             third = 1
45.
46.         # Calculate averages for each third
47.         first_avg = sum(pipeline_orders[:third]) / third
48.         second_avg = sum(pipeline_orders[third:2*third]) / third if 2*third <= L else
sum(pipeline_orders[third:]) / (L - third)
49.         third_avg = sum(pipeline_orders[2*third:]) / (L - 2*third) if 2*third < L else 0
50.
51.         # Detect pattern
52.         if third_avg > 0:
53.             trend_ratio = (third_avg - first_avg) / third_avg
54.         else:
55.             trend_ratio = 0
56.
57.         # Calculate volatility
58.         pipeline_mean = sum(pipeline_orders) / L
59.         if pipeline_mean > 0 and L > 1:
60.             variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
61.             volatility = variance ** 0.5 / pipeline_mean
62.         else:
63.             volatility = 0.0
64.
65.         # Pattern classification and adjustment
66.         if abs(trend_ratio) > 0.3:
67.             # Strong trend detected
68.             if trend_ratio > 0:
69.                 # Increasing trend - anticipate higher demand
70.                 pattern_adjustment = pattern_sensitivity * trend_ratio * base_stock
71.             else:
72.                 # Decreasing trend - anticipate lower demand
73.                 pattern_adjustment = pattern_sensitivity * trend_ratio * base_stock * 0.5
74.         elif volatility > 0.5:
75.             # High volatility - spike pattern
76.             pattern_adjustment = pattern_sensitivity * volatility * base_stock * 0.3
77.         else:
78.             # Stable pattern
79.             pattern_adjustment = 0
80.     else:
81.         pattern_adjustment = 0
82.         volatility = 0.0
83.
84.     # NEW STRUCTURAL ELEMENT: Newsvendor-inspired adjustment
85.     # Use critical ratio (p/(p+h)) to adjust safety stock
86.     # Higher critical ratio → more aggressive ordering
87.     newsvendor_factor = critical_ratio * 2 - 1 # Maps [0.5, 0.9] to [0, 0.8]

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned}
 & \text{third} = \left\lfloor \frac{L}{3} \right\rfloor \\
 & \text{third} = \begin{cases} 1, & \text{third} = 0 \\ \text{third}, & \text{otherwise} \end{cases} \\
 & \text{first_avg} = \frac{\sum_{i=0}^{\text{third}-1} q_i}{\text{third}} \\
 & \text{second_avg} = \begin{cases} \frac{\sum_{i=\text{third}}^{(2 \cdot \text{third})-1} q_i}{\text{third}}, & (2 \cdot \text{third}) \leq L \\ \frac{\sum_{i=\text{third}}^{L-1} q_i}{(L - \text{third})}, & \text{otherwise} \end{cases} \\
 & \text{third_avg} = \begin{cases} \frac{\sum_{i=(2 \cdot \text{third})}^{L-1} q_i}{(L - (2 \cdot \text{third}))}, & (2 \cdot \text{third}) < L \\ 0, & \text{otherwise} \end{cases} \\
 & \text{trend_ratio} = \begin{cases} \frac{\text{third_avg} - \text{first_avg}}{\text{third_avg}}, & \text{third_avg} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in q} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \\
 & \text{volatility} = 0.0 \quad \neg((\text{pipeline_mean} > 0 \wedge L > 1)) \\
 & \text{pattern_adjustment} = \begin{cases} ((0.2 \cdot \text{trend_ratio}) \cdot 303.5), & \text{trend_ratio} > 0 \\ (((0.2 \cdot \text{trend_ratio}) \cdot 303.5) \cdot 0.5), & \text{otherwise} \end{cases} \quad (|\text{trend_ratio}| > 0.3) \\
 & \text{pattern_adjustment} = \begin{cases} (((0.2 \cdot \text{volatility}) \cdot 303.5) \cdot 0.3), & \text{volatility} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (\neg(|\text{trend_ratio}| > 0.3)) \\
 & \text{pattern_adjustment} = 0 \quad \neg(L \geq 3) \\
 & \text{volatility} = 0.0 \\
 & \text{newsvendor_factor} = ((0.9 \cdot 2) - 1) \\
 & \text{safety_stock} = ((303.5 \cdot 0.2) \cdot ((1 + \text{newsvendor_factor}) + \text{volatility})) \\
 & \text{adjusted_base_stock} = ((303.5 + \text{pattern_adjustment}) + \text{safety_stock}) \\
 & \text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 & \text{raw_order} = \max(0, (\text{adjusted_base_stock} - \text{net_inventory})) \\
 & \text{weighted_ref} = 0 \\
 & \text{total_weight} = 0 \\
 & \text{weighted_ref} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right) \\
 & \text{reference_order} = \begin{cases} \frac{\text{weighted_ref}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \text{raw_order}, & \text{otherwise} \end{cases} \\
 & \text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{reference_order})) \\
 & \text{smoothed_order} = \text{raw_order} \quad \neg(L > 0)
 \end{aligned} \right\} \quad (L \geq 3)$$

Python source lines 88-121

```

88.
89. # Calculate safety stock with volatility adjustment
90. safety_stock = base_stock * 0.2 * (1 + newsvendor_factor + volatility)
91.
92. # Calculate total adjustment
93. adjusted_base_stock = base_stock + pattern_adjustment + safety_stock
94.
95. # Calculate current inventory position
96. net_inventory = on_hand_inventory + sum(pipeline_orders)
97.
98. # Basic order calculation
99. raw_order = max(0, adjusted_base_stock - net_inventory)
100.
101. # Apply smoothing using weighted average of pipeline as reference
102. if L > 0:
103.     # Weight recent pipeline orders more heavily
104.     weighted_ref = 0
105.     total_weight = 0
106.     for i, q in enumerate(pipeline_orders):
107.         weight = (L - i) / L # Linear weights
108.         weighted_ref += q * weight
109.         total_weight += weight
110.
111.     reference_order = weighted_ref / total_weight if total_weight > 0 else raw_order
112.
113. # Smooth towards reference
114. smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * reference_order
115. else:
116.     smoothed_order = raw_order
117.
118. # Apply constraints
119. order_amount = int(round(max(min_order, min(max_order, smoothed_order))))
120.
121. return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned}
 & \text{third} = \left\lfloor \frac{L}{3} \right\rfloor \\
 & \text{third} = \begin{cases} 1, & \text{third} = 0 \\ \text{third}, & \text{otherwise} \end{cases} \\
 & \text{first_avg} = \frac{\sum_{i=0}^{(\text{third})-1} q_i}{\text{third}} \\
 & \text{second_avg} = \begin{cases} \frac{\sum_{i=\text{third}}^{((2 \cdot \text{third})-1)} q_i}{\text{third}}, & (2 \cdot \text{third}) \leq L \\ \frac{\sum_{i=L-\text{third}}^{L-1} q_i}{(L-\text{third})}, & \text{otherwise} \end{cases} \\
 & \text{third_avg} = \begin{cases} \frac{\sum_{i=(2 \cdot \text{third})}^{L-1} q_i}{(L-(2 \cdot \text{third}))}, & (2 \cdot \text{third}) < L \\ 0, & \text{otherwise} \end{cases} \\
 & \text{trend_ratio} = \begin{cases} \frac{(\text{third_avg} - \text{first_avg})}{\text{third_avg}}, & \text{third_avg} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \end{aligned} \right\} \begin{aligned} & ((\text{pipeline_mean} > 0 \wedge L > 1)) \\ & \text{volatility} = 0.0 \} \quad (\neg ((\text{pipeline_mean} > 0 \wedge L > 1))) \end{aligned} \\
 & \text{pattern_adjustment} = \begin{cases} ((0.2 \cdot \text{trend_ratio}) \cdot 303.5), & \text{trend_ratio} > 0 \\ (((0.2 \cdot \text{trend_ratio}) \cdot 303.5) \cdot 0.5), & \text{otherwise} \end{cases} \quad (|\text{trend_ratio}| > 0.3) \\
 & \text{pattern_adjustment} = \begin{cases} (((0.2 \cdot \text{volatility}) \cdot 303.5) \cdot 0.3), & \text{volatility} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (\neg (|\text{trend_ratio}| > 0.3)) \\
 & \text{pattern_adjustment} = 0 \} \quad (\neg (L \geq 3)) \\
 & \text{volatility} = 0.0 \\
 & \text{newsvendor_factor} = ((0.9 \cdot 2) - 1) \\
 & \text{safety_stock} = ((303.5 \cdot 0.2) \cdot ((1 + \text{newsvendor_factor}) + \text{volatility})) \\
 & \text{adjusted_base_stock} = ((303.5 + \text{pattern_adjustment}) + \text{safety_stock}) \\
 & \text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 & \text{raw_order} = \max(0, (\text{adjusted_base_stock} - \text{net_inventory})) \\
 & \text{weighted_ref} = 0 \\
 & \text{total_weight} = 0 \\
 & \text{weighted_ref} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right) \\
 & \text{reference_order} = \begin{cases} \frac{\text{weighted_ref}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \text{raw_order}, & \text{otherwise} \end{cases} \\
 & \text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{reference_order})) \end{aligned} \right\} (L > 0)$$

Python source lines 1-41

```

1. # sample_id: 010
2. # folder: deepseek-chat_normal_std10_L6_c1_2_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: normal_std10_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 6
6. # objective: 725.9
7. # test_objective: 771.368
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 503.37232950779367 # OPT_PARAM: {"initial": 503.37232950779367, "min": 400, "max":
700, "type": "float"}
14.     safety_stock = 86.71422918452267 # OPT_PARAM: {"initial": 86.71422918452267, "min": 50, "max":
150, "type": "float"}
15.     demand_estimate = 95.30195205059414 # OPT_PARAM: {"initial": 95.30195205059414, "min": 80,
"max": 120, "type": "float"}
16.     smoothing_factor = 0.01 # OPT_PARAM: {"initial": 0.01, "min": 0.01, "max": 0.3, "type": "float"}
17.     order_cap_multiplier = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 1.0, "max": 3.0, "type":
"float"}
18.     pipeline_weight = 0.13743730510232083 # OPT_PARAM: {"initial": 0.13743730510232083, "min": 0.1,
"max": 1.0, "type": "float"}
19.     lost_sales_weight = 2.0 # OPT_PARAM: {"initial": 2.0, "min": 0.5, "max": 2.0, "type": "float"}
20.     demand_adjustment = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.5, "max": 1.5, "type": "float"}
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected demand during lead time
26.     lead_time = len(pipeline_orders)
27.     expected_lead_time_demand = demand_estimate * lead_time * demand_adjustment
28.
29.     # Simplified safety stock adjustment
30.     if len(pipeline_orders) > 0:
31.         avg_pipeline = sum(pipeline_orders) / lead_time
32.         pipeline_variability = abs(avg_pipeline - demand_estimate) / (demand_estimate + 1e-6)
33.         adjusted_safety = safety_stock * (1 + pipeline_weight * pipeline_variability)
34.     else:
35.         adjusted_safety = safety_stock
36.
37.     # Cost-adjusted safety stock
38.     cost_adjusted_safety = adjusted_safety * lost_sales_weight
39.
40.     # Calculate target inventory position
41.     target_position = base_stock + cost_adjusted_safety

```

Declarative mathematical formula rows 1-14

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

lead_time = L

expected_lead_time_demand = $((95.3 \cdot \text{lead_time}) \cdot 0.8)$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{\text{lead_time}} \\ \text{pipeline_variability} &= \frac{|(\text{avg_pipeline} - 95.3)|}{(95.3 + 1.0e - 06)} \end{aligned} \right\} (L > 0)$$

adjusted_safety = $(86.7 \cdot (1 + (0.1 \cdot \text{pipeline_variability})))$

adjusted_safety = 86.7 $\neg (L > 0)$

cost_adjusted_safety = $(\text{adjusted_safety} \cdot 2.0)$

target_position = $(503.4 + \text{cost_adjusted_safety})$

order_amount = $\max(0, (\text{target_position} - \text{inventory_position}))$

smoothed_demand = $(95.3 + (1.0e - 02 \cdot \text{cost_adjusted_safety}))$

max_order = $(1.0 \cdot \text{smoothed_demand})$

order_amount = $\min(\text{order_amount}, \text{max_order})$

order_amount = $\text{round}_{\text{even}}(\text{order_amount})$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 42-54

```

42.
43.     # Calculate order amount
44.     order_amount = max(0, target_position - inventory_position)
45.
46.     # Dynamic order cap
47.     smoothed_demand = demand_estimate + smoothing_factor * cost_adjusted_safety
48.     max_order = order_cap_multiplier * smoothed_demand
49.     order_amount = min(order_amount, max_order)
50.
51.     # Ensure integer order amount
52.     order_amount = int(round(order_amount))
53.
54.     return order_amount

```

Declarative mathematical formula rows 1-14

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{lead_time} = L$

$\text{expected_lead_time_demand} = ((95.3 \cdot \text{lead_time}) \cdot 0.8)$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{\text{lead_time}} \\ \text{pipeline_variability} &= \frac{|(\text{avg_pipeline} - 95.3)|}{(95.3 + 1.0e - 06)} \end{aligned} \right\} (L > 0)$$

$\text{adjusted_safety} = (86.7 \cdot (1 + (0.1 \cdot \text{pipeline_variability})))$

$\text{adjusted_safety} = 86.7 \} \quad (\neg (L > 0))$

$\text{cost_adjusted_safety} = (\text{adjusted_safety} \cdot 2.0)$

$\text{target_position} = (503.4 + \text{cost_adjusted_safety})$

$\text{order_amount} = \max(0, (\text{target_position} - \text{inventory_position}))$

$\text{smoothed_demand} = (95.3 + (1.0e - 02 \cdot \text{cost_adjusted_safety}))$

$\text{max_order} = (1.0 \cdot \text{smoothed_demand})$

$\text{order_amount} = \min(\text{order_amount}, \text{max_order})$

$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 1-41

```

1. # sample_id: 011
2. # folder: deepseek-chat_exponential_L6_c1_5_50_plain_processed_scipy_15_default_m2plural_2_r6
3. # distribution: exponential_L6_c1_5
4. # generation: 7
5. # rank_in_population_file: 2
6. # objective: 11107.66
7. # test_objective: 11209.965
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 564.7744400524856 # OPT_PARAM: {"initial": 564.7744400524856, "min": 400, "max":
    800, "type": "float"}
14.     safety_stock = 104.87444005248125 # OPT_PARAM: {"initial": 104.87444005248125, "min": 50, "max":
    250, "type": "float"}
15.     lead_time = len(pipeline_orders)
16.
17.     # Calculate inventory position
18.     inventory_position = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Calculate expected demand during lead time using recent arrivals
21.     # Use first 3 pipeline orders (recently arrived) as proxy for demand
22.     if len(pipeline_orders) >= 3:
23.         recent_demand = pipeline_orders[0] + pipeline_orders[1] + pipeline_orders[2]
24.         avg_recent_demand = recent_demand / 3.0
25.     else:
26.         avg_recent_demand = sum(pipeline_orders) / max(len(pipeline_orders), 1)
27.
28.     # Dynamic adjustment based on recent demand
29.     demand_adjustment_factor = 0.7326478951298112 # OPT_PARAM: {"initial": 0.7326478951298112,
    "min": 0.3, "max": 1.5, "type": "float"}
30.     demand_adjustment = avg_recent_demand * lead_time * demand_adjustment_factor
31.
32.     # Calculate target inventory position
33.     target_position = base_stock + safety_stock + demand_adjustment
34.
35.     # Calculate raw order needed
36.     raw_order = max(0, target_position - inventory_position)
37.
38.     # Apply aggressive smoothing to prevent over-ordering
39.     smoothing_factor = 0.16978525457711363 # OPT_PARAM: {"initial": 0.16978525457711363, "min": 0.1,
    "max": 0.5, "type": "float"}
40.     smoothed_order = smoothing_factor * raw_order
41.

```

Declarative mathematical formula rows 1-11

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{recent_demand} &= ((q_0 + q_1) + q_2) \\ \text{avg_recent_demand} &= \frac{\text{recent_demand}}{3.0} \end{aligned} \right\} \quad (L \geq 3)$$

$$\text{avg_recent_demand} = \frac{\sum_{i=0}^{L-1} q_i}{\max(L, 1)} \quad (\neg (L \geq 3))$$

$$\text{demand_adjustment} = ((\text{avg_recent_demand} \cdot \text{lead_time}) \cdot 0.7)$$

$$\text{target_position} = ((564.8 + 104.9) + \text{demand_adjustment})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = (0.2 \cdot \text{raw_order})$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 42-45

```
42.     # Round to nearest integer
43.     order_amount = int(round(smoothed_order))
44.
45.     return order_amount
```

Declarative mathematical formula rows 1-11

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{lead_time} = L$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{array}{l} \text{recent_demand} = ((q_0 + q_1) + q_2) \\ \text{avg_recent_demand} = \frac{\text{recent_demand}}{3.0} \end{array} \right\} \quad (L \geq 3)$$

$$\text{avg_recent_demand} = \frac{\sum_{i=0}^{L-1} q_i}{\max(L, 1)} \quad (\neg (L \geq 3))$$

$\text{demand_adjustment} = ((\text{avg_recent_demand} \cdot \text{lead_time}) \cdot 0.7)$

$\text{target_position} = ((564.8 + 104.9) + \text{demand_adjustment})$

$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$

$\text{smoothed_order} = (0.2 \cdot \text{raw_order})$

$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-40

```

1. # sample_id: 012
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: normal_std30_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 5
6. # objective: 4090.6
7. # test_objective: 4114.324
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    pipeline_weighting;order_up_to;state_dependent_target;partial_adjustment;order_smoothing
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 530.022331273577 # OPT_PARAM: {"initial": 530.022331273577, "min": 400, "max": 700,
    "type": "float"}
14.     safety_stock = 50.0 # OPT_PARAM: {"initial": 50.0, "min": 20, "max": 100, "type": "float"}
15.     demand_estimate = 115.0 # OPT_PARAM: {"initial": 115.0, "min": 90, "max": 140, "type": "float"}
16.     pipeline_weight = 0.9500996931856073 # OPT_PARAM: {"initial": 0.9500996931856073, "min": 0.8,
    "max": 1.0, "type": "float"}
17.     smoothing_factor = 0.9002990795568219 # OPT_PARAM: {"initial": 0.9002990795568219, "min": 0.5,
    "max": 0.95, "type": "float"}
18.     cost_ratio_adjust = 0.667 # OPT_PARAM: {"initial": 0.667, "min": 0.5, "max": 0.8, "type":
    "float"}
19.     pipeline_discount = 0.980039877242429 # OPT_PARAM: {"initial": 0.980039877242429, "min": 0.9,
    "max": 1.0, "type": "float"}
20.
21.     # Calculate weighted pipeline inventory with time-based discounting
22.     weighted_pipeline = sum(p * (pipeline_weight ** i) * (pipeline_discount ** i)
23.                             for i, p in enumerate(reversed(pipeline_orders)))
24.
25.     # Calculate net inventory position
26.     net_inventory = on_hand_inventory + weighted_pipeline
27.
28.     # Adjust safety stock based on cost ratio p/(p+h) = 2/3
29.     adjusted_safety_stock = safety_stock * cost_ratio_adjust
30.
31.     # Calculate expected demand coverage for lead time
32.     lead_time_demand = demand_estimate * len(pipeline_orders)
33.
34.     # Calculate target inventory position
35.     target_position = lead_time_demand + adjusted_safety_stock
36.
37.     # Calculate raw order amount
38.     raw_order = max(0, target_position - net_inventory)
39.
40.     # Apply base stock as upper bound

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_{L-1-i} \cdot (1.0)^i) \cdot (1.0)^i)$$

$$\text{net_inventory} = (h + \text{weighted_pipeline})$$

$$\text{adjusted_safety_stock} = (50.0 \cdot 0.7)$$

$$\text{lead_time_demand} = (115.0 \cdot L)$$

$$\text{target_position} = (\text{lead_time_demand} + \text{adjusted_safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{net_inventory}))$$

$$\text{base_stock_order} = \max(0, (530.0 - \text{net_inventory}))$$

$$\text{order_amount} = \min(\text{raw_order}, \text{base_stock_order})$$

$$\left. \begin{array}{l} \text{previous_order} = q_{-(1)} \\ \text{order_amount} = ((0.9 \cdot \text{order_amount}) + ((1 - 0.9) \cdot \text{previous_order})) \end{array} \right\} (L > 0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 41-52

```

41.     base_stock_order = max(0, base_stock - net_inventory)
42.     order_amount = min(raw_order, base_stock_order)
43.
44.     # Smooth ordering using previous order if available
45.     if len(pipeline_orders) > 0:
46.         previous_order = pipeline_orders[-1]
47.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * previous_order
48.
49.     # Ensure non-negative integer order
50.     order_amount = max(0, int(round(order_amount)))
51.
52.     return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_{L-1-i} \cdot (1.0)^i) \cdot (1.0)^i)$$

$$\text{net_inventory} = (h + \text{weighted_pipeline})$$

$$\text{adjusted_safety_stock} = (50.0 \cdot 0.7)$$

$$\text{lead_time_demand} = (115.0 \cdot L)$$

$$\text{target_position} = (\text{lead_time_demand} + \text{adjusted_safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{net_inventory}))$$

$$\text{base_stock_order} = \max(0, (530.0 - \text{net_inventory}))$$

$$\text{order_amount} = \min(\text{raw_order}, \text{base_stock_order})$$

$$\left. \begin{array}{l} \text{previous_order} = q_{-(1)} \\ \text{order_amount} = ((0.9 \cdot \text{order_amount}) + ((1 - 0.9) \cdot \text{previous_order})) \end{array} \right\} \quad (L > 0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-42

```

1. # sample_id: 013
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_e1-e1-e1_4_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 9
5. # rank_in_population_file: 2
6. # objective: 6027.72
7. # test_objective: 6152.459
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 303.5203530187662 # OPT_PARAM: {"initial": 303.5203530187662, "min": 100, "max": 800, "type": "float"}
15.
16.     # Critical ratio for newsvendor-like adjustment
17.     critical_ratio = 0.9 # OPT_PARAM: {"initial": 0.9, "min": 0.5, "max": 0.9, "type": "float"}
18.
19.     # Pipeline pattern recognition sensitivity
20.     pattern_sensitivity = 0.20257569496074482 # OPT_PARAM: {"initial": 0.20257569496074482, "min": 0.0, "max": 2.0, "type": "float"}
21.
22.     # Order smoothing factor
23.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.0, "max": 1.0, "type": "float"}
24.
25.     # Minimum order quantity
26.     min_order = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 50, "type": "float"}
27.
28.     # Maximum order quantity
29.     max_order = 800.0 # OPT_PARAM: {"initial": 800.0, "min": 200, "max": 1500, "type": "float"}
30.
31.     # NEW STRUCTURAL ELEMENT: Pattern-based demand forecasting
32.     # Instead of just averaging pipeline, we detect specific patterns:
33.     # 1. Increasing trend
34.     # 2. Decreasing trend
35.     # 3. Spike pattern
36.     # 4. Stable pattern
37.
38.     L = len(pipeline_orders)
39.
40.     if L >= 3:
41.         # Split pipeline into thirds for pattern analysis
42.         third = L // 3

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 & \text{third} = \left\lfloor \frac{L}{3} \right\rfloor \\
 & \text{third} = \begin{cases} 1, & \text{third} = 0 \\ \text{third}, & \text{otherwise} \end{cases} \\
 & \text{first_avg} = \frac{\sum_{i=0}^{(\text{third})-1} q_i}{\text{third}} \\
 & \text{second_avg} = \begin{cases} \frac{\sum_{i=\text{third}}^{(2 \cdot \text{third})-1} q_i}{\text{third}}, & (2 \cdot \text{third}) \leq L \\ \frac{\sum_{i=\text{third}}^{L-1} q_i}{(L - \text{third})}, & \text{otherwise} \end{cases} \\
 & \text{third_avg} = \begin{cases} \frac{\sum_{i=(2 \cdot \text{third})}^{L-1} q_i}{(L - (2 \cdot \text{third}))}, & (2 \cdot \text{third}) < L \\ 0, & \text{otherwise} \end{cases} \\
 & \text{trend_ratio} = \begin{cases} \frac{(\text{third_avg} - \text{first_avg})}{\text{third_avg}}, & \text{third_avg} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \\
 & \text{volatility} = 0.0 \quad \{ \neg ((\text{pipeline_mean} > 0 \wedge L > 1)) \} \\
 & \text{pattern_adjustment} = \begin{cases} ((0.2 \cdot \text{trend_ratio}) \cdot 303.5), & \text{trend_ratio} > 0 \\ (((0.2 \cdot \text{trend_ratio}) \cdot 303.5) \cdot 0.5), & \text{otherwise} \end{cases} \quad \{ |\text{trend_ratio}| > 0.3 \} \\
 & \text{pattern_adjustment} = \begin{cases} (((0.2 \cdot \text{volatility}) \cdot 303.5) \cdot 0.3), & \text{volatility} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad \{ \neg (|\text{trend_ratio}| > 0.3) \} \\
 & \text{pattern_adjustment} = 0 \quad \{ \neg (L \geq 3) \} \\
 & \text{volatility} = 0.0 \quad \{ \neg (L \geq 3) \} \\
 & \text{newsvendor_factor} = ((0.9 \cdot 2) - 1) \\
 & \text{safety_stock} = ((303.5 \cdot 0.2) \cdot ((1 + \text{newsvendor_factor}) + \text{volatility})) \\
 & \text{adjusted_base_stock} = ((303.5 + \text{pattern_adjustment}) + \text{safety_stock}) \\
 & \text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 & \text{raw_order} = \max(0, (\text{adjusted_base_stock} - \text{net_inventory})) \\
 & \text{weighted_ref} = 0 \\
 & \text{total_weight} = 0 \\
 & \text{weighted_ref} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right) \\
 & \text{reference_order} = \begin{cases} \frac{\text{weighted_ref}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \text{raw_order}, & \text{otherwise} \end{cases} \\
 & \text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{reference_order})) \\
 & \text{smoothed_order} = \text{raw_order} \quad \{ \neg (L > 0) \} \\
 & \text{order_amount} = \text{round}_{\text{even}}(\max(10.0, \min(800.0, \text{smoothed_order}))) \\
 & Q(h, \mathbf{q}) = \text{order_amount}
 \end{aligned}$$

Python source lines 43-87

```

43.         if third == 0:
44.             third = 1
45.
46.         # Calculate averages for each third
47.         first_avg = sum(pipeline_orders[:third]) / third
48.         second_avg = sum(pipeline_orders[third:2*third]) / third if 2*third <= L else
sum(pipeline_orders[third:]) / (L - third)
49.         third_avg = sum(pipeline_orders[2*third:]) / (L - 2*third) if 2*third < L else 0
50.
51.         # Detect pattern
52.         if third_avg > 0:
53.             trend_ratio = (third_avg - first_avg) / third_avg
54.         else:
55.             trend_ratio = 0
56.
57.         # Calculate volatility
58.         pipeline_mean = sum(pipeline_orders) / L
59.         if pipeline_mean > 0 and L > 1:
60.             variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
61.             volatility = variance ** 0.5 / pipeline_mean
62.         else:
63.             volatility = 0.0
64.
65.         # Pattern classification and adjustment
66.         if abs(trend_ratio) > 0.3:
67.             # Strong trend detected
68.             if trend_ratio > 0:
69.                 # Increasing trend - anticipate higher demand
70.                 pattern_adjustment = pattern_sensitivity * trend_ratio * base_stock
71.             else:
72.                 # Decreasing trend - anticipate lower demand
73.                 pattern_adjustment = pattern_sensitivity * trend_ratio * base_stock * 0.5
74.         elif volatility > 0.5:
75.             # High volatility - spike pattern
76.             pattern_adjustment = pattern_sensitivity * volatility * base_stock * 0.3
77.         else:
78.             # Stable pattern
79.             pattern_adjustment = 0
80.     else:
81.         pattern_adjustment = 0
82.         volatility = 0.0
83.
84.     # NEW STRUCTURAL ELEMENT: Newsvendor-inspired adjustment
85.     # Use critical ratio (p/(p+h)) to adjust safety stock
86.     # Higher critical ratio → more aggressive ordering
87.     newsvendor_factor = critical_ratio * 2 - 1 # Maps [0.5, 0.9] to [0, 0.8]

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned}
 & \text{third} = \left\lfloor \frac{L}{3} \right\rfloor \\
 & \text{third} = \begin{cases} 1, & \text{third} = 0 \\ \text{third}, & \text{otherwise} \end{cases} \\
 & \text{first_avg} = \frac{\sum_{i=0}^{\text{third}-1} q_i}{\text{third}} \\
 & \text{second_avg} = \begin{cases} \frac{\sum_{i=\text{third}}^{(2 \cdot \text{third})-1} q_i}{\text{third}}, & (2 \cdot \text{third}) \leq L \\ \frac{\sum_{i=\text{third}}^{L-1} q_i}{(L - \text{third})}, & \text{otherwise} \end{cases} \\
 & \text{third_avg} = \begin{cases} \frac{\sum_{i=(2 \cdot \text{third})}^{L-1} q_i}{(L - (2 \cdot \text{third}))}, & (2 \cdot \text{third}) < L \\ 0, & \text{otherwise} \end{cases} \\
 & \text{trend_ratio} = \begin{cases} \frac{\text{third_avg} - \text{first_avg}}{\text{third_avg}}, & \text{third_avg} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in q} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \\
 & \text{volatility} = 0.0 \quad \neg((\text{pipeline_mean} > 0 \wedge L > 1)) \\
 & \text{pattern_adjustment} = \begin{cases} ((0.2 \cdot \text{trend_ratio}) \cdot 303.5), & \text{trend_ratio} > 0 \\ (((0.2 \cdot \text{trend_ratio}) \cdot 303.5) \cdot 0.5), & \text{otherwise} \end{cases} \quad (|\text{trend_ratio}| > 0.3) \\
 & \text{pattern_adjustment} = \begin{cases} (((0.2 \cdot \text{volatility}) \cdot 303.5) \cdot 0.3), & \text{volatility} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (\neg(|\text{trend_ratio}| > 0.3)) \\
 & \text{pattern_adjustment} = 0 \quad \neg(L \geq 3) \\
 & \text{volatility} = 0.0 \\
 & \text{newsvendor_factor} = ((0.9 \cdot 2) - 1) \\
 & \text{safety_stock} = ((303.5 \cdot 0.2) \cdot ((1 + \text{newsvendor_factor}) + \text{volatility})) \\
 & \text{adjusted_base_stock} = ((303.5 + \text{pattern_adjustment}) + \text{safety_stock}) \\
 & \text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 & \text{raw_order} = \max(0, (\text{adjusted_base_stock} - \text{net_inventory})) \\
 & \text{weighted_ref} = 0 \\
 & \text{total_weight} = 0 \\
 & \text{weighted_ref} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right) \\
 & \text{reference_order} = \begin{cases} \frac{\text{weighted_ref}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \text{raw_order}, & \text{otherwise} \end{cases} \\
 & \text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{reference_order})) \\
 & \text{smoothed_order} = \text{raw_order} \quad \neg(L > 0)
 \end{aligned} \right\} \quad (L \geq 3)$$

Python source lines 88-121

```

88.
89. # Calculate safety stock with volatility adjustment
90. safety_stock = base_stock * 0.2 * (1 + newsvendor_factor + volatility)
91.
92. # Calculate total adjustment
93. adjusted_base_stock = base_stock + pattern_adjustment + safety_stock
94.
95. # Calculate current inventory position
96. net_inventory = on_hand_inventory + sum(pipeline_orders)
97.
98. # Basic order calculation
99. raw_order = max(0, adjusted_base_stock - net_inventory)
100.
101. # Apply smoothing using weighted average of pipeline as reference
102. if L > 0:
103.     # Weight recent pipeline orders more heavily
104.     weighted_ref = 0
105.     total_weight = 0
106.     for i, q in enumerate(pipeline_orders):
107.         weight = (L - i) / L # Linear weights
108.         weighted_ref += q * weight
109.         total_weight += weight
110.
111.     reference_order = weighted_ref / total_weight if total_weight > 0 else raw_order
112.
113. # Smooth towards reference
114. smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * reference_order
115. else:
116.     smoothed_order = raw_order
117.
118. # Apply constraints
119. order_amount = int(round(max(min_order, min(max_order, smoothed_order))))
120.
121. return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned}
 & \text{third} = \left\lfloor \frac{L}{3} \right\rfloor \\
 & \text{third} = \begin{cases} 1, & \text{third} = 0 \\ \text{third}, & \text{otherwise} \end{cases} \\
 & \text{first_avg} = \frac{\sum_{i=0}^{(\text{third})-1} q_i}{\text{third}} \\
 & \text{second_avg} = \begin{cases} \frac{\sum_{i=\text{third}}^{((2 \cdot \text{third})-1)} q_i}{\text{third}}, & (2 \cdot \text{third}) \leq L \\ \frac{\sum_{i=L-\text{third}}^{L-1} q_i}{(L-\text{third})}, & \text{otherwise} \end{cases} \\
 & \text{third_avg} = \begin{cases} \frac{\sum_{i=(2 \cdot \text{third})}^{L-1} q_i}{(L-(2 \cdot \text{third}))}, & (2 \cdot \text{third}) < L \\ 0, & \text{otherwise} \end{cases} \\
 & \text{trend_ratio} = \begin{cases} \frac{(\text{third_avg} - \text{first_avg})}{\text{third_avg}}, & \text{third_avg} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \end{aligned} \right\} ((\text{pipeline_mean} > 0 \wedge L > 1))$$

$$\left. \begin{aligned}
 & \text{volatility} = 0.0 \quad \neg((\text{pipeline_mean} > 0 \wedge L > 1)) \\
 & \text{pattern_adjustment} = \begin{cases} ((0.2 \cdot \text{trend_ratio}) \cdot 303.5), & \text{trend_ratio} > 0 \\ (((0.2 \cdot \text{trend_ratio}) \cdot 303.5) \cdot 0.5), & \text{otherwise} \end{cases} \quad (|\text{trend_ratio}| > 0.3) \\
 & \text{pattern_adjustment} = \begin{cases} (((0.2 \cdot \text{volatility}) \cdot 303.5) \cdot 0.3), & \text{volatility} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (\neg(|\text{trend_ratio}| > 0.3)) \\
 & \text{pattern_adjustment} = 0 \quad \neg(L \geq 3) \\
 & \text{volatility} = 0.0 \quad \neg(L \geq 3) \\
 & \text{newsvendor_factor} = ((0.9 \cdot 2) - 1) \\
 & \text{safety_stock} = ((303.5 \cdot 0.2) \cdot ((1 + \text{newsvendor_factor}) + \text{volatility})) \\
 & \text{adjusted_base_stock} = ((303.5 + \text{pattern_adjustment}) + \text{safety_stock}) \\
 & \text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 & \text{raw_order} = \max(0, (\text{adjusted_base_stock} - \text{net_inventory})) \\
 & \text{weighted_ref} = 0 \\
 & \text{total_weight} = 0 \\
 & \text{weighted_ref} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right) \\
 & \text{reference_order} = \begin{cases} \frac{\text{weighted_ref}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \text{raw_order}, & \text{otherwise} \end{cases} \\
 & \text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{reference_order})) \\
 & \text{smoothed_order} = \text{raw_order} \quad \neg(L > 0)
 \end{aligned} \right\} (L > 0)$$

Python source lines 1-42

```

1. # sample_id: 014
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: poisson_L6_c1_5
4. # generation: 8
5. # rank_in_population_file: 1
6. # objective: 1234.48
7. # test_objective: 1268.566
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 583.5806013968356 # OPT_PARAM: {"initial": 583.5806013968356, "min": 400, "max":
    800, "type": "float"}
15.
16.     # Safety stock adjustment
17.     safety_stock = 122.3200350131986 # OPT_PARAM: {"initial": 122.3200350131986, "min": 20, "max":
    150, "type": "float"}
18.
19.     # Demand anticipation factor
20.     anticipation_factor = 1.5 # OPT_PARAM: {"initial": 1.5, "min": 0.5, "max": 1.5, "type": "float"}
21.
22.     # Order smoothing
23.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.8, "type": "float"}
24.
25.     # Order bounds
26.     min_order = 96.64831031968099 # OPT_PARAM: {"initial": 96.64831031968099, "min": 20, "max": 100,
    "type": "float"}
27.     max_order = 200.0 # OPT_PARAM: {"initial": 200.0, "min": 100, "max": 300, "type": "float"}
28.
29.     # Pipeline urgency adjustment
30.     urgency_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.5, "type": "float"}
31.     urgency_threshold = 50.0 # OPT_PARAM: {"initial": 50.0, "min": 10, "max": 150, "type": "float"}
32.
33.     # Lead time demand coverage
34.     coverage_periods = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 2.0, "type": "float"}
35.
36.     # NEW: Cost-ratio adjustment factor
37.     cost_ratio_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.5, "type": "float"}
38.
39.     # Calculate inventory position
40.     inventory_position = on_hand_inventory + sum(pipeline_orders)
41.
42.     # Estimate near-term demand using weighted average of recent pipeline arrivals

```

Declarative mathematical formula rows 1-20

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{weighted_demand_estimate} = 0$

$\text{total_weight} = 0$

$$\text{weighted_demand_estimate} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right)$$

$$\text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right)$$

$$\text{avg_weighted_demand} = \begin{cases} \frac{\text{weighted_demand_estimate}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 100.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_base_stock} = (583.6 + (1.5 \cdot (\text{avg_weighted_demand} - 100.0)))$$

$$\text{near_pipeline} = \sum_{i=0}^{(\min(2,L))-1} q_i$$

$$\text{urgency_adjustment} = \left(1.0 + \left(\frac{(50.0 - \text{near_pipeline})}{50.0} \cdot (0.5 - 1.0) \right) \right) \Bigg\} \quad (\text{near_pipeline} < 50.0)$$

$$\text{adjusted_base_stock} = (\text{adjusted_base_stock} \cdot \text{urgency_adjustment})$$

$$\text{lead_time_demand} = (\text{avg_weighted_demand} \cdot L)$$

$$\text{coverage_adjustment} = (\text{lead_time_demand} \cdot 0.5)$$

$$\text{final_base_stock} = \max(\text{adjusted_base_stock}, \text{coverage_adjustment})$$

$$\text{cost_adjusted_base_stock} = (\text{final_base_stock} \cdot 0.5)$$

$$\text{target_inventory_position} = (\text{cost_adjusted_base_stock} + 122.3)$$

$$\text{raw_order} = \max(0, (\text{target_inventory_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot 96.6))$$

$$\text{bounded_order} = \max(96.6, \min(200.0, \text{smoothed_order}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{bounded_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 43-87

```

43.     weighted_demand_estimate = 0
44.     total_weight = 0
45.     L = len(pipeline_orders)
46.
47.     for i, q in enumerate(pipeline_orders):
48.         # Weight decreases for older pipeline orders
49.         weight = (L - i) / L
50.         weighted_demand_estimate += q * weight
51.         total_weight += weight
52.
53.     if total_weight > 0:
54.         avg_weighted_demand = weighted_demand_estimate / total_weight
55.     else:
56.         avg_weighted_demand = 100.0 # Default estimate
57.
58.     # Adjust base stock based on demand anticipation
59.     adjusted_base_stock = base_stock + anticipation_factor * (avg_weighted_demand - 100.0)
60.
61.     # Apply pipeline urgency adjustment
62.     near_pipeline = sum(pipeline_orders[:min(2, L)])
63.     if near_pipeline < urgency_threshold:
64.         urgency_adjustment = 1.0 + (urgency_threshold - near_pipeline) / urgency_threshold *
65. (urgency_factor - 1.0)
66.         adjusted_base_stock *= urgency_adjustment
67.
68.     # Ensure lead time demand coverage
69.     lead_time_demand = avg_weighted_demand * L
70.     coverage_adjustment = lead_time_demand * coverage_periods
71.     final_base_stock = max(adjusted_base_stock, coverage_adjustment)
72.
73.     # NEW: Adjust for cost ratio (p/h = 5/1 = 5)
74.     # Higher p/h ratio favors higher inventory to avoid stockouts
75.     cost_adjusted_base_stock = final_base_stock * cost_ratio_factor
76.
77.     # Add safety stock
78.     target_inventory_position = cost_adjusted_base_stock + safety_stock
79.
80.     # Calculate raw order
81.     raw_order = max(0, target_inventory_position - inventory_position)
82.
83.     # Apply smoothing
84.     smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * min_order
85.
86.     # Apply bounds
87.     bounded_order = max(min_order, min(max_order, smoothed_order))

```

Declarative mathematical formula rows 1-20

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{weighted_demand_estimate} = 0$$

$$\text{total_weight} = 0$$

$$\text{weighted_demand_estimate} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right)$$

$$\text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right)$$

$$\text{avg_weighted_demand} = \begin{cases} \frac{\text{weighted_demand_estimate}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 100.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_base_stock} = (583.6 + (1.5 \cdot (\text{avg_weighted_demand} - 100.0)))$$

$$\text{near_pipeline} = \sum_{i=0}^{(\min(2,L))-1} q_i$$

$$\text{urgency_adjustment} = \left(1.0 + \left(\frac{(50.0 - \text{near_pipeline})}{50.0} \cdot (0.5 - 1.0) \right) \right) \left\} \quad (\text{near_pipeline} < 50.0)$$

$$\text{adjusted_base_stock} = (\text{adjusted_base_stock} \cdot \text{urgency_adjustment})$$

$$\text{lead_time_demand} = (\text{avg_weighted_demand} \cdot L)$$

$$\text{coverage_adjustment} = (\text{lead_time_demand} \cdot 0.5)$$

$$\text{final_base_stock} = \max(\text{adjusted_base_stock}, \text{coverage_adjustment})$$

$$\text{cost_adjusted_base_stock} = (\text{final_base_stock} \cdot 0.5)$$

$$\text{target_inventory_position} = (\text{cost_adjusted_base_stock} + 122.3)$$

$$\text{raw_order} = \max(0, (\text{target_inventory_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot 96.6))$$

$$\text{bounded_order} = \max(96.6, \min(200.0, \text{smoothed_order}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{bounded_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 88-91

```

88.     # Round to nearest integer
89.     order_amount = int(round(bounded_order))
90.
91.     return order_amount

```

Declarative mathematical formula rows 1-20

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{weighted_demand_estimate} = 0$$

$$\text{total_weight} = 0$$

$$\text{weighted_demand_estimate} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{(L-i)}{L} \right) \right)$$

$$\text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{(L-i)}{L} \right)$$

$$\text{avg_weighted_demand} = \begin{cases} \frac{\text{weighted_demand_estimate}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 100.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_base_stock} = (583.6 + (1.5 \cdot (\text{avg_weighted_demand} - 100.0)))$$

$$\text{near_pipeline} = \sum_{i=0}^{(\min(2,L))-1} q_i$$

$$\text{urgency_adjustment} = \left(1.0 + \left(\frac{(50.0 - \text{near_pipeline})}{50.0} \cdot (0.5 - 1.0) \right) \right) \left\} \quad (\text{near_pipeline} < 50.0)$$

$$\text{adjusted_base_stock} = (\text{adjusted_base_stock} \cdot \text{urgency_adjustment})$$

$$\text{lead_time_demand} = (\text{avg_weighted_demand} \cdot L)$$

$$\text{coverage_adjustment} = (\text{lead_time_demand} \cdot 0.5)$$

$$\text{final_base_stock} = \max(\text{adjusted_base_stock}, \text{coverage_adjustment})$$

$$\text{cost_adjusted_base_stock} = (\text{final_base_stock} \cdot 0.5)$$

$$\text{target_inventory_position} = (\text{cost_adjusted_base_stock} + 122.3)$$

$$\text{raw_order} = \max(0, (\text{target_inventory_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot 96.6))$$

$$\text{bounded_order} = \max(96.6, \min(200.0, \text{smoothed_order}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{bounded_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-34

```
1. # sample_id: 015
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_4_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 1
5. # rank_in_population_file: 3
6. # objective: 2785.04
7. # test_objective: 2787.582
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 637.9351014652013 # OPT_PARAM: {"initial": 637.9351014652013, "min": 10, "max":
1000, "type": "float"}
14.     safety_stock = 23.945552227041272 # OPT_PARAM: {"initial": 23.945552227041272, "min": 0, "max":
200, "type": "float"}
15.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.9, "type": "float"}
16.
17.     # Calculate inventory position
18.     inventory_position = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Calculate expected demand based on recent pipeline arrivals
21.     recent_arrivals = pipeline_orders[:3] if len(pipeline_orders) >= 3 else pipeline_orders
22.     avg_recent_demand = sum(recent_arrivals) / len(recent_arrivals) if recent_arrivals else 0
23.
24.     # Adjust base stock based on recent demand pattern
25.     adjusted_base = base_stock + (avg_recent_demand - 100) * smoothing_factor
26.
27.     # Calculate target inventory position
28.     target_position = adjusted_base + safety_stock
29.
30.     # Calculate order amount with smoothing
31.     raw_order = max(0, target_position - inventory_position)
32.     order_amount = int(round(raw_order))
33.
34.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\max \left(0, \left(\left(637.9 + \left(\left(\frac{\sum \left(\begin{cases} q_3, & L \geq 3 \\ q_1, & \text{otherwise} \end{cases} \right)}{\text{len} \left(\begin{cases} q_3, & L \geq 3 \\ q_1, & \text{otherwise} \end{cases} \right)} \right), \begin{cases} q_3, & L \geq 3 \\ q, & \text{otherwise} - 100 \end{cases} \cdot 0.1 \right) + 23.9 \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right).$$

Python source lines 1-40

```

1. # sample_id: 016
2. # folder: deepseek-chat_exponential_L6_c1_5_50_plain_processed_scipy_15_default_m2_2_r6
3. # distribution: exponential_L6_c1_5
4. # generation: 6
5. # rank_in_population_file: 8
6. # objective: 11232.3442
7. # test_objective: 11214.91281
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 431.45521403586514 # OPT_PARAM: {"initial": 431.45521403586514, "min": 200, "max":
14.     800, "type": "float"}
15.     safety_stock = 166.96209077603805 # OPT_PARAM: {"initial": 166.96209077603805, "min": 100,
16.     "max": 300, "type": "float"}
17.     demand_smoothing = 0.14593966226801802 # OPT_PARAM: {"initial": 0.14593966226801802, "min":
18.     0.05, "max": 0.3, "type": "float"}
19.     pipeline_weight = 0.6078122160613739 # OPT_PARAM: {"initial": 0.6078122160613739, "min": 0.3,
20.     "max": 1.0, "type": "float"}
21.     order_smoothing = 0.1296107641229217 # OPT_PARAM: {"initial": 0.1296107641229217, "min": 0.1,
22.     "max": 0.8, "type": "float"}
23.
24.     # Calculate inventory position
25.     inventory_position = on_hand_inventory + sum(pipeline_orders)
26.
27.     # Estimate demand from recent pipeline arrivals (more stable than just first few)
28.     # Use weighted average with more weight on recent arrivals
29.     if pipeline_orders:
30.         weights = [0.1, 0.2, 0.3, 0.4] # Increasing weights for more recent
31.         weighted_sum = 0
32.         total_weight = 0
33.         for i, qty in enumerate(pipeline_orders[-4:] if len(pipeline_orders) >= 4 else
34. pipeline_orders):
35.             weight = weights[i] if i < len(weights) else 0.2
36.             weighted_sum += qty * weight
37.             total_weight += weight
38.         estimated_demand = weighted_sum / total_weight if total_weight > 0 else 0
39.     else:
40.         estimated_demand = 0
41.
42.     # Smooth demand estimate
43.     smoothed_demand = demand_smoothing * estimated_demand + (1 - demand_smoothing) * (base_stock /
44. 10)
45.
46.     # Calculate target inventory position considering pipeline
47.     pipeline_effect = pipeline_weight * smoothed_demand * len(pipeline_orders)

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} & \text{weights} = [0.1, 0.2, 0.3, 0.4] \\ & \text{weighted_sum} = 0 \\ & \text{total_weight} = 0 \\ & \text{weighted_sum} = \sum_{i=0}^{L-1} \left(\left(\left(\left(\begin{cases} q_{-(4)}, & L \geq 4 \\ \mathbf{q}, & \text{otherwise} \end{cases} \right)^{L-1-i} \right) \cdot \begin{cases} \left(\begin{cases} q_{-(4)}, & L \geq 4 \\ \mathbf{q}, & \text{otherwise} \end{cases} \right)_i \cdot \begin{cases} \text{weights}_i, & i < \text{len}(\text{weights}) \\ 0.2, & \text{otherwise} \end{cases} \end{cases} \right) \right) \right) \\ & \text{total_weight} = \sum_{i=0}^{L-1} \left(\begin{cases} \left(\begin{cases} q_{-(4)}, & L \geq 4 \\ \mathbf{q}, & \text{otherwise} \end{cases} \right)^{L-1-i} \\ \left(\begin{cases} \text{weights}_i, & i < \text{len}(\text{weights}) \\ 0.2, & \text{otherwise} \end{cases} \right) \end{cases} \right) \\ & \text{estimated_demand} = \begin{cases} \frac{\text{weighted_sum}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 0, & \text{otherwise} \end{cases} \end{aligned} \right\} (\mathbf{q})$$

$$\text{estimated_demand} = 0 \quad (\neg(\mathbf{q}))$$

$$\text{smoothed_demand} = \left((0.1 \cdot \text{estimated_demand}) + \left((1 - 0.1) \cdot \frac{431.5}{10} \right) \right)$$

$$\text{pipeline_effect} = ((0.6 \cdot \text{smoothed_demand}) \cdot L)$$

$$\text{target_position} = ((431.5 + 167.0) + \text{pipeline_effect})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{order_amount} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{smoothed_demand}))$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 41-48

```

41.     target_position = base_stock + safety_stock + pipeline_effect
42.
43.     # Calculate order amount with smoothing
44.     raw_order = max(0, target_position - inventory_position)
45.     # Apply order smoothing to reduce volatility
46.     order_amount = order_smoothing * raw_order + (1 - order_smoothing) * smoothed_demand
47.
48.     return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 \text{inventory_position} &= \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 \left. \begin{aligned}
 &\text{weights} = [0.1, 0.2, 0.3, 0.4] \\
 &\text{weighted_sum} = 0 \\
 &\text{total_weight} = 0 \\
 &\text{weighted_sum} = \left\{ \begin{aligned} &q_{-(4):}, & L \geq 4 \\ &\mathbf{q}, & \text{otherwise} \end{aligned} \right\}^{L-1} \sum_{i=0} \left(\left(\left(\left\{ \begin{aligned} &q_{-(4):}, & L \geq 4 \\ &\mathbf{q}, & \text{otherwise} \end{aligned} \right\}_i \cdot \left\{ \begin{aligned} &\text{weights}_i, & i < \text{len}(\text{weights}) \\ &0.2, & \text{otherwise} \end{aligned} \right\} \right) \right) \right) \end{aligned} \right\} (\mathbf{q}) \\
 &\text{total_weight} = \left\{ \begin{aligned} &q_{-(4):}, & L \geq 4 \\ &\mathbf{q}, & \text{otherwise} \end{aligned} \right\}^{L-1} \sum_{i=0} \left(\left\{ \begin{aligned} &\text{weights}_i, & i < \text{len}(\text{weights}) \\ &0.2, & \text{otherwise} \end{aligned} \right\} \right) \\
 &\text{estimated_demand} = \begin{cases} \frac{\text{weighted_sum}}{\text{total_weight}}, & \text{total_weight} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 &\text{estimated_demand} = 0 \quad (\neg(\mathbf{q})) \\
 &\text{smoothed_demand} = \left((0.1 \cdot \text{estimated_demand}) + \left((1 - 0.1) \cdot \frac{431.5}{10} \right) \right) \\
 &\text{pipeline_effect} = ((0.6 \cdot \text{smoothed_demand}) \cdot L) \\
 &\text{target_position} = ((431.5 + 167.0) + \text{pipeline_effect}) \\
 &\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position})) \\
 &\text{order_amount} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{smoothed_demand})) \\
 &\boxed{Q(h, \mathbf{q}) = \text{order_amount}}
 \end{aligned}$$

Python source lines 1-39

```
1. # sample_id: 017
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_m2_4_r6
3. # distribution: poisson_L4_c1_5
4. # generation: 7
5. # rank_in_population_file: 1
6. # objective: 2043.12
7. # test_objective: 2034.491
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;order_up_to;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 580.0 # OPT_PARAM: {"initial": 580.0, "min": 400, "max": 700, "type": "float"}
14.     safety_stock = 229.86830524581194 # OPT_PARAM: {"initial": 229.86830524581194, "min": 80, "max": 250, "type": "float"}
15.     demand_estimate = 120.0 # OPT_PARAM: {"initial": 120.0, "min": 80, "max": 120, "type": "float"}
16.     adjustment_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.0, "type": "float"}
17.     lead_time = len(pipeline_orders)
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand during lead time
23.     expected_lead_time_demand = demand_estimate * lead_time
24.
25.     # Calculate target inventory position
26.     target_position = expected_lead_time_demand + safety_stock
27.
28.     # Calculate order amount with adjustment
29.     order_amount = max(0, target_position - inventory_position)
30.     order_amount = order_amount * adjustment_factor
31.
32.     # Apply base stock as upper bound
33.     max_order = max(0, base_stock - inventory_position)
34.     order_amount = min(order_amount, max_order)
35.
36.     # Round to nearest integer
37.     order_amount = int(round(order_amount))
38.
39.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\min \left(\left(\max \left(0, \left((120.0 \cdot L) + 229.9 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \cdot 0.5 \right), \max \left(0, \left(580.0 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right) \right).$$

Python source lines 1-36

```
1. # sample_id: 018
2. # folder: deepseek-chat_exponential_L4_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: exponential_L4_c1_5
4. # generation: 4
5. # rank_in_population_file: 9
6. # objective: 11188.91926
7. # test_objective: 11348.70347
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 354.2408066643192 # OPT_PARAM: {"initial": 354.2408066643192, "min": 200, "max":
600, "type": "float"}
14.     safety_stock = 59.47477215640345 # OPT_PARAM: {"initial": 59.47477215640345, "min": 0, "max":
150, "type": "float"}
15.     demand_estimate = 119.23299630310831 # OPT_PARAM: {"initial": 119.23299630310831, "min": 80,
"max": 200, "type": "float"}
16.     pipeline_weight = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 0.3, "type": "float"}
17.     smoothing_factor = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.3, "max": 0.9, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected shortfall
23.     shortfall = max(0, base_stock - inventory_position)
24.
25.     # Adjust for pipeline coverage
26.     pipeline_coverage = sum(pipeline_orders)
27.     expected_pipeline_needs = demand_estimate * len(pipeline_orders)
28.     pipeline_adjustment = max(0, expected_pipeline_needs - pipeline_coverage) * pipeline_weight
29.
30.     # Smooth the order amount to avoid large fluctuations
31.     smoothed_shortfall = shortfall * smoothing_factor + safety_stock
32.
33.     # Final order amount
34.     order_amount = max(0, smoothed_shortfall + pipeline_adjustment)
35.
36.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(\left(\left(\max \left(0, 354.2 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \cdot 0.3 \right) + 59.5 \right) + \left(\max \left(0, \left((119.2 \cdot L) - \sum_{i=0}^{L-1} q_i \right) \cdot 0.0 \right) \right) \right) \right).$$

Python source lines 1-42

```

1. # sample_id: 019
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: poisson_L6_c1_2
4. # generation: 5
5. # rank_in_population_file: 6
6. # objective: 1053.82
7. # test_objective: 1032.595
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 630.0044857452867 # OPT_PARAM: {"initial": 630.0044857452867, "min": 550, "max":
700, "type": "float"}
14.     safety_stock = 40.0 # OPT_PARAM: {"initial": 40.0, "min": 25, "max": 55, "type": "float"}
15.     demand_forecast = 97.53915439102583 # OPT_PARAM: {"initial": 97.53915439102583, "min": 90,
"max": 104, "type": "float"}
16.     smoothing_factor = 0.05 # OPT_PARAM: {"initial": 0.05, "min": 0.05, "max": 0.15, "type":
"float"}
17.     order_threshold = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 5, "max": 20, "type": "float"}
18.     cost_ratio_factor = 0.7 # OPT_PARAM: {"initial": 0.7, "min": 0.6, "max": 0.75, "type": "float"}
19.     pipeline_weight = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.1, "max": 0.5, "type": "float"}
20.
21.     # Calculate inventory position
22.     inventory_position = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Calculate expected demand during lead time
25.     lead_time = len(pipeline_orders)
26.     expected_lead_time_demand = demand_forecast * lead_time
27.
28.     # Adjust safety stock based on cost ratio (p/(p+h) = 2/3 ≈ 0.667)
29.     adjusted_safety_stock = safety_stock * cost_ratio_factor
30.
31.     # Calculate target inventory level with pipeline consideration
32.     # Give more weight to near-term pipeline arrivals
33.     pipeline_imbalance = sum(p * (1 - pipeline_weight * i) for i, p in enumerate(pipeline_orders)) /
lead_time if lead_time > 0 else 0
34.     target_inventory = expected_lead_time_demand + adjusted_safety_stock - max(0, pipeline_imbalance
- demand_forecast)
35.
36.     # Use the maximum of base_stock and target_inventory as order-up-to level
37.     order_up_to = max(base_stock, target_inventory)
38.
39.     # Calculate raw order amount
40.     raw_order = max(0, order_up_to - inventory_position)
41.
42.     # Apply smoothing to reduce order volatility

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{lead_time} = L$

$\text{expected_lead_time_demand} = (97.5 \cdot \text{lead_time})$

$\text{adjusted_safety_stock} = (40.0 \cdot 0.7)$

$$\text{pipeline_imbalance} = \begin{cases} \frac{\sum_{i=0}^{L-1} (q_i \cdot (1 - (0.3^i)))}{\text{lead_time}}, & \text{lead_time} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$\text{target_inventory} = ((\text{expected_lead_time_demand} + \text{adjusted_safety_stock}) - \max(0, (\text{pipeline_imbalance} - 97.5)))$

$\text{order_up_to} = \max(630.0, \text{target_inventory})$

$\text{raw_order} = \max(0, (\text{order_up_to} - \text{inventory_position}))$

$$\text{smoothed_order} = \begin{cases} ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot 97.5)), & \text{raw_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \text{round}_{\text{even}}(\text{smoothed_order}), & \text{otherwise} \end{cases}$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 43-54

```

43.     if raw_order > 0:
44.         smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * demand_forecast
45.     else:
46.         smoothed_order = 0
47.
48.     # Apply order threshold to avoid small orders
49.     if smoothed_order < order_threshold:
50.         order_amount = 0
51.     else:
52.         order_amount = int(round(smoothed_order))
53.
54.     return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{lead_time} = L$

$\text{expected_lead_time_demand} = (97.5 \cdot \text{lead_time})$

$\text{adjusted_safety_stock} = (40.0 \cdot 0.7)$

$$\text{pipeline_imbalance} = \begin{cases} \frac{\sum_{i=0}^{L-1} (q_i \cdot (1 - (0.3^i)))}{\text{lead_time}}, & \text{lead_time} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$\text{target_inventory} = ((\text{expected_lead_time_demand} + \text{adjusted_safety_stock}) - \max(0, (\text{pipeline_imbalance} - 97.5)))$

$\text{order_up_to} = \max(630.0, \text{target_inventory})$

$\text{raw_order} = \max(0, (\text{order_up_to} - \text{inventory_position}))$

$\text{smoothed_order} = \begin{cases} ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot 97.5)), & \text{raw_order} > 0 \\ 0, & \text{otherwise} \end{cases}$

$\text{order_amount} = \begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \text{round}_{\text{even}}(\text{smoothed_order}), & \text{otherwise} \end{cases}$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```

1. # sample_id: 020
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: exponential_L6_c1_2
4. # generation: 1
5. # rank_in_population_file: 9
6. # objective: 7317.1
7. # test_objective: 7436.453
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 326.7318458492169 # OPT_PARAM: {"initial": 326.7318458492169, "min": 10, "max":
1000, "type": "float"}
14.     safety_stock = 19.771688406626556 # OPT_PARAM: {"initial": 19.771688406626556, "min": 0, "max":
200, "type": "float"}
15.     demand_forecast_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 2.0, "type":
"float"}
16.
17.     # Calculate inventory position
18.     inventory_position = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Estimate upcoming demand based on pipeline orders (as proxy for recent orders)
21.     # Use average of recent pipeline orders as demand forecast
22.     if len(pipeline_orders) > 0:
23.         recent_orders = pipeline_orders[-3:] if len(pipeline_orders) >= 3 else pipeline_orders
24.         forecast_demand = sum(recent_orders) / len(recent_orders) * demand_forecast_factor
25.     else:
26.         forecast_demand = 0
27.
28.     # Adjust base stock level based on forecast
29.     adjusted_base_stock = base_stock + forecast_demand + safety_stock
30.
31.     # Calculate order amount
32.     order_amount = max(0, adjusted_base_stock - inventory_position)
33.
34.     # Round to nearest integer (since order amounts should be integers)
35.     order_amount = int(round(order_amount))
36.
37.     return order_amount

```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{recent_orders} = \left\{ \begin{array}{ll} q_{-(3)}, & L \geq 3 \\ \mathbf{q}, & \text{otherwise} \end{array} \right\} \quad (L > 0)$$

$$\text{forecast_demand} = \left(\frac{\text{sum}(\text{recent_orders})}{\text{len}(\text{recent_orders})} \cdot 0.1 \right)$$

$$\text{forecast_demand} = 0 \quad (\neg (L > 0))$$

$$\text{adjusted_base_stock} = ((326.7 + \text{forecast_demand}) + 19.8)$$

$$\text{order_amount} = \max(0, (\text{adjusted_base_stock} - \text{inventory_position}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-22

```
1. # sample_id: 021
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_no_15_default_m2_10_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 18
5. # rank_in_population_file: 1
6. # objective: 7242.76
7. # test_objective: 7384.309
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 310 # OPT_PARAM: {'initial': 310, 'min': 10, 'max': 1000, 'type': 'float'}
14.     safety_stock = 45 # OPT_PARAM: {'initial': 45, 'min': 0, 'max': 200, 'type': 'float'}
15.
16.     # Calculate net inventory position
17.     net_inventory = on_hand_inventory + sum(pipeline_orders)
18.
19.     # Order up to base_stock plus safety_stock adjustment
20.     order_amount = max(0, base_stock + safety_stock - net_inventory)
21.
22.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left((310 + 45) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right).$$

Python source lines 1-15

```
1. # sample_id: 022
2. # folder: deepseek-chat_normal_std50_L2_c1_5_50_plain_processed_scipy_15_default_e1-e2-m2_2_r1
3. # distribution: normal_std50_L2_c1_5
4. # generation: 0
5. # rank_in_population_file: 1
6. # objective: 5738.975
7. # test_objective: 5705.41496
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs:
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 345.925000000000797 # OPT_PARAM: {"initial": 345.925000000000797, "min": 10, "max":
14.     1000, "type": "float"}
15.     order_amount = max(0, base_stock - on_hand_inventory - sum(pipeline_orders))
16.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(345.9 - h - \sum_{i=0}^{L-1} q_i \right) \right).$$

Python source lines 1-15

```
1. # sample_id: 023
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_no_15_default_m2_10_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 4
5. # rank_in_population_file: 5
6. # objective: 4885.1
7. # test_objective: 4872.183
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs:
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 450 # OPT_PARAM: {'initial': 450, 'min': 10, 'max': 1000, 'type': 'float'}
14.     order_amount = max(0, base_stock - on_hand_inventory - sum(pipeline_orders))
15.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(450 - h - \sum_{i=0}^{L-1} q_i \right) \right).$$

Python source lines 1-37

```
1. # sample_id: 024
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_8_r6
3. # distribution: poisson_L6_c1_2
4. # generation: 3
5. # rank_in_population_file: 6
6. # objective: 781.98108
7. # test_objective: 757.94413
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 663.9895492381544 # OPT_PARAM: {"initial": 663.9895492381544, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 69.29783740818819 # OPT_PARAM: {"initial": 69.29783740818819, "min": 0, "max":
16.     200, "type": "float"}
17.     demand_forecast = 150.0 # OPT_PARAM: {"initial": 150.0, "min": 50, "max": 150, "type": "float"}
18.     max_order = 95.00028797791165 # OPT_PARAM: {"initial": 95.00028797791165, "min": 50, "max": 500,
19.     "type": "float"}
20.     smoothing_factor = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.1, "max": 1.0, "type": "float"}
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected demand during lead time
26.     expected_lead_time_demand = demand_forecast * len(pipeline_orders)
27.
28.     # Calculate target inventory position
29.     target_inventory = expected_lead_time_demand + safety_stock
30.
31.     # Calculate raw order amount
32.     raw_order = max(0, target_inventory - inventory_position)
33.
34.     # Apply smoothing to reduce order volatility
35.     smoothed_order = smoothing_factor * raw_order
36.
37.     # Cap order amount to avoid excessive ordering
38.     order_amount = min(smoothed_order, max_order)
39.
40.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \min \left(\left(1.0 \cdot \max \left(0, \left(((150.0 \cdot L) + 69.3) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right), 95.0 \right).$$

Python source lines 1-43

```

1. # sample_id: 025
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: poisson_L6_c1_5
4. # generation: 8
5. # rank_in_population_file: 10
6. # objective: 1326.06
7. # test_objective: 1381.561
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 641.7661796828763 # OPT_PARAM: {"initial": 641.7661796828763, "min": 500, "max":
14.     700, "type": "float"}
15.     safety_stock = 70.0 # OPT_PARAM: {"initial": 70.0, "min": 50, "max": 150, "type": "float"}
16.     demand_estimate = 97.29811863847306 # OPT_PARAM: {"initial": 97.29811863847306, "min": 95,
17.     "max": 105, "type": "float"}
18.     lead_time_multiplier = 1.2 # OPT_PARAM: {"initial": 1.2, "min": 1.0, "max": 1.3, "type":
19.     "float"}
20.     smoothing_factor = 0.01 # OPT_PARAM: {"initial": 0.01, "min": 0.01, "max": 0.1, "type": "float"}
21.     order_threshold = 20.0 # OPT_PARAM: {"initial": 20.0, "min": 10, "max": 50, "type": "float"}
22.
23.     # Calculate inventory position
24.     inventory_position = on_hand_inventory + sum(pipeline_orders)
25.
26.     # Calculate expected demand during lead time
27.     lead_time = len(pipeline_orders)
28.     lead_time_demand = demand_estimate * lead_time * lead_time_multiplier
29.
30.     # Calculate target inventory level
31.     target_inventory = lead_time_demand + safety_stock
32.
33.     # Use the minimum of base_stock and target_inventory
34.     order_up_to = min(base_stock, target_inventory)
35.
36.     # Calculate order amount
37.     order_amount = max(0, order_up_to - inventory_position)
38.
39.     # Apply smoothing only when order amount is above threshold
40.     if order_amount > order_threshold:
41.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * demand_estimate
42.
43.     # Round to nearest integer
44.     order_amount = int(round(order_amount))
45.
46.     return order_amount

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{lead_time} = L$

$\text{lead_time_demand} = ((97.3 \cdot \text{lead_time}) \cdot 1.2)$

$\text{target_inventory} = (\text{lead_time_demand} + 70.0)$

$\text{order_up_to} = \min(641.8, \text{target_inventory})$

$\text{order_amount} = \max(0, (\text{order_up_to} - \text{inventory_position}))$

$\text{order_amount} = \begin{cases} ((1.0e - 02 \cdot \text{order_amount}) + ((1 - 1.0e - 02) \cdot 97.3)), & \text{order_amount} > 20.0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$

$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-40

```
1. # sample_id: 026
2. # folder: deepseek-chat_poisson_L2_c1_2_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: poisson_L2_c1_2
4. # generation: 3
5. # rank_in_population_file: 3
6. # objective: 735.84
7. # test_objective: 747.41
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level parameters
14.     base_stock = 258.4217014907287 # OPT_PARAM: {"initial": 258.4217014907287, "min": 150, "max": 350, "type": "float"}
15.     safety_stock = 22.3071877883226 # OPT_PARAM: {"initial": 22.3071877883226, "min": 5, "max": 40, "type": "float"}
16.
17.     # Demand estimation parameters
18.     demand_estimation_factor = 0.7446704670998904 # OPT_PARAM: {"initial": 0.7446704670998904, "min": 0.7, "max": 1.0, "type": "float"}
19.
20.     # Order adjustment parameters
21.     pipeline_adjustment_factor = 0.32566760957648905 # OPT_PARAM: {"initial": 0.32566760957648905, "min": 0.1, "max": 0.5, "type": "float"}
22.     order_smoothing = 0.09328880135760655 # OPT_PARAM: {"initial": 0.09328880135760655, "min": 0.0, "max": 0.3, "type": "float"}
23.
24.     # Order constraints
25.     min_order = 0 # OPT_PARAM: {"initial": 0, "min": 0, "max": 10, "type": "int"}
26.     max_order = 120 # OPT_PARAM: {"initial": 120, "min": 80, "max": 200, "type": "int"}
27.
28.     # Calculate inventory position
29.     inventory_position = on_hand_inventory + sum(pipeline_orders)
30.
31.     # Estimate demand based on base stock level
32.     estimated_demand = base_stock * demand_estimation_factor / 2.0
33.
34.     # Calculate pipeline adjustment
35.     if len(pipeline_orders) > 0:
36.         avg_pipeline = sum(pipeline_orders) / len(pipeline_orders)
37.         pipeline_adjustment = pipeline_adjustment_factor * (estimated_demand - avg_pipeline)
38.     else:
39.         pipeline_adjustment = 0
40.
```

Declarative mathematical formula rows 1-11

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{estimated_demand} = \frac{(258.4 \cdot 0.7)}{2.0}$$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_adjustment} &= (0.3 \cdot (\text{estimated_demand} - \text{avg_pipeline})) \end{aligned} \right\} (L > 0)$$
$$\text{pipeline_adjustment} = 0 \quad (\neg (L > 0))$$

$$\text{target_inventory_position} = ((258.4 + 22.3) + \text{pipeline_adjustment})$$

$$\text{suggested_order} = \max(0, (\text{target_inventory_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{suggested_order}) + ((1 - 0.1) \cdot \text{estimated_demand}))$$

$$\text{constrained_order} = \max(0, \min(120, \text{smoothed_order}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{constrained_order})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 41-56

```

41. # Calculate target inventory position
42. target_inventory_position = base_stock + safety_stock + pipeline_adjustment
43.
44. # Calculate suggested order
45. suggested_order = max(0, target_inventory_position - inventory_position)
46.
47. # Apply smoothing
48. smoothed_order = order_smoothing * suggested_order + (1 - order_smoothing) * estimated_demand
49.
50. # Apply constraints
51. constrained_order = max(min_order, min(max_order, smoothed_order))
52.
53. # Round to nearest integer
54. order_amount = int(round(constrained_order))
55.
56. return order_amount

```

Declarative mathematical formula rows 1-11

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{estimated_demand} = \frac{(258.4 \cdot 0.7)}{2.0}$$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_adjustment} &= (0.3 \cdot (\text{estimated_demand} - \text{avg_pipeline})) \end{aligned} \right\} (L > 0)$$

$$\text{pipeline_adjustment} = 0 \quad (\neg (L > 0))$$

$$\text{target_inventory_position} = ((258.4 + 22.3) + \text{pipeline_adjustment})$$

$$\text{suggested_order} = \max(0, (\text{target_inventory_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{suggested_order}) + ((1 - 0.1) \cdot \text{estimated_demand}))$$

$$\text{constrained_order} = \max(0, \min(120, \text{smoothed_order}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{constrained_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-38

```

1. # sample_id: 027
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: poisson_L4_c1_5
4. # generation: 4
5. # rank_in_population_file: 1
6. # objective: 1123.42
7. # test_objective: 1186.7
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs:
    pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 418.4796855242302 # OPT_PARAM: {"initial": 418.4796855242302, "min": 350, "max":
    500, "type": "float"}
15.
16.     # Safety stock multiplier
17.     safety_multiplier = 1.913108986256217 # OPT_PARAM: {"initial": 1.913108986256217, "min": 1.0,
    "max": 2.5, "type": "float"}
18.
19.     # Demand parameters
20.     typical_demand = 99.80989497864154 # OPT_PARAM: {"initial": 99.80989497864154, "min": 80, "max":
    120, "type": "float"}
21.     demand_std = 17.329580214699273 # OPT_PARAM: {"initial": 17.329580214699273, "min": 8, "max":
    20, "type": "float"}
22.
23.     # Pipeline weighting parameters
24.     pipeline_weight = 0.9104567322084133 # OPT_PARAM: {"initial": 0.9104567322084133, "min": 0.8,
    "max": 1.2, "type": "float"}
25.
26.     # Order smoothing parameters
27.     max_order_change = 40 # OPT_PARAM: {"initial": 40, "min": 20, "max": 80, "type": "int"}
28.     typical_order = 60.2 # OPT_PARAM: {"initial": 60.2, "min": 40, "max": 90, "type": "float"}
29.
30.     # New: Pipeline age weighting
31.     age_weight_power = 0.6154647503059288 # OPT_PARAM: {"initial": 0.6154647503059288, "min": 0.1,
    "max": 2.0, "type": "float"}
32.
33.     # New: Demand anticipation factor
34.     demand_anticipation = 0.2353085324847689 # OPT_PARAM: {"initial": 0.2353085324847689, "min":
    0.0, "max": 1.0, "type": "float"}
35.
36.     # Calculate weighted pipeline with age-based discounting
37.     lead_time = len(pipeline_orders)
38.     weighted_pipeline = 0.0

```

Declarative mathematical formula rows 1-14

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{lead_time} = L$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot ((i+1))^{0.6}))$$

$$\text{total_age_weight} = \sum_{i \in \text{range}(\text{lead_time})} ((i+1))^{0.6}$$

$$\text{weighted_pipeline} = \left(\left(\frac{\text{weighted_pipeline}}{\text{total_age_weight}} \cdot \text{lead_time} \right) \cdot 0.9 \right)$$

$\text{review_period} = 1$

$$\text{safety_stock} = ((1.9 \cdot 17.3) \cdot ((\text{lead_time} + \text{review_period}))^{0.5})$$

$$\text{adjusted_base_stock} = \left(418.5 \cdot \left(1 + \frac{(0.2 \cdot (99.8 - 100))}{100} \right) \right)$$

$\text{target_position} = (\text{adjusted_base_stock} + \text{safety_stock})$

$\text{current_position} = (h + \text{weighted_pipeline})$

$\text{raw_order} = (\text{target_position} - \text{current_position})$

$$\left. \begin{array}{l} \text{upper_bound} = (60.2 + 40) \\ \text{lower_bound} = \max(0, (60.2 - (40 \cdot 0.5))) \\ \text{smoothed_order} = \min(\max(\text{raw_order}, \text{lower_bound}), \text{upper_bound}) \\ \text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order})) \end{array} \right\} (\text{raw_order} > 0)$$

$\text{order_amount} = 0 \} (\neg(\text{raw_order} > 0))$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 39-73

```

39.     for i, order in enumerate(pipeline_orders):
40.         # Older orders get higher weight (closer to arrival)
41.         age_weight = (i + 1) ** age_weight_power
42.         weighted_pipeline += order * age_weight
43.
44.     # Normalize by sum of age weights
45.     total_age_weight = sum((i + 1) ** age_weight_power for i in range(lead_time))
46.     weighted_pipeline = weighted_pipeline / total_age_weight * lead_time * pipeline_weight
47.
48.     # Calculate safety stock with lead time consideration
49.     review_period = 1
50.     safety_stock = safety_multiplier * demand_std * ((lead_time + review_period) ** 0.5)
51.
52.     # Adjust base stock based on demand anticipation
53.     adjusted_base_stock = base_stock * (1 + demand_anticipation * (typical_demand - 100) / 100)
54.
55.     # Calculate target inventory position
56.     target_position = adjusted_base_stock + safety_stock
57.
58.     # Calculate current inventory position
59.     current_position = on_hand_inventory + weighted_pipeline
60.
61.     # Calculate raw order amount
62.     raw_order = target_position - current_position
63.
64.     # Apply order smoothing with asymmetric bounds
65.     if raw_order > 0:
66.         upper_bound = typical_order + max_order_change
67.         lower_bound = max(0, typical_order - max_order_change * 0.5) # New: asymmetric smoothing
68.         smoothed_order = min(max(raw_order, lower_bound), upper_bound)
69.         order_amount = max(0, int(round(smoothed_order)))
70.     else:
71.         order_amount = 0
72.
73.     return order_amount

```

Declarative mathematical formula rows 1-14

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{lead_time} = L$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot ((i+1))^{0.6}))$$

$$\text{total_age_weight} = \sum_{i \in \text{range}(\text{lead_time})} ((i+1))^{0.6}$$

$$\text{weighted_pipeline} = \left(\left(\frac{\text{weighted_pipeline}}{\text{total_age_weight}} \cdot \text{lead_time} \right) \cdot 0.9 \right)$$

$\text{review_period} = 1$

$$\text{safety_stock} = ((1.9 \cdot 17.3) \cdot ((\text{lead_time} + \text{review_period}))^{0.5})$$

$$\text{adjusted_base_stock} = \left(418.5 \cdot \left(1 + \frac{(0.2 \cdot (99.8 - 100))}{100} \right) \right)$$

$\text{target_position} = (\text{adjusted_base_stock} + \text{safety_stock})$

$\text{current_position} = (h + \text{weighted_pipeline})$

$\text{raw_order} = (\text{target_position} - \text{current_position})$

$$\left. \begin{array}{l} \text{upper_bound} = (60.2 + 40) \\ \text{lower_bound} = \max(0, (60.2 - (40 \cdot 0.5))) \\ \text{smoothed_order} = \min(\max(\text{raw_order}, \text{lower_bound}), \text{upper_bound}) \\ \text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order})) \end{array} \right\} \quad (\text{raw_order} > 0)$$

$\text{order_amount} = 0 \} \quad (\neg(\text{raw_order} > 0))$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 1-42

```

1. # sample_id: 028
2. # folder: deepseek-chat_exponential_L4_c1_2_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: exponential_L4_c1_2
4. # generation: 6
5. # rank_in_population_file: 6
6. # objective: 6118.04
7. # test_objective: 6175.142
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 192.40570617005534 # OPT_PARAM: {"initial": 192.40570617005534, "min": 100, "max":
300, "type": "float"}
14.     safety_stock = 42.40570617005488 # OPT_PARAM: {"initial": 42.40570617005488, "min": 10, "max":
80, "type": "float"}
15.     demand_forecast_factor = 1.3013911932804665 # OPT_PARAM: {"initial": 1.3013911932804665, "min":
0.8, "max": 1.5, "type": "float"}
16.     smoothing_factor = 0.2 # OPT_PARAM: {"initial": 0.2, "min": 0.2, "max": 0.8, "type": "float"}
17.     pipeline_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.3, "max": 1.0, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Use more recent pipeline orders for demand estimation (last 3 periods)
23.     recent_arrivals = pipeline_orders[:3]
24.     if any(recent_arrivals):
25.         # Weight recent arrivals more heavily
26.         weights = [0.5, 0.3, 0.2] # Decreasing weights for older orders
27.         weighted_sum = sum(w * val for w, val in zip(weights, recent_arrivals))
28.         avg_recent = weighted_sum / sum(weights[:len(recent_arrivals)])
29.         recent_demand_estimate = avg_recent * demand_forecast_factor
30.     else:
31.         recent_demand_estimate = 0
32.
33.     # Adjust base stock based on recent demand and safety stock
34.     adjusted_base_stock = base_stock + safety_stock + recent_demand_estimate
35.
36.     # Calculate order amount using inventory position
37.     order_amount = max(0, adjusted_base_stock - inventory_position)
38.
39.     # Apply smoothing with pipeline-aware adjustment
40.     if len(pipeline_orders) > 0:
41.         # Weighted average of pipeline orders
42.         avg_pipeline = sum(pipeline_orders) / len(pipeline_orders)

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{recent_arrivals} = q_{:3}$

$$\left. \begin{aligned} \text{weights} &= [0.5, 0.3, 0.2] \\ \text{weighted_sum} &= \sum_{(w, \text{val}) \in \text{zip}(\text{weights}, \text{recent_arrivals})} (w \cdot \text{val}) \\ \text{avg_recent} &= \frac{\text{weighted_sum}}{\text{sum}(\text{weights}.\text{len}(\text{recent_arrivals}))} \\ \text{recent_demand_estimate} &= (\text{avg_recent} \cdot 1.3) \end{aligned} \right\} \text{(any(recent_arrivals))}$$

$\text{recent_demand_estimate} = 0$ $\neg$ (any(recent_arrivals))

$\text{adjusted_base_stock} = ((192.4 + 42.4) + \text{recent_demand_estimate})$

$\text{order_amount} = \max(0, (\text{adjusted_base_stock} - \text{inventory_position}))$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{smoothed_order} &= ((0.2 \cdot \text{order_amount}) + (((1 - 0.2) \cdot \text{avg_pipeline}) \cdot 1.0)) \end{aligned} \right\} (L > 0)$$

$\text{order_amount} = \max(0, \text{smoothed_order})$

$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 43-51

```

43.         # Blend order amount with pipeline average
44.         smoothed_order = (smoothing_factor * order_amount +
45.                           (1 - smoothing_factor) * avg_pipeline * pipeline_weight)
46.         order_amount = max(0, smoothed_order)
47.
48.     # Round to nearest integer
49.     order_amount = int(round(order_amount))
50.
51.     return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

recent_arrivals = $q_{:3}$

$$\left. \begin{aligned} \text{weights} &= [0.5, 0.3, 0.2] \\ \text{weighted_sum} &= \sum_{(w, \text{val}) \in \text{zip}(\text{weights}, \text{recent_arrivals})} (w \cdot \text{val}) \\ \text{avg_recent} &= \frac{\text{weighted_sum}}{\text{sum}(\text{weights}.\text{len}(\text{recent_arrivals}))} \\ \text{recent_demand_estimate} &= (\text{avg_recent} \cdot 1.3) \end{aligned} \right\} \text{(any(recent_arrivals))}$$

recent_demand_estimate = 0} $(\neg(\text{any}(\text{recent_arrivals})))$

adjusted_base_stock = $((192.4 + 42.4) + \text{recent_demand_estimate})$

order_amount = $\max(0, (\text{adjusted_base_stock} - \text{inventory_position}))$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{smoothed_order} &= ((0.2 \cdot \text{order_amount}) + (((1 - 0.2) \cdot \text{avg_pipeline}) \cdot 1.0)) \\ \text{order_amount} &= \max(0, \text{smoothed_order}) \end{aligned} \right\} (L > 0)$$

order_amount = $\text{round}_{\text{even}}(\text{order_amount})$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-43

```

1. # sample_id: 029
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: poisson_L6_c1_5
4. # generation: 9
5. # rank_in_population_file: 10
6. # objective: 1326.06
7. # test_objective: 1381.561
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 641.7661796828763 # OPT_PARAM: {"initial": 641.7661796828763, "min": 500, "max":
14.     700, "type": "float"}
15.     safety_stock = 70.0 # OPT_PARAM: {"initial": 70.0, "min": 50, "max": 150, "type": "float"}
16.     demand_estimate = 97.29811863847306 # OPT_PARAM: {"initial": 97.29811863847306, "min": 95,
17.     "max": 105, "type": "float"}
18.     lead_time_multiplier = 1.2 # OPT_PARAM: {"initial": 1.2, "min": 1.0, "max": 1.3, "type":
19.     "float"}
20.     smoothing_factor = 0.01 # OPT_PARAM: {"initial": 0.01, "min": 0.01, "max": 0.1, "type": "float"}
21.     order_threshold = 20.0 # OPT_PARAM: {"initial": 20.0, "min": 10, "max": 50, "type": "float"}
22.
23.     # Calculate inventory position
24.     inventory_position = on_hand_inventory + sum(pipeline_orders)
25.
26.     # Calculate expected demand during lead time
27.     lead_time = len(pipeline_orders)
28.     lead_time_demand = demand_estimate * lead_time * lead_time_multiplier
29.
30.     # Calculate target inventory level
31.     target_inventory = lead_time_demand + safety_stock
32.
33.     # Use the minimum of base_stock and target_inventory
34.     order_up_to = min(base_stock, target_inventory)
35.
36.     # Calculate order amount
37.     order_amount = max(0, order_up_to - inventory_position)
38.
39.     # Apply smoothing only when order amount is above threshold
40.     if order_amount > order_threshold:
41.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * demand_estimate
42.
43.     # Round to nearest integer
44.     order_amount = int(round(order_amount))
45.
46.     return order_amount

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{lead_time} = L$

$\text{lead_time_demand} = ((97.3 \cdot \text{lead_time}) \cdot 1.2)$

$\text{target_inventory} = (\text{lead_time_demand} + 70.0)$

$\text{order_up_to} = \min(641.8, \text{target_inventory})$

$\text{order_amount} = \max(0, (\text{order_up_to} - \text{inventory_position}))$

$\text{order_amount} = \begin{cases} ((1.0e - 02 \cdot \text{order_amount}) + ((1 - 1.0e - 02) \cdot 97.3)), & \text{order_amount} > 20.0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$

$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-38

```

1. # sample_id: 030
2. # folder: deepeek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: exponential_L6_c1_2
4. # generation: 9
5. # rank_in_population_file: 5
6. # objective: 6003.88807
7. # test_objective: 6125.17677
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;emergen
    cy_or_shortage_boost
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 349.93516183417404 # OPT_PARAM: {"initial": 349.93516183417404, "min": 250, "max":
    450, "type": "float"}
14.     safety_stock = 44.935161834169534 # OPT_PARAM: {"initial": 44.935161834169534, "min": 30, "max":
    80, "type": "float"}
15.     smoothing_factor = 0.10743259195308223 # OPT_PARAM: {"initial": 0.10743259195308223, "min":
    0.05, "max": 0.3, "type": "float"}
16.     demand_forecast = 99.83223131359875 # OPT_PARAM: {"initial": 99.83223131359875, "min": 80,
    "max": 120, "type": "float"}
17.     forecast_weight = 1.2051163998324366 # OPT_PARAM: {"initial": 1.2051163998324366, "min": 0.9,
    "max": 1.3, "type": "float"}
18.     pipeline_weight = 0.4512560796313605 # OPT_PARAM: {"initial": 0.4512560796313605, "min": 0.0,
    "max": 1.0, "type": "float"}
19.     min_order = 20.0 # OPT_PARAM: {"initial": 20.0, "min": 10, "max": 40, "type": "float"}
20.     emergency_multiplier = 1.5 # OPT_PARAM: {"initial": 1.5, "min": 1.0, "max": 2.0, "type":
    "float"}
21.     recent_window = 2 # OPT_PARAM: {"initial": 2, "min": 1, "max": 4, "type": "int"}
22.
23.     # Calculate inventory position
24.     inventory_position = on_hand_inventory + sum(pipeline_orders)
25.
26.     # Estimate recent demand from pipeline arrivals
27.     if len(pipeline_orders) >= recent_window:
28.         recent_arrivals = pipeline_orders[:recent_window]
29.         avg_recent_demand = sum(recent_arrivals) / len(recent_arrivals)
30.     else:
31.         avg_recent_demand = demand_forecast
32.
33.     # Blend static forecast with recent demand signal
34.     blended_forecast = (1 - pipeline_weight) * demand_forecast + pipeline_weight * avg_recent_demand
35.
36.     # Calculate expected lead time demand
37.     lead_time = len(pipeline_orders)
38.     expected_lead_time_demand = blended_forecast * lead_time

```

Declarative mathematical formula rows 1-14

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{array}{l} \text{recent_arrivals} = q_{:2} \\ \text{avg_recent_demand} = \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})} \end{array} \right\} \quad (L \geq 2)$$

$$\text{avg_recent_demand} = 99.8 \quad (\neg (L \geq 2))$$

$$\text{blended_forecast} = (((1 - 0.5) \cdot 99.8) + (0.5 \cdot \text{avg_recent_demand}))$$

$$\text{lead_time} = L$$

$$\text{expected_lead_time_demand} = (\text{blended_forecast} \cdot \text{lead_time})$$

$$\text{dynamic_target} = (349.9 + (1.2 \cdot \text{expected_lead_time_demand}))$$

$$\text{target_inventory} = (\text{dynamic_target} + 44.9)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{order_amount} = (0.1 \cdot \text{raw_order})$$

$$\left. \begin{array}{l} \text{emergency_order} = (1.5 \cdot \text{blended_forecast}) \\ \text{order_amount} = \max(\text{order_amount}, \text{emergency_order}) \end{array} \right\} \quad (\text{inventory_position} < 20.0)$$

$$\text{order_amount} = \begin{cases} \max(\text{order_amount}, 20.0), & (\text{raw_order} > 0 \wedge \text{order_amount} < 20.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 39-62

```

39.
40.     # Dynamic target: base stock + forecast-adjusted component
41.     dynamic_target = base_stock + forecast_weight * expected_lead_time_demand
42.
43.     # Final target with safety stock
44.     target_inventory = dynamic_target + safety_stock
45.
46.     # Calculate raw order amount
47.     raw_order = max(0, target_inventory - inventory_position)
48.
49.     # Apply smoothing
50.     order_amount = smoothing_factor * raw_order
51.
52.     # Emergency ordering for critically low inventory
53.     if inventory_position < min_order:
54.         emergency_order = emergency_multiplier * blended_forecast
55.         order_amount = max(order_amount, emergency_order)
56.
57.     # Ensure minimum order when raw order is significant
58.     if raw_order > 0 and order_amount < min_order:
59.         order_amount = max(order_amount, min_order)
60.
61.     # Round to nearest integer
62.     return order_amount

```

Declarative mathematical formula rows 1-14

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{array}{l} \text{recent_arrivals} = q_{:2} \\ \text{avg_recent_demand} = \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})} \end{array} \right\} \quad (L \geq 2)$$

$$\text{avg_recent_demand} = 99.8 \quad (\neg (L \geq 2))$$

$$\text{blended_forecast} = (((1 - 0.5) \cdot 99.8) + (0.5 \cdot \text{avg_recent_demand}))$$

$$\text{lead_time} = L$$

$$\text{expected_lead_time_demand} = (\text{blended_forecast} \cdot \text{lead_time})$$

$$\text{dynamic_target} = (349.9 + (1.2 \cdot \text{expected_lead_time_demand}))$$

$$\text{target_inventory} = (\text{dynamic_target} + 44.9)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{order_amount} = (0.1 \cdot \text{raw_order})$$

$$\left. \begin{array}{l} \text{emergency_order} = (1.5 \cdot \text{blended_forecast}) \\ \text{order_amount} = \max(\text{order_amount}, \text{emergency_order}) \end{array} \right\} \quad (\text{inventory_position} < 20.0)$$

$$\text{order_amount} = \begin{cases} \max(\text{order_amount}, 20.0), & (\text{raw_order} > 0 \wedge \text{order_amount} < 20.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```

1. # sample_id: 031
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: exponential_L6_c1_2
4. # generation: 5
5. # rank_in_population_file: 5
6. # objective: 6011.41212
7. # test_objective: 6126.22791
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;emergen
    cy_or_shortage_boost
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 319.94837008016606 # OPT_PARAM: {"initial": 319.94837008016606, "min": 250, "max":
    400, "type": "float"}
14.     safety_stock = 44.94837008016602 # OPT_PARAM: {"initial": 44.94837008016602, "min": 30, "max":
    80, "type": "float"}
15.     smoothing_factor = 0.13367404564819818 # OPT_PARAM: {"initial": 0.13367404564819818, "min": 0.1,
    "max": 0.3, "type": "float"}
16.     demand_forecast = 99.85949394768903 # OPT_PARAM: {"initial": 99.85949394768903, "min": 80,
    "max": 120, "type": "float"}
17.     forecast_weight = 1.1015656359447512 # OPT_PARAM: {"initial": 1.1015656359447512, "min": 0.9,
    "max": 1.2, "type": "float"}
18.     pipeline_weight = 0.5380871201690676 # OPT_PARAM: {"initial": 0.5380871201690676, "min": 0.0,
    "max": 1.0, "type": "float"}
19.     recent_demand_window = 2 # OPT_PARAM: {"initial": 2, "min": 1, "max": 4, "type": "int"}
20.     min_order_threshold = 20.0 # OPT_PARAM: {"initial": 20.0, "min": 10, "max": 40, "type": "float"}
21.     emergency_order_multiplier = 1.2 # OPT_PARAM: {"initial": 1.2, "min": 1.0, "max": 2.0, "type":
    "float"}
22.
23.     # Calculate inventory position
24.     inventory_position = on_hand_inventory + sum(pipeline_orders)
25.
26.     # Estimate recent demand from pipeline arrivals
27.     if len(pipeline_orders) >= recent_demand_window:
28.         recent_arrivals = pipeline_orders[:recent_demand_window]
29.         avg_recent_demand = sum(recent_arrivals) / len(recent_arrivals)
30.     else:
31.         avg_recent_demand = demand_forecast
32.
33.     # Blend static forecast with recent demand signal
34.     blended_forecast = (1 - pipeline_weight) * demand_forecast + pipeline_weight * avg_recent_demand
35.
36.     # Calculate expected lead time demand
37.     lead_time = len(pipeline_orders)

```

Declarative mathematical formula rows 1-13

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{array}{l} \text{recent_arrivals} = q_{:2} \\ \text{avg_recent_demand} = \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})} \end{array} \right\} \quad (L \geq 2)$$

$$\text{avg_recent_demand} = 99.9 \quad (\neg (L \geq 2))$$

$$\text{blended_forecast} = (((1 - 0.5) \cdot 99.9) + (0.5 \cdot \text{avg_recent_demand}))$$

$$\text{lead_time} = L$$

$$\text{expected_lead_time_demand} = (\text{blended_forecast} \cdot \text{lead_time})$$

$$\text{dynamic_target} = (319.9 + (1.1 \cdot \text{expected_lead_time_demand}))$$

$$\text{target_inventory} = (\text{dynamic_target} + 44.9)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{order_amount} = (0.1 \cdot \text{raw_order})$$

$$\left. \begin{array}{l} \text{emergency_order} = (1.2 \cdot \text{blended_forecast}) \\ \text{order_amount} = \max(\text{order_amount}, \text{emergency_order}) \end{array} \right\} \quad (\text{inventory_position} < 20.0)$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 38-58

```

38.     expected_lead_time_demand = blended_forecast * lead_time
39.
40.     # Dynamic target: base stock + forecast-adjusted component
41.     dynamic_target = base_stock + forecast_weight * expected_lead_time_demand
42.
43.     # Final target with safety stock
44.     target_inventory = dynamic_target + safety_stock
45.
46.     # Calculate raw order amount
47.     raw_order = max(0, target_inventory - inventory_position)
48.
49.     # Apply smoothing with emergency ordering for critically low inventory
50.     order_amount = smoothing_factor * raw_order
51.
52.     # Emergency ordering when inventory is very low
53.     if inventory_position < min_order_threshold:
54.         emergency_order = emergency_order_multiplier * blended_forecast
55.         order_amount = max(order_amount, emergency_order)
56.
57.     # Ensure integer output
58.     return order_amount

```

Declarative mathematical formula rows 1-13

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{array}{l} \text{recent_arrivals} = q_{:2} \\ \text{avg_recent_demand} = \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})} \end{array} \right\} \quad (L \geq 2)$$

$$\text{avg_recent_demand} = 99.9 \quad (\neg (L \geq 2))$$

$$\text{blended_forecast} = (((1 - 0.5) \cdot 99.9) + (0.5 \cdot \text{avg_recent_demand}))$$

$$\text{lead_time} = L$$

$$\text{expected_lead_time_demand} = (\text{blended_forecast} \cdot \text{lead_time})$$

$$\text{dynamic_target} = (319.9 + (1.1 \cdot \text{expected_lead_time_demand}))$$

$$\text{target_inventory} = (\text{dynamic_target} + 44.9)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{order_amount} = (0.1 \cdot \text{raw_order})$$

$$\left. \begin{array}{l} \text{emergency_order} = (1.2 \cdot \text{blended_forecast}) \\ \text{order_amount} = \max(\text{order_amount}, \text{emergency_order}) \end{array} \right\} \quad (\text{inventory_position} < 20.0)$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```

1. # sample_id: 032
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_e1-e2_6_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 5
5. # rank_in_population_file: 5
6. # objective: 6068.0
7. # test_objective: 6159.841
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping;order_smoothing
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 389.9862343918401 # OPT_PARAM: {"initial": 389.9862343918401, "min": 200.0, "max": 600.0, "type": "float"}
15.
16.     # Demand anticipation factor (lookahead adjustment)
17.     lookahead_factor = 0.8545657923765599 # OPT_PARAM: {"initial": 0.8545657923765599, "min": 0.3, "max": 1.5, "type": "float"}
18.
19.     # Pipeline shape penalty (penalizes uneven distribution)
20.     shape_penalty = 0.40742876509371084 # OPT_PARAM: {"initial": 0.40742876509371084, "min": 0.0, "max": 0.5, "type": "float"}
21.
22.     # Order smoothing factor
23.     smoothing = 0.13403118760093807 # OPT_PARAM: {"initial": 0.13403118760093807, "min": 0.0, "max": 0.8, "type": "float"}
24.
25.     # Minimum order threshold
26.     min_order = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0.0, "max": 30.0, "type": "float"}
27.
28.     # Critical ratio adjustment (newsvendor-like)
29.     critical_ratio = 0.645682997278703 # OPT_PARAM: {"initial": 0.645682997278703, "min": 0.5, "max": 0.9, "type": "float"}
30.
31.     # Pipeline variability safety factor
32.     safety_multiplier = 0.29881913103080476 # OPT_PARAM: {"initial": 0.29881913103080476, "min": 0.1, "max": 2.0, "type": "float"}
33.
34.     # Pipeline trend factor for demand prediction
35.     trend_factor = 0.9932500500007628 # OPT_PARAM: {"initial": 0.9932500500007628, "min": 0.3, "max": 1.5, "type": "float"}
36.
37.     # Calculate total pipeline inventory

```

Declarative mathematical formula rows 1-17

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\left. \begin{aligned} \text{split} &= \left\lfloor \frac{L}{2} \right\rfloor \\ \text{first_half} &= \sum_{i=0}^{(\text{split})-1} q_i \\ \text{second_half} &= \sum_{i=\text{split}}^{L-1} q_i \end{aligned} \right\} (L \geq 2)$$

$$\text{imbalance} = \begin{cases} \frac{|(\text{first_half} - \text{second_half})|}{(\text{first_half} + \text{second_half})}, & (\text{first_half} + \text{second_half}) > 0 \\ 0.0, & \text{otherwise} \end{cases}$$

$$\text{shape_adjustment} = ((0.4 \cdot \text{imbalance}) \cdot 390.0)$$

$$\text{shape_adjustment} = 0.0 \quad (\neg(L \geq 2))$$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.9)^{(L-i-1)}))$$

$$\text{trend_adjustment} = 0.0$$

$$\left. \begin{aligned} \text{recent_periods} &= \min(3, L) \\ \text{recent_sum} &= \sum_{i=0}^{(\text{recent_periods})-1} q_i \\ \text{older_sum} &= \begin{cases} \sum_{i=\text{recent_periods}}^{L-1} q_i, & L > \text{recent_periods} \\ 0, & \text{otherwise} \end{cases} \end{aligned} \right\} (L \geq 2)$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\text{recent_sum}}{\text{recent_periods}} \\ \text{older_avg} &= \begin{cases} \frac{\text{older_sum}}{(L - \text{recent_periods})}, & L > \text{recent_periods} \\ \text{recent_avg}, & \text{otherwise} \end{cases} \end{aligned} \right\} ((\text{older_sum} > 0 \wedge \text{recent_periods} > 0))$$

$$\text{demand_trend} = (\text{recent_avg} - \text{older_avg})$$

$$\text{trend_adjustment} = ((1.0 \cdot \text{demand_trend}) \cdot 2.0)$$

$$\text{effective_inventory} = (h + \text{weighted_pipeline})$$

$$\text{ratio_adjustment} = ((0.6 - 0.7) \cdot 100.0)$$

$$\left. \begin{aligned} \text{mean_pipeline} &= \frac{\text{total_pipeline}}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{mean_pipeline}))^2}{L} \\ \text{pipeline_std} &= (\text{variance})^{0.5} \\ \text{safety_stock} &= (0.3 \cdot \text{pipeline_std}) \end{aligned} \right\} (L > 1)$$

$$\text{safety_stock} = 0.0 \quad (\neg(L > 1))$$

$$\text{target_inventory} = (((390.0 + \text{ratio_adjustment}) - \text{shape_adjustment}) + \text{safety_stock}) + \text{trend_adjustment}$$

$$\text{basic_order} = \max(0, (\text{target_inventory} - \text{effective_inventory}))$$

$$\left. \begin{aligned} \text{previous_order} &= q_{-1} \\ \text{smoothed_order} &= ((0.1 \cdot \text{basic_order}) + ((1 - 0.1) \cdot \text{previous_order})) \end{aligned} \right\} (L > 0)$$

$$\text{smoothed_order} = \text{basic_order} \quad (\neg(L > 0))$$

Python source lines 38-83

```

38.     total_pipeline = sum(pipeline_orders)
39.
40.     # Calculate pipeline shape penalty
41.     L = len(pipeline_orders)
42.     if L >= 2:
43.         # Split into first half and second half
44.         split = L // 2
45.         first_half = sum(pipeline_orders[:split])
46.         second_half = sum(pipeline_orders[split:])
47.
48.         # Calculate imbalance (0 = balanced, >0 = imbalanced)
49.         if first_half + second_half > 0:
50.             imbalance = abs(first_half - second_half) / (first_half + second_half)
51.         else:
52.             imbalance = 0.0
53.
54.         # Apply penalty proportional to imbalance
55.         shape_adjustment = shape_penalty * imbalance * base_stock
56.     else:
57.         shape_adjustment = 0.0
58.
59.     # Calculate lookahead adjustment with exponential decay
60.     weighted_pipeline = 0.0
61.     for i, q in enumerate(pipeline_orders):
62.         weight = lookahead_factor ** (L - i - 1)
63.         weighted_pipeline += q * weight
64.
65.     # NEW: Calculate pipeline trend for demand prediction
66.     trend_adjustment = 0.0
67.     if L >= 2:
68.         # Compare recent vs older pipeline orders to estimate demand trend
69.         recent_periods = min(3, L)
70.         recent_sum = sum(pipeline_orders[:recent_periods])
71.         older_sum = sum(pipeline_orders[recent_periods:]) if L > recent_periods else 0
72.
73.         if older_sum > 0 and recent_periods > 0:
74.             recent_avg = recent_sum / recent_periods
75.             older_avg = older_sum / (L - recent_periods) if L > recent_periods else recent_avg
76.             demand_trend = recent_avg - older_avg
77.             trend_adjustment = trend_factor * demand_trend * 2.0 # Scale adjustment
78.
79.     # Calculate effective inventory position with lookahead
80.     effective_inventory = on_hand_inventory + weighted_pipeline
81.
82.     # Calculate critical ratio adjustment
83.     ratio_adjustment = (critical_ratio - 0.667) * 100.0

```

Declarative mathematical formula rows 1-17

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\left. \begin{aligned} \text{split} &= \left\lfloor \frac{L}{2} \right\rfloor \\ \text{first_half} &= \sum_{i=0}^{(\text{split})-1} q_i \\ \text{second_half} &= \sum_{i=\text{split}}^{L-1} q_i \\ \text{imbalance} &= \begin{cases} \frac{|(\text{first_half} - \text{second_half})|}{(\text{first_half} + \text{second_half})}, & (\text{first_half} + \text{second_half}) > 0 \\ 0.0, & \text{otherwise} \end{cases} \\ \text{shape_adjustment} &= ((0.4 \cdot \text{imbalance}) \cdot 390.0) \end{aligned} \right\} (L \geq 2)$$

$$\text{shape_adjustment} = 0.0 \quad (\neg(L \geq 2))$$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} (q_i \cdot (0.9)^{(L-i-1)})$$

$$\text{trend_adjustment} = 0.0$$

$$\left. \begin{aligned} \text{recent_periods} &= \min(3, L) \\ \text{recent_sum} &= \sum_{i=0}^{(\text{recent_periods})-1} q_i \\ \text{older_sum} &= \begin{cases} \sum_{i=\text{recent_periods}}^{L-1} q_i, & L > \text{recent_periods} \\ 0, & \text{otherwise} \end{cases} \\ \text{recent_avg} &= \frac{\text{recent_sum}}{\text{recent_periods}} \\ \text{older_avg} &= \begin{cases} \frac{\text{older_sum}}{(L - \text{recent_periods})}, & L > \text{recent_periods} \\ \text{recent_avg}, & \text{otherwise} \end{cases} \\ \text{demand_trend} &= (\text{recent_avg} - \text{older_avg}) \\ \text{trend_adjustment} &= ((1.0 \cdot \text{demand_trend}) \cdot 2.0) \end{aligned} \right\} ((\text{older_sum} > 0 \wedge \text{recent_periods} > 0))$$

$$\text{effective_inventory} = (h + \text{weighted_pipeline})$$

$$\text{ratio_adjustment} = ((0.6 - 0.7) \cdot 100.0)$$

$$\left. \begin{aligned} \text{mean_pipeline} &= \frac{\text{total_pipeline}}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{mean_pipeline}))^2}{L} \\ \text{pipeline_std} &= (\text{variance})^{0.5} \\ \text{safety_stock} &= (0.3 \cdot \text{pipeline_std}) \end{aligned} \right\} (L > 1)$$

$$\text{safety_stock} = 0.0 \quad (\neg(L > 1))$$

$$\text{target_inventory} = (((390.0 + \text{ratio_adjustment}) - \text{shape_adjustment}) + \text{safety_stock}) + \text{trend_adjustment}$$

$$\text{basic_order} = \max(0, (\text{target_inventory} - \text{effective_inventory}))$$

$$\text{previous_order} = q_{-1}$$

$$(L < 0)$$

Python source lines 84-113

```

84.
85.     # Calculate pipeline variability safety stock
86.     if len(pipeline_orders) > 1:
87.         mean_pipeline = total_pipeline / len(pipeline_orders)
88.         variance = sum((q - mean_pipeline) ** 2 for q in pipeline_orders) / len(pipeline_orders)
89.         pipeline_std = variance ** 0.5
90.         safety_stock = safety_multiplier * pipeline_std
91.     else:
92.         safety_stock = 0.0
93.
94.     # Calculate target inventory position with all adjustments
95.     target_inventory = base_stock + ratio_adjustment - shape_adjustment + safety_stock +
trend_adjustment
96.
97.     # Calculate basic order amount
98.     basic_order = max(0, target_inventory - effective_inventory)
99.
100.    # Apply smoothing with previous order (if available)
101.    if L > 0:
102.        previous_order = pipeline_orders[-1]
103.        smoothed_order = smoothing * basic_order + (1 - smoothing) * previous_order
104.    else:
105.        smoothed_order = basic_order
106.
107.    # Apply minimum order threshold
108.    if smoothed_order < min_order:
109.        order_amount = 0
110.    else:
111.        order_amount = int(round(smoothed_order))
112.
113.    return order_amount

```

Declarative mathematical formula rows 1-17

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\left. \begin{aligned} \text{split} &= \left\lfloor \frac{L}{2} \right\rfloor \\ \text{first_half} &= \sum_{i=0}^{(\text{split})-1} q_i \\ \text{second_half} &= \sum_{i=\text{split}}^{L-1} q_i \\ \text{imbalance} &= \begin{cases} \frac{|(\text{first_half} - \text{second_half})|}{(\text{first_half} + \text{second_half})}, & (\text{first_half} + \text{second_half}) > 0 \\ 0.0, & \text{otherwise} \end{cases} \\ \text{shape_adjustment} &= ((0.4 \cdot \text{imbalance}) \cdot 390.0) \end{aligned} \right\} (L \geq 2)$$

$$\text{shape_adjustment} = 0.0 \quad (\neg (L \geq 2))$$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.9)^{(L-i)-1}))$$

$$\text{trend_adjustment} = 0.0$$

$$\left. \begin{aligned} \text{recent_periods} &= \min(3, L) \\ \text{recent_sum} &= \sum_{i=0}^{(\text{recent_periods})-1} q_i \\ \text{older_sum} &= \begin{cases} \sum_{i=\text{recent_periods}}^{L-1} q_i, & L > \text{recent_periods} \\ 0, & \text{otherwise} \end{cases} \\ \text{recent_avg} &= \frac{\text{recent_sum}}{\text{recent_periods}} \\ \text{older_avg} &= \begin{cases} \frac{\text{older_sum}}{(L - \text{recent_periods})}, & L > \text{recent_periods} \\ \text{recent_avg}, & \text{otherwise} \end{cases} \\ \text{demand_trend} &= (\text{recent_avg} - \text{older_avg}) \\ \text{trend_adjustment} &= ((1.0 \cdot \text{demand_trend}) \cdot 2.0) \end{aligned} \right\} ((\text{older_sum} > 0 \wedge \text{recent_periods} > 0)) \quad (L \geq 2)$$

$$\text{effective_inventory} = (h + \text{weighted_pipeline})$$

$$\text{ratio_adjustment} = ((0.6 - 0.7) \cdot 100.0)$$

$$\left. \begin{aligned} \text{mean_pipeline} &= \frac{\text{total_pipeline}}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{mean_pipeline})^2)}{L} \\ \text{pipeline_std} &= (\text{variance})^{0.5} \\ \text{safety_stock} &= (0.3 \cdot \text{pipeline_std}) \end{aligned} \right\} (L > 1)$$

$$\text{safety_stock} = 0.0 \quad (\neg (L > 1))$$

$$\text{target_inventory} = (((390.0 + \text{ratio_adjustment}) - \text{shape_adjustment}) + \text{safety_stock}) + \text{trend_adjustment}$$

$$\text{basic_order} = \max(0, (\text{target_inventory} - \text{effective_inventory}))$$

$$\text{previous_order} = q_{-1} \quad (L > 0)$$

Python source lines 1-37

```
1. # sample_id: 033
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r6
3. # distribution: poisson_L6_c1_5
4. # generation: 6
5. # rank_in_population_file: 7
6. # objective: 1169.10026
7. # test_objective: 1237.16751
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;order_up_to;state_dependent_target;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 711.3122885869875 # OPT_PARAM: {"initial": 711.3122885869875, "min": 400, "max":
    800, "type": "float"}
14.     safety_stock = 80.97700274721207 # OPT_PARAM: {"initial": 80.97700274721207, "min": 30, "max":
    150, "type": "float"}
15.     demand_forecast = 105.6294716556608 # OPT_PARAM: {"initial": 105.6294716556608, "min": 80,
    "max": 120, "type": "float"}
16.     max_order = 98.96898380462649 # OPT_PARAM: {"initial": 98.96898380462649, "min": 80, "max": 200,
    "type": "float"}
17.
18.     # Calculate inventory position
19.     inventory_position = on_hand_inventory + sum(pipeline_orders)
20.
21.     # Calculate expected demand during lead time plus review period
22.     expected_demand = demand_forecast * (len(pipeline_orders) + 1)
23.
24.     # Calculate target inventory position
25.     target_position = expected_demand + safety_stock
26.
27.     # Calculate order amount
28.     order_amount = max(0, target_position - inventory_position)
29.
30.     # Apply maximum order constraint
31.     order_amount = min(order_amount, max_order)
32.
33.     # Apply base stock as upper bound
34.     if inventory_position + order_amount > base_stock:
35.         order_amount = max(0, base_stock - inventory_position)
36.
37.     return order_amount
```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_demand} = (105.6 \cdot (L + 1))$$

$$\text{target_position} = (\text{expected_demand} + 81.0)$$

$$\text{order_amount} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{order_amount} = \min(\text{order_amount}, 99.0)$$

$$\text{order_amount} = \begin{cases} \max(0, (711.3 - \text{inventory_position})), & (\text{inventory_position} + \text{order_amount}) > 711.3 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-31

```
1. # sample_id: 034
2. # folder: deepseek-chat_exponential_L6_c1_5_50_plain_processed_scipy_15_default_m2_2_r6
3. # distribution: exponential_L6_c1_5
4. # generation: 1
5. # rank_in_population_file: 2
6. # objective: 12927.64416
7. # test_objective: 13114.4267
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 477.5417156010887 # OPT_PARAM: {"initial": 477.5417156010887, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 77.1165950245596 # OPT_PARAM: {"initial": 77.1165950245596, "min": 50, "max":
16.     400, "type": "float"}
17.     demand_estimate = 72.05912542391138 # OPT_PARAM: {"initial": 72.05912542391138, "min": 50,
18.     "max": 300, "type": "float"}
19.     pipeline_weight = 0.9975758916253765 # OPT_PARAM: {"initial": 0.9975758916253765, "min": 0.1,
20.     "max": 1.5, "type": "float"}
21.
22.     # Calculate effective inventory position
23.     effective_pipeline = sum(pipeline_orders) * pipeline_weight
24.     inventory_position = on_hand_inventory + effective_pipeline
25.
26.     # Calculate target inventory level
27.     target_inventory = base_stock + safety_stock
28.
29.     # Calculate order amount
30.     order_amount = max(0, target_inventory - inventory_position)
31.
32.     # Add demand estimate adjustment
33.     order_amount = max(order_amount, demand_estimate - sum(pipeline_orders[-2:])) if
34.     len(pipeline_orders) >= 2 else demand_estimate
35.
36.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(\max \left(0, \left((477.5 + 77.1) - \left(h + \left(\sum_{i=0}^{L-1} q_i \cdot 1.0 \right) \right) \right) \right), \begin{cases} (72.1 - \sum_{i=L-(2)}^{L-1} q_i), & L \geq 2 \\ 72.1, & \text{otherwise} \end{cases} \right).$$

Python source lines 1-34

```

1. # sample_id: 035
2. # folder: deepseek-chat_poisson_L2_c1_2_50_plain_processed_scipy_15_default_m2_10_r2
3. # distribution: poisson_L2_c1_2
4. # generation: 1
5. # rank_in_population_file: 3
6. # objective: 1031.2
7. # test_objective: 1033.424
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 294.00072750770767 # OPT_PARAM: {"initial": 294.00072750770767, "min": 10, "max":
1000, "type": "float"}
14.     safety_stock = 15.0 # OPT_PARAM: {"initial": 15.0, "min": 0, "max": 100, "type": "float"}
15.     demand_estimate = 100.0 # OPT_PARAM: {"initial": 100.0, "min": 50, "max": 150, "type": "float"}
16.     adjustment_factor = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.1, "max": 1.0, "type": "float"}
17.
18.     # Calculate current inventory position
19.     inventory_position = on_hand_inventory + sum(pipeline_orders)
20.
21.     # Calculate expected demand during lead time (L=2 periods)
22.     expected_lead_time_demand = demand_estimate * 2
23.
24.     # Calculate target inventory position
25.     target_position = expected_lead_time_demand + safety_stock
26.
27.     # Calculate order-up-to level
28.     order_up_to = max(base_stock, target_position)
29.
30.     # Calculate order quantity with adjustment
31.     raw_order = max(0, order_up_to - inventory_position)
32.     order_amount = int(round(raw_order * adjustment_factor))
33.
34.     return order_amount

```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\left(\max \left(0, \left(\max(294.0, ((100.0 \cdot 2) + 15.0)) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \cdot 1.0 \right) \right).$$

Python source lines 1-41

```

1. # sample_id: 036
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_m2_4_r7
3. # distribution: poisson_L4_c1_5
4. # generation: 4
5. # rank_in_population_file: 7
6. # objective: 1504.18025
7. # test_objective: 1526.05274
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 460.6106521270296 # OPT_PARAM: {"initial": 460.6106521270296, "min": 400, "max":
550, "type": "float"}
14.     safety_stock = 51.02889525595642 # OPT_PARAM: {"initial": 51.02889525595642, "min": 40, "max":
100, "type": "float"}
15.     demand_forecast = 94.63494303930437 # OPT_PARAM: {"initial": 94.63494303930437, "min": 90,
"max": 110, "type": "float"}
16.     alpha = 0.7 # OPT_PARAM: {"initial": 0.7, "min": 0.7, "max": 0.95, "type": "float"}
17.     beta = 0.25 # OPT_PARAM: {"initial": 0.25, "min": 0.15, "max": 0.5, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand during lead time
23.     lead_time = len(pipeline_orders)
24.     lead_time_demand = demand_forecast * lead_time
25.
26.     # Target inventory position: base stock adjusted for lead time demand
27.     target_inventory = base_stock + safety_stock
28.
29.     # Calculate order needed to reach target
30.     order_needed = target_inventory - inventory_position
31.
32.     if order_needed > 0:
33.         # Order smoothing: mix of needed amount and forecasted demand
34.         order_amount = alpha * order_needed + (1 - alpha) * demand_forecast
35.         # Additional smoothing to prevent over-ordering
36.         order_amount = beta * order_amount + (1 - beta) * demand_forecast
37.     else:
38.         order_amount = 0
39.
40.     # Ensure non-negative integer order
41.     return order_amount

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$\text{lead_time} = L$

$\text{lead_time_demand} = (94.6 \cdot \text{lead_time})$

$\text{target_inventory} = (460.6 + 51.0)$

$\text{order_needed} = (\text{target_inventory} - \text{inventory_position})$

$\text{order_amount} = ((0.7 \cdot \text{order_needed}) + ((1 - 0.7) \cdot 94.6)) \}$ (order_needed > 0)

$\text{order_amount} = ((0.2 \cdot \text{order_amount}) + ((1 - 0.2) \cdot 94.6)) \}$

$\text{order_amount} = 0 \}$ ($\neg(\text{order_needed} > 0)$)

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-38

```

1. # sample_id: 037
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-m2_6_r2
3. # distribution: normal_std30_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 4
6. # objective: 2404.84
7. # test_objective: 2406.131
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;nonlinear_pipeline_composition;order_clipping
11. # extra_motifs:
    safety_stock_buffer;threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Core inventory parameters
14.     base_stock = 598.921328373424 # OPT_PARAM: {"initial": 598.921328373424, "min": 300, "max": 900, "type": "float"}
15.     safety_stock = 150.59173522735384 # OPT_PARAM: {"initial": 150.59173522735384, "min": 50, "max": 300, "type": "float"}
16.
17.     # NEW STRUCTURAL FEATURE: Pipeline volatility assessment
18.     volatility_sensitivity = 0.2574739274873779 # OPT_PARAM: {"initial": 0.2574739274873779, "min": 0.1, "max": 1.0, "type": "float"}
19.     smoothing_factor = 0.08936848187184449 # OPT_PARAM: {"initial": 0.08936848187184449, "min": 0.05, "max": 0.5, "type": "float"}
20.
21.     # NEW STRUCTURAL FEATURE: Asymmetric response parameters
22.     shortage_response = 1.6258079301377846 # OPT_PARAM: {"initial": 1.6258079301377846, "min": 1.0, "max": 2.0, "type": "float"}
23.     excess_response = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.5, "max": 1.0, "type": "float"}
24.
25.     # NEW STRUCTURAL FEATURE: Pipeline pattern recognition
26.     pattern_weight = 0.421263036256311 # OPT_PARAM: {"initial": 0.421263036256311, "min": 0.1, "max": 0.5, "type": "float"}
27.     min_order_threshold = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0.0, "max": 30.0, "type": "float"}
28.
29.     # Calculate pipeline volatility - NEW DECISION LOGIC
30.     # Measures variability in pipeline orders (standard deviation relative to mean)
31.     L = len(pipeline_orders)
32.     if L > 1:
33.         pipeline_mean = sum(pipeline_orders) / L
34.         if pipeline_mean > 0:
35.             variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
36.             volatility = (variance ** 0.5) / pipeline_mean
37.         else:
38.             volatility = 0.0

```

Declarative mathematical formula rows 1-17

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{variance} &= \frac{\sum_{q \in q} ((q - \text{pipeline_mean}))^2}{L} \\ \text{volatility} &= \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \end{aligned} \right\} \begin{aligned} &(\text{pipeline_mean} > 0) \\ &(\text{pipeline_mean} > 0) \end{aligned} \quad (L > 1)$$

$$\text{volatility} = 0.0 \quad (\neg(\text{pipeline_mean} > 0))$$

$$\text{volatility} = 0.0 \quad (\neg(L > 1))$$

$$\left. \begin{aligned} x_mean &= \frac{(L-1)}{2} \\ y_mean &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{numerator} &= \sum_{i=0}^{L-1} ((i - x_mean) \cdot (q_i - y_mean)) \\ \text{denominator} &= \sum_{i \in \text{range}(L)} ((i - x_mean))^2 \\ \text{trend_slope} &= \frac{\text{numerator}}{\text{denominator}} \\ \text{max_possible_slope} &= \begin{cases} \max(\{|q| \mid q \in q\}), & q \\ 1, & \text{otherwise} \end{cases} \\ \text{normalized_trend} &= \frac{\text{trend_slope}}{(\text{max_possible_slope} + 1.0e-06)} \end{aligned} \right\} \begin{aligned} &(\text{denominator} > 0) \\ &(\text{denominator} > 0) \end{aligned} \quad (L \geq 3)$$

$$\text{normalized_trend} = \max(-(1.0), \min(1.0, \text{normalized_trend}))$$

$$\text{normalized_trend} = 0.0 \quad (\neg(\text{denominator} > 0))$$

$$\text{normalized_trend} = 0.0 \quad (\neg(L \geq 3))$$

$$\text{volatility_adjusted_safety} = (150.6 \cdot (1.0 + (0.3 \cdot \text{volatility})))$$

$$\left. \begin{aligned} \text{weights} &= \left[\left(1.0 - \frac{(i \cdot |\text{normalized_trend}|)}{L} \right) \mid i \in \text{range}(L) \right] \\ \text{weights} &= [\max(0.1, w) \mid w \in \text{weights}] \end{aligned} \right\} \quad (\text{normalized_trend} < 0)$$

$$\left. \begin{aligned} \text{weights} &= \left\{ \left[\left(0.5 + \frac{(i \cdot \text{normalized_trend})}{L} \right) \mid i \in \text{range}(L) \right], \quad \text{normalized_trend} > 0 \right. \\ &\quad \left. ([1.0] \cdot L), \quad \text{otherwise} \right\} \end{aligned} \right\} \quad (\neg(\text{normalized_trend} < 0)) \quad (L > 0)$$

$$\text{weight_sum} = \text{sum}(\text{weights})$$

$$\text{effective_pipeline} = \begin{cases} \frac{\sum_{q \in q} \text{weight_q}(w \cdot q)}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{effective_pipeline} = 0.0 \quad (\neg(L > 0))$$

pattern_adjusted_pipeline = (effective_pipeline · (1.0 + (0.4 · normalized_trend)))

effective_inventory = (h + pattern_adjusted_pipeline)

target_inventory = (598.9 + volatility_adjusted_safety)

order_gap = (target_inventory - effective_inventory)

response_factor = 1.6 (order_gap > 0)

response_factor = 0.8 (order_gap > 0)

order_gap = 0 (order_gap > 0)

smoothed_order = ((0.1 · response_factor) · order_gap)

order_amount = $\begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \max(0, \text{round}_{\text{even}}(\text{smoothed_order})), & \text{otherwise} \end{cases}$

$Q(h, q) = \text{order_amount}$

Python source lines 39-83

```

39.     else:
40.         volatility = 0.0
41.
42.     # NEW STRUCTURAL FEATURE: Detect pipeline patterns
43.     # Identify increasing/decreasing trends in pipeline
44.     if L >= 3:
45.         # Calculate trend using linear regression slope
46.         x_mean = (L - 1) / 2
47.         y_mean = sum(pipeline_orders) / L
48.
49.         numerator = sum((i - x_mean) * (q - y_mean) for i, q in enumerate(pipeline_orders))
50.         denominator = sum((i - x_mean) ** 2 for i in range(L))
51.
52.         if denominator > 0:
53.             trend_slope = numerator / denominator
54.             # Normalize trend to [-1, 1] range
55.             max_possible_slope = max(abs(q) for q in pipeline_orders) if pipeline_orders else 1
56.             normalized_trend = trend_slope / (max_possible_slope + 1e-6)
57.             normalized_trend = max(-1.0, min(1.0, normalized_trend))
58.         else:
59.             normalized_trend = 0.0
60.     else:
61.         normalized_trend = 0.0
62.
63.     # Adjust safety stock based on pipeline volatility
64.     # Higher volatility → higher safety stock
65.     volatility_adjusted_safety = safety_stock * (1.0 + volatility_sensitivity * volatility)
66.
67.     # Calculate effective inventory position with pattern adjustment
68.     # Trend-aware weighting: discount future orders if pipeline is decreasing
69.     if L > 0:
70.         if normalized_trend < 0: # Decreasing pipeline
71.             # Give more weight to near-term orders
72.             weights = [1.0 - i * abs(normalized_trend) / L for i in range(L)]
73.             weights = [max(0.1, w) for w in weights] # Ensure positive weights
74.         elif normalized_trend > 0: # Increasing pipeline
75.             # Give more weight to future orders
76.             weights = [0.5 + i * normalized_trend / L for i in range(L)]
77.         else: # No clear trend
78.             weights = [1.0] * L
79.
80.     # Normalize weights
81.     weight_sum = sum(weights)
82.     effective_pipeline = sum(w * q for w, q in zip(weights, pipeline_orders)) / weight_sum if
83.     weight_sum > 0 else 0
84.     else:

```

Declarative mathematical formula rows 1-17

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned}
 &\text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 &\text{variance} = \frac{\sum_{q \in q} ((q - \text{pipeline_mean}))^2}{L} \\
 &\text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}} \\
 &\text{volatility} = 0.0 \} \quad (\neg(\text{pipeline_mean} > 0)) \\
 &\text{volatility} = 0.0 \} \quad (\neg(L > 1))
 \end{aligned} \right\} \quad (\text{pipeline_mean} > 0) \quad (L > 1)$$

$$\left. \begin{aligned}
 &x_mean = \frac{(L-1)}{2} \\
 &y_mean = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 &\text{numerator} = \sum_{i=0}^{L-1} ((i - x_mean) \cdot (q_i - y_mean)) \\
 &\text{denominator} = \sum_{i \in \text{range}(L)} ((i - x_mean))^2 \\
 &\text{trend_slope} = \frac{\text{numerator}}{\text{denominator}} \\
 &\text{max_possible_slope} = \begin{cases} \max(\{|q| \mid q \in q\}), & q \\ 1, & \text{otherwise} \end{cases} \\
 &\text{normalized_trend} = \frac{\text{trend_slope}}{(\text{max_possible_slope} + 1.0e-06)} \\
 &\text{normalized_trend} = \max(-1.0, \min(1.0, \text{normalized_trend})) \\
 &\text{normalized_trend} = 0.0 \} \quad (\neg(\text{denominator} > 0)) \\
 &\text{normalized_trend} = 0.0 \} \quad (\neg(L \geq 3))
 \end{aligned} \right\} \quad (\text{denominator} > 0) \quad (L \geq 3)$$

$$\left. \begin{aligned}
 &\text{normalized_trend} = 0.0 \} \quad (\neg(L \geq 3)) \\
 &\text{volatility_adjusted_safety} = (150.6 \cdot (1.0 + (0.3 \cdot \text{volatility}))) \\
 &\text{weights} = \left[\left(1.0 - \frac{(i \cdot |\text{normalized_trend}|)}{L} \right) \mid i \in \text{range}(L) \right] \} \quad (\text{normalized_trend} < 0) \\
 &\text{weights} = \left[\left(0.5 + \frac{(i \cdot \text{normalized_trend})}{L} \right) \mid i \in \text{range}(L) \right], \quad \text{normalized_trend} > 0 \} \quad (\neg(\text{normalized_trend} < 0)) \\
 &\text{weights} = \left[\left(1.0 \right) \cdot L, \right] \} \quad (\neg(\text{normalized_trend} < 0))
 \end{aligned} \right\} \quad (L > 0)$$

$$\left. \begin{aligned}
 &\text{weight_sum} = \sum(\text{weights}) \\
 &\text{effective_pipeline} = \begin{cases} \frac{\sum_{(w,q) \in \text{zip}(\text{weights}, q)} (w \cdot q)}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 &\text{effective_pipeline} = 0.0 \} \quad (\neg(L > 0)) \\
 &\text{pattern_adjusted_pipeline} = (\text{effective_pipeline} \cdot (1.0 + (0.4 \cdot \text{normalized_trend}))) \\
 &\text{effective_inventory} = (h + \text{pattern_adjusted_pipeline}) \\
 &\text{target_inventory} = (598.9 + \text{volatility_adjusted_safety}) \\
 &\text{order_gap} = (\text{target_inventory} - \text{effective_inventory}) \\
 &\text{response_factor} = 1.6 \} \quad (\text{order_gap} > 0) \\
 &\text{response_factor} = 0.8 \} \quad (\neg(\text{order_gap} > 0)) \\
 &\text{order_gap} = 0 \\
 &\text{smoothed_order} = ((0.1 \cdot \text{response_factor}) \cdot \text{order_gap}) \\
 &\text{order_amount} = \begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \max(0, \text{round}_{\text{even}}(\text{smoothed_order})), & \text{otherwise} \end{cases} \\
 &Q(h, q) = \text{order_amount}
 \end{aligned} \right\}$$

Python source lines 84-114

```

84.         effective_pipeline = 0.0
85.
86.     # Apply pattern-based adjustment to pipeline estimate
87.     pattern_adjusted_pipeline = effective_pipeline * (1.0 + pattern_weight * normalized_trend)
88.
89.     effective_inventory = on_hand_inventory + pattern_adjusted_pipeline
90.
91.     # Calculate target inventory
92.     target_inventory = base_stock + volatility_adjusted_safety
93.
94.     # Calculate order gap with asymmetric response
95.     order_gap = target_inventory - effective_inventory
96.
97.     # NEW STRUCTURAL FEATURE: Asymmetric smoothing
98.     # Respond more aggressively to shortages, less aggressively to excess
99.     if order_gap > 0: # Shortage situation
100.         response_factor = shortage_response
101.     else: # Excess inventory situation (order_gap <= 0)
102.         response_factor = excess_response
103.         order_gap = 0 # No ordering when we have excess
104.
105.     # Apply smoothing with asymmetric response
106.     smoothed_order = smoothing_factor * response_factor * order_gap
107.
108.     # Apply minimum order threshold
109.     if smoothed_order < min_order_threshold:
110.         order_amount = 0
111.     else:
112.         order_amount = max(0, int(round(smoothed_order)))
113.
114.     return order_amount

```

Declarative mathematical formula rows 1-17

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 & \left. \begin{aligned}
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{variance} = \frac{\sum_{q \in q} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{volatility} = \frac{(\text{variance})^{0.5}}{\text{pipeline_mean}}
 \end{aligned} \right\} \begin{aligned} & (\text{pipeline_mean} > 0) \\ & (\text{pipeline_mean} > 0) \end{aligned} \quad (L > 1) \\
 & \text{volatility} = 0.0 \} \quad (\neg (\text{pipeline_mean} > 0)) \\
 & \text{volatility} = 0.0 \} \quad (\neg (L > 1)) \\
 & \left. \begin{aligned}
 & \text{x_mean} = \frac{(L-1)}{2} \\
 & \text{y_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{numerator} = \sum_{i=0}^{L-1} ((i - \text{x_mean}) \cdot (q_i - \text{y_mean})) \\
 & \text{denominator} = \sum_{i \in \text{range}(L)} ((i - \text{x_mean}))^2 \\
 & \text{trend_slope} = \frac{\text{numerator}}{\text{denominator}} \\
 & \text{max_possible_slope} = \begin{cases} \max(\{|q| \mid q \in q\}), & q \\ 1, & \text{otherwise} \end{cases} \\
 & \text{normalized_trend} = \frac{\text{trend_slope}}{(\text{max_possible_slope} + 1.0e-06)} \\
 & \text{normalized_trend} = \max(-1.0, \min(1.0, \text{normalized_trend})) \\
 & \text{normalized_trend} = 0.0 \} \quad (\neg (\text{denominator} > 0)) \\
 & \text{normalized_trend} = 0.0 \} \quad (\neg (L \geq 3))
 \end{aligned} \right\} (L \geq 3) \\
 & \text{volatility_adjusted_safety} = (150.6 \cdot (1.0 + (0.3 \cdot \text{volatility}))) \\
 & \left. \begin{aligned}
 & \text{weights} = \left[\left(1.0 - \frac{(i \cdot |\text{normalized_trend}|)}{L} \right) \mid i \in \text{range}(L) \right] \\
 & \text{weights} = [\max(0.1, w) \mid w \in \text{weights}] \\
 & \text{weights} = \begin{cases} \left[\left(0.5 + \frac{(i \cdot \text{normalized_trend})}{L} \right) \mid i \in \text{range}(L) \right], & \text{normalized_trend} > 0 \\ [1.0] \cdot L, & \text{otherwise} \end{cases} \\
 & \text{weight_sum} = \text{sum}(\text{weights}) \\
 & \text{effective_pipeline} = \begin{cases} \frac{\sum_{(w,q) \in \text{weights_q}} (w \cdot q)}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 0, & \text{otherwise} \end{cases}
 \end{aligned} \right\} \begin{aligned} & (\text{normalized_trend} < 0) \\ & (\neg (\text{normalized_trend} < 0)) \end{aligned} \quad (L > 0) \\
 & \text{effective_pipeline} = 0.0 \} \quad (\neg (L > 0)) \\
 & \text{pattern_adjusted_pipeline} = (\text{effective_pipeline} \cdot (1.0 + (0.4 \cdot \text{normalized_trend}))) \\
 & \text{effective_inventory} = (h + \text{pattern_adjusted_pipeline}) \\
 & \text{target_inventory} = (598.9 + \text{volatility_adjusted_safety}) \\
 & \text{order_gap} = (\text{target_inventory} - \text{effective_inventory}) \\
 & \text{response_factor} = 1.6 \} \quad (\text{order_gap} > 0) \\
 & \text{response_factor} = 0.8 \} \quad (\neg (\text{order_gap} > 0)) \\
 & \text{order_gap} = 0 \\
 & \text{smoothed_order} = ((0.1 \cdot \text{response_factor}) \cdot \text{order_gap}) \\
 & \text{order_amount} = \begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \max(0, \text{round}_{\text{even}}(\text{smoothed_order})), & \text{otherwise} \end{cases} \\
 & Q(h, q) = \text{order_amount}
 \end{aligned}$$

Python source lines 1-39

```

1. # sample_id: 038
2. # folder: deepseek-chat_poisson_L2_c1_2_50_plain_processed_scipy_15_default_m2_2_r6
3. # distribution: poisson_L2_c1_2
4. # generation: 9
5. # rank_in_population_file: 1
6. # objective: 758.44
7. # test_objective: 766.956
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping;order_
    smoothing
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer
    _rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 195.06472893772462 # OPT_PARAM: {"initial": 195.06472893772462, "min": 150, "max":
    250, "type": "float"}
14.     safety_multiplier = 1.2008583079563495 # OPT_PARAM: {"initial": 1.2008583079563495, "min": 0.8,
    "max": 2.0, "type": "float"}
15.     smoothing_factor = 0.28326705351321924 # OPT_PARAM: {"initial": 0.28326705351321924, "min":
    0.05, "max": 0.5, "type": "float"}
    lead_time_days = 2
16.
17.
18.     # Calculate net inventory position
19.     net_inventory = on_hand_inventory + sum(pipeline_orders)
20.
21.     # Use more accurate demand estimate based on historical data analysis
22.     demand_estimate = 100.13111443020748 # OPT_PARAM: {"initial": 100.13111443020748, "min": 90,
    "max": 110, "type": "float"}
23.
24.     # Calculate safety stock with improved formula
25.     safety_stock = safety_multiplier * demand_estimate * (lead_time_days ** 0.5)
26.
27.     # Adjust target based on pipeline composition - simplified
28.     pipeline_adjustment_factor = 0.7422589966119842 # OPT_PARAM: {"initial": 0.7422589966119842,
    "min": 0.5, "max": 1.2, "type": "float"}
    pipeline_adjustment = pipeline_adjustment_factor * sum(pipeline_orders)
29.
30.
31.     # Target inventory position with pipeline adjustment
32.     target = base_stock + safety_stock - pipeline_adjustment
33.
34.     # Calculate raw order quantity
35.     raw_order = max(0, target - net_inventory)
36.
37.     # Apply smoothing with minimum order threshold
38.     min_order_threshold = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 30, "type": "float"}
39.

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{lead_time_days} = 2$

$$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{safety_stock} = \left((1.2 \cdot 100.1) \cdot (\text{lead_time_days})^{0.5} \right)$$

$$\text{pipeline_adjustment} = \left(0.7 \cdot \sum_{i=0}^{L-1} q_i \right)$$

$$\text{target} = ((195.1 + \text{safety_stock}) - \text{pipeline_adjustment})$$

$$\text{raw_order} = \max(0, (\text{target} - \text{net_inventory}))$$

$$\left. \begin{array}{l} \text{recent_order} = q_{-(1)} \\ \text{smoothed_order} = ((0.3 \cdot \text{raw_order}) + ((1 - 0.3) \cdot \text{recent_order})) \end{array} \right\} \quad (L > 0)$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg (L > 0))$$

$$\text{smoothed_order} = \begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \text{smoothed_order}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 40-53

```

40.     if len(pipeline_orders) > 0:
41.         recent_order = pipeline_orders[-1]
42.         smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * recent_order
43.     else:
44.         smoothed_order = raw_order
45.
46.     # Apply minimum order threshold
47.     if smoothed_order < min_order_threshold:
48.         smoothed_order = 0
49.
50.     # Round to nearest integer
51.     order_amount = int(round(smoothed_order))
52.
53.     return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{lead_time_days} = 2$

$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$

$\text{safety_stock} = \left((1.2 \cdot 100.1) \cdot (\text{lead_time_days})^{0.5} \right)$

$\text{pipeline_adjustment} = \left(0.7 \cdot \sum_{i=0}^{L-1} q_i \right)$

$\text{target} = ((195.1 + \text{safety_stock}) - \text{pipeline_adjustment})$

$\text{raw_order} = \max(0, (\text{target} - \text{net_inventory}))$

$\left. \begin{array}{l} \text{recent_order} = q_{-(1)} \\ \text{smoothed_order} = ((0.3 \cdot \text{raw_order}) + ((1 - 0.3) \cdot \text{recent_order})) \end{array} \right\} \quad (L > 0)$

$\text{smoothed_order} = \text{raw_order} \quad (\neg (L > 0))$

$\text{smoothed_order} = \begin{cases} 0, & \text{smoothed_order} < 10.0 \\ \text{smoothed_order}, & \text{otherwise} \end{cases}$

$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-43

```

1. # sample_id: 039
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_e1-m2_6_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 6
6. # objective: 914.48
7. # test_objective: 901.753
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer_rounding;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 645.7134146565843 # OPT_PARAM: {"initial": 645.7134146565843, "min": 400, "max": 1000, "type": "float"}
15.
16.     # Demand forecasting parameters
17.     forecast_horizon = 3 # OPT_PARAM: {"initial": 3, "min": 1, "max": 6, "type": "int"}
18.     forecast_weight = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 1.0, "type": "float"}
19.
20.     # Pipeline risk assessment parameters
21.     risk_threshold = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.1, "max": 0.5, "type": "float"}
22.     risk_multiplier = 1.5 # OPT_PARAM: {"initial": 1.5, "min": 1.0, "max": 3.0, "type": "float"}
23.
24.     # Order smoothing parameters
25.     smooth_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.5, "type": "float"}
26.     min_order = 5 # OPT_PARAM: {"initial": 5, "min": 0, "max": 20, "type": "int"}
27.
28.     L = len(pipeline_orders)
29.
30.     # 1. Demand forecasting from pipeline pattern
31.     # Use pipeline orders as proxy for future demand (since they reflect past ordering decisions)
32.     if L >= forecast_horizon:
33.         # Extract recent pipeline orders for forecasting
34.         recent_pipeline = pipeline_orders[-forecast_horizon:] if forecast_horizon > 0 else []
35.         if recent_pipeline:
36.             # Weighted average with more weight on recent periods
37.             weights = [0.5 ** (forecast_horizon - i - 1) for i in range(forecast_horizon)]
38.             weight_sum = sum(weights)
39.             forecast_demand = sum(q * w for q, w in zip(recent_pipeline, weights)) / weight_sum
40.         else:
41.             forecast_demand = 0
42.     else:
43.         forecast_demand = sum(pipeline_orders) / L if L > 0 else 0

```

Declarative mathematical formula rows 1-16

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned} \text{recent_pipeline} &= \begin{cases} q_{-(3):}, & 3 > 0 \\ [], & \text{otherwise} \end{cases} \\ \text{weights} &= [(0.5)^{((3-i)-1)} \mid i \in \text{range}(3)] \\ \text{weight_sum} &= \text{sum}(\text{weights}) \\ \text{forecast_demand} &= \frac{\sum_{(q,w) \in \text{zip}(\text{recent_pipeline}, \text{weights})} (q \cdot w)}{\text{weight_sum}} \end{aligned} \right\} \text{ (recent_pipeline)} \quad (L \geq 3)$$

$$\text{forecast_demand} = 0 \quad (\neg(\text{recent_pipeline}))$$

$$\text{forecast_demand} = \begin{cases} \frac{\sum_{i=0}^{L-1} q_i}{L}, & L > 0 \\ 0, & \text{otherwise} \end{cases} \quad (\neg(L \geq 3))$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ \text{pipeline_std} &= (\text{pipeline_variance})^{0.5} \end{aligned} \right\} (L > 1)$$

$$\text{risk_level} = \begin{cases} \min(1.0, \frac{\text{pipeline_std}}{\text{pipeline_mean}}), & \text{pipeline_mean} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{risk_level} = 0 \quad (\neg(L > 1))$$

$$\text{safety_stock} = (27.9 \cdot 0.5) \quad (\text{risk_level} > 0.5)$$

$$\left. \begin{aligned} \text{safety_reduction} &= \frac{(0.5 - \text{risk_level})}{0.5} \\ \text{safety_stock} &= (27.9 \cdot (1.0 - (0.5 \cdot \text{safety_reduction}))) \end{aligned} \right\} (\neg(\text{risk_level} > 0.5))$$

$$\text{simple_inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{forecast_adjustment} = (\text{forecast_demand} \cdot 0.1)$$

$$\text{adjusted_inventory_position} = (\text{simple_inventory_position} - \text{forecast_adjustment})$$

$$\text{target_position} = (645.7 + \text{safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{adjusted_inventory_position}))$$

$$\left. \begin{aligned} \text{pipeline_avg} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{smoothed_order} &= ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{pipeline_avg})) \end{aligned} \right\} (L > 0)$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L > 0))$$

$$\text{order_amount} = \begin{cases} \max(5, \text{round}_{\text{even}}(\text{smoothed_order})), & \text{smoothed_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 44-87

```

44.
45.     # 2. Pipeline risk assessment
46.     # Calculate pipeline variability as risk indicator
47.     if L > 1:
48.         pipeline_mean = sum(pipeline_orders) / L
49.         pipeline_variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
50.         pipeline_std = pipeline_variance ** 0.5
51.
52.         # Normalized risk measure (0 to 1)
53.         if pipeline_mean > 0:
54.             risk_level = min(1.0, pipeline_std / pipeline_mean)
55.         else:
56.             risk_level = 0
57.     else:
58.         risk_level = 0
59.
60.     # 3. Dynamic safety stock based on pipeline risk
61.     base_safety_stock = 27.91616435187665 # OPT_PARAM: {"initial": 27.91616435187665, "min": 10,
62. "max": 150, "type": "float"}
63.
64.     if risk_level > risk_threshold:
65.         safety_stock = base_safety_stock * 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.1, "max": 0.9,
66. "type": "float"}
67.     else:
68.         # Linear interpolation between base and reduced safety stock
69.         safety_reduction = (risk_threshold - risk_level) / risk_threshold
70.         safety_stock = base_safety_stock * (1.0 - 0.5 * safety_reduction)
71.
72.     # 4. Calculate inventory position with forecast adjustment
73.     simple_inventory_position = on_hand_inventory + sum(pipeline_orders)
74.
75.     # Adjust inventory position based on demand forecast
76.     # If forecast is high, effectively reduce perceived inventory
77.     forecast_adjustment = forecast_demand * forecast_weight
78.     adjusted_inventory_position = simple_inventory_position - forecast_adjustment
79.
80.     # 5. Calculate target with dynamic safety stock
81.     target_position = base_stock + safety_stock
82.
83.     # 6. Calculate raw order amount
84.     raw_order = max(0, target_position - adjusted_inventory_position)
85.
86.     # 7. Apply smoothing to avoid drastic order changes
87.     # Use pipeline average as reference for smoothing
88.     if L > 0:
89.         pipeline_avg = sum(pipeline_orders) / L

```

Declarative mathematical formula rows 1-16

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned} \text{recent_pipeline} &= \begin{cases} q_{-(3)}, & 3 > 0 \\ [], & \text{otherwise} \end{cases} \\ \text{weights} &= [(0.5)^{((3-i)-1)} \mid i \in \text{range}(3)] \\ \text{weight_sum} &= \text{sum}(\text{weights}) \\ \text{forecast_demand} &= \frac{\sum_{(q,w) \in \text{zip}(\text{recent_pipeline}, \text{weights})} (q \cdot w)}{\text{weight_sum}} \end{aligned} \right\} \text{ (recent_pipeline)} \quad (L \geq 3)$$

$$\text{forecast_demand} = \begin{cases} \frac{\sum_{i=0}^{L-1} q_i}{L}, & L > 0 \\ 0, & \text{otherwise} \end{cases} \quad (\neg(L \geq 3))$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ \text{pipeline_std} &= (\text{pipeline_variance})^{0.5} \end{aligned} \right\} (L > 1)$$

$$\text{risk_level} = \begin{cases} \min(1.0, \frac{\text{pipeline_std}}{\text{pipeline_mean}}), & \text{pipeline_mean} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{risk_level} = 0 \quad (\neg(L > 1))$$

$$\text{safety_stock} = (27.9 \cdot 0.5) \quad (\text{risk_level} > 0.5)$$

$$\left. \begin{aligned} \text{safety_reduction} &= \frac{(0.5 - \text{risk_level})}{0.5} \\ \text{safety_stock} &= (27.9 \cdot (1.0 - (0.5 \cdot \text{safety_reduction}))) \end{aligned} \right\} (\neg(\text{risk_level} > 0.5))$$

$$\text{simple_inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{forecast_adjustment} = (\text{forecast_demand} \cdot 0.1)$$

$$\text{adjusted_inventory_position} = (\text{simple_inventory_position} - \text{forecast_adjustment})$$

$$\text{target_position} = (645.7 + \text{safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{adjusted_inventory_position}))$$

$$\left. \begin{aligned} \text{pipeline_avg} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \end{aligned} \right\} (L > 0)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{pipeline_avg}))$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L > 0))$$

$$\text{order_amount} = \begin{cases} \max(5, \text{round}_{\text{even}}(\text{smoothed_order})), & \text{smoothed_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 88-98

```

88.         smoothed_order = smooth_factor * raw_order + (1 - smooth_factor) * pipeline_avg
89.     else:
90.         smoothed_order = raw_order
91.
92.     # 8. Apply minimum order constraint
93.     if smoothed_order > 0:
94.         order_amount = max(min_order, int(round(smoothed_order)))
95.     else:
96.         order_amount = 0
97.
98.     return order_amount

```

Declarative mathematical formula rows 1-16

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned} \text{recent_pipeline} &= \begin{cases} q_{-(3):}, & 3 > 0 \\ [], & \text{otherwise} \end{cases} \\ \text{weights} &= [(0.5)^{((3-i)-1)} \mid i \in \text{range}(3)] \\ \text{weight_sum} &= \text{sum}(\text{weights}) \\ \text{forecast_demand} &= \frac{\sum_{(q,w) \in \text{zip}(\text{recent_pipeline}, \text{weights})} (q \cdot w)}{\text{weight_sum}} \end{aligned} \right\} \text{ (recent_pipeline)} \quad (L \geq 3)$$

$$\text{forecast_demand} = \begin{cases} 0, & \neg(\text{recent_pipeline}) \end{cases}$$

$$\text{forecast_demand} = \begin{cases} \frac{\sum_{i=0}^{L-1} q_i}{L}, & L > 0 \\ 0, & \text{otherwise} \end{cases} \quad (\neg(L \geq 3))$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ \text{pipeline_std} &= (\text{pipeline_variance})^{0.5} \end{aligned} \right\} (L > 1)$$

$$\text{risk_level} = \begin{cases} \min(1.0, \frac{\text{pipeline_std}}{\text{pipeline_mean}}), & \text{pipeline_mean} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{risk_level} = 0 \quad (\neg(L > 1))$$

$$\text{safety_stock} = (27.9 \cdot 0.5) \quad (\text{risk_level} > 0.5)$$

$$\left. \begin{aligned} \text{safety_reduction} &= \frac{(0.5 - \text{risk_level})}{0.5} \\ \text{safety_stock} &= (27.9 \cdot (1.0 - (0.5 \cdot \text{safety_reduction}))) \end{aligned} \right\} (\neg(\text{risk_level} > 0.5))$$

$$\text{simple_inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{forecast_adjustment} = (\text{forecast_demand} \cdot 0.1)$$

$$\text{adjusted_inventory_position} = (\text{simple_inventory_position} - \text{forecast_adjustment})$$

$$\text{target_position} = (645.7 + \text{safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{adjusted_inventory_position}))$$

$$\left. \begin{aligned} \text{pipeline_avg} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \end{aligned} \right\} (L > 0)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{pipeline_avg}))$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L > 0))$$

$$\text{order_amount} = \begin{cases} \max(5, \text{round}_{\text{even}}(\text{smoothed_order})), & \text{smoothed_order} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 1-39

```

1. # sample_id: 040
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_8_r6
3. # distribution: exponential_L6_c1_2
4. # generation: 3
5. # rank_in_population_file: 6
6. # objective: 6145.5
7. # test_objective: 6290.117
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 353.1678105626499 # OPT_PARAM: {"initial": 353.1678105626499, "min": 10, "max": 1000, "type": "float"}
14.     safety_stock = 46.207982248122036 # OPT_PARAM: {"initial": 46.207982248122036, "min": 0, "max": 200, "type": "float"}
15.     demand_forecast = 50.59999999999991 # OPT_PARAM: {"initial": 50.59999999999991, "min": 10, "max": 500, "type": "float"}
16.     smoothing_factor = 0.06586766248144076 # OPT_PARAM: {"initial": 0.06586766248144076, "min": 0.01, "max": 1.0, "type": "float"}
17.     pipeline_weight = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.0, "max": 1.0, "type": "float"}
18.     adjustment_factor = 0.9224417153025465 # OPT_PARAM: {"initial": 0.9224417153025465, "min": 0.0, "max": 1.0, "type": "float"}
19.     demand_adjustment = 0.3000000000004861 # OPT_PARAM: {"initial": 0.3000000000004861, "min": 0.0, "max": 0.5, "type": "float"}
20.
21.     # Calculate inventory position
22.     inventory_position = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Calculate weighted pipeline (emphasize near-term arrivals)
25.     weighted_pipeline = 0
26.     for i, order in enumerate(pipeline_orders):
27.         weight = pipeline_weight ** i
28.         weighted_pipeline += order * weight
29.
30.     # Adjust base stock based on recent demand pattern
31.     recent_arrivals = pipeline_orders[0] if pipeline_orders else 0
32.     if recent_arrivals > 0:
33.         demand_ratio = min(2.0, recent_arrivals / max(1.0, demand_forecast))
34.         adjusted_base = base_stock * (1.0 + demand_adjustment * (demand_ratio - 1.0))
35.     else:
36.         adjusted_base = base_stock
37.
38.     # Apply pipeline composition adjustment
39.     adjusted_base = adjusted_base * adjustment_factor

```

Declarative mathematical formula rows 1-13

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.8)^i))$$

$$\text{recent_arrivals} = \begin{cases} q_0, & \mathbf{q} \\ 0, & \text{otherwise} \end{cases}$$

$$\text{demand_ratio} = \min \left(2.0, \frac{\text{recent_arrivals}}{\max(1.0, 50.6)} \right) \quad \left. \vphantom{\frac{\text{recent_arrivals}}{\max(1.0, 50.6)}} \right\} \quad (\text{recent_arrivals} > 0)$$

$$\text{adjusted_base} = (353.2 \cdot (1.0 + (0.3 \cdot (\text{demand_ratio} - 1.0)))) \quad \left. \vphantom{(353.2 \cdot (1.0 + (0.3 \cdot (\text{demand_ratio} - 1.0))))} \right\} \quad (\text{recent_arrivals} > 0)$$

$$\text{adjusted_base} = 353.2 \quad (\neg (\text{recent_arrivals} > 0))$$

$$\text{adjusted_base} = (\text{adjusted_base} \cdot 0.9)$$

$$\text{target_inventory} = (\text{adjusted_base} + 46.2)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{recent_avg} = \frac{\sum_{i=L-(\min(3,L))}^{L-1} q_i}{\min(3, L)} \quad \left. \vphantom{\frac{\sum_{i=L-(\min(3,L))}^{L-1} q_i}{\min(3, L)}} \right\} \quad (L > 0)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{recent_avg})) \quad \left. \vphantom{((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{recent_avg}))} \right\} \quad (L > 0)$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg (L > 0))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 40-57

```

40.
41.     # Add safety stock buffer
42.     target_inventory = adjusted_base + safety_stock
43.
44.     # Calculate raw order amount
45.     raw_order = max(0, target_inventory - inventory_position)
46.
47.     # Apply smoothing using recent pipeline orders
48.     if len(pipeline_orders) > 0:
49.         recent_avg = sum(pipeline_orders[-min(3, len(pipeline_orders)):]) / min(3,
len(pipeline_orders))
50.         smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * recent_avg
51.     else:
52.         smoothed_order = raw_order
53.
54.     # Round to nearest integer
55.     order_amount = int(round(smoothed_order))
56.
57.     return order_amount

```

Declarative mathematical formula rows 1-13

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.8)^i))$$

$$\text{recent_arrivals} = \begin{cases} q_0, & \mathbf{q} \\ 0, & \text{otherwise} \end{cases}$$

$$\text{demand_ratio} = \min \left(2.0, \frac{\text{recent_arrivals}}{\max(1.0, 50.6)} \right) \left. \vphantom{\frac{\text{recent_arrivals}}{\max(1.0, 50.6)}} \right\} \quad (\text{recent_arrivals} > 0)$$

$$\text{adjusted_base} = (353.2 \cdot (1.0 + (0.3 \cdot (\text{demand_ratio} - 1.0))))$$

$$\text{adjusted_base} = 353.2 \quad (\neg (\text{recent_arrivals} > 0))$$

$$\text{adjusted_base} = (\text{adjusted_base} \cdot 0.9)$$

$$\text{target_inventory} = (\text{adjusted_base} + 46.2)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{recent_avg} = \frac{\sum_{i=L-\min(3,L)}^{L-1} q_i}{\min(3, L)} \left. \vphantom{\frac{\sum_{i=L-\min(3,L)}^{L-1} q_i}{\min(3, L)}} \right\} \quad (L > 0)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{recent_avg}))$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg (L > 0))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```
1. # sample_id: 041
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r5
3. # distribution: poisson_L6_c1_2
4. # generation: 3
5. # rank_in_population_file: 6
6. # objective: 1642.5
7. # test_objective: 1638.292
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 663.9895492381544 # OPT_PARAM: {"initial": 663.9895492381544, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 87.16738388927891 # OPT_PARAM: {"initial": 87.16738388927891, "min": 0, "max":
16.     200, "type": "float"}
17.     demand_forecast = 100.21320601001209 # OPT_PARAM: {"initial": 100.21320601001209, "min": 50,
18.     "max": 150, "type": "float"}
19.     smoothing_factor = 0.1134087362976288 # OPT_PARAM: {"initial": 0.1134087362976288, "min": 0.1,
20.     "max": 0.9, "type": "float"}
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected demand during lead time
26.     expected_lead_time_demand = demand_forecast * len(pipeline_orders)
27.
28.     # Calculate target inventory level
29.     target_inventory = expected_lead_time_demand + safety_stock
30.
31.     # Calculate order quantity
32.     order_amount = max(0, target_inventory - inventory_position)
33.
34.     # Apply smoothing to avoid large order fluctuations
35.     if order_amount > 0:
36.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * demand_forecast
37.
38.     # Round to nearest integer (as required by output type)
39.     order_amount = int(round(order_amount))
40.
41.     return order_amount
```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (100.2 \cdot L)$$

$$\text{target_inventory} = (\text{expected_lead_time_demand} + 87.2)$$

$$\text{order_amount} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{order_amount} = \begin{cases} ((0.1 \cdot \text{order_amount}) + ((1 - 0.1) \cdot 100.2)), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-43

```

1. # sample_id: 042
2. # folder: deepseek-chat_normal_std30_L2_c1_2_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: normal_std30_L2_c1_2
4. # generation: 1
5. # rank_in_population_file: 4
6. # objective: 2318.88
7. # test_objective: 2265.944
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 303.7139198304675 # OPT_PARAM: {"initial": 303.7139198304675, "min": 100, "max":
    500, "type": "float"}
15.
16.     # Safety stock multiplier
17.     safety_multiplier = 1.2 # OPT_PARAM: {"initial": 1.2, "min": 0.5, "max": 2.5, "type": "float"}
18.
19.     # Order smoothing factor (0-1, higher = more smoothing)
20.     smoothing_factor = 0.5672492181352733 # OPT_PARAM: {"initial": 0.5672492181352733, "min": 0.1,
    "max": 1.0, "type": "float"}
21.
22.     # Pipeline imbalance threshold
23.     imbalance_threshold = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.1, "max": 0.5, "type": "float"}
24.
25.     # Pipeline weight for inventory position calculation
26.     pipeline_weight = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.0, "type": "float"}
27.
28.     # Calculate weighted inventory position
29.     weighted_pipeline = pipeline_weight * sum(pipeline_orders)
30.     inventory_position = on_hand_inventory + weighted_pipeline
31.
32.     # Calculate average pipeline order (excluding current arrival)
33.     if len(pipeline_orders) > 1:
34.         future_pipeline = pipeline_orders[1:] # Exclude q_{t,1} which arrives now
35.         avg_future = sum(future_pipeline) / len(future_pipeline)
36.     else:
37.         avg_future = 0
38.
39.     # Detect pipeline imbalance
40.     pipeline_imbalance = False
41.     if len(pipeline_orders) >= 2 and avg_future > 0:
42.         immediate_coverage = sum(pipeline_orders[:min(2, len(pipeline_orders))])
43.         if immediate_coverage < avg_future * (1 - imbalance_threshold):

```

Declarative mathematical formula rows 1-14

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \left(0.5 \cdot \sum_{i=0}^{L-1} q_i \right)$$

$\text{inventory_position} = (h + \text{weighted_pipeline})$

$$\left. \begin{array}{l} \text{future_pipeline} = q_1: \\ \text{avg_future} = \frac{\text{sum}(\text{future_pipeline})}{\text{len}(\text{future_pipeline})} \end{array} \right\} (L > 1)$$

$\text{avg_future} = 0 \} \quad (\neg (L > 1))$

$\text{pipeline_imbalance} = \text{False}$

$$\left. \begin{array}{l} \text{immediate_coverage} = \sum_{i=0}^{(\min(2,L))-1} q_i \\ \text{pipeline_imbalance} = \begin{cases} \text{True}, & \text{immediate_coverage} < (\text{avg_future} \cdot (1 - 0.3)) \\ \text{pipeline_imbalance}, & \text{otherwise} \end{cases} \end{array} \right\} ((L \geq 2 \wedge \text{avg_future} > 0))$$

$$\text{pipeline_variance} = \begin{cases} (\max(\mathbf{q}) - \min(\mathbf{q})), & L > 1 \\ 0, & \text{otherwise} \end{cases} \quad (L > 0)$$

$\text{safety_stock} = (1.2 \cdot (\text{avg_future} + (\text{pipeline_variance} \cdot 0.1)))$

$\text{safety_stock} = ((1.2 \cdot 303.7) \cdot 0.1) \} \quad (\neg (L > 0))$

$\text{target_position} = \begin{cases} (303.7 + (\text{safety_stock} \cdot 1.5)), & \text{pipeline_imbalance} \\ (303.7 + \text{safety_stock}), & \text{otherwise} \end{cases}$

$\text{raw_order_needed} = (\text{target_position} - \text{inventory_position})$

$\text{order_amount} = \begin{cases} (\text{raw_order_needed} \cdot 0.6), & \text{raw_order_needed} > 0 \\ 0, & \text{otherwise} \end{cases}$

$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 44-71

```

44.         pipeline_imbalance = True
45.
46.     # Calculate safety stock
47.     if len(pipeline_orders) > 0:
48.         pipeline_variance = max(pipeline_orders) - min(pipeline_orders) if len(pipeline_orders) > 1
49.     else 0
50.         safety_stock = safety_multiplier * (avg_future + pipeline_variance * 0.1)
51.     else:
52.         safety_stock = safety_multiplier * base_stock * 0.1
53.
54.     # Determine target inventory position
55.     if pipeline_imbalance:
56.         target_position = base_stock + safety_stock * 1.5
57.     else:
58.         target_position = base_stock + safety_stock
59.
60.     # Calculate raw order needed
61.     raw_order_needed = target_position - inventory_position
62.
63.     # Apply smoothing: order a smoothed version of what's needed
64.     if raw_order_needed > 0:
65.         order_amount = raw_order_needed * smoothing_factor
66.     else:
67.         order_amount = 0
68.
69.     # Round to nearest integer
70.     order_amount = int(round(order_amount))
71.
72.     return order_amount

```

Declarative mathematical formula rows 1-14

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \left(0.5 \cdot \sum_{i=0}^{L-1} q_i \right)$$

$$\text{inventory_position} = (h + \text{weighted_pipeline})$$

$$\left. \begin{array}{l} \text{future_pipeline} = q_1: \\ \text{avg_future} = \frac{\text{sum}(\text{future_pipeline})}{\text{len}(\text{future_pipeline})} \end{array} \right\} (L > 1)$$

$$\text{avg_future} = 0 \} (\neg (L > 1))$$

$$\text{pipeline_imbalance} = \text{False}$$

$$\left. \begin{array}{l} \text{immediate_coverage} = \sum_{i=0}^{(\min(2,L))-1} q_i \\ \text{pipeline_imbalance} = \begin{cases} \text{True}, & \text{immediate_coverage} < (\text{avg_future} \cdot (1 - 0.3)) \\ \text{pipeline_imbalance}, & \text{otherwise} \end{cases} \end{array} \right\} ((L \geq 2 \wedge \text{avg_future} > 0))$$

$$\text{pipeline_variance} = \begin{cases} (\max(\mathbf{q}) - \min(\mathbf{q})), & L > 1 \\ 0, & \text{otherwise} \end{cases} (L > 0)$$

$$\text{safety_stock} = (1.2 \cdot (\text{avg_future} + (\text{pipeline_variance} \cdot 0.1)))$$

$$\text{safety_stock} = ((1.2 \cdot 303.7) \cdot 0.1) \} (\neg (L > 0))$$

$$\text{target_position} = \begin{cases} (303.7 + (\text{safety_stock} \cdot 1.5)), & \text{pipeline_imbalance} \\ (303.7 + \text{safety_stock}), & \text{otherwise} \end{cases}$$

$$\text{raw_order_needed} = (\text{target_position} - \text{inventory_position})$$

$$\text{order_amount} = \begin{cases} (\text{raw_order_needed} \cdot 0.6), & \text{raw_order_needed} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-38

```

1. # sample_id: 043
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_2_r7
3. # distribution: poisson_L6_c1_5
4. # generation: 5
5. # rank_in_population_file: 1
6. # objective: 3074.22
7. # test_objective: 3077.277
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 650.0 # OPT_PARAM: {"initial": 650.0, "min": 400, "max": 900, "type": "float"}
14.     safety_stock = 155.24240621033815 # OPT_PARAM: {"initial": 155.24240621033815, "min": 100,
15.     "max": 250, "type": "float"}
16.     demand_forecast = 105.69007241806933 # OPT_PARAM: {"initial": 105.69007241806933, "min": 80,
17.     "max": 120, "type": "float"}
18.     smoothing_factor = 0.4915613763933389 # OPT_PARAM: {"initial": 0.4915613763933389, "min": 0.0,
19.     "max": 1.0, "type": "float"}
20.     lead_time = len(pipeline_orders)
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected demand during lead time plus one period
26.     expected_lead_time_demand = demand_forecast * (lead_time + 1)
27.
28.     # Calculate target inventory level
29.     target_inventory = expected_lead_time_demand + safety_stock
30.
31.     # Use the maximum of base_stock and target_inventory as order-up-to level
32.     order_up_to = max(base_stock, target_inventory)
33.
34.     # Calculate order amount
35.     order_amount = max(0, order_up_to - inventory_position)
36.
37.     # Apply smoothing only when order amount is positive
38.     if order_amount > 0:
39.         order_amount = int(order_amount * smoothing_factor + 0.5)
40.
41.     return order_amount

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

expected_lead_time_demand = $(105.7 \cdot (\text{lead_time} + 1))$

target_inventory = $(\text{expected_lead_time_demand} + 155.2)$

order_up_to = $\max(650.0, \text{target_inventory})$

order_amount = $\max(0, (\text{order_up_to} - \text{inventory_position}))$

order_amount = $\begin{cases} \text{int}(((\text{order_amount} \cdot 0.5) + 0.5)), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-15

```
1. # sample_id: 044
2. # folder: deepseek-chat_normal_std30_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r7
3. # distribution: normal_std30_L6_c1_5
4. # generation: 0
5. # rank_in_population_file: 1
6. # objective: 6033.8008
7. # test_objective: 5897.01244
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs:
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 692.0049957614882 # OPT_PARAM: {"initial": 692.0049957614882, "min": 10, "max":
14.     1000, "type": "float"}
15.     order_amount = max(0, base_stock - on_hand_inventory - sum(pipeline_orders))
16.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left((692.0 - h) - \sum_{i=0}^{L-1} q_i \right) \right).$$

Python source lines 1-25

```
1. # sample_id: 045
2. # folder: grok-4.1-fast_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_2_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 1
5. # rank_in_population_file: 6
6. # objective: 2781.10132
7. # test_objective: 2783.54765
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     L = 6
14.     lookback_k = 3 # OPT_PARAM: {"initial": 3, "min": 1, "max": 6, "type": "int"}
15.     prior_mu = 102.48823456990216 # OPT_PARAM: {"initial": 102.48823456990216, "min": 80, "max":
120, "type": "float"}
16.     prior_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.0, "max": 1.0, "type": "float"}
17.     safety_stock = 49.06507394056868 # OPT_PARAM: {"initial": 49.06507394056868, "min": 0.0, "max":
100.0, "type": "float"}
18.     recent_orders = pipeline_orders[-lookback_k:]
19.     recent_avg = sum(recent_orders) / lookback_k
20.     blended_mu = prior_weight * prior_mu + (1 - prior_weight) * recent_avg
21.     lt_forecast = blended_mu * L
22.     base_stock = lt_forecast + safety_stock
23.     total_ip = on_hand_inventory + sum(pipeline_orders)
24.     order_amount = max(0, base_stock - total_ip)
25.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(\left(\left(\left((1.0 \cdot 102.5) + \left((1 - 1.0) \cdot \frac{\text{sum}(q_{-(3):})}{3} \right) \right) \cdot 6 \right) + 49.1 \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right).$$

Python source lines 1-41

```

1. # sample_id: 046
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-m2_6_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 6
5. # rank_in_population_file: 5
6. # objective: 2345.88
7. # test_objective: 2339.654
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding;emergency_or_shor
    tage_boost
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Core inventory parameters
14.     base_stock = 536.8920410178813 # OPT_PARAM: {"initial": 536.8920410178813, "min": 300, "max":
    800, "type": "float"}
15.     safety_stock = 20.756009431915693 # OPT_PARAM: {"initial": 20.756009431915693, "min": 20, "max":
    150, "type": "float"}
16.
17.     # Cost-aware adjustment parameters
18.     holding_weight = 0.8160040850045172 # OPT_PARAM: {"initial": 0.8160040850045172, "min": 0.1,
    "max": 2.0, "type": "float"}
19.     shortage_weight = 2.134635819833547 # OPT_PARAM: {"initial": 2.134635819833547, "min": 0.1,
    "max": 3.0, "type": "float"}
20.
21.     # Calculate total pipeline inventory (unweighted sum)
22.     total_pipeline = sum(pipeline_orders)
23.
24.     # Calculate current inventory position
25.     current_position = on_hand_inventory + total_pipeline
26.
27.     # Calculate target inventory position
28.     target_position = base_stock + safety_stock
29.
30.     # Calculate imbalance
31.     imbalance = target_position - current_position
32.
33.     # Apply cost-aware adjustment based on imbalance direction
34.     if imbalance > 0:
35.         # Risk of shortage - apply shortage weight
36.         adjusted_order = imbalance * shortage_weight
37.     else:
38.         # Risk of excess inventory - apply holding weight
39.         adjusted_order = imbalance * holding_weight
40.
41.     # Simple smoothing based on recent demand pattern

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\text{current_position} = (h + \text{total_pipeline})$$

$$\text{target_position} = (536.9 + 20.8)$$

$$\text{imbalance} = (\text{target_position} - \text{current_position})$$

$$\text{adjusted_order} = \begin{cases} (\text{imbalance} \cdot 2.1), & \text{imbalance} > 0 \\ (\text{imbalance} \cdot 0.8), & \text{otherwise} \end{cases}$$

$$\text{recent_avg} = \frac{(q_0 + q_1)}{2.0} \quad \left. \vphantom{\frac{(q_0 + q_1)}{2.0}} \right\} (L \geq 2)$$

$$\text{smoothed_order} = ((\text{adjusted_order} \cdot 0.1) + (\text{recent_avg} \cdot (1 - 0.1)))$$

$$\text{smoothed_order} = \text{adjusted_order} \quad (\neg (L \geq 2))$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 42-52

```

42.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 1.0, "type": "float"}
43.     if len(pipeline_orders) >= 2:
44.         recent_avg = (pipeline_orders[0] + pipeline_orders[1]) / 2.0
45.         smoothed_order = adjusted_order * smoothing_factor + recent_avg * (1 - smoothing_factor)
46.     else:
47.         smoothed_order = adjusted_order
48.
49.     # Ensure non-negative and round to integer
50.     order_amount = max(0, int(round(smoothed_order)))
51.
52.     return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\text{current_position} = (h + \text{total_pipeline})$$

$$\text{target_position} = (536.9 + 20.8)$$

$$\text{imbalance} = (\text{target_position} - \text{current_position})$$

$$\text{adjusted_order} = \begin{cases} (\text{imbalance} \cdot 2.1), & \text{imbalance} > 0 \\ (\text{imbalance} \cdot 0.8), & \text{otherwise} \end{cases}$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{(q_0 + q_1)}{2.0} \\ \text{smoothed_order} &= ((\text{adjusted_order} \cdot 0.1) + (\text{recent_avg} \cdot (1 - 0.1))) \end{aligned} \right\} \quad (L \geq 2)$$

$$\text{smoothed_order} = \text{adjusted_order} \quad (\neg (L \geq 2))$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-41

```

1. # sample_id: 047
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 6
5. # rank_in_population_file: 5
6. # objective: 6041.04
7. # test_objective: 6154.832
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 367.35214483887654 # OPT_PARAM: {"initial": 367.35214483887654, "min": 300, "max":
14.     450, "type": "float"}
15.     safety_stock = 45.60091931543199 # OPT_PARAM: {"initial": 45.60091931543199, "min": 40, "max":
16.     120, "type": "float"}
17.     pipeline_weight = 0.9944093066156077 # OPT_PARAM: {"initial": 0.9944093066156077, "min": 0.5,
18.     "max": 1.0, "type": "float"}
19.     order_smoothing = 0.11124913391084858 # OPT_PARAM: {"initial": 0.11124913391084858, "min": 0.05,
20.     "max": 0.3, "type": "float"}
21.     demand_forecast_factor = 7.806710259511079e-14 # OPT_PARAM: {"initial": 7.806710259511079e-14,
22.     "min": 0.0, "max": 0.3, "type": "float"}
23.
24.     # Calculate inventory position
25.     inventory_position = on_hand_inventory + sum(pipeline_orders)
26.
27.     # Calculate weighted average of recent pipeline orders as demand forecast
28.     if pipeline_orders:
29.         weights = [pipeline_weight ** i for i in range(len(pipeline_orders))]
30.         weights.reverse()
31.         weighted_sum = sum(w * q for w, q in zip(weights, pipeline_orders))
32.         weight_sum = sum(weights)
33.         forecast_demand = weighted_sum / weight_sum if weight_sum > 0 else 0
34.     else:
35.         forecast_demand = 0
36.
37.     # Adjust base stock level based on demand forecast
38.     adjusted_base = base_stock + safety_stock
39.     if forecast_demand > 0:
40.         # Adjust based on forecast deviation from average expected demand
41.         avg_expected_demand = base_stock / 6

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 \text{inventory_position} &= \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 \left. \begin{aligned}
 \text{weights} &= [(1.0)^i \mid i \in \text{range}(L)] \\
 \text{weights.reverse}() \\
 \text{weighted_sum} &= \sum_{(w,q) \in \text{zip}(\text{weights}, \mathbf{q})} (w \cdot q) \\
 \text{weight_sum} &= \text{sum}(\text{weights})
 \end{aligned} \right\} (\mathbf{q}) \\
 \text{forecast_demand} &= \begin{cases} \frac{\text{weighted_sum}}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 \text{forecast_demand} &= 0 \quad (\neg(\mathbf{q})) \\
 \text{adjusted_base} &= (367.4 + 45.6)
 \end{aligned}$$

$$\begin{aligned}
 \text{avg_expected_demand} &= \frac{367.4}{6} \\
 \left. \begin{aligned}
 \text{demand_adjustment} &= ((\text{forecast_demand} - \text{avg_expected_demand}) \cdot 7.8e - 14) \\
 \text{demand_adjustment} &= \max(-80, \min(80, \text{demand_adjustment})) \\
 \text{adjusted_base} &= (\text{adjusted_base} + \text{demand_adjustment})
 \end{aligned} \right\} (\text{forecast_demand} > 0) \\
 \text{raw_order} &= \max(0, (\text{adjusted_base} - \text{inventory_position})) \\
 \text{smoothed_order} &= ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{forecast_demand})) \\
 \text{order_amount} &= \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))
 \end{aligned}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 42-51

```

42. # Calculate raw order amount
43. raw_order = max(0, adjusted_base - inventory_position)
44.
45. # Apply smoothing with forecast demand as anchor
46. smoothed_order = order_smoothing * raw_order + (1 - order_smoothing) * forecast_demand
47.
48. # Ensure non-negative and round to integer
49. order_amount = int(round(max(0, smoothed_order)))
50.
51. return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 \text{inventory_position} &= \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 \left. \begin{aligned}
 \text{weights} &= [(1.0)^i \mid i \in \text{range}(L)] \\
 \text{weights.reverse}() \\
 \text{weighted_sum} &= \sum_{(w,q) \in \text{zip}(\text{weights}, \mathbf{q})} (w \cdot q) \\
 \text{weight_sum} &= \text{sum}(\text{weights})
 \end{aligned} \right\} (\mathbf{q}) \\
 \text{forecast_demand} &= \begin{cases} \frac{\text{weighted_sum}}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 0, & \text{otherwise} \end{cases} \\
 \text{forecast_demand} &= 0 \quad (\neg(\mathbf{q})) \\
 \text{adjusted_base} &= (367.4 + 45.6)
 \end{aligned}$$

$$\begin{aligned}
 \text{avg_expected_demand} &= \frac{367.4}{6} \\
 \left. \begin{aligned}
 \text{demand_adjustment} &= ((\text{forecast_demand} - \text{avg_expected_demand}) \cdot 7.8e - 14) \\
 \text{demand_adjustment} &= \max(-80, \min(80, \text{demand_adjustment})) \\
 \text{adjusted_base} &= (\text{adjusted_base} + \text{demand_adjustment})
 \end{aligned} \right\} (\text{forecast_demand} > 0) \\
 \text{raw_order} &= \max(0, (\text{adjusted_base} - \text{inventory_position})) \\
 \text{smoothed_order} &= ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{forecast_demand})) \\
 \text{order_amount} &= \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))
 \end{aligned}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-42

```

1. # sample_id: 048
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: poisson_L4_c1_5
4. # generation: 2
5. # rank_in_population_file: 5
6. # objective: 2183.7834
7. # test_objective: 2221.19349
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_smoothing
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 522.6543634227193 # OPT_PARAM: {"initial": 522.6543634227193, "min": 300, "max":
14.     700, "type": "float"}
    safety_stock = 142.99766910765004 # OPT_PARAM: {"initial": 142.99766910765004, "min": 20, "max":
15.     150, "type": "float"}
    demand_forecast = 120.0 # OPT_PARAM: {"initial": 120.0, "min": 80, "max": 120, "type": "float"}
16.     smoothing_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.2, "max": 0.8, "type": "float"}
17.     pipeline_weight = 0.7 # OPT_PARAM: {"initial": 0.7, "min": 0.3, "max": 1.0, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand during lead time
23.     expected_lead_time_demand = demand_forecast * len(pipeline_orders)
24.
25.     # Calculate target inventory level with adjusted safety stock
26.     target_inventory = expected_lead_time_demand + safety_stock
27.
28.     # Use minimum of base_stock and target_inventory to avoid over-ordering
29.     order_up_to = min(base_stock, target_inventory)
30.
31.     # Calculate order amount
32.     order_amount = max(0, order_up_to - inventory_position)
33.
34.     # Apply smoothing with pipeline consideration
35.     if pipeline_orders:
36.         # Weight recent pipeline orders more heavily
37.         weighted_pipeline = pipeline_weight * pipeline_orders[-1] + (1 - pipeline_weight) *
pipeline_orders[0]
38.         smoothed_order = smoothing_factor * order_amount + (1 - smoothing_factor) * weighted_pipeline
39.     else:
40.         smoothed_order = order_amount
41.
42.     # Round to nearest integer for practical ordering

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (120.0 \cdot L)$$

$$\text{target_inventory} = (\text{expected_lead_time_demand} + 143.0)$$

$$\text{order_up_to} = \min(522.7, \text{target_inventory})$$

$$\text{order_amount} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{weighted_pipeline} &= ((0.7 \cdot q_{-(1)}) + ((1 - 0.7) \cdot q_0)) \\ \text{smoothed_order} &= ((0.5 \cdot \text{order_amount}) + ((1 - 0.5) \cdot \text{weighted_pipeline})) \end{aligned} \right\} \quad (\mathbf{q})$$

$$\text{smoothed_order} = \text{order_amount} \quad (\neg(\mathbf{q}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 43-43

```
43.     return order_amount
```

Declarative mathematical formula rows 1-9

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$
$$\text{expected_lead_time_demand} = (120.0 \cdot L)$$
$$\text{target_inventory} = (\text{expected_lead_time_demand} + 143.0)$$
$$\text{order_up_to} = \min(522.7, \text{target_inventory})$$
$$\text{order_amount} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$
$$\text{weighted_pipeline} = ((0.7 \cdot q_{-(1)}) + ((1 - 0.7) \cdot q_0))$$
$$\text{smoothed_order} = ((0.5 \cdot \text{order_amount}) + ((1 - 0.5) \cdot \text{weighted_pipeline})) \quad (\mathbf{q})$$
$$\text{smoothed_order} = \text{order_amount} \quad (\neg(\mathbf{q}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```

1. # sample_id: 049
2. # folder: deepseek-chat_poisson_L4_c1_2_50_plain_processed_scipy_15_default_m2_10_r2
3. # distribution: poisson_L4_c1_2
4. # generation: 1
5. # rank_in_population_file: 9
6. # objective: 1829.22
7. # test_objective: 1842.556
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 415.75040145040623 # OPT_PARAM: {"initial": 415.75040145040623, "min": 300, "max":
600, "type": "float"}
14.     safety_stock = 45.75040145039649 # OPT_PARAM: {"initial": 45.75040145039649, "min": 20, "max":
150, "type": "float"}
15.     demand_forecast_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.2, "type":
"float"}
16.
17.     # Calculate inventory position
18.     inventory_position = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Estimate upcoming demand based on recent pipeline arrivals
21.     # Use average of recent pipeline orders as demand forecast
22.     if len(pipeline_orders) > 0:
23.         recent_orders = pipeline_orders[-min(3, len(pipeline_orders)):] # Last 3 orders
24.         forecast_demand = sum(recent_orders) / len(recent_orders) * demand_forecast_factor
25.     else:
26.         forecast_demand = 100.0 # Default forecast
27.
28.     # Adjust base stock level based on forecast
29.     adjusted_base_stock = base_stock + safety_stock + forecast_demand * 0.5
30.
31.     # Calculate order amount
32.     order_amount = max(0, adjusted_base_stock - inventory_position)
33.
34.     # Round to nearest integer (since order amounts should be integers)
35.     order_amount = int(round(order_amount))
36.
37.     return order_amount

```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{array}{l} \text{recent_orders} = q_{-(\min(3, L))} \\ \text{forecast_demand} = \left(\frac{\text{sum}(\text{recent_orders})}{\text{len}(\text{recent_orders})} \cdot 0.5 \right) \end{array} \right\} \quad (L > 0)$$

$$\text{forecast_demand} = 100.0 \quad (\neg (L > 0))$$

$$\text{adjusted_base_stock} = ((415.8 + 45.8) + (\text{forecast_demand} \cdot 0.5))$$

$$\text{order_amount} = \max(0, (\text{adjusted_base_stock} - \text{inventory_position}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```

1. # sample_id: 050
2. # folder: deepseek-chat_poisson_L2_c1_5_50_plain_processed_scipy_15_default_m2_2_r6
3. # distribution: poisson_L2_c1_5
4. # generation: 8
5. # rank_in_population_file: 3
6. # objective: 1220.58
7. # test_objective: 1215.692
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment;order_smoothing
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 291.40237332064817 # OPT_PARAM: {"initial": 291.40237332064817, "min": 10, "max":
1000, "type": "float"}
14.     safety_stock = 32.420891839163716 # OPT_PARAM: {"initial": 32.420891839163716, "min": 0, "max":
200, "type": "float"}
15.     pipeline_weight = 1.1817895463385606 # OPT_PARAM: {"initial": 1.1817895463385606, "min": 0.1,
"max": 1.5, "type": "float"}
16.
17.     # Calculate effective inventory position with weighted pipeline
18.     effective_pipeline = pipeline_weight * sum(pipeline_orders)
19.     inventory_position = on_hand_inventory + effective_pipeline
20.
21.     # Adjust base stock based on safety stock
22.     adjusted_base_stock = base_stock + safety_stock
23.
24.     # Calculate order amount
25.     order_amount = max(0, adjusted_base_stock - inventory_position)
26.
27.     # Apply smoothing to avoid extreme order variations
28.     smoothing_factor = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.3, "max": 1.0, "type": "float"}
29.     if len(pipeline_orders) > 0:
30.         recent_order = pipeline_orders[-1]
31.         smoothed_order = smoothing_factor * order_amount + (1 - smoothing_factor) * recent_order
32.         order_amount = max(0, smoothed_order)
33.
34.     # Round to nearest integer since order amounts should be integers
35.     order_amount = int(round(order_amount))
36.
37.     return order_amount

```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{effective_pipeline} = \left(1.2 \cdot \sum_{i=0}^{L-1} q_i \right)$$

$$\text{inventory_position} = (h + \text{effective_pipeline})$$

$$\text{adjusted_base_stock} = (291.4 + 32.4)$$

$$\text{order_amount} = \max(0, (\text{adjusted_base_stock} - \text{inventory_position}))$$

$$\left. \begin{array}{l} \text{recent_order} = q_{-(1)} \\ \text{smoothed_order} = ((0.3 \cdot \text{order_amount}) + ((1 - 0.3) \cdot \text{recent_order})) \end{array} \right\} (L > 0)$$

$$\text{order_amount} = \max(0, \text{smoothed_order})$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-42

```

1. # sample_id: 051
2. # folder: deepseek-chat_normal_std50_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r6
3. # distribution: normal_std50_L6_c1_5
4. # generation: 2
5. # rank_in_population_file: 9
6. # objective: 7995.64
7. # test_objective: 7874.44
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 548.5150245677041 # OPT_PARAM: {"initial": 548.5150245677041, "min": 100, "max":
    800, "type": "float"}
14.     safety_stock = 119.0735286707627 # OPT_PARAM: {"initial": 119.0735286707627, "min": 50, "max":
    200, "type": "float"}
15.     demand_forecast_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.2, "type":
    "float"}
16.     pipeline_weight = 0.01 # OPT_PARAM: {"initial": 0.01, "min": 0.01, "max": 0.2, "type": "float"}
17.     holding_weight = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.5, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate base target
23.     target_inventory = base_stock + safety_stock
24.
25.     # Adjust for pipeline: reduce target when large incoming orders
26.     pipeline_adjustment = pipeline_weight * pipeline_orders[0] if pipeline_orders else 0
27.     adjusted_target = target_inventory - pipeline_adjustment
28.
29.     # Calculate base order
30.     order_amount = max(0, adjusted_target - inventory_position)
31.
32.     # Apply demand forecast adjustment
33.     order_amount = order_amount * demand_forecast_factor
34.
35.     # Reduce order when on-hand inventory is high (holding cost consideration)
36.     if on_hand_inventory > 0:
37.         order_amount = order_amount * (1 - holding_weight * min(1.0, on_hand_inventory / base_stock))
38.
39.     # Round to nearest integer
40.     order_amount = int(round(order_amount))
41.
42.     return order_amount

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{target_inventory} = (548.5 + 119.1)$$

$$\text{pipeline_adjustment} = \begin{cases} (1.0e - 02 \cdot q_0), & \mathbf{q} \\ 0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_target} = (\text{target_inventory} - \text{pipeline_adjustment})$$

$$\text{order_amount} = \max(0, (\text{adjusted_target} - \text{inventory_position}))$$

$$\text{order_amount} = (\text{order_amount} \cdot 0.5)$$

$$\text{order_amount} = \begin{cases} (\text{order_amount} \cdot (1 - (0.1 \cdot \min(1.0, \frac{h}{548.5}))))), & h > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-39

```
1. # sample_id: 052
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r7
3. # distribution: poisson_L6_c1_5
4. # generation: 3
5. # rank_in_population_file: 2
6. # objective: 2474.85996
7. # test_objective: 2323.33752
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 647.9228012866358 # OPT_PARAM: {"initial": 647.9228012866358, "min": 10, "max":
    1000, "type": "float"}
14.     safety_stock = 98.02280128663519 # OPT_PARAM: {"initial": 98.02280128663519, "min": 0, "max":
    300, "type": "float"}
15.     smoothing_factor = 0.4406616715947892 # OPT_PARAM: {"initial": 0.4406616715947892, "min": 0.1,
    "max": 1.0, "type": "float"}
16.     demand_forecast_factor = 0.7696949899212768 # OPT_PARAM: {"initial": 0.7696949899212768, "min":
    0.5, "max": 1.2, "type": "float"}
17.     pipeline_weight = 0.4176510997345094 # OPT_PARAM: {"initial": 0.4176510997345094, "min": 0.0,
    "max": 0.8, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand based on recent pipeline arrivals
23.     # Use average of recent pipeline arrivals as demand proxy
24.     recent_arrivals = pipeline_orders[:3] if len(pipeline_orders) >= 3 else pipeline_orders
25.     expected_demand = sum(recent_arrivals) / max(len(recent_arrivals), 1) * demand_forecast_factor
26.
27.     # Adjust target based on expected demand and pipeline composition
28.     pipeline_variability = max(pipeline_orders) - min(pipeline_orders) if pipeline_orders else 0
29.     pipeline_adjustment = pipeline_variability * pipeline_weight
30.
31.     # Dynamic target that responds to demand patterns
32.     dynamic_target = base_stock + safety_stock + expected_demand - pipeline_adjustment
33.
34.     # Calculate order with smoothing
35.     raw_order = dynamic_target - inventory_position
36.     order_amount = max(0, smoothing_factor * raw_order)
37.
38.     # Round to nearest integer
39.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, q) = \max \left(0, 0.4 \cdot \left(\left((647.9 + 98.0) + \frac{\sum \left(\begin{matrix} q_i, & L \geq 3 \\ & \text{otherwise} \end{matrix} \right)}{\max \left(\begin{matrix} \left(\begin{matrix} q_i, & L \geq 3 \\ & \text{otherwise} \end{matrix} \right), 1 \end{matrix} \right)} \cdot 0.8 \right) - \left(\begin{matrix} (\max(q) - \min(q)), & q \\ 0, & \text{otherwise} \end{matrix} \cdot 0.4 \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right)$$

Python source lines 1-42

```

1. # sample_id: 053
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_10_r3
3. # distribution: poisson_L6_c1_5
4. # generation: 4
5. # rank_in_population_file: 10
6. # objective: 1759.60213
7. # test_objective: 1580.4407
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 534.6011611483775 # OPT_PARAM: {"initial": 534.6011611483775, "min": 400, "max":
700, "type": "float"}
14.     safety_stock = 44.65053849986418 # OPT_PARAM: {"initial": 44.65053849986418, "min": 20, "max":
80, "type": "float"}
15.     demand_forecast = 102.24532485660083 # OPT_PARAM: {"initial": 102.24532485660083, "min": 80,
"max": 120, "type": "float"}
16.     pipeline_weight = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.8, "max": 1.1, "type": "float"}
17.     order_smoothing_min = 0.9 # OPT_PARAM: {"initial": 0.9, "min": 0.5, "max": 0.9, "type": "float"}
18.     order_smoothing_max = 1.1 # OPT_PARAM: {"initial": 1.1, "min": 1.1, "max": 1.5, "type": "float"}
19.     lost_sales_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.5, "max": 1.0, "type": "float"}
20.
21.     # Calculate effective inventory position with full pipeline weight
22.     effective_pipeline = sum(pipeline_orders) * pipeline_weight
23.     inventory_position = on_hand_inventory + effective_pipeline
24.
25.     # Adjust base stock based on lost sales cost ratio
26.     adjusted_base_stock = base_stock * lost_sales_weight
27.
28.     # Calculate order-up-to level
29.     order_up_to = adjusted_base_stock + safety_stock
30.
31.     # Calculate raw order amount
32.     raw_order = max(0, order_up_to - inventory_position)
33.
34.     # Apply smoother ordering bounds based on forecast
35.     if raw_order > 0:
36.         min_order = demand_forecast * order_smoothing_min
37.         max_order = demand_forecast * order_smoothing_max
38.         order_amount = max(min_order, min(raw_order, max_order))
39.     else:
40.         order_amount = 0
41.
42.     return order_amount

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{effective_pipeline} = \left(\sum_{i=0}^{L-1} q_i \cdot 0.8 \right)$$

$$\text{inventory_position} = (h + \text{effective_pipeline})$$

$$\text{adjusted_base_stock} = (534.6 \cdot 1.0)$$

$$\text{order_up_to} = (\text{adjusted_base_stock} + 44.7)$$

$$\text{raw_order} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$

$$\left. \begin{array}{l} \text{min_order} = (102.2 \cdot 0.9) \\ \text{max_order} = (102.2 \cdot 1.1) \end{array} \right\} (\text{raw_order} > 0)$$

$$\text{order_amount} = \max(\text{min_order}, \min(\text{raw_order}, \text{max_order}))$$

$$\text{order_amount} = 0 \} (\neg (\text{raw_order} > 0))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-36

```
1. # sample_id: 054
2. # folder: deepseek-chat_normal_std30_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r9
3. # distribution: normal_std30_L6_c1_5
4. # generation: 7
5. # rank_in_population_file: 8
6. # objective: 4044.71097
7. # test_objective: 3960.15521
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;partial_adjustment
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 686.812636924258 # OPT_PARAM: {"initial": 686.812636924258, "min": 600, "max": 850,
    "type": "float"}
14.     safety_stock = 89.54758273145202 # OPT_PARAM: {"initial": 89.54758273145202, "min": 80, "max":
    140, "type": "float"}
15.     pipeline_coverage = 0.8624625103570247 # OPT_PARAM: {"initial": 0.8624625103570247, "min": 0.7,
    "max": 1.0, "type": "float"}
16.     demand_multiplier = 1.0420790590982434 # OPT_PARAM: {"initial": 1.0420790590982434, "min": 1.0,
    "max": 1.3, "type": "float"}
17.     smoothing_factor = 0.15029012827558405 # OPT_PARAM: {"initial": 0.15029012827558405, "min": 0.1,
    "max": 0.5, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate effective pipeline coverage
23.     pipeline_sum = sum(pipeline_orders)
24.     effective_pipeline = pipeline_sum * pipeline_coverage
25.
26.     # Calculate target inventory level
27.     target_inventory = base_stock + safety_stock
28.
29.     # Calculate order amount with smoothing
30.     raw_order = max(0, target_inventory - inventory_position + effective_pipeline)
31.
32.     # Apply demand multiplier and smoothing
33.     order_amount = raw_order * demand_multiplier * smoothing_factor
34.
35.     # Round to nearest integer
36.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \left(\left(\max \left(0, \left((686.8 + 89.5) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) + \left(\sum_{i=0}^{L-1} q_i \cdot 0.9 \right) \right) \right) \cdot 1.0 \right) \cdot 0.2$$

Python source lines 1-39

```
1. # sample_id: 055
2. # folder: deepseek-chat_normal_std30_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: normal_std30_L6_c1_5
4. # generation: 8
5. # rank_in_population_file: 8
6. # objective: 3872.97736
7. # test_objective: 3848.71283
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 979.5288931101101 # OPT_PARAM: {"initial": 979.5288931101101, "min": 800, "max":
14.     1100, "type": "float"}
15.     safety_stock = 201.36307898735558 # OPT_PARAM: {"initial": 201.36307898735558, "min": 100,
16.     "max": 250, "type": "float"}
17.     demand_estimate = 92.62607902796263 # OPT_PARAM: {"initial": 92.62607902796263, "min": 90,
18.     "max": 110, "type": "float"}
19.     smoothing_factor = 0.05 # OPT_PARAM: {"initial": 0.05, "min": 0.05, "max": 0.25, "type":
20.     "float"}
21.     order_multiplier = 0.95 # OPT_PARAM: {"initial": 0.95, "min": 0.95, "max": 1.15, "type":
22.     "float"}
23.
24.     # Calculate inventory position
25.     inventory_position = on_hand_inventory + sum(pipeline_orders)
26.
27.     # Calculate expected demand during lead time
28.     lead_time = len(pipeline_orders)
29.     expected_lead_time_demand = demand_estimate * lead_time * order_multiplier
30.
31.     # Calculate target inventory position
32.     target_position = expected_lead_time_demand + safety_stock
33.
34.     # Calculate order amount
35.     raw_order = max(0, target_position - inventory_position)
36.
37.     # Apply smoothing
38.     smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * demand_estimate
39.
40.     # Apply base stock constraint
41.     order_amount = min(smoothed_order, max(0, base_stock - inventory_position))
42.
43.     # Round to nearest integer
44.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \min \left(\left(\left(0.1 \cdot \max \left(0, \left(((92.6 \cdot L) \cdot 0.9) + 201.4) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right) + ((1 - 0.1) \cdot 92.6) \right), \max \left(0, \left(979.5 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right).$$

Python source lines 1-22

```
1. # sample_id: 056
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_no_15_default_m2_10_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 13
5. # rank_in_population_file: 2
6. # objective: 7244.18
7. # test_objective: 7383.881
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 290 # OPT_PARAM: {'initial': 290, 'min': 10, 'max': 1000, 'type': 'float'}
14.     safety_stock = 60 # OPT_PARAM: {'initial': 60, 'min': 0, 'max': 200, 'type': 'float'}
15.
16.     # Calculate net inventory position
17.     net_inventory = on_hand_inventory + sum(pipeline_orders)
18.
19.     # Order up to base_stock plus safety_stock adjustment
20.     order_amount = max(0, base_stock + safety_stock - net_inventory)
21.
22.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left((290 + 60) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right).$$

Python source lines 1-43

```

1. # sample_id: 057
2. # folder: deepseek-chat_normal_std10_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r6
3. # distribution: normal_std10_L6_c1_5
4. # generation: 5
5. # rank_in_population_file: 1
6. # objective: 1318.78
7. # test_objective: 1311.369
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 589.9262169191956 # OPT_PARAM: {"initial": 589.9262169191956, "min": 450, "max":
14.     700, "type": "float"}
15.     safety_stock = 104.7989098814106 # OPT_PARAM: {"initial": 104.7989098814106, "min": 40, "max":
16.     120, "type": "float"}
17.     smoothing_factor = 0.2 # OPT_PARAM: {"initial": 0.2, "min": 0.2, "max": 0.9, "type": "float"}
18.     demand_adjustment = 0.9404108089646986 # OPT_PARAM: {"initial": 0.9404108089646986, "min": 0.8,
19.     "max": 3.0, "type": "float"}
20.     min_order = 20.0 # OPT_PARAM: {"initial": 20.0, "min": 10, "max": 50, "type": "float"}
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate demand estimate using all pipeline orders (better signal)
26.     if len(pipeline_orders) > 0:
27.         # Weight recent orders more heavily
28.         weights = [0.4, 0.3, 0.2, 0.1, 0.0, 0.0][:len(pipeline_orders)]
29.         weighted_sum = sum(w * q for w, q in zip(weights, pipeline_orders))
30.         weight_sum = sum(weights[:len(pipeline_orders)])
31.         demand_estimate = weighted_sum / weight_sum if weight_sum > 0 else 100.0
32.     else:
33.         demand_estimate = 100.0
34.
35.     # Adjust base stock based on demand estimate
36.     adjusted_base = base_stock + (demand_estimate - 100) * demand_adjustment
37.
38.     # Calculate target inventory position
39.     target_position = adjusted_base + safety_stock
40.
41.     # Calculate order needed
42.     order_needed = target_position - inventory_position
43.
44.     # Apply smoothing only when ordering
45.     if order_needed > min_order:
46.         smoothed_order = smoothing_factor * order_needed + (1 - smoothing_factor) * demand_estimate

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{weights} = [0.4, 0.3, 0.2, 0.1, 0.0, 0.0]_{:L}$$

$$\text{weighted_sum} = \sum_{(w,q) \in \text{zip}(\text{weights}, \mathbf{q})} (w \cdot q)$$

$$\text{weight_sum} = \text{sum}(\text{weights}_{:L})$$

$$\text{demand_estimate} = \left\{ \begin{array}{ll} \frac{\text{weighted_sum}}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 100.0, & \text{otherwise} \end{array} \right\} \quad (L > 0)$$

$$\text{demand_estimate} = 100.0 \quad (\neg (L > 0))$$

$$\text{adjusted_base} = (589.9 + ((\text{demand_estimate} - 100) \cdot 0.9))$$

$$\text{target_position} = (\text{adjusted_base} + 104.8)$$

$$\text{order_needed} = (\text{target_position} - \text{inventory_position})$$

$$\text{smoothed_order} = \left\{ \begin{array}{ll} ((0.2 \cdot \text{order_needed}) + ((1 - 0.2) \cdot \text{demand_estimate})), & \text{order_needed} > 20.0 \\ \max(0, \text{order_needed}), & \text{otherwise} \end{array} \right.$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 44-50

```

44.     else:
45.         smoothed_order = max(0, order_needed)
46.
47.     # Round to integer and ensure non-negative
48.     order_amount = max(0, int(round(smoothed_order)))
49.
50.     return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{weights} = [0.4, 0.3, 0.2, 0.1, 0.0, 0.0]_{:L}$$

$$\text{weighted_sum} = \sum_{(w,q) \in \text{zip}(\text{weights}, \mathbf{q})} (w \cdot q)$$

$$\text{weight_sum} = \text{sum}(\text{weights}_{:L})$$

$$\text{demand_estimate} = \left\{ \begin{array}{ll} \frac{\text{weighted_sum}}{\text{weight_sum}}, & \text{weight_sum} > 0 \\ 100.0, & \text{otherwise} \end{array} \right\} \quad (L > 0)$$

$$\text{demand_estimate} = 100.0 \quad (\neg(L > 0))$$

$$\text{adjusted_base} = (589.9 + ((\text{demand_estimate} - 100) \cdot 0.9))$$

$$\text{target_position} = (\text{adjusted_base} + 104.8)$$

$$\text{order_needed} = (\text{target_position} - \text{inventory_position})$$

$$\text{smoothed_order} = \left\{ \begin{array}{ll} ((0.2 \cdot \text{order_needed}) + ((1 - 0.2) \cdot \text{demand_estimate})), & \text{order_needed} > 20.0 \\ \max(0, \text{order_needed}), & \text{otherwise} \end{array} \right.$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-42

```
1. # sample_id: 058
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: poisson_L6_c1_5
4. # generation: 7
5. # rank_in_population_file: 6
6. # objective: 2947.05251
7. # test_objective: 2977.16457
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 709.6025870100856 # OPT_PARAM: {"initial": 709.6025870100856, "min": 500, "max":
    800, "type": "float"}
14.     safety_stock = 200.0 # OPT_PARAM: {"initial": 200.0, "min": 100, "max": 200, "type": "float"}
15.     demand_forecast = 107.5524968773776 # OPT_PARAM: {"initial": 107.5524968773776, "min": 90,
    "max": 110, "type": "float"}
16.     adjustment_factor = 0.7 # OPT_PARAM: {"initial": 0.7, "min": 0.7, "max": 1.0, "type": "float"}
17.     pipeline_weight = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.1, "max": 0.5, "type": "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Calculate expected demand during lead time with pipeline consideration
23.     lead_time = len(pipeline_orders)
24.     expected_lead_time_demand = demand_forecast * lead_time
25.
26.     # Adjust target based on pipeline variability
27.     pipeline_sum = sum(pipeline_orders)
28.     pipeline_avg = pipeline_sum / max(1, lead_time)
29.     pipeline_adjustment = max(0, pipeline_avg - demand_forecast) * pipeline_weight * lead_time
30.
31.     # Calculate target inventory position
32.     target_inventory = expected_lead_time_demand + safety_stock - pipeline_adjustment
33.
34.     # Calculate order amount with adjustment factor
35.     order_needed = target_inventory - inventory_position
36.     order_amount = max(0, order_needed * adjustment_factor)
37.
38.     # Apply base stock as upper bound
39.     order_amount = min(order_amount, max(0, base_stock - inventory_position))
40.
41.     # Round to nearest integer since order amounts should be integers
42.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, q) = \min \left(\max \left(0, \left(\left(((107.6 \cdot L) + 200.0) - \left(\max \left(0, \left(\frac{\sum_{t=0}^{L-1} q_t}{\max(1, L)} - 107.6 \right) \cdot 0.5 \right) \cdot L \right) - \left(h + \sum_{t=0}^{L-1} q_t \right) \cdot 0.7 \right) \right), \max \left(0, \left(709.6 - \left(h + \sum_{t=0}^{L-1} q_t \right) \right) \right) \right) \right).$$

Python source lines 1-38

```

1. # sample_id: 059
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_e1-m2_6_r2
3. # distribution: poisson_L6_c1_2
4. # generation: 10
5. # rank_in_population_file: 2
6. # objective: 1033.1
7. # test_objective: 976.627
8. # is_top10_by_distribution: False
9. # is_final_generation: True
10. # table_motifs:
    pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping;order_ssmoothing
11. # extra_motifs:
    pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Core policy parameters
14.     base_target = 581.8060760782333 # OPT_PARAM: {"initial": 581.8060760782333, "min": 400, "max": 900, "type": "float"}
15.     demand_estimate = 80.0 # OPT_PARAM: {"initial": 80.0, "min": 80, "max": 120, "type": "float"}
16.     safety_factor = 0.8088798033541513 # OPT_PARAM: {"initial": 0.8088798033541513, "min": 0.8, "max": 2.0, "type": "float"}
17.     pipeline_decay = 0.8356706894043239 # OPT_PARAM: {"initial": 0.8356706894043239, "min": 0.5, "max": 1.0, "type": "float"}
18.
19.     # Novel structural component: demand pattern recognition
20.     pipeline_gradient_threshold = 0.30615656003069885 # OPT_PARAM: {"initial": 0.30615656003069885, "min": 0.1, "max": 0.8, "type": "float"}
21.     gradient_response = 2.986613877539136 # OPT_PARAM: {"initial": 2.986613877539136, "min": 1.0, "max": 3.0, "type": "float"}
22.
23.     # Calculate weighted pipeline with exponential decay
24.     weighted_pipeline = 0.0
25.     for i, order in enumerate(pipeline_orders):
26.         weight = pipeline_decay ** (len(pipeline_orders) - i - 1)
27.         weighted_pipeline += order * weight
28.
29.     # Novel structural logic: Detect pipeline depletion patterns
30.     pipeline_gradient = 0.0
31.     if len(pipeline_orders) >= 2:
32.         # Calculate slope of pipeline orders (recent vs older)
33.         recent_avg = sum(pipeline_orders[-2:]) / 2
34.         older_avg = sum(pipeline_orders[:-2]) / (len(pipeline_orders) - 2) if len(pipeline_orders) > 2 else recent_avg
35.         pipeline_gradient = (recent_avg - older_avg) / max(1.0, older_avg)
36.
37.     # Adjust target based on pipeline gradient
38.     gradient_adjustment = 1.0

```

Declarative mathematical formula rows 1-19

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.8)^{(L-i)-1}))$$

$$\text{pipeline_gradient} = 0.0$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\sum_{i=L-(2)}^{L-1} q_i}{2} \\ \text{older_avg} &= \begin{cases} \frac{\sum_{i=0}^{L-(2)-1} q_i}{(L-2)}, & L > 2 \\ \text{recent_avg}, & \text{otherwise} \end{cases} \end{aligned} \right\} (L \geq 2)$$

$$\text{pipeline_gradient} = \frac{(\text{recent_avg} - \text{older_avg})}{\max(1.0, \text{older_avg})}$$

$$\text{gradient_adjustment} = 1.0$$

$$\text{gradient_adjustment} = 3.0 \quad (\text{pipeline_gradient} < -(0.3))$$

$$\text{gradient_adjustment} = \begin{cases} \frac{1.0}{3.0}, & \text{pipeline_gradient} > 0.3 \\ \text{gradient_adjustment}, & \text{otherwise} \end{cases} \quad (\neg(\text{pipeline_gradient} < -(0.3)))$$

$$\text{effective_position} = (h + \text{weighted_pipeline})$$

$$\text{adjusted_target} = (581.8 \cdot \text{gradient_adjustment})$$

$$\text{base_order} = \max(0, (\text{adjusted_target} - \text{effective_position}))$$

$$\text{demand_component} = (80.0 \cdot 0.8)$$

$$\text{pipeline_coverage_ratio} = \begin{cases} \frac{\text{weighted_pipeline}}{(80.0 \cdot L)}, & L > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{blend_weight} = 0.8 \quad (\text{pipeline_coverage_ratio} < 0.8)$$

$$\text{blend_weight} = \begin{cases} 0.3, & \text{pipeline_coverage_ratio} > 1.2 \\ 0.5, & \text{otherwise} \end{cases} \quad (\neg(\text{pipeline_coverage_ratio} < 0.8))$$

$$\text{order_amount} = ((\text{blend_weight} \cdot \text{base_order}) + ((1 - \text{blend_weight}) \cdot \text{demand_component}))$$

$$\left. \begin{aligned} \text{last_order} &= q_{-(1)} \\ \text{order_amount} &= (\text{last_order} + (0.3 \cdot (\text{order_amount} - \text{last_order}))) \end{aligned} \right\} (L > 0)$$

$$\text{order_amount} = \max(0, \text{order_amount})$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 39-82

```

39.     if pipeline_gradient < -pipeline_gradient_threshold:
40.         # Pipeline is decreasing sharply - increase orders
41.         gradient_adjustment = gradient_response
42.     elif pipeline_gradient > pipeline_gradient_threshold:
43.         # Pipeline is increasing - reduce orders
44.         gradient_adjustment = 1.0 / gradient_response
45.
46.     # Calculate effective inventory position
47.     effective_position = on_hand_inventory + weighted_pipeline
48.
49.     # Calculate adjusted target
50.     adjusted_target = base_target * gradient_adjustment
51.
52.     # Base order calculation
53.     base_order = max(0, adjusted_target - effective_position)
54.
55.     # Add demand-responsive component
56.     demand_component = demand_estimate * safety_factor
57.
58.     # Novel structural logic: Blend based on pipeline coverage
59.     pipeline_coverage_ratio = weighted_pipeline / (demand_estimate * len(pipeline_orders)) if
len(pipeline_orders) > 0 else 0
60.
61.     if pipeline_coverage_ratio < 0.8:
62.         # Low pipeline coverage - emphasize demand component
63.         blend_weight = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.3, "max": 0.9, "type": "float"}
64.     elif pipeline_coverage_ratio > 1.2:
65.         # High pipeline coverage - emphasize base stock adjustment
66.         blend_weight = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.1, "max": 0.7, "type": "float"}
67.     else:
68.         # Moderate coverage - balanced approach
69.         blend_weight = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.2, "max": 0.8, "type": "float"}
70.
71.     # Final blended order
72.     order_amount = blend_weight * base_order + (1 - blend_weight) * demand_component
73.
74.     # Apply non-linear smoothing
75.     smoothing_factor = 0.25747925090182155 # OPT_PARAM: {"initial": 0.25747925090182155, "min":
0.05, "max": 0.5, "type": "float"}
76.     if len(pipeline_orders) > 0:
77.         last_order = pipeline_orders[-1]
78.         order_amount = last_order + smoothing_factor * (order_amount - last_order)
79.
80.     # Ensure non-negative and integer
81.     order_amount = max(0, order_amount)
82.     order_amount = int(round(order_amount))

```

Declarative mathematical formula rows 1-19

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.8)^{(L-i)-1}))$$

pipeline_gradient = 0.0

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\sum_{i=L-(2)}^{L-1} q_i}{2} \\ \text{older_avg} &= \begin{cases} \frac{\sum_{i=0}^{L-(2)-1} q_i}{(L-2)}, & L > 2 \\ \text{recent_avg}, & \text{otherwise} \end{cases} \end{aligned} \right\} (L \geq 2)$$

$$\text{pipeline_gradient} = \frac{(\text{recent_avg} - \text{older_avg})}{\max(1.0, \text{older_avg})}$$

gradient_adjustment = 1.0

gradient_adjustment = 3.0} (pipeline_gradient < -(0.3))

$$\text{gradient_adjustment} = \begin{cases} \frac{1.0}{3.0}, & \text{pipeline_gradient} > 0.3 \\ \text{gradient_adjustment}, & \text{otherwise} \end{cases} \quad (\neg(\text{pipeline_gradient} < -(0.3)))$$

effective_position = (h + weighted_pipeline)

adjusted_target = (581.8 · gradient_adjustment)

base_order = max(0, (adjusted_target - effective_position))

demand_component = (80.0 · 0.8)

$$\text{pipeline_coverage_ratio} = \begin{cases} \frac{\text{weighted_pipeline}}{(80.0 \cdot L)}, & L > 0 \\ 0, & \text{otherwise} \end{cases}$$

blend_weight = 0.8} (pipeline_coverage_ratio < 0.8)

$$\text{blend_weight} = \begin{cases} 0.3, & \text{pipeline_coverage_ratio} > 1.2 \\ 0.5, & \text{otherwise} \end{cases} \quad (\neg(\text{pipeline_coverage_ratio} < 0.8))$$

order_amount = ((blend_weight · base_order) + ((1 - blend_weight) · demand_component))

$$\left. \begin{aligned} \text{last_order} &= q_{-(1)} \\ \text{order_amount} &= (\text{last_order} + (0.3 \cdot (\text{order_amount} - \text{last_order}))) \end{aligned} \right\} (L > 0)$$

order_amount = max(0, order_amount)

order_amount = round_{even}(order_amount)

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 83-84

```

83.
84.     return order_amount

```

Declarative mathematical formula rows 1-19

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (0.8)^{(L-i)-1}))$$

$$\text{pipeline_gradient} = 0.0$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\sum_{i=L-(2)}^{L-1} q_i}{2} \\ \text{older_avg} &= \begin{cases} \frac{\sum_{i=0}^{L-(2)-1} q_i}{(L-2)}, & L > 2 \\ \text{recent_avg}, & \text{otherwise} \end{cases} \\ \text{pipeline_gradient} &= \frac{(\text{recent_avg} - \text{older_avg})}{\max(1.0, \text{older_avg})} \end{aligned} \right\} (L \geq 2)$$

$$\text{gradient_adjustment} = 1.0$$

$$\text{gradient_adjustment} = 3.0 \quad (\text{pipeline_gradient} < -(0.3))$$

$$\text{gradient_adjustment} = \begin{cases} \frac{1.0}{3.0}, & \text{pipeline_gradient} > 0.3 \\ \text{gradient_adjustment}, & \text{otherwise} \end{cases} \quad (\neg(\text{pipeline_gradient} < -(0.3)))$$

$$\text{effective_position} = (h + \text{weighted_pipeline})$$

$$\text{adjusted_target} = (581.8 \cdot \text{gradient_adjustment})$$

$$\text{base_order} = \max(0, (\text{adjusted_target} - \text{effective_position}))$$

$$\text{demand_component} = (80.0 \cdot 0.8)$$

$$\text{pipeline_coverage_ratio} = \begin{cases} \frac{\text{weighted_pipeline}}{(80.0 \cdot L)}, & L > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{blend_weight} = 0.8 \quad (\text{pipeline_coverage_ratio} < 0.8)$$

$$\text{blend_weight} = \begin{cases} 0.3, & \text{pipeline_coverage_ratio} > 1.2 \\ 0.5, & \text{otherwise} \end{cases} \quad (\neg(\text{pipeline_coverage_ratio} < 0.8))$$

$$\text{order_amount} = ((\text{blend_weight} \cdot \text{base_order}) + ((1 - \text{blend_weight}) \cdot \text{demand_component}))$$

$$\left. \begin{aligned} \text{last_order} &= q_{-(1)} \\ \text{order_amount} &= (\text{last_order} + (0.3 \cdot (\text{order_amount} - \text{last_order}))) \end{aligned} \right\} (L > 0)$$

$$\text{order_amount} = \max(0, \text{order_amount})$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-41

```

1. # sample_id: 060
2. # folder: deepseek-chat_exponential_L6_c1_5_50_plain_processed_scipy_15_default_m2plural_2_r6
3. # distribution: exponential_L6_c1_5
4. # generation: 4
5. # rank_in_population_file: 4
6. # objective: 11112.78
7. # test_objective: 11227.361
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 517.9333077084599 # OPT_PARAM: {"initial": 517.9333077084599, "min": 400, "max":
    700, "type": "float"}
14.     safety_stock = 118.14102434624172 # OPT_PARAM: {"initial": 118.14102434624172, "min": 80, "max":
    200, "type": "float"}
15.     lead_time = len(pipeline_orders)
16.
17.     # Calculate inventory position
18.     inventory_position = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Calculate expected demand during lead time using recent arrivals
21.     # Use first 3 pipeline orders (recently arrived) as proxy for demand
22.     if len(pipeline_orders) >= 3:
23.         recent_demand = pipeline_orders[0] + pipeline_orders[1] + pipeline_orders[2]
24.         avg_recent_demand = recent_demand / 3.0
25.     else:
26.         avg_recent_demand = sum(pipeline_orders) / max(len(pipeline_orders), 1)
27.
28.     # Dynamic adjustment based on recent demand
29.     demand_adjustment_factor = 0.8334838024453708 # OPT_PARAM: {"initial": 0.8334838024453708,
    "min": 0.5, "max": 1.2, "type": "float"}
30.     demand_adjustment = avg_recent_demand * lead_time * demand_adjustment_factor
31.
32.     # Calculate target inventory position
33.     target_position = base_stock + safety_stock + demand_adjustment
34.
35.     # Calculate raw order needed
36.     raw_order = max(0, target_position - inventory_position)
37.
38.     # Apply moderate smoothing to balance responsiveness and stability
39.     smoothing_factor = 0.16708542784709138 # OPT_PARAM: {"initial": 0.16708542784709138, "min":
    0.15, "max": 0.4, "type": "float"}
40.     smoothed_order = smoothing_factor * raw_order
41.

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{recent_demand} &= ((q_0 + q_1) + q_2) \\ \text{avg_recent_demand} &= \frac{\text{recent_demand}}{3.0} \end{aligned} \right\} \quad (L \geq 3)$$

$$\text{avg_recent_demand} = \frac{\sum_{i=0}^{L-1} q_i}{\max(L, 1)} \quad (\neg (L \geq 3))$$

$$\text{demand_adjustment} = ((\text{avg_recent_demand} \cdot \text{lead_time}) \cdot 0.8)$$

$$\text{target_position} = ((517.9 + 118.1) + \text{demand_adjustment})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = (0.2 \cdot \text{raw_order})$$

$$\left. \begin{aligned} \text{pipeline_variance} &= (\max(\mathbf{q}) - \min(\mathbf{q})) \\ \text{variance_factor} &= 2.4e - 02 \\ \text{variance_adjustment} &= (\text{pipeline_variance} \cdot \text{variance_factor}) \end{aligned} \right\} \quad (L \geq 2)$$

$$\text{smoothed_order} = (\text{smoothed_order} + \text{variance_adjustment})$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 42-52

```

42.     # Add small adjustment based on pipeline variability
43.     if len(pipeline_orders) >= 2:
44.         pipeline_variance = max(pipeline_orders) - min(pipeline_orders)
45.         variance_factor = 0.023639962573094874 # OPT_PARAM: {"initial": 0.023639962573094874, "min":
0.01, "max": 0.15, "type": "float"}
46.         variance_adjustment = pipeline_variance * variance_factor
47.         smoothed_order += variance_adjustment
48.
49.     # Round to nearest integer
50.     order_amount = int(round(smoothed_order))
51.
52.     return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{recent_demand} &= ((q_0 + q_1) + q_2) \\ \text{avg_recent_demand} &= \frac{\text{recent_demand}}{3.0} \end{aligned} \right\} \quad (L \geq 3)$$

$$\text{avg_recent_demand} = \frac{\sum_{i=0}^{L-1} q_i}{\max(L, 1)} \quad (\neg (L \geq 3))$$

$$\text{demand_adjustment} = ((\text{avg_recent_demand} \cdot \text{lead_time}) \cdot 0.8)$$

$$\text{target_position} = ((517.9 + 118.1) + \text{demand_adjustment})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = (0.2 \cdot \text{raw_order})$$

$$\left. \begin{aligned} \text{pipeline_variance} &= (\max(\mathbf{q}) - \min(\mathbf{q})) \\ \text{variance_factor} &= 2.4e - 02 \\ \text{variance_adjustment} &= (\text{pipeline_variance} \cdot \text{variance_factor}) \\ \text{smoothed_order} &= (\text{smoothed_order} + \text{variance_adjustment}) \end{aligned} \right\} \quad (L \geq 2)$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-37

```

1. # sample_id: 061
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_m2_4_r9
3. # distribution: poisson_L4_c1_5
4. # generation: 2
5. # rank_in_population_file: 5
6. # objective: 2035.5205
7. # test_objective: 2034.89705
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 437.50330001294367 # OPT_PARAM: {"initial": 437.50330001294367, "min": 400, "max":
    600, "type": "float"}
14.     safety_stock = 47.503300012927305 # OPT_PARAM: {"initial": 47.503300012927305, "min": 20, "max":
    150, "type": "float"}
15.     smoothing_factor = 0.31732836224038596 # OPT_PARAM: {"initial": 0.31732836224038596, "min": 0.1,
    "max": 0.9, "type": "float"}
16.     lead_time = 4 # OPT_PARAM: {"initial": 4, "min": 1, "max": 8, "type": "int"}
17.     demand_forecast_factor = 0.6 # OPT_PARAM: {"initial": 0.6, "min": 0.5, "max": 1.2, "type":
    "float"}
18.
19.     # Calculate inventory position
20.     inventory_position = on_hand_inventory + sum(pipeline_orders)
21.
22.     # Estimate upcoming demand based on recent pipeline usage
23.     # Use average of recent orders as demand forecast
24.     if len(pipeline_orders) > 0:
25.         recent_orders_avg = sum(pipeline_orders) / len(pipeline_orders)
26.         forecast_adjustment = demand_forecast_factor * recent_orders_avg
27.     else:
28.         forecast_adjustment = 0
29.
30.     # Adjust target based on forecast and lead time
31.     adjusted_target = base_stock + safety_stock + forecast_adjustment * lead_time
32.
33.     # Calculate order amount with smoothing
34.     order_amount = max(0, smoothing_factor * (adjusted_target - inventory_position))
35.
36.     # Round to nearest integer
37.     return order_amount

```

Declarative mathematical formula rows 1-7

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{recent_orders_avg} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \end{aligned} \right\} (L > 0)$$

$$\text{forecast_adjustment} = (0.6 \cdot \text{recent_orders_avg})$$

$$\text{forecast_adjustment} = 0 \quad (\neg (L > 0))$$

$$\text{adjusted_target} = ((437.5 + 47.5) + (\text{forecast_adjustment} \cdot 4))$$

$$\text{order_amount} = \max(0, (0.3 \cdot (\text{adjusted_target} - \text{inventory_position})))$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 1-41

```

1. # sample_id: 062
2. # folder: deepseek-chat_normal_std50_L6_c1_2_50_plain_processed_scipy_15_default_e1-e2-m2_2_r1
3. # distribution: normal_std50_L6_c1_2
4. # generation: 12
5. # rank_in_population_file: 1
6. # objective: 3793.52
7. # test_objective: 3791.56
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;nonlinear_pipeline_composition;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding;emergency_or_shor
    tage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 642.5315011025823 # OPT_PARAM: {"initial": 642.5315011025823, "min": 400, "max":
    900, "type": "float"}
14.     safety_stock_multiplier = 1.3757177793014963 # OPT_PARAM: {"initial": 1.3757177793014963, "min":
    1.0, "max": 3.0, "type": "float"}
15.     demand_smoothing = 0.35 # OPT_PARAM: {"initial": 0.35, "min": 0.1, "max": 0.5, "type": "float"}
16.     pipeline_decay = 0.7183866101559002 # OPT_PARAM: {"initial": 0.7183866101559002, "min": 0.4,
    "max": 0.9, "type": "float"}
17.     order_aggressiveness = 0.7099305808723362 # OPT_PARAM: {"initial": 0.7099305808723362, "min":
    0.5, "max": 1.0, "type": "float"}
18.
19.     min_order_qty = 20 # OPT_PARAM: {"initial": 20, "min": 10, "max": 30, "type": "int"}
20.     max_order_qty = 220 # OPT_PARAM: {"initial": 220, "min": 150, "max": 300, "type": "int"}
21.
22.     pipeline_lookback = 4 # OPT_PARAM: {"initial": 4, "min": 2, "max": 6, "type": "int"}
23.     demand_estimation_window = 5 # OPT_PARAM: {"initial": 5, "min": 3, "max": 8, "type": "int"}
24.
25.     # Calculate weighted pipeline inventory with exponential decay
26.     weighted_pipeline = 0.0
27.     current_weight = 1.0
28.     for i in range(min(pipeline_lookback, len(pipeline_orders))):
29.         weighted_pipeline += pipeline_orders[i] * current_weight
30.         current_weight *= pipeline_decay
31.
32.     # Estimate demand using recent pipeline orders (proxy for recent sales)
33.     if len(pipeline_orders) >= demand_estimation_window:
34.         recent_orders = pipeline_orders[:demand_estimation_window]
35.         estimated_demand = sum(recent_orders) / len(recent_orders)
36.     else:
37.         estimated_demand = 100.0 # fallback estimate
38.
39.     # Calculate safety stock based on estimated demand
40.     safety_stock = safety_stock_multiplier * estimated_demand
41.

```

Declarative mathematical formula rows 1-13

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{weighted_pipeline} = 0.0$

$\text{current_weight} = 1.0$

$$\text{weighted_pipeline} = \sum_{i=0}^{\min(4,L)-1} ((q_i \cdot (1.0) (0.7)^i))$$

$$\left. \begin{array}{l} \text{recent_orders} = q_{:5} \\ \text{estimated_demand} = \frac{\text{sum}(\text{recent_orders})}{\text{len}(\text{recent_orders})} \end{array} \right\} (L \geq 5)$$

$\text{estimated_demand} = 100.0 \} (\neg(L \geq 5))$

$\text{safety_stock} = (1.4 \cdot \text{estimated_demand})$

$\text{target_inv_level} = (642.5 + \text{safety_stock})$

$\text{current_inv_position} = (h + \text{weighted_pipeline})$

$\text{order_gap} = (\text{target_inv_level} - \text{current_inv_position})$

$$\left. \begin{array}{l} \text{smoothed_order} = (\text{order_gap})^{0.7} \\ \text{order_amount} = \max(20, \min(220, \text{int}(\text{smoothed_order}))) \end{array} \right\} (\text{order_gap} > 0)$$

$\text{order_amount} = 0 \} (\neg(\text{order_gap} > 0))$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 42-60

```

42. # Target inventory level considering base stock and safety stock
43. target_inv_level = base_stock + safety_stock
44.
45. # Current inventory position (on-hand + weighted pipeline)
46. current_inv_position = on_hand_inventory + weighted_pipeline
47.
48. # Calculate order gap
49. order_gap = target_inv_level - current_inv_position
50.
51. # Apply smoothed ordering with constraints
52. if order_gap > 0:
53.     # Use power function for order smoothing
54.     smoothed_order = order_gap ** order_aggressiveness
55.     # Apply min/max constraints and round to integer
56.     order_amount = max(min_order_qty, min(max_order_qty, int(smoothed_order)))
57. else:
58.     order_amount = 0
59.
60. return order_amount

```

Declarative mathematical formula rows 1-13

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{weighted_pipeline} = 0.0$

$\text{current_weight} = 1.0$

$$\text{weighted_pipeline} = \sum_{i=0}^{\min(4,L)-1} ((q_i \cdot (1.0) (0.7)^i))$$

$$\left. \begin{array}{l} \text{recent_orders} = q_{:5} \\ \text{estimated_demand} = \frac{\text{sum}(\text{recent_orders})}{\text{len}(\text{recent_orders})} \end{array} \right\} (L \geq 5)$$

$\text{estimated_demand} = 100.0 \} (\neg(L \geq 5))$

$\text{safety_stock} = (1.4 \cdot \text{estimated_demand})$

$\text{target_inv_level} = (642.5 + \text{safety_stock})$

$\text{current_inv_position} = (h + \text{weighted_pipeline})$

$\text{order_gap} = (\text{target_inv_level} - \text{current_inv_position})$

$$\left. \begin{array}{l} \text{smoothed_order} = (\text{order_gap})^{0.7} \\ \text{order_amount} = \max(20, \min(220, \text{int}(\text{smoothed_order}))) \end{array} \right\} (\text{order_gap} > 0)$$

$\text{order_amount} = 0 \} (\neg(\text{order_gap} > 0))$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-33

```
1. # sample_id: 063
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: poisson_L6_c1_2
4. # generation: 5
5. # rank_in_population_file: 2
6. # objective: 761.06286
7. # test_objective: 746.02634
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 663.9895492381544 # OPT_PARAM: {"initial": 663.9895492381544, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 58.68920289468093 # OPT_PARAM: {"initial": 58.68920289468093, "min": 0, "max":
16.     200, "type": "float"}
17.     demand_estimate = 125.95458012582644 # OPT_PARAM: {"initial": 125.95458012582644, "min": 50,
18.     "max": 150, "type": "float"}
19.
20.     # Calculate net inventory position
21.     net_inventory = on_hand_inventory + sum(pipeline_orders)
22.
23.     # Calculate expected demand during lead time
24.     expected_lead_time_demand = demand_estimate * len(pipeline_orders)
25.
26.     # Calculate target inventory position
27.     target_position = expected_lead_time_demand + safety_stock
28.
29.     # Calculate order amount
30.     order_amount = max(0, target_position - net_inventory)
31.
32.     # Cap order amount to avoid excessive ordering
33.     max_order = 95.92075836350209 # OPT_PARAM: {"initial": 95.92075836350209, "min": 50, "max": 500,
34.     "type": "float"}
35.     order_amount = min(order_amount, max_order)
36.
37.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \min \left(\max \left(0, \left((126.0 \cdot L) + 58.7 \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right), 95.9 \right).$$

Python source lines 1-20

```
1. # sample_id: 064
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_no_15_default_m2_10_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 4
6. # objective: 3001.4
7. # test_objective: 2994.197
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 510 # OPT_PARAM: {'initial': 510, 'min': 10, 'max': 1000, 'type': 'float'}
14.     safety_stock = 110 # OPT_PARAM: {'initial': 110, 'min': 0, 'max': 200, 'type': 'float'}
15.
16.     inventory_position = on_hand_inventory + sum(pipeline_orders)
17.     target_inventory = base_stock + safety_stock
18.
19.     order_amount = max(0, target_inventory - inventory_position)
20.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left((510 + 110) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right).$$

Python source lines 1-36

```

1. # sample_id: 065
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 10
5. # rank_in_population_file: 3
6. # objective: 839.74
7. # test_objective: 817.593
8. # is_top10_by_distribution: False
9. # is_final_generation: True
10. # table_motifs:
    pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer
    _rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 735.7792788064165 # OPT_PARAM: {"initial": 735.7792788064165, "min": 400, "max":
    900, "type": "float"}
15.
16.     # Demand anticipation parameters
17.     demand_anticipation_factor = 0.2058491381515156 # OPT_PARAM: {"initial": 0.2058491381515156,
    "min": 0.1, "max": 0.8, "type": "float"}
18.     recent_demand_weight = 0.6 # OPT_PARAM: {"initial": 0.6, "min": 0.3, "max": 0.9, "type":
    "float"}
19.
20.     # Pipeline urgency factor
21.     pipeline_urgency_factor = 0.22842984269653896 # OPT_PARAM: {"initial": 0.22842984269653896,
    "min": 0.1, "max": 0.8, "type": "float"}
22.
23.     # Cost-balancing parameters
24.     holding_cost_weight = 0.9368933615395726 # OPT_PARAM: {"initial": 0.9368933615395726, "min":
    0.3, "max": 1.2, "type": "float"}
25.     lost_sales_cost_weight = 1.252120939146047 # OPT_PARAM: {"initial": 1.252120939146047, "min":
    0.8, "max": 2.0, "type": "float"}
26.
27.     # Order smoothing parameters
28.     smoothing_factor = 0.11157585598691623 # OPT_PARAM: {"initial": 0.11157585598691623, "min": 0.1,
    "max": 0.5, "type": "float"}
29.     min_order_size = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 30, "type": "float"}
30.
31.     # Pipeline variability adjustment
32.     pipeline_variability_factor = 6.165600208464511e-06 # OPT_PARAM: {"initial": 6.165600208464511e-
    06, "min": 0.0, "max": 0.8, "type": "float"}
33.
34.     # Safety stock buffer factor
35.     safety_buffer_factor = 0.062341482774576434 # OPT_PARAM: {"initial": 0.062341482774576434,
    "min": 0.0, "max": 0.5, "type": "float"}
36.

```

Declarative mathematical formula rows 1-22

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 & \text{weighted_pipeline} = 0.0 \\
 & \text{total_weight} = 0.0 \\
 & \text{weighted_pipeline} = \sum_{i=0}^{L-1} \left(\left(q_i \cdot \frac{1.0}{(1.0 + i)} \right) \right) \\
 & \text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{1.0}{(1.0 + i)} \right) \\
 & \text{effective_pipeline} = \begin{cases} \frac{\text{weighted_pipeline}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \sum_{i=0}^{L-1} q_i, & \text{otherwise} \end{cases} \\
 & \text{pipeline_urgency} = 0.0 \\
 & \text{near_pipeline} = \sum_{i=0}^{\lfloor \frac{L}{2} \rfloor - 1} q_i \\
 & \text{far_pipeline} = \sum_{i=\lfloor \frac{L}{2} \rfloor}^{L-1} q_i \\
 & \text{total_pipeline} = (\text{near_pipeline} + \text{far_pipeline}) \\
 & \text{pipeline_urgency} = \begin{cases} \frac{\text{far_pipeline}}{\text{total_pipeline}}, & \text{total_pipeline} > 0 \\ \text{pipeline_urgency}, & \text{otherwise} \end{cases} \\
 & \text{effective_pipeline} = 0.0 \\
 & \text{pipeline_urgency} = 0.0 \\
 & \text{pipeline_variability} = 0.0
 \end{aligned}
 \quad \left. \begin{array}{l} (L > 0) \\ (L \geq 2) \end{array} \right\}$$

$$\begin{aligned}
 & \text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 & \text{pipeline_variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\
 & \text{cv_squared} = \frac{\text{pipeline_variance}}{(\text{pipeline_mean})^2} \\
 & \text{pipeline_variability} = \text{cv_squared}
 \end{aligned}
 \quad \left. \begin{array}{l} (L > 1) \end{array} \right\} \quad (\text{pipeline_mean} > 0)$$

inventory_position = (h + effective_pipeline)

dynamic_safety_stock = ($735.8 \cdot (1.0 + (\text{pipeline_urgency} \cdot 0.2))$)

variability_adjustment = ($1.0 + (\text{pipeline_variability} \cdot 6.2e - 06)$)

dynamic_safety_stock = (dynamic_safety_stock · variability_adjustment)

inventory_ratio = $\frac{\text{inventory_position}}{\text{dynamic_safety_stock}}$

dynamic_safety_stock = $\begin{cases} (\text{dynamic_safety_stock} \cdot 1.3), & \text{inventory_ratio} < 0.3 \\ \text{dynamic_safety_stock}, & \text{otherwise} \end{cases}$

cost_ratio = $\frac{(1.3 \cdot 1.5)}{0.9}$

cost_balanced_level = ($\text{dynamic_safety_stock} \cdot (1.0 + (0.2 \cdot (\text{cost_ratio} - 1.0)))$)

safety_buffer = 0.0

safety_buffer = $\begin{cases} (0.1 \cdot (\text{dynamic_safety_stock} - \text{inventory_position})), & \text{inventory_position} < (\text{dynamic_safety_stock} \cdot 0.5) \\ \text{safety_buffer}, & \text{otherwise} \end{cases}$

cost_balanced_level = (cost_balanced_level + safety_buffer)

raw_order = max(0.0, (cost_balanced_level - inventory_position))

urgency_adjusted_smoothing = ($0.1 \cdot (1.0 - (\text{pipeline_urgency} \cdot 0.5))$)

smoothed_order = (raw_order · urgency_adjusted_smoothing)

smoothed_order = $\begin{cases} 0.0, & \text{smoothed_order} < 10.0 \\ \text{smoothed_order}, & \text{otherwise} \end{cases}$

Python source lines 37-79

```

37. # NEW: Aggressive ordering factor for low inventory
38. low_inventory_boost = 1.333272947835163 # OPT_PARAM: {"initial": 1.333272947835163, "min": 1.0,
    "max": 3.0, "type": "float"}
39.
40. # NEW: Lost sales penalty multiplier
41. lost_sales_penalty = 1.5076284285199317 # OPT_PARAM: {"initial": 1.5076284285199317, "min": 1.0,
    "max": 3.0, "type": "float"}
42.
43. # Calculate pipeline urgency
44. L = len(pipeline_orders)
45. if L > 0:
46.     weighted_pipeline = 0.0
47.     total_weight = 0.0
48.
49.     for i, q in enumerate(pipeline_orders):
50.         weight = 1.0 / (1.0 + i)
51.         weighted_pipeline += q * weight
52.         total_weight += weight
53.
54.     effective_pipeline = weighted_pipeline / total_weight if total_weight > 0 else
    sum(pipeline_orders)
55.
56.     pipeline_urgency = 0.0
57.     if L >= 2:
58.         near_pipeline = sum(pipeline_orders[:L//2])
59.         far_pipeline = sum(pipeline_orders[L//2:])
60.         total_pipeline = near_pipeline + far_pipeline
61.
62.         if total_pipeline > 0:
63.             pipeline_urgency = far_pipeline / total_pipeline
64.     else:
65.         effective_pipeline = 0.0
66.         pipeline_urgency = 0.0
67.
68. # Calculate pipeline variability
69. pipeline_variability = 0.0
70. if L > 1:
71.     pipeline_mean = sum(pipeline_orders) / L
72.     if pipeline_mean > 0:
73.         pipeline_variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
74.         cv_squared = pipeline_variance / (pipeline_mean ** 2)
75.         pipeline_variability = cv_squared
76.
77. # Calculate inventory position
78. inventory_position = on_hand_inventory + effective_pipeline
79.

```

Declarative mathematical formula rows 1-22

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned} &\text{weighted_pipeline} = 0.0 \\ &\text{total_weight} = 0.0 \\ &\text{weighted_pipeline} = \sum_{i=0}^{L-1} \left(q_i \cdot \frac{1.0}{(1.0 + i)} \right) \\ &\text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{1.0}{(1.0 + i)} \right) \\ &\text{effective_pipeline} = \begin{cases} \frac{\text{weighted_pipeline}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \sum_{i=0}^{L-1} q_i, & \text{otherwise} \end{cases} \\ &\text{pipeline_urgency} = 0.0 \end{aligned} \right\} (L > 0)$$

$$\left. \begin{aligned} &\text{near_pipeline} = \sum_{i=0}^{\lfloor \frac{L}{2} \rfloor - 1} q_i \\ &\text{far_pipeline} = \sum_{i=\lfloor \frac{L}{2} \rfloor}^{L-1} q_i \\ &\text{total_pipeline} = (\text{near_pipeline} + \text{far_pipeline}) \\ &\text{pipeline_urgency} = \begin{cases} \frac{\text{far_pipeline}}{\text{total_pipeline}}, & \text{total_pipeline} > 0 \\ \text{pipeline_urgency}, & \text{otherwise} \end{cases} \end{aligned} \right\} (L \geq 2)$$

$$\left. \begin{aligned} &\text{effective_pipeline} = 0.0 \\ &\text{pipeline_urgency} = 0.0 \end{aligned} \right\} (\neg (L > 0))$$

$\text{pipeline_variability} = 0.0$

$$\left. \begin{aligned} &\text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\ &\text{pipeline_variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ &\text{cv_squared} = \frac{\text{pipeline_variance}}{(\text{pipeline_mean})^2} \\ &\text{pipeline_variability} = \text{cv_squared} \end{aligned} \right\} (\text{pipeline_mean} > 0) (L > 1)$$

$$\text{inventory_position} = (h + \text{effective_pipeline})$$

$$\text{dynamic_safety_stock} = (735.8 \cdot (1.0 + (\text{pipeline_urgency} \cdot 0.2)))$$

$$\text{variability_adjustment} = (1.0 + (\text{pipeline_variability} \cdot 6.2e - 06))$$

$$\text{dynamic_safety_stock} = (\text{dynamic_safety_stock} \cdot \text{variability_adjustment})$$

$$\text{inventory_ratio} = \frac{\text{inventory_position}}{\text{dynamic_safety_stock}}$$

$$\text{dynamic_safety_stock} = \begin{cases} (\text{dynamic_safety_stock} \cdot 1.3), & \text{inventory_ratio} < 0.3 \\ \text{dynamic_safety_stock}, & \text{otherwise} \end{cases}$$

$$\text{cost_ratio} = \frac{(1.3 \cdot 1.5)}{0.9}$$

$$\text{cost_balanced_level} = (\text{dynamic_safety_stock} \cdot (1.0 + (0.2 \cdot (\text{cost_ratio} - 1.0))))$$

$$\text{safety_buffer} = 0.0$$

$$\text{safety_buffer} = \begin{cases} (0.1 \cdot (\text{dynamic_safety_stock} - \text{inventory_position})), & \text{inventory_position} < (\text{dynamic_safety_stock} \cdot 0.5) \\ \text{safety_buffer}, & \text{otherwise} \end{cases}$$

$$\text{cost_balanced_level} = (\text{cost_balanced_level} + \text{safety_buffer})$$

$$\text{raw_order} = \max(0.0, (\text{cost_balanced_level} - \text{inventory_position}))$$

$$\text{urgency_adjusted_smoothing} = (0.1 \cdot (1.0 - (\text{pipeline_urgency} \cdot 0.5)))$$

$$\text{smoothed_order} = (\text{raw_order} \cdot \text{urgency_adjusted_smoothing})$$

Python source lines 80-116

```

80.     # Dynamic safety stock
81.     dynamic_safety_stock = base_stock * (1.0 + pipeline_urgency * pipeline_urgency_factor)
82.
83.     # Adjust for pipeline variability
84.     variability_adjustment = 1.0 + pipeline_variability * pipeline_variability_factor
85.     dynamic_safety_stock *= variability_adjustment
86.
87.     # NEW: Apply low inventory boost
88.     inventory_ratio = inventory_position / dynamic_safety_stock
89.     if inventory_ratio < 0.3:
90.         dynamic_safety_stock *= low_inventory_boost
91.
92.     # Cost-balanced order-up-to level with lost sales penalty
93.     cost_ratio = (lost_sales_cost_weight * lost_sales_penalty) / holding_cost_weight
94.     cost_balanced_level = dynamic_safety_stock * (1.0 + demand_anticipation_factor * (cost_ratio -
1.0))
95.
96.     # Add safety buffer
97.     safety_buffer = 0.0
98.     if inventory_position < dynamic_safety_stock * 0.5:
99.         safety_buffer = safety_buffer_factor * (dynamic_safety_stock - inventory_position)
100.     cost_balanced_level += safety_buffer
101.
102.     # Calculate raw order amount
103.     raw_order = max(0.0, cost_balanced_level - inventory_position)
104.
105.     # Apply smoothing
106.     urgency_adjusted_smoothing = smoothing_factor * (1.0 - pipeline_urgency * 0.5)
107.     smoothed_order = raw_order * urgency_adjusted_smoothing
108.
109.     # Apply minimum order threshold
110.     if smoothed_order < min_order_size:
111.         smoothed_order = 0.0
112.
113.     # Round to nearest integer
114.     order_amount = int(round(smoothed_order))
115.
116.     return order_amount

```

Declarative mathematical formula rows 1-22

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\left. \begin{aligned} &\text{weighted_pipeline} = 0.0 \\ &\text{total_weight} = 0.0 \\ &\text{weighted_pipeline} = \sum_{i=0}^{L-1} \left(q_i \cdot \frac{1.0}{(1.0 + i)} \right) \\ &\text{total_weight} = \sum_{i=0}^{L-1} \left(\frac{1.0}{(1.0 + i)} \right) \\ &\text{effective_pipeline} = \begin{cases} \frac{\text{weighted_pipeline}}{\text{total_weight}}, & \text{total_weight} > 0 \\ \sum_{i=0}^{L-1} q_i, & \text{otherwise} \end{cases} \\ &\text{pipeline_urgency} = 0.0 \end{aligned} \right\} (L > 0)$$

$$\left. \begin{aligned} &\text{near_pipeline} = \sum_{i=0}^{\lfloor \frac{L}{2} \rfloor - 1} q_i \\ &\text{far_pipeline} = \sum_{i=\lfloor \frac{L}{2} \rfloor}^{L-1} q_i \\ &\text{total_pipeline} = (\text{near_pipeline} + \text{far_pipeline}) \\ &\text{pipeline_urgency} = \begin{cases} \frac{\text{far_pipeline}}{\text{total_pipeline}}, & \text{total_pipeline} > 0 \\ \text{pipeline_urgency}, & \text{otherwise} \end{cases} \end{aligned} \right\} (L \geq 2)$$

$$\left. \begin{aligned} &\text{effective_pipeline} = 0.0 \\ &\text{pipeline_urgency} = 0.0 \end{aligned} \right\} (\neg (L > 0))$$

$\text{pipeline_variability} = 0.0$

$$\left. \begin{aligned} &\text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L} \\ &\text{pipeline_variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean})^2)}{L} \\ &\text{cv_squared} = \frac{\text{pipeline_variance}}{(\text{pipeline_mean})^2} \\ &\text{pipeline_variability} = \text{cv_squared} \end{aligned} \right\} (\text{pipeline_mean} > 0) (L > 1)$$

$$\text{inventory_position} = (h + \text{effective_pipeline})$$

$$\text{dynamic_safety_stock} = (735.8 \cdot (1.0 + (\text{pipeline_urgency} \cdot 0.2)))$$

$$\text{variability_adjustment} = (1.0 + (\text{pipeline_variability} \cdot 6.2e - 06))$$

$$\text{dynamic_safety_stock} = (\text{dynamic_safety_stock} \cdot \text{variability_adjustment})$$

$$\text{inventory_ratio} = \frac{\text{inventory_position}}{\text{dynamic_safety_stock}}$$

$$\text{dynamic_safety_stock} = \begin{cases} (\text{dynamic_safety_stock} \cdot 1.3), & \text{inventory_ratio} < 0.3 \\ \text{dynamic_safety_stock}, & \text{otherwise} \end{cases}$$

$$\text{cost_ratio} = \frac{(1.3 \cdot 1.5)}{0.9}$$

$$\text{cost_balanced_level} = (\text{dynamic_safety_stock} \cdot (1.0 + (0.2 \cdot (\text{cost_ratio} - 1.0))))$$

$$\text{safety_buffer} = 0.0$$

$$\text{safety_buffer} = \begin{cases} (0.1 \cdot (\text{dynamic_safety_stock} - \text{inventory_position})), & \text{inventory_position} < (\text{dynamic_safety_stock} \cdot 0.5) \\ \text{safety_buffer}, & \text{otherwise} \end{cases}$$

$$\text{cost_balanced_level} = (\text{cost_balanced_level} + \text{safety_buffer})$$

$$\text{raw_order} = \max(0.0, (\text{cost_balanced_level} - \text{inventory_position}))$$

$$\text{urgency_adjusted_smoothing} = (0.1 \cdot (1.0 - (\text{pipeline_urgency} \cdot 0.5)))$$

$$\text{smoothed_order} = (\text{raw_order} \cdot \text{urgency_adjusted_smoothing})$$

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, q) = \text{round}_{\text{ave}} \left(\max \left(0, \left(\max \left(0, \left((294.0 + 20.0) + \min \left(50, \max \left(-(50), \left(\frac{\sum_{i=1}^{L-1} q_i}{L} \right)^2, L > 0 \right) \cdot 0.1 \right) \right) \right) - \left(h + \sum_{i=1}^L q_i \right) \right) \cdot 0.7 + \left(\frac{\sum_{i=1}^{L-1} q_i}{\max(1, L)} \cdot 0.9 \cdot 0.3 \right) \right) \right).$$

Python source lines 1-15

```
1. # sample_id: 067
2. # folder: deepseek-chat_exponential_L6_c1_5_50_plain_processed_scipy_15_default_m2plural_2_r6
3. # distribution: exponential_L6_c1_5
4. # generation: 0
5. # rank_in_population_file: 1
6. # objective: 13069.62249
7. # test_objective: 13284.31768
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs:
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 553.9875397341209 # OPT_PARAM: {"initial": 553.9875397341209, "min": 10, "max":
14.     1000, "type": "float"}
15.     order_amount = max(0, base_stock - on_hand_inventory - sum(pipeline_orders))
16.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(554.0 - h - \sum_{i=0}^{L-1} q_i \right) \right).$$

Python source lines 1-40

```
1. # sample_id: 068
2. # folder: deepseek-chat_poisson_L4_c1_5_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: poisson_L4_c1_5
4. # generation: 2
5. # rank_in_population_file: 1
6. # objective: 1940.21194
7. # test_objective: 1965.16912
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 537.5676918637181 # OPT_PARAM: {"initial": 537.5676918637181, "min": 400, "max":
700, "type": "float"}
14.     safety_stock = 35.0 # OPT_PARAM: {"initial": 35.0, "min": 20, "max": 80, "type": "float"}
15.     demand_estimate = 98.5 # OPT_PARAM: {"initial": 98.5, "min": 85, "max": 115, "type": "float"}
16.     adjustment_factor = 0.4 # OPT_PARAM: {"initial": 0.4, "min": 0.4, "max": 0.9, "type": "float"}
17.     pipeline_weight = 0.5351670816616867 # OPT_PARAM: {"initial": 0.5351670816616867, "min": 0.5,
"max": 1.0, "type": "float"}
18.
19.     # Calculate inventory position with weighted pipeline
20.     weighted_pipeline = sum(p * pipeline_weight**((len(pipeline_orders)-i-1)
21.                                     for i, p in enumerate(pipeline_orders))
22.                               inventory_position = on_hand_inventory + weighted_pipeline
23.
24.     # Calculate expected demand during lead time
25.     expected_lead_time_demand = demand_estimate * len(pipeline_orders)
26.
27.     # Calculate target inventory position
28.     target_position = expected_lead_time_demand + safety_stock
29.
30.     # Use the maximum of base_stock and target_position
31.     order_up_to = max(base_stock, target_position)
32.
33.     # Calculate order amount
34.     order_amount = max(0, order_up_to - inventory_position)
35.
36.     # Apply adjustment factor
37.     order_amount = order_amount * adjustment_factor
38.
39.     # Round to nearest integer (practical ordering)
40.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \left(\max \left(0, \left(\max(537.6, ((98.5 \cdot L) + 35.0)) - \left(h + \sum_{i=0}^{L-1} (q_i \cdot (0.5)^{((L-i)-1)}) \right) \right) \right) \right) \cdot 0.4.$$

Python source lines 1-40

```

1. # sample_id: 069
2. # folder: deepseek-chat_exponential_L6_c1_2_50_plain_processed_scipy_15_default_e1-e2_6_r2
3. # distribution: exponential_L6_c1_2
4. # generation: 1
5. # rank_in_population_file: 2
6. # objective: 6376.28
7. # test_objective: 6445.733
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;order_up_to;partial_adjustment;order_smoothing
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding;emergency_or_sh
    ortage_boost
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level - fundamental parameter
14.     base_stock = 326.0462767090309 # OPT_PARAM: {"initial": 326.0462767090309, "min": 100, "max":
    800, "type": "float"}
15.
16.     # Safety stock multiplier - adjusts for demand variability
17.     safety_multiplier = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 3.0, "type": "float"}
18.
19.     # Pipeline weighting factor - gives more weight to imminent arrivals
20.     pipeline_weight = 1.044462251668397 # OPT_PARAM: {"initial": 1.044462251668397, "min": 0.1,
    "max": 1.5, "type": "float"}
21.
22.     # Emergency threshold for aggressive ordering
23.     emergency_threshold = 150.0 # OPT_PARAM: {"initial": 150.0, "min": 50, "max": 300, "type":
    "float"}
24.
25.     # Emergency boost multiplier
26.     emergency_boost = 1.1712478055383213 # OPT_PARAM: {"initial": 1.1712478055383213, "min": 1.1,
    "max": 2.0, "type": "float"}
27.
28.     # Calculate weighted pipeline inventory
29.     # More weight to orders arriving soon, less to distant ones
30.     weighted_pipeline = 0
31.     for i, order in enumerate(pipeline_orders):
32.         weight = pipeline_weight ** (i + 1) # Exponential decay
33.         weighted_pipeline += order * weight
34.
35.     # Calculate effective inventory position
36.     effective_inventory = on_hand_inventory + weighted_pipeline
37.
38.     # Calculate demand coverage period (how many periods of average demand we want to cover)
39.     coverage_periods = 1.1001181735173573 # OPT_PARAM: {"initial": 1.1001181735173573, "min": 1.0,
    "max": 5.0, "type": "float"}
40.

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (1.0)^{(i+1)}))$$

$$\text{effective_inventory} = (h + \text{weighted_pipeline})$$

$$\text{avg_estimated_demand} = \begin{cases} 99.1, & \sum_{i=0}^{L-1} q_i > 0 \\ 100.0, & \text{otherwise} \end{cases}$$

$$\text{safety_stock} = ((0.5 \cdot \text{avg_estimated_demand}) \cdot 1.1)$$

$$\text{target_inventory} = (326.0 + \text{safety_stock})$$

$$\text{target_inventory} = \begin{cases} (\text{target_inventory} \cdot 1.2), & \text{effective_inventory} < 150.0 \\ \text{target_inventory}, & \text{otherwise} \end{cases}$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{effective_inventory}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{previous_order})) \quad \left. \begin{matrix} \text{previous_order} = q_{-(1)} \\ \text{smoothed_order} = \text{raw_order} \end{matrix} \right\} \quad (L > 0)$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L > 0))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 41-74

```

41.     # Estimate average demand from pipeline pattern (assuming recent orders reflect demand)
42.     if sum(pipeline_orders) > 0:
43.         avg_estimated_demand = 99.10971369816312 # Optimized
44.     else:
45.         avg_estimated_demand = 100.0 # OPT_PARAM: {"initial": 100.0, "min": 50.0, "max": 300.0,
"type": "float"}
46.
47.     # Calculate safety stock based on coverage and variability
48.     safety_stock = safety_multiplier * avg_estimated_demand * coverage_periods
49.
50.     # Calculate target inventory level
51.     target_inventory = base_stock + safety_stock
52.
53.     # Apply emergency boost if effective inventory is critically low
54.     if effective_inventory < emergency_threshold:
55.         target_inventory *= emergency_boost
56.
57.     # Calculate order amount with smoothing
58.     raw_order = max(0, target_inventory - effective_inventory)
59.
60.     # Order smoothing factor - prevents large order swings
61.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 1.0, "type": "float"}
62.
63.     # Apply smoothing: blend with previous order if available
64.     # Since we can't store state, we estimate previous order from pipeline
65.     if len(pipeline_orders) > 0:
66.         previous_order = pipeline_orders[-1] # Most recent order placed
67.         smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * previous_order
68.     else:
69.         smoothed_order = raw_order
70.
71.     # Round to nearest integer (realistic ordering)
72.     order_amount = int(round(max(0, smoothed_order)))
73.
74.     return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_i \cdot (1.0)^{(i+1)}))$$

$$\text{effective_inventory} = (h + \text{weighted_pipeline})$$

$$\text{avg_estimated_demand} = \begin{cases} 99.1, & \sum_{i=0}^{L-1} q_i > 0 \\ 100.0, & \text{otherwise} \end{cases}$$

$$\text{safety_stock} = ((0.5 \cdot \text{avg_estimated_demand}) \cdot 1.1)$$

$$\text{target_inventory} = (326.0 + \text{safety_stock})$$

$$\text{target_inventory} = \begin{cases} (\text{target_inventory} \cdot 1.2), & \text{effective_inventory} < 150.0 \\ \text{target_inventory}, & \text{otherwise} \end{cases}$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{effective_inventory}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{previous_order})) \quad (L > 0)$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L > 0))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-34

```
1. # sample_id: 070
2. # folder: deepseek-chat_exponential_L4_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: exponential_L4_c1_5
4. # generation: 2
5. # rank_in_population_file: 2
6. # objective: 11182.29596
7. # test_objective: 11335.45953
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 427.05501776950115 # OPT_PARAM: {"initial": 427.05501776950115, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 34.79592594200072 # OPT_PARAM: {"initial": 34.79592594200072, "min": 0, "max":
16.     300, "type": "float"}
17.     demand_estimate = 84.78492538308028 # OPT_PARAM: {"initial": 84.78492538308028, "min": 50,
18.     "max": 400, "type": "float"}
19.     adjustment_factor = 0.25047133377839226 # OPT_PARAM: {"initial": 0.25047133377839226, "min":
20.     0.1, "max": 1.5, "type": "float"}
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected shortfall considering pipeline
26.     expected_shortfall = max(0, base_stock - inventory_position)
27.
28.     # Adjust for demand variability with safety stock
29.     adjusted_shortfall = expected_shortfall * adjustment_factor + safety_stock
30.
31.     # Consider upcoming demand in pipeline
32.     upcoming_demand_estimate = demand_estimate * len(pipeline_orders)
33.     pipeline_adjustment = max(0, upcoming_demand_estimate - sum(pipeline_orders)) * 0.3
34.
35.     # Final order amount
36.     order_amount = max(0, adjusted_shortfall + pipeline_adjustment)
37.
38.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(\left(\left(\max \left(0, 427.1 - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \cdot 0.3 \right) + 34.8 \right) + \left(\max \left(0, \left(84.8 \cdot L - \sum_{i=0}^{L-1} q_i \right) \cdot 0.3 \right) \right) \right) \right).$$

Python source lines 1-41

```

1. # sample_id: 071
2. # folder: deepseek-chat_exponential_L2_c1_5_50_plain_processed_scipy_15_default_m2_10_r2
3. # distribution: exponential_L2_c1_5
4. # generation: 9
5. # rank_in_population_file: 9
6. # objective: 10168.88
7. # test_objective: 10598.171
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 284.2684349440411 # OPT_PARAM: {"initial": 284.2684349440411, "min": 200, "max":
14.     450, "type": "float"}
15.     safety_factor = 1.7620210737068769 # OPT_PARAM: {"initial": 1.7620210737068769, "min": 1.0,
16.     "max": 3.0, "type": "float"}
17.     smoothing_factor = 0.37959881581168164 # OPT_PARAM: {"initial": 0.37959881581168164, "min": 0.3,
18.     "max": 0.8, "type": "float"}
19.     pipeline_adjust = 0.35 # OPT_PARAM: {"initial": 0.35, "min": 0.1, "max": 0.6, "type": "float"}
20.     lost_sales_multiplier = 0.86505310919691 # OPT_PARAM: {"initial": 0.86505310919691, "min": 0.8,
21.     "max": 2.0, "type": "float"}
22.
23.     # Calculate inventory position
24.     inventory_position = on_hand_inventory + sum(pipeline_orders)
25.
26.     # Estimate demand from recent pipeline arrivals (weighted average)
27.     if len(pipeline_orders) >= 2:
28.         recent_demand_estimate = (pipeline_orders[0] * 0.7 + pipeline_orders[1] * 0.3)
29.     else:
30.         recent_demand_estimate = 0
31.
32.     # Dynamic safety stock based on demand variability estimate
33.     safety_stock = base_stock * safety_factor * (1 + pipeline_adjust * (recent_demand_estimate /
34.     base_stock))
35.
36.     # Adjust target based on lost sales penalty
37.     target_inventory = safety_stock * lost_sales_multiplier
38.
39.     # Calculate order gap
40.     gap = target_inventory - inventory_position
41.
42.     # Apply smoothing to avoid large order swings
43.     if gap > 0:
44.         order_amount = gap * smoothing_factor
45.     else:
46.         order_amount = 0

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{recent_demand_estimate} = \begin{cases} ((q_0 \cdot 0.7) + (q_1 \cdot 0.3)), & L \geq 2 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{safety_stock} = \left((284.3 \cdot 1.8) \cdot \left(1 + \left(0.3 \cdot \frac{\text{recent_demand_estimate}}{284.3} \right) \right) \right)$$

$$\text{target_inventory} = (\text{safety_stock} \cdot 0.9)$$

$$\text{gap} = (\text{target_inventory} - \text{inventory_position})$$

$$\text{order_amount} = \begin{cases} (\text{gap} \cdot 0.4), & \text{gap} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 42-46

```
42.  
43.     # Round to nearest integer and ensure non-negative  
44.     order_amount = max(0, int(round(order_amount)))  
45.  
46.     return order_amount
```

Declarative mathematical formula rows 1-9

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{recent_demand_estimate} = \begin{cases} ((q_0 \cdot 0.7) + (q_1 \cdot 0.3)), & L \geq 2 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{safety_stock} = \left((284.3 \cdot 1.8) \cdot \left(1 + \left(0.3 \cdot \frac{\text{recent_demand_estimate}}{284.3} \right) \right) \right)$$

$$\text{target_inventory} = (\text{safety_stock} \cdot 0.9)$$

$$\text{gap} = (\text{target_inventory} - \text{inventory_position})$$

$$\text{order_amount} = \begin{cases} (\text{gap} \cdot 0.4), & \text{gap} > 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-43

```

1. # sample_id: 072
2. # folder: deepseek-chat_poisson_L4_c1_2_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: poisson_L4_c1_2
4. # generation: 6
5. # rank_in_population_file: 9
6. # objective: 1009.1
7. # test_objective: 1014.849
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 441.87238367408924 # OPT_PARAM: {"initial": 441.87238367408924, "min": 300, "max":
14.     600, "type": "float"}
15.     safety_stock = 20.003514553741276 # OPT_PARAM: {"initial": 20.003514553741276, "min": 10, "max":
16.     80, "type": "float"}
17.     demand_estimate = 93.33801940498932 # OPT_PARAM: {"initial": 93.33801940498932, "min": 80,
18.     "max": 120, "type": "float"}
19.     smoothing_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.9, "type": "float"}
20.     order_threshold = 10.1 # OPT_PARAM: {"initial": 10.1, "min": 5, "max": 30, "type": "float"}
21.     lead_time = len(pipeline_orders)
22.
23.     # Calculate inventory position
24.     inventory_position = on_hand_inventory + sum(pipeline_orders)
25.
26.     # Calculate expected demand over lead time plus one period
27.     expected_lead_time_demand = demand_estimate * (lead_time + 1)
28.
29.     # Calculate target inventory position
30.     target_position = base_stock + safety_stock
31.
32.     # Calculate order amount using base-stock policy
33.     order_amount = max(0, target_position - inventory_position)
34.
35.     # Apply smoothing only when order is significant
36.     if order_amount > demand_estimate * 0.5:
37.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * demand_estimate
38.
39.     # Apply order threshold
40.     if order_amount < order_threshold:
41.         order_amount = 0
42.
43.     # Round to nearest integer
44.     order_amount = int(round(order_amount))
45.
46.     return order_amount

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (93.3 \cdot (\text{lead_time} + 1))$$

$$\text{target_position} = (441.9 + 20.0)$$

$$\text{order_amount} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{order_amount} = \begin{cases} ((0.1 \cdot \text{order_amount}) + ((1 - 0.1) \cdot 93.3)), & \text{order_amount} > (93.3 \cdot 0.5) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \begin{cases} 0, & \text{order_amount} < 10.1 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-41

```

1. # sample_id: 073
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-e1-e1_4_r4
3. # distribution: normal_std30_L6_c1_2
4. # generation: 5
5. # rank_in_population_file: 4
6. # objective: 2753.3
7. # test_objective: 2740.264
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;partial_adjustment;order_clipping
11. # extra_motifs:
    threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Dual-threshold policy with adaptive risk adjustment
14.     # This policy uses two distinct thresholds: one for normal operation and one for emergency
15.     # Risk level adapts based on pipeline volatility and recent inventory coverage
16.
17.     # Base parameters
18.     normal_threshold = 800.0 # OPT_PARAM: {"initial": 800.0, "min": 200, "max": 800, "type":
    "float"}
19.     emergency_threshold = 229.69832788330328 # OPT_PARAM: {"initial": 229.69832788330328, "min": 50,
    "max": 300, "type": "float"}
20.     volatility_sensitivity = 0.9446073433321022 # OPT_PARAM: {"initial": 0.9446073433321022, "min":
    0.5, "max": 3.0, "type": "float"}
21.     coverage_weight = 0.14167033675094412 # OPT_PARAM: {"initial": 0.14167033675094412, "min": 0.1,
    "max": 1.5, "type": "float"}
22.     order_aggressiveness = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.3, "max": 1.2, "type":
    "float"}
23.     min_order_size = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0, "max": 50, "type": "float"}
24.
25.     L = len(pipeline_orders)
26.     if L == 0:
27.         return 0
28.
29.     # Calculate inventory position
30.     inventory_position = on_hand_inventory + sum(pipeline_orders)
31.
32.     # Calculate pipeline volatility (new structural element)
33.     if L > 1:
34.         pipeline_mean = sum(pipeline_orders) / L
35.         variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
36.         pipeline_volatility = variance ** 0.5
37.     else:
38.         pipeline_volatility = 0.0
39.
40.     # Calculate coverage ratio (new structural element)
41.     # How many periods of average pipeline flow can current inventory cover?

```

Declarative mathematical formula rows 1-16

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$Q(h, \mathbf{q}) = 0 \quad (L = 0)$$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ \text{pipeline_volatility} &= (\text{variance})^{0.5} \end{aligned} \right\} \quad (L > 1)$$

$$\text{pipeline_volatility} = 0.0 \quad (\neg(L > 1))$$

$$\text{coverage_ratio} = \begin{cases} \frac{h}{\text{pipeline_mean}}, & \text{pipeline_mean} > 0 \\ \text{float}(\text{inf}), & \text{otherwise} \end{cases}$$

$$\text{volatility_risk} = \begin{cases} \min\left(1.0, \frac{\text{pipeline_volatility}}{100.0}\right), & \text{pipeline_volatility} > 0 \\ 0.0, & \text{otherwise} \end{cases}$$

$$\text{coverage_risk} = \max\left(0.0, \left(1.0 - \min\left(1.0, \frac{\text{coverage_ratio}}{3.0}\right)\right)\right)$$

$$\text{combined_risk} = \frac{((\text{volatility_risk} \cdot 0.9) + (\text{coverage_risk} \cdot 0.1))}{2.0}$$

$$\text{risk_level} = \min(1.0, \max(0.0, \text{combined_risk}))$$

$$\text{risk_adjustment} = (1.0 - \text{risk_level})$$

$$\text{adjusted_threshold} = (229.7 + ((800.0 - 229.7) \cdot \text{risk_adjustment}))$$

$$\text{gap} = (\text{adjusted_threshold} - \text{inventory_position})$$

$$\left. \begin{aligned} \text{order_amount} &= (\text{gap} \cdot 0.3) \\ \text{order_amount} &= \max(0, \text{int}((\text{order_amount} + 0.5))) \end{aligned} \right\} \quad (\text{gap} > 0.0)$$

$$\text{order_amount} = 0 \quad (\neg(\text{gap} > 0.0))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 42-70

```

42.     if pipeline_mean > 0:
43.         coverage_ratio = on_hand_inventory / pipeline_mean
44.     else:
45.         coverage_ratio = float('inf')
46.
47.     # Determine risk level based on volatility and coverage (new structural element)
48.     # Higher volatility increases risk, lower coverage increases risk
49.     volatility_risk = min(1.0, pipeline_volatility / 100.0) if pipeline_volatility > 0 else 0.0
50.     coverage_risk = max(0.0, 1.0 - min(1.0, coverage_ratio / 3.0))
51.
52.     combined_risk = (volatility_risk * volatility_sensitivity + coverage_risk * coverage_weight) /
53. 2.0
54.     risk_level = min(1.0, max(0.0, combined_risk))
55.
56.     # Adjust threshold based on risk level (new structural element)
57.     # Higher risk → lower threshold to be more conservative
58.     risk_adjustment = 1.0 - risk_level
59.     adjusted_threshold = emergency_threshold + (normal_threshold - emergency_threshold) *
60. risk_adjustment
61.
62.     # Calculate order gap
63.     gap = adjusted_threshold - inventory_position
64.
65.     # Apply order aggressiveness
66.     if gap > min_order_size:
67.         order_amount = gap * order_aggressiveness
68.         order_amount = max(0, int(order_amount + 0.5))
69.     else:
70.         order_amount = 0
71.
72.     return order_amount

```

Declarative mathematical formula rows 1-16

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$Q(h, \mathbf{q}) = 0 \quad (L = 0)$$

$$\left. \begin{aligned} \text{inventory_position} &= \left(h + \sum_{i=0}^{L-1} q_i \right) \\ \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean})^2)}{L} \\ \text{pipeline_volatility} &= (\text{variance})^{0.5} \end{aligned} \right\} \quad (L > 1)$$

$$\text{pipeline_volatility} = 0.0 \quad (\neg (L > 1))$$

$$\text{coverage_ratio} = \begin{cases} \frac{h}{\text{pipeline_mean}}, & \text{pipeline_mean} > 0 \\ \text{float}(\text{inf}), & \text{otherwise} \end{cases}$$

$$\text{volatility_risk} = \begin{cases} \min\left(1.0, \frac{\text{pipeline_volatility}}{100.0}\right), & \text{pipeline_volatility} > 0 \\ 0.0, & \text{otherwise} \end{cases}$$

$$\text{coverage_risk} = \max\left(0.0, \left(1.0 - \min\left(1.0, \frac{\text{coverage_ratio}}{3.0}\right)\right)\right)$$

$$\text{combined_risk} = \frac{((\text{volatility_risk} \cdot 0.9) + (\text{coverage_risk} \cdot 0.1))}{2.0}$$

$$\text{risk_level} = \min(1.0, \max(0.0, \text{combined_risk}))$$

$$\text{risk_adjustment} = (1.0 - \text{risk_level})$$

$$\text{adjusted_threshold} = (229.7 + ((800.0 - 229.7) \cdot \text{risk_adjustment}))$$

$$\text{gap} = (\text{adjusted_threshold} - \text{inventory_position})$$

$$\left. \begin{aligned} \text{order_amount} &= (\text{gap} \cdot 0.3) \\ \text{order_amount} &= \max(0, \text{int}((\text{order_amount} + 0.5))) \end{aligned} \right\} \quad (\text{gap} > 0.0)$$

$$\text{order_amount} = 0 \quad (\neg (\text{gap} > 0.0))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-38

```

1. # sample_id: 074
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_2_r7
3. # distribution: poisson_L6_c1_5
4. # generation: 10
5. # rank_in_population_file: 1
6. # objective: 3074.22
7. # test_objective: 3077.277
8. # is_top10_by_distribution: False
9. # is_final_generation: True
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 650.0 # OPT_PARAM: {"initial": 650.0, "min": 400, "max": 900, "type": "float"}
14.     safety_stock = 155.24240621033815 # OPT_PARAM: {"initial": 155.24240621033815, "min": 100,
15.     "max": 250, "type": "float"}
16.     demand_forecast = 105.69007241806933 # OPT_PARAM: {"initial": 105.69007241806933, "min": 80,
17.     "max": 120, "type": "float"}
18.     smoothing_factor = 0.4915613763933389 # OPT_PARAM: {"initial": 0.4915613763933389, "min": 0.0,
19.     "max": 1.0, "type": "float"}
20.     lead_time = len(pipeline_orders)
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected demand during lead time plus one period
26.     expected_lead_time_demand = demand_forecast * (lead_time + 1)
27.
28.     # Calculate target inventory level
29.     target_inventory = expected_lead_time_demand + safety_stock
30.
31.     # Use the maximum of base_stock and target_inventory as order-up-to level
32.     order_up_to = max(base_stock, target_inventory)
33.
34.     # Calculate order amount
35.     order_amount = max(0, order_up_to - inventory_position)
36.
37.     # Apply smoothing only when order amount is positive
38.     if order_amount > 0:
39.         order_amount = int(order_amount * smoothing_factor + 0.5)
40.
41.     return order_amount

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

expected_lead_time_demand = $(105.7 \cdot (\text{lead_time} + 1))$

target_inventory = $(\text{expected_lead_time_demand} + 155.2)$

order_up_to = $\max(650.0, \text{target_inventory})$

order_amount = $\max(0, (\text{order_up_to} - \text{inventory_position}))$

order_amount = $\begin{cases} \text{int}(((\text{order_amount} \cdot 0.5) + 0.5)), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-41

```

1. # sample_id: 075
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_4_r7
3. # distribution: poisson_L6_c1_2
4. # generation: 9
5. # rank_in_population_file: 9
6. # objective: 972.22
7. # test_objective: 951.996
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target;partial_a
    djustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 530.8053146215685 # OPT_PARAM: {"initial": 530.8053146215685, "min": 450, "max":
    600, "type": "float"}
14.     safety_stock = 52.00531462156949 # OPT_PARAM: {"initial": 52.00531462156949, "min": 20, "max":
    80, "type": "float"}
15.     demand_forecast = 94.67244908223753 # OPT_PARAM: {"initial": 94.67244908223753, "min": 90,
    "max": 110, "type": "float"}
16.     smoothing_factor = 0.05 # OPT_PARAM: {"initial": 0.05, "min": 0.05, "max": 0.3, "type": "float"}
17.     pipeline_weight = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.0, "type": "float"}
18.     lost_sales_weight = 1.1 # OPT_PARAM: {"initial": 1.1, "min": 1.0, "max": 1.5, "type": "float"}
19.
20.     # Calculate effective pipeline (weighted average)
21.     weighted_pipeline = 0
22.     total_weight = 0
23.     for i, q in enumerate(pipeline_orders):
24.         weight = pipeline_weight ** (len(pipeline_orders) - i - 1)
25.         weighted_pipeline += q * weight
26.         total_weight += weight
27.
28.     if total_weight > 0:
29.         effective_pipeline = weighted_pipeline / total_weight * len(pipeline_orders)
30.     else:
31.         effective_pipeline = sum(pipeline_orders)
32.
33.     # Calculate net inventory position
34.     net_inventory = on_hand_inventory + effective_pipeline
35.
36.     # Adjust safety stock based on pipeline coverage
37.     pipeline_coverage = sum(pipeline_orders) / (demand_forecast * len(pipeline_orders)) if
    demand_forecast > 0 else 1.0
    adjusted_safety = safety_stock * (1.0 + 0.3 * (1.0 - min(1.0, pipeline_coverage)))
38.
39.
40.     # Calculate target with lost-sales bias
41.     target_inventory = base_stock + adjusted_safety * lost_sales_weight

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} \left((q_i \cdot (0.5)^{(L-i)-1}) \right)$$

$$\text{total_weight} = \sum_{i=0}^{L-1} \left((0.5)^{(L-i)-1} \right)$$

$$\text{effective_pipeline} = \begin{cases} \left(\frac{\text{weighted_pipeline}}{\text{total_weight}} \cdot L \right), & \text{total_weight} > 0 \\ \sum_{i=0}^{L-1} q_i, & \text{otherwise} \end{cases}$$

$$\text{net_inventory} = (h + \text{effective_pipeline})$$

$$\text{pipeline_coverage} = \begin{cases} \frac{\sum_{i=0}^{L-1} q_i}{(94.7 \cdot L)}, & 94.7 > 0 \\ 1.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_safety} = (52.0 \cdot (1.0 + (0.3 \cdot (1.0 - \min(1.0, \text{pipeline_coverage}))))))$$

$$\text{target_inventory} = (530.8 + (\text{adjusted_safety} \cdot 1.1))$$

$$\text{order_up_to} = \max(0, (\text{target_inventory} - \text{net_inventory}))$$

$$\text{smoothed_order} = ((0.1 \cdot \text{order_up_to}) + ((1 - 0.1) \cdot 94.7))$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 42-52

```
42.
43.     # Calculate order-up-to level
44.     order_up_to = max(0, target_inventory - net_inventory)
45.
46.     # Apply smoothing
47.     smoothed_order = smoothing_factor * order_up_to + (1 - smoothing_factor) * demand_forecast
48.
49.     # Round to nearest integer
50.     order_amount = max(0, int(round(smoothed_order)))
51.
52.     return order_amount
```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} \left((q_i \cdot (0.5)^{(L-i)-1}) \right)$$
$$\text{total_weight} = \sum_{i=0}^{L-1} \left((0.5)^{(L-i)-1} \right)$$
$$\text{effective_pipeline} = \begin{cases} \left(\frac{\text{weighted_pipeline}}{\text{total_weight}} \cdot L \right), & \text{total_weight} > 0 \\ \sum_{i=0}^{L-1} q_i, & \text{otherwise} \end{cases}$$
$$\text{net_inventory} = (h + \text{effective_pipeline})$$
$$\text{pipeline_coverage} = \begin{cases} \frac{\sum_{i=0}^{L-1} q_i}{(94.7 \cdot L)}, & 94.7 > 0 \\ 1.0, & \text{otherwise} \end{cases}$$
$$\text{adjusted_safety} = (52.0 \cdot (1.0 + (0.3 \cdot (1.0 - \min(1.0, \text{pipeline_coverage}))))))$$
$$\text{target_inventory} = (530.8 + (\text{adjusted_safety} \cdot 1.1))$$
$$\text{order_up_to} = \max(0, (\text{target_inventory} - \text{net_inventory}))$$
$$\text{smoothed_order} = ((0.1 \cdot \text{order_up_to}) + ((1 - 0.1) \cdot 94.7))$$
$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-42

```

1. # sample_id: 076
2. # folder: deepseek-chat_poisson_L2_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: poisson_L2_c1_5
4. # generation: 8
5. # rank_in_population_file: 7
6. # objective: 1116.9
7. # test_objective: 1093.328
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 310.17160011102334 # OPT_PARAM: {"initial": 310.17160011102334, "min": 250, "max":
320, "type": "float"}
14.     safety_stock = 63.44756030622884 # OPT_PARAM: {"initial": 63.44756030622884, "min": 30, "max":
70, "type": "float"}
15.     adjustment_factor = 0.7 # OPT_PARAM: {"initial": 0.7, "min": 0.6, "max": 1.0, "type": "float"}
16.     demand_estimate = 100.01298553345889 # OPT_PARAM: {"initial": 100.01298553345889, "min": 95,
"max": 105, "type": "float"}
17.     pipeline_weight = 0.2 # OPT_PARAM: {"initial": 0.2, "min": 0.2, "max": 0.4, "type": "float"}
18.     min_order_multiplier = 0.2 # OPT_PARAM: {"initial": 0.2, "min": 0.1, "max": 0.3, "type":
"float"}
19.     max_order_multiplier = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 1.0, "max": 2.0, "type":
"float"}
20.
21.     # Calculate inventory position
22.     inventory_position = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Calculate expected pipeline contribution
25.     expected_pipeline = sum(pipeline_orders) * pipeline_weight
26.
27.     # Target inventory level
28.     target_inventory = base_stock + safety_stock - expected_pipeline
29.
30.     # Calculate base order amount
31.     order_amount = max(0, (target_inventory - inventory_position) * adjustment_factor)
32.
33.     # Apply order limits
34.     max_order = demand_estimate * max_order_multiplier
35.     min_order = demand_estimate * min_order_multiplier
36.
37.     if order_amount > 0:
38.         # Ensure reasonable order sizes
39.         order_amount = max(min_order, min(order_amount, max_order))
40.
41.     # Round to nearest integer
42.     order_amount = int(round(order_amount))

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_pipeline} = \left(\sum_{i=0}^{L-1} q_i \cdot 0.2 \right)$$

$$\text{target_inventory} = ((310.2 + 63.4) - \text{expected_pipeline})$$

$$\text{order_amount} = \max(0, ((\text{target_inventory} - \text{inventory_position}) \cdot 0.7))$$

$$\text{max_order} = (100.0 \cdot 1.0)$$

$$\text{min_order} = (100.0 \cdot 0.2)$$

$$\text{order_amount} = \begin{cases} \max(\text{min_order}, \min(\text{order_amount}, \text{max_order})), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 43-44

```
43.  
44.     return order_amount
```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$
$$\text{expected_pipeline} = \left(\sum_{i=0}^{L-1} q_i \cdot 0.2 \right)$$
$$\text{target_inventory} = ((310.2 + 63.4) - \text{expected_pipeline})$$
$$\text{order_amount} = \max(0, ((\text{target_inventory} - \text{inventory_position}) \cdot 0.7))$$
$$\text{max_order} = (100.0 \cdot 1.0)$$
$$\text{min_order} = (100.0 \cdot 0.2)$$
$$\text{order_amount} = \begin{cases} \max(\text{min_order}, \min(\text{order_amount}, \text{max_order})), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$
$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-37

```

1. # sample_id: 077
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 12
5. # rank_in_population_file: 8
6. # objective: 2243.71088
7. # test_objective: 2236.31464
8. # is_top10_by_distribution: True
9. # is_final_generation: True
10. # table_motifs: pipeline_weighting;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;emergen
    cy_or_shortage_boost
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Fixed constants from dataset analysis
14.     mean_demand = 100.0
15.     L = len(pipeline_orders)
16.
17.     # Base stock level - tuned for lead time demand coverage
18.     base_stock = 566.185973326278 # OPT_PARAM: {"initial": 566.185973326278, "min": 400, "max": 800,
    "type": "float"}
19.
20.     # Pipeline discount factors - more aggressive discounting for later arrivals
21.     early_discount = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.8, "max": 1.0, "type": "float"}
22.     mid_discount = 0.9 # OPT_PARAM: {"initial": 0.9, "min": 0.5, "max": 0.9, "type": "float"}
23.     late_discount = 0.6541439085216577 # OPT_PARAM: {"initial": 0.6541439085216577, "min": 0.2,
    "max": 0.7, "type": "float"}
24.
25.     # Order smoothing parameters
26.     min_order = 9.863642334015735 # OPT_PARAM: {"initial": 9.863642334015735, "min": 5, "max": 30,
    "type": "float"}
27.     max_order = 87.72136344324525 # OPT_PARAM: {"initial": 87.72136344324525, "min": 80, "max": 200,
    "type": "float"}
28.
29.     # Aggressiveness adjustment thresholds
30.     low_stock_threshold = 50.0 # OPT_PARAM: {"initial": 50.0, "min": 30, "max": 100, "type":
    "float"}
31.     high_stock_threshold = 199.9908165611701 # OPT_PARAM: {"initial": 199.9908165611701, "min": 100,
    "max": 250, "type": "float"}
32.
33.     # Aggressiveness multipliers
34.     low_stock_multiplier = 1.25 # OPT_PARAM: {"initial": 1.25, "min": 1.1, "max": 1.5, "type":
    "float"}
35.     high_stock_multiplier = 0.8690541317887209 # OPT_PARAM: {"initial": 0.8690541317887209, "min":
    0.5, "max": 0.9, "type": "float"}
36.
37.     # Safety stock buffer for demand variability

```

Declarative mathematical formula rows 1-19

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{mean_demand} = 100.0$

$$\text{discounted_pipeline} = \sum_{i=0}^{L-1} (1_{\{i < 2\}} ((q_i \cdot 1.0)) + 1_{\{(\neg(i < 2)) \wedge (i < 4)\}} ((q_i \cdot 0.9)) + 1_{\{(\neg(i < 2)) \wedge (\neg(i < 4))\}} ((q_i \cdot 0.7)))$$

$$\text{pipeline_roughness} = \sum_{i=0}^{(L-1)-1} (|(q_i - q_{(i+1)})|)$$

$$\text{discounted_pipeline} = \begin{cases} (\text{discounted_pipeline} \cdot 0.9), & \text{pipeline_roughness} > 100 \\ \text{discounted_pipeline}, & \text{otherwise} \end{cases}$$

$\text{dynamic_safety} = 66.4$

$$\text{dynamic_safety} = \begin{cases} (\text{dynamic_safety} \cdot (1.0 + (0.9 \cdot \frac{\text{pipeline_roughness}}{500.0}))), & \text{pipeline_roughness} > 50 \\ \text{dynamic_safety}, & \text{otherwise} \end{cases}$$

$\text{adjusted_base_stock} = (566.2 \cdot 1.0)$

$\text{effective_ip} = (h + \text{discounted_pipeline})$

$\text{target_level} = (\text{adjusted_base_stock} + \text{dynamic_safety})$

$\text{immediate_coverage} = (h + q_0)$

$$\text{target_level} = \begin{cases} (\text{target_level} + 25.0), & \text{immediate_coverage} < \text{mean_demand} \\ \text{target_level}, & \text{otherwise} \end{cases}$$

$\text{order_amount} = \max(0.0, (\text{target_level} - \text{effective_ip}))$

$\text{availability} = (h + q_0)$

$\text{order_amount} = (\text{order_amount} \cdot 1.5) \quad (\text{availability} < 30.0)$

$\text{order_amount} = (\text{order_amount} \cdot 1.2) \quad (\text{availability} < 50.0)$

$$\text{order_amount} = \begin{cases} (\text{order_amount} \cdot 0.9), & \text{availability} > 200.0 \\ \text{order_amount}, & \text{otherwise} \end{cases} \quad (\neg(\text{availability} < 50.0)) \quad \left. \vphantom{\begin{cases} (\text{order_amount} \cdot 0.9), \\ \text{order_amount}, \end{cases}} \right\} \quad (\neg(\text{availability} < 30.0))$$

$$\text{order_amount} = \begin{cases} 9.9, & (\text{order_amount} > 0 \wedge \text{order_amount} < 9.9) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$\text{order_amount} = \min(\text{order_amount}, 87.7)$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 38-79

```

38.     safety_stock = 66.36535573531937 # OPT_PARAM: {"initial": 66.36535573531937, "min": 20, "max":
    150, "type": "float"}
39.
40.     # Pipeline shape penalty for uneven distribution
41.     pipeline_roughness_penalty = 0.8569745712739317 # OPT_PARAM: {"initial": 0.8569745712739317,
    "min": 0.5, "max": 1.2, "type": "float"}
42.
43.     # Dynamic safety stock based on pipeline variability
44.     pipeline_variability_factor = 0.8793791645850786 # OPT_PARAM: {"initial": 0.8793791645850786,
    "min": 0.5, "max": 1.5, "type": "float"}
45.
46.     # Aggressive ordering when immediate availability is critically low
47.     critical_threshold = 30.0 # OPT_PARAM: {"initial": 30.0, "min": 10, "max": 50, "type": "float"}
48.     critical_multiplier = 1.5 # OPT_PARAM: {"initial": 1.5, "min": 1.2, "max": 2.0, "type": "float"}
49.
50.     # NEW: Demand forecast adjustment factor
51.     demand_adjustment = 1.0083668144408222 # OPT_PARAM: {"initial": 1.0083668144408222, "min": 0.9,
    "max": 1.2, "type": "float"}
52.
53.     # NEW: Immediate coverage buffer
54.     immediate_buffer = 25.0 # OPT_PARAM: {"initial": 25.0, "min": 10, "max": 50, "type": "float"}
55.
56.     # Calculate discounted pipeline value with slot-specific discounts
57.     discounted_pipeline = 0.0
58.
59.     for i in range(L):
60.         if i < 2: # Arriving in next 2 periods
61.             discounted_pipeline += pipeline_orders[i] * early_discount
62.         elif i < 4: # Arriving in periods 3-4
63.             discounted_pipeline += pipeline_orders[i] * mid_discount
64.         else: # Arriving in periods 5-6
65.             discounted_pipeline += pipeline_orders[i] * late_discount
66.
67.     # Calculate pipeline roughness (variability in arrivals)
68.     pipeline_roughness = 0.0
69.     for i in range(L-1):
70.         pipeline_roughness += abs(pipeline_orders[i] - pipeline_orders[i+1])
71.
72.     # Apply roughness penalty to discounted pipeline
73.     if pipeline_roughness > 100: # Significant variability threshold
74.         discounted_pipeline *= pipeline_roughness_penalty
75.
76.     # Calculate dynamic safety stock based on pipeline variability
77.     dynamic_safety = safety_stock
78.     if pipeline_roughness > 50: # Moderate variability
79.         dynamic_safety *= (1.0 + pipeline_variability_factor * (pipeline_roughness / 500.0))

```

Declarative mathematical formula rows 1-19

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

mean_demand = 100.0

$$\text{discounted_pipeline} = \sum_{i=0}^{L-1} (\mathbf{1}_{\{i < 2\}} ((q_i \cdot 1.0)) + \mathbf{1}_{\{(\neg(i < 2)) \wedge (i < 4)\}} ((q_i \cdot 0.9)) + \mathbf{1}_{\{(\neg(i < 2)) \wedge (\neg(i < 4))\}} ((q_i \cdot 0.7)))$$

$$\text{pipeline_roughness} = \sum_{i=0}^{(L-1)-1} (|(q_i - q_{(i+1)})|)$$

$$\text{discounted_pipeline} = \begin{cases} (\text{discounted_pipeline} \cdot 0.9), & \text{pipeline_roughness} > 100 \\ \text{discounted_pipeline}, & \text{otherwise} \end{cases}$$

dynamic_safety = 66.4

$$\text{dynamic_safety} = \begin{cases} (\text{dynamic_safety} \cdot (1.0 + (0.9 \cdot \frac{\text{pipeline_roughness}}{500.0}))), & \text{pipeline_roughness} > 50 \\ \text{dynamic_safety}, & \text{otherwise} \end{cases}$$

adjusted_base_stock = (566.2 · 1.0)

effective_ip = (h + discounted_pipeline)

target_level = (adjusted_base_stock + dynamic_safety)

immediate_coverage = (h + q_0)

$$\text{target_level} = \begin{cases} (\text{target_level} + 25.0), & \text{immediate_coverage} < \text{mean_demand} \\ \text{target_level}, & \text{otherwise} \end{cases}$$

order_amount = max(0.0, (target_level – effective_ip))

availability = (h + q_0)

order_amount = (order_amount · 1.5) } (availability < 30.0)

order_amount = (order_amount · 1.2) } (availability < 50.0)

$$\text{order_amount} = \begin{cases} (\text{order_amount} \cdot 0.9), & \text{availability} > 200.0 \\ \text{order_amount}, & \text{otherwise} \end{cases} \quad \left. \begin{matrix} \text{ } \\ \text{ } \end{matrix} \right\} (\neg(\text{availability} < 50.0)) \quad \left. \begin{matrix} \text{ } \\ \text{ } \end{matrix} \right\} (\neg(\text{availability} < 30.0))$$

$$\text{order_amount} = \begin{cases} 9.9, & (\text{order_amount} > 0 \wedge \text{order_amount} < 9.9) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

order_amount = min(order_amount, 87.7)

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 80-117

```

80.
81. # NEW: Adjust base stock based on demand forecast
82. adjusted_base_stock = base_stock * demand_adjustment
83.
84. # Calculate effective inventory position with dynamic safety stock
85. effective_ip = on_hand_inventory + discounted_pipeline
86. target_level = adjusted_base_stock + dynamic_safety
87.
88. # NEW: Add immediate coverage consideration
89. immediate_coverage = on_hand_inventory + pipeline_orders[0]
90. if immediate_coverage < mean_demand:
91.     target_level += immediate_buffer
92.
93. # Base order: target level minus effective inventory position
94. order_amount = max(0.0, target_level - effective_ip)
95.
96. # Adjust aggressiveness based on current availability
97. availability = on_hand_inventory + pipeline_orders[0]
98.
99. # Critical stock handling - most aggressive when immediate stock is very low
100. if availability < critical_threshold:
101.     order_amount = order_amount * critical_multiplier
102. elif availability < low_stock_threshold:
103.     # More aggressive when stock is low
104.     order_amount = order_amount * low_stock_multiplier
105. elif availability > high_stock_threshold:
106.     # Less aggressive when stock is high
107.     order_amount = order_amount * high_stock_multiplier
108.
109. # Apply order smoothing
110. if order_amount > 0 and order_amount < min_order:
111.     order_amount = min_order
112.
113. # Cap order amount
114. order_amount = min(order_amount, max_order)
115.
116. # Round to nearest integer (orders must be integer quantities)
117. return order_amount

```

Declarative mathematical formula rows 1-19

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

mean_demand = 100.0

$$\text{discounted_pipeline} = \sum_{i=0}^{L-1} (1_{\{i < 2\}} ((q_i \cdot 1.0)) + 1_{\{(\neg(i < 2)) \wedge (i < 4)\}} ((q_i \cdot 0.9)) + 1_{\{(\neg(i < 2)) \wedge (\neg(i < 4))\}} ((q_i \cdot 0.7)))$$

$$\text{pipeline_roughness} = \sum_{i=0}^{(L-1)-1} (|(q_i - q_{(i+1)})|)$$

$$\text{discounted_pipeline} = \begin{cases} (\text{discounted_pipeline} \cdot 0.9), & \text{pipeline_roughness} > 100 \\ \text{discounted_pipeline}, & \text{otherwise} \end{cases}$$

dynamic_safety = 66.4

$$\text{dynamic_safety} = \begin{cases} (\text{dynamic_safety} \cdot (1.0 + (0.9 \cdot \frac{\text{pipeline_roughness}}{500.0}))), & \text{pipeline_roughness} > 50 \\ \text{dynamic_safety}, & \text{otherwise} \end{cases}$$

adjusted_base_stock = (566.2 · 1.0)

effective_ip = (h + discounted_pipeline)

target_level = (adjusted_base_stock + dynamic_safety)

immediate_coverage = (h + q₀)

$$\text{target_level} = \begin{cases} (\text{target_level} + 25.0), & \text{immediate_coverage} < \text{mean_demand} \\ \text{target_level}, & \text{otherwise} \end{cases}$$

order_amount = max(0.0, (target_level - effective_ip))

availability = (h + q₀)

order_amount = (order_amount · 1.5) } (availability < 30.0)

order_amount = (order_amount · 1.2) } (availability < 50.0)

$$\text{order_amount} = \begin{cases} (\text{order_amount} \cdot 0.9), & \text{availability} > 200.0 \\ \text{order_amount}, & \text{otherwise} \end{cases} \quad \left. \vphantom{\text{order_amount}} \right\} (\neg(\text{availability} < 50.0)) \quad \left. \vphantom{\text{order_amount}} \right\} (\neg(\text{availability} < 30.0))$$

$$\text{order_amount} = \begin{cases} 9.9, & (\text{order_amount} > 0 \wedge \text{order_amount} < 9.9) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

order_amount = min(order_amount, 87.7)

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-39

```

1. # sample_id: 078
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_8_r6
3. # distribution: poisson_L6_c1_2
4. # generation: 6
5. # rank_in_population_file: 2
6. # objective: 759.98
7. # test_objective: 745.637
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping;order_
    smoothing
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 663.9895492381544 # OPT_PARAM: {"initial": 663.9895492381544, "min": 10, "max":
    1000, "type": "float"}
14.     safety_stock = 58.640380519374744 # OPT_PARAM: {"initial": 58.640380519374744, "min": 0, "max":
    200, "type": "float"}
15.     demand_forecast = 125.84085101407936 # OPT_PARAM: {"initial": 125.84085101407936, "min": 50,
    "max": 150, "type": "float"}
16.     max_order = 96.02075836350208 # OPT_PARAM: {"initial": 96.02075836350208, "min": 50, "max": 500,
    "type": "float"}
17.     smoothing_factor = 0.6628825194762518 # OPT_PARAM: {"initial": 0.6628825194762518, "min": 0.1,
    "max": 0.9, "type": "float"}
18.     min_order = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 50, "type": "float"}
19.
20.     # Calculate inventory position
21.     inventory_position = on_hand_inventory + sum(pipeline_orders)
22.
23.     # Calculate expected demand during lead time
24.     expected_lead_time_demand = demand_forecast * len(pipeline_orders)
25.
26.     # Calculate target inventory position
27.     target_inventory = expected_lead_time_demand + safety_stock
28.
29.     # Calculate raw order amount
30.     raw_order = max(0, target_inventory - inventory_position)
31.
32.     # Apply smoothing to reduce volatility
33.     smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * (pipeline_orders[-1] if
    pipeline_orders else 0)
34.
35.     # Apply minimum order quantity to reduce small orders
36.     if smoothed_order > 0 and smoothed_order < min_order:
37.         smoothed_order = min_order
38.
39.     # Cap order amount to avoid excessive ordering

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (125.8 \cdot L)$$

$$\text{target_inventory} = (\text{expected_lead_time_demand} + 58.6)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{smoothed_order} = \left((0.7 \cdot \text{raw_order}) + \left((1 - 0.7) \cdot \begin{cases} q_{-1}, & \mathbf{q} \\ 0, & \text{otherwise} \end{cases} \right) \right)$$

$$\text{smoothed_order} = \begin{cases} 10.0, & (\text{smoothed_order} > 0 \wedge \text{smoothed_order} < 10.0) \\ \text{smoothed_order}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \min(\text{smoothed_order}, 96.0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 40-45

```

40.     order_amount = min(smoothed_order, max_order)
41.
42.     # Ensure non-negative integer
43.     order_amount = max(0, int(round(order_amount)))
44.
45.     return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (125.8 \cdot L)$$

$$\text{target_inventory} = (\text{expected_lead_time_demand} + 58.6)$$

$$\text{raw_order} = \max(0, (\text{target_inventory} - \text{inventory_position}))$$

$$\text{smoothed_order} = \left((0.7 \cdot \text{raw_order}) + \left((1 - 0.7) \cdot \begin{cases} q_{-(1)}, & \mathbf{q} \\ 0, & \text{otherwise} \end{cases} \right) \right)$$

$$\text{smoothed_order} = \begin{cases} 10.0, & (\text{smoothed_order} > 0 \wedge \text{smoothed_order} < 10.0) \\ \text{smoothed_order}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \min(\text{smoothed_order}, 96.0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-23

```
1. # sample_id: 079
2. # folder: grok-4.1-fast_exponential_L6_c1_2_50_plain_processed_scipy_15_default_m2_2_r1
3. # distribution: exponential_L6_c1_2
4. # generation: 8
5. # rank_in_population_file: 5
6. # objective: 6193.19816
7. # test_objective: 6337.68845
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: pipeline_weighting;order_up_to
11. # extra_motifs:
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     bs = 252.76461729338735 # OPT_PARAM: {"initial": 252.76461729338735, "min": 50.0, "max": 1000.0,
    "type": "float"}
14.     w_i = 0.06301367498820724 # OPT_PARAM: {"initial": 0.06301367498820724, "min": 0.0, "max": 2.0,
    "type": "float"}
15.     w_pipe = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.5, "type": "float"}
16.     discount = 1.2 # OPT_PARAM: {"initial": 1.2, "min": 0.8, "max": 1.2, "type": "float"}
17.     ip = w_i * on_hand_inventory
18.     w = w_pipe
19.     for q in pipeline_orders:
20.         ip += w * q
21.         w *= discount
22.     order_amount = max(0.0, bs - ip)
23.     return order_amount
```

Declarative mathematical formula rows 1-6

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$ip = (0.1 \cdot h)$$

$$w = 0.5$$

$$ip = (0.1 \cdot h) + \sum_{i=0}^{L-1} \left(((0.5) (1.2)^i \cdot q_i) \right)$$

$$\text{order_amount} = \max(0.0, (252.8 - ip))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-39

```
1. # sample_id: 080
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_4_r7
3. # distribution: normal_std30_L6_c1_2
4. # generation: 1
5. # rank_in_population_file: 7
6. # objective: 4247.42
7. # test_objective: 4226.674
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;state_dependent_target
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 571.8118933345869 # OPT_PARAM: {"initial": 571.8118933345869, "min": 10, "max":
1000, "type": "float"}
14.     safety_stock = 26.823248836796438 # OPT_PARAM: {"initial": 26.823248836796438, "min": 0, "max":
200, "type": "float"}
15.     demand_adjustment_factor = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 2.0, "type":
"float"}
16.
17.     # Estimate upcoming demand based on pipeline arrivals
18.     upcoming_arrivals = sum(pipeline_orders)
19.
20.     # Calculate net inventory position
21.     net_inventory = on_hand_inventory + upcoming_arrivals
22.
23.     # Adjust base stock based on recent pipeline pattern
24.     recent_orders = pipeline_orders[-3:] if len(pipeline_orders) >= 3 else pipeline_orders
25.     avg_recent = sum(recent_orders) / len(recent_orders) if recent_orders else 0
26.
27.     # Dynamic adjustment: if recent orders are high, anticipate higher future demand
28.     dynamic_adjustment = avg_recent * demand_adjustment_factor
29.
30.     # Calculate target inventory position
31.     target_inventory = base_stock + safety_stock + dynamic_adjustment
32.
33.     # Calculate order amount
34.     order_amount = max(0, target_inventory - net_inventory)
35.
36.     # Round to nearest integer (as order amounts should be integers)
37.     order_amount = int(round(order_amount))
38.
39.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}} \left(\max \left(0, \left((571.8 + 26.8) + \frac{\sum_{\text{len}} \left(\begin{cases} q_{-(3)}, & L \geq 3 \\ \mathbf{q}, & \text{otherwise} \end{cases} }{\begin{cases} q_{-(3)}, & L \geq 3 \\ \mathbf{q}, & \text{otherwise} \end{cases}} \right), \begin{cases} q_{-(3)}, & L \geq 3 \\ \mathbf{q}, & \text{otherwise} \cdot 0.1 \end{cases} \right) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right).$$

Python source lines 1-28

```
1. # sample_id: 081
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r4
3. # distribution: poisson_L6_c1_2
4. # generation: 2
5. # rank_in_population_file: 3
6. # objective: 2556.99901
7. # test_objective: 2552.16349
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;partial_adjustment
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 665.8919289780819 # OPT_PARAM: {"initial": 665.8919289780819, "min": 10, "max":
14.     1000, "type": "float"}
15.     safety_stock = 51.902379739927916 # OPT_PARAM: {"initial": 51.902379739927916, "min": 0, "max":
16.     200, "type": "float"}
17.     smoothing_factor = 0.5093579167278066 # OPT_PARAM: {"initial": 0.5093579167278066, "min": 0,
18.     "max": 1, "type": "float"}
19.
20.     # Calculate inventory position
21.     inventory_position = on_hand_inventory + sum(pipeline_orders)
22.
23.     # Smooth adjustment to avoid extreme order fluctuations
24.     target_position = base_stock + safety_stock
25.     adjustment = smoothing_factor * (target_position - inventory_position)
26.
27.     # Ensure non-negative order
28.     order_amount = max(0, adjustment)
29.
30.     # Round to nearest integer (since order amounts should be integers)
31.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left(0.5 \cdot \left((665.9 + 51.9) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right) \right).$$

Python source lines 1-44

```

1. # sample_id: 082
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: poisson_L6_c1_2
4. # generation: 3
5. # rank_in_population_file: 6
6. # objective: 1195.96
7. # test_objective: 1187.877
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 550.0 # OPT_PARAM: {"initial": 550.0, "min": 300, "max": 800, "type": "float"}
14.     safety_stock = 83.79999999999248 # OPT_PARAM: {"initial": 83.79999999999248, "min": 20, "max":
150, "type": "float"}
15.     demand_estimate = 120.0 # OPT_PARAM: {"initial": 120.0, "min": 80, "max": 120, "type": "float"}
16.
17.     # Calculate net inventory position
18.     net_inventory = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Calculate expected demand during lead time
21.     expected_lead_time_demand = demand_estimate * len(pipeline_orders)
22.
23.     # Calculate target inventory position with dynamic adjustment
24.     target_position = expected_lead_time_demand + safety_stock
25.
26.     # Calculate order amount
27.     order_amount = max(0, target_position - net_inventory)
28.
29.     # Apply smoothing factor to reduce order volatility
30.     smoothing_factor = 0.9 # OPT_PARAM: {"initial": 0.9, "min": 0.3, "max": 1.0, "type": "float"}
31.     if order_amount > 0:
32.         order_amount = order_amount * smoothing_factor
33.
34.     # Cap order amount based on recent demand pattern
35.     max_order = 120.0 # OPT_PARAM: {"initial": 120.0, "min": 80, "max": 200, "type": "float"}
36.     order_amount = min(order_amount, max_order)
37.
38.     # Minimum order quantity to avoid tiny orders
39.     min_order = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 30, "type": "float"}
40.     if 0 < order_amount < min_order:
41.         order_amount = min_order
42.
43.     # Round to nearest integer since order amount must be integer
44.     order_amount = int(round(order_amount))

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (120.0 \cdot L)$$

$$\text{target_position} = (\text{expected_lead_time_demand} + 83.8)$$

$$\text{order_amount} = \max(0, (\text{target_position} - \text{net_inventory}))$$

$$\text{order_amount} = \begin{cases} (\text{order_amount} \cdot 0.9), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \min(\text{order_amount}, 120.0)$$

$$\text{order_amount} = \begin{cases} 10.0, & 0 < \text{order_amount} < 10.0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 45-46

```
45.  
46.     return order_amount
```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$
$$\text{expected_lead_time_demand} = (120.0 \cdot L)$$
$$\text{target_position} = (\text{expected_lead_time_demand} + 83.8)$$
$$\text{order_amount} = \max(0, (\text{target_position} - \text{net_inventory}))$$
$$\text{order_amount} = \begin{cases} (\text{order_amount} \cdot 0.9), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$
$$\text{order_amount} = \min(\text{order_amount}, 120.0)$$
$$\text{order_amount} = \begin{cases} 10.0, & 0 < \text{order_amount} < 10.0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$
$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-39

```

1. # sample_id: 083
2. # folder: deepeek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-m2_6_r2
3. # distribution: normal_std30_L6_c1_2
4. # generation: 9
5. # rank_in_population_file: 3
6. # objective: 2398.26
7. # test_objective: 2414.279
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment
11. # extra_motifs: integer_rounding;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Core base stock level
14.     base_stock = 541.0430695860339 # OPT_PARAM: {"initial": 541.0430695860339, "min": 300, "max":
    900, "type": "float"}
15.
16.     # Cost-ratio adjustment factor (p/h = 2)
17.     cost_ratio_factor = 1.377841154054285 # OPT_PARAM: {"initial": 1.377841154054285, "min": 1.0,
    "max": 3.0, "type": "float"}
18.
19.     # Pipeline variability adjustment
20.     variability_threshold = 0.3961856822731612 # OPT_PARAM: {"initial": 0.3961856822731612, "min":
    0.1, "max": 0.5, "type": "float"}
21.     variability_multiplier = 1.109110084850649 # OPT_PARAM: {"initial": 1.109110084850649, "min":
    1.0, "max": 3.0, "type": "float"}
22.
23.     # Order smoothing parameters
24.     smoothing_base = 0.115182788573315 # OPT_PARAM: {"initial": 0.115182788573315, "min": 0.1,
    "max": 0.8, "type": "float"}
25.     smoothing_adjust = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.05, "max": 0.3, "type": "float"}
26.
27.     # Calculate inventory position
28.     inventory_position = on_hand_inventory + sum(pipeline_orders)
29.
30.     # Calculate pipeline variability (new structural element)
31.     if len(pipeline_orders) > 1:
32.         pipeline_mean = sum(pipeline_orders) / len(pipeline_orders)
33.         pipeline_variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) /
    len(pipeline_orders)
34.         variability_ratio = pipeline_variance / (pipeline_mean + 1e-6)
35.     else:
36.         variability_ratio = 0.0
37.
38.     # Adjust base stock based on cost ratio and pipeline variability
39.     adjusted_base = base_stock * cost_ratio_factor

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ \text{variability_ratio} &= \frac{\text{pipeline_variance}}{(\text{pipeline_mean} + 1.0e - 06)} \end{aligned} \right\} \quad (L > 1)$$

$$\text{variability_ratio} = 0.0 \quad (\neg (L > 1))$$

$$\text{adjusted_base} = (541.0 \cdot 1.4)$$

$$\text{adjusted_base} = \begin{cases} (\text{adjusted_base} \cdot 1.1), & \text{variability_ratio} > 0.4 \\ \text{adjusted_base}, & \text{otherwise} \end{cases}$$

$$\text{order_gap} = \max(0, (\text{adjusted_base} - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{gap_ratio} &= \frac{\text{order_gap}}{(\text{adjusted_base} + 1.0e - 06)} \\ \text{dynamic_smoothing} &= (0.1 + (0.3 \cdot (1.0 - \text{gap_ratio}))) \end{aligned} \right\} \quad (\text{order_gap} > 0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}((\text{dynamic_smoothing} \cdot \text{order_gap})))$$

$$\text{order_amount} = 0 \quad (\neg (\text{order_gap} > 0))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 40-54

```

40.     if variability_ratio > variability_threshold:
41.         adjusted_base *= variability_multiplier
42.
43.     # Calculate order-up-to gap
44.     order_gap = max(0, adjusted_base - inventory_position)
45.
46.     # Dynamic smoothing based on gap size (new structural element)
47.     if order_gap > 0:
48.         gap_ratio = order_gap / (adjusted_base + 1e-6)
49.         dynamic_smoothing = smoothing_base + smoothing_adjust * (1.0 - gap_ratio)
50.         order_amount = max(0, int(round(dynamic_smoothing * order_gap)))
51.     else:
52.         order_amount = 0
53.
54.     return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{pipeline_variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L} \\ \text{variability_ratio} &= \frac{\text{pipeline_variance}}{(\text{pipeline_mean} + 1.0e - 06)} \end{aligned} \right\} \quad (L > 1)$$

$$\text{variability_ratio} = 0.0 \quad (\neg (L > 1))$$

$$\text{adjusted_base} = (541.0 \cdot 1.4)$$

$$\text{adjusted_base} = \begin{cases} (\text{adjusted_base} \cdot 1.1), & \text{variability_ratio} > 0.4 \\ \text{adjusted_base}, & \text{otherwise} \end{cases}$$

$$\text{order_gap} = \max(0, (\text{adjusted_base} - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{gap_ratio} &= \frac{\text{order_gap}}{(\text{adjusted_base} + 1.0e - 06)} \\ \text{dynamic_smoothing} &= (0.1 + (0.3 \cdot (1.0 - \text{gap_ratio}))) \\ \text{order_amount} &= \max(0, \text{round}_{\text{even}}((\text{dynamic_smoothing} \cdot \text{order_gap}))) \\ \text{order_amount} &= 0 \end{aligned} \right\} \quad (\text{order_gap} > 0) \quad (\neg (\text{order_gap} > 0))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-44

```

1. # sample_id: 084
2. # folder: deepseek-chat_normal_std10_L6_c1_5_50_plain_processed_scipy_15_default_m2_4_r6
3. # distribution: normal_std10_L6_c1_5
4. # generation: 4
5. # rank_in_population_file: 5
6. # objective: 1365.78
7. # test_objective: 1344.857
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 585.6428063424911 # OPT_PARAM: {"initial": 585.6428063424911, "min": 450, "max":
14.     700, "type": "float"}
15.     safety_stock = 107.01607779666783 # OPT_PARAM: {"initial": 107.01607779666783, "min": 40, "max":
16.     120, "type": "float"}
17.     smoothing_factor = 0.2 # OPT_PARAM: {"initial": 0.2, "min": 0.2, "max": 0.9, "type": "float"}
18.     demand_adjustment = 0.8 # OPT_PARAM: {"initial": 0.8, "min": 0.8, "max": 3.0, "type": "float"}
19.     min_order = 15.0 # OPT_PARAM: {"initial": 15.0, "min": 10, "max": 50, "type": "float"}
20.     pipeline_weight = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.0, "type": "float"}
21.
22.     # Calculate inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate demand estimate using weighted average of recent pipeline orders
26.     if len(pipeline_orders) > 0:
27.         # Simple exponential weighting: recent orders get more weight
28.         weights = [pipeline_weight ** i for i in range(len(pipeline_orders))]
29.         weights = [w / sum(weights) for w in weights] # Normalize
30.         demand_estimate = sum(w * q for w, q in zip(weights, pipeline_orders))
31.     else:
32.         demand_estimate = 100.0
33.
34.     # Adjust base stock based on demand estimate
35.     adjusted_base = base_stock + (demand_estimate - 100) * demand_adjustment
36.
37.     # Calculate target inventory position
38.     target_position = adjusted_base + safety_stock
39.
40.     # Calculate order needed
41.     order_needed = target_position - inventory_position
42.
43.     # Apply smoothing to reduce order volatility
44.     if order_needed > min_order:
45.         smoothed_order = smoothing_factor * order_needed + (1 - smoothing_factor) * demand_estimate
46.     else:

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{weights} &= \left[(0.5)^i \mid i \in \text{range}(L) \right] \\ \text{weights} &= \left[\frac{w}{\text{sum}(\text{weights})} \mid w \in \text{weights} \right] \\ \text{demand_estimate} &= \sum_{(w,q) \in \text{zip}(\text{weights}, \mathbf{q})} (w \cdot q) \end{aligned} \right\} \quad (L > 0)$$

$$\text{demand_estimate} = 100.0 \quad (\neg(L > 0))$$

$$\text{adjusted_base} = (585.6 + ((\text{demand_estimate} - 100) \cdot 0.8))$$

$$\text{target_position} = (\text{adjusted_base} + 107.0)$$

$$\text{order_needed} = (\text{target_position} - \text{inventory_position})$$

$$\text{smoothed_order} = \begin{cases} ((0.2 \cdot \text{order_needed}) + ((1 - 0.2) \cdot \text{demand_estimate})), & \text{order_needed} > 15.0 \\ \max(0, \text{order_needed}), & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 45-50

```

45.         smoothed_order = max(0, order_needed)
46.
47.         # Round to integer and ensure non-negative
48.         order_amount = max(0, int(round(smoothed_order)))
49.
50.         return order_amount

```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{weights} &= [(0.5)^i \mid i \in \text{range}(L)] \\ \text{weights} &= \left[\frac{w}{\text{sum}(\text{weights})} \mid w \in \text{weights} \right] \\ \text{demand_estimate} &= \sum_{(w,q) \in \text{zip}(\text{weights}, \mathbf{q})} (w \cdot q) \end{aligned} \right\} \quad (L > 0)$$

$$\text{demand_estimate} = 100.0 \quad (\neg (L > 0))$$

$$\text{adjusted_base} = (585.6 + ((\text{demand_estimate} - 100) \cdot 0.8))$$

$$\text{target_position} = (\text{adjusted_base} + 107.0)$$

$$\text{order_needed} = (\text{target_position} - \text{inventory_position})$$

$$\text{smoothed_order} = \begin{cases} ((0.2 \cdot \text{order_needed}) + ((1 - 0.2) \cdot \text{demand_estimate})), & \text{order_needed} > 15.0 \\ \max(0, \text{order_needed}), & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-40

```

1. # sample_id: 085
2. # folder: deepseek-chat_normal_std30_L4_c1_2_50_plain_processed_scipy_15_default_e1-e2-m2_4_r1
3. # distribution: normal_std30_L4_c1_2
4. # generation: 9
5. # rank_in_population_file: 2
6. # objective: 2148.37423
7. # test_objective: 2214.44297
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 421.6446046499665 # OPT_PARAM: {"initial": 421.6446046499665, "min": 300, "max":
600, "type": "float"}
14.     safety_stock = 10.112929000695772 # OPT_PARAM: {"initial": 10.112929000695772, "min": 10, "max":
50, "type": "float"}
15.     pipeline_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.7, "max": 1.0, "type": "float"}
16.     demand_forecast_factor = 0.5157159963654885 # OPT_PARAM: {"initial": 0.5157159963654885, "min":
0.5, "max": 1.2, "type": "float"}
17.     smoothing_factor = 0.23388874095375883 # OPT_PARAM: {"initial": 0.23388874095375883, "min": 0.1,
"max": 0.5, "type": "float"}
18.     min_order = 20.0 # OPT_PARAM: {"initial": 20.0, "min": 10, "max": 40, "type": "float"}
19.     max_order = 88.51464456153231 # OPT_PARAM: {"initial": 88.51464456153231, "min": 80, "max": 200,
"type": "float"}
20.
21.     # Calculate average pipeline demand (excluding zeros in planning phase)
22.     non_zero_pipeline = [q for q in pipeline_orders if q > 0]
23.     if non_zero_pipeline:
24.         avg_pipeline = sum(non_zero_pipeline) / len(non_zero_pipeline)
25.     else:
26.         avg_pipeline = 100.0 # default estimate
27.
28.     # Forecast demand based on pipeline average
29.     forecast_demand = avg_pipeline * demand_forecast_factor
30.
31.     # Calculate effective inventory position
32.     effective_pipeline = sum(pipeline_orders) * pipeline_weight
33.     inventory_position = on_hand_inventory + effective_pipeline
34.
35.     # Target inventory level
36.     target_inventory = base_stock + safety_stock + forecast_demand
37.
38.     # Calculate order with smoothing
39.     order_needed = target_inventory - inventory_position
40.     smoothed_order = smoothing_factor * order_needed + (1 - smoothing_factor) * avg_pipeline

```

Declarative mathematical formula rows 1-11

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{non_zero_pipeline} = [q \mid q \in \mathbf{q}, q > 0]$

$$\text{avg_pipeline} = \begin{cases} \frac{\text{sum}(\text{non_zero_pipeline})}{\text{len}(\text{non_zero_pipeline})}, & \text{non_zero_pipeline} \\ 100.0, & \text{otherwise} \end{cases}$$

$\text{forecast_demand} = (\text{avg_pipeline} \cdot 0.5)$

$$\text{effective_pipeline} = \left(\sum_{i=0}^{L-1} q_i \cdot 1.0 \right)$$

$\text{inventory_position} = (h + \text{effective_pipeline})$

$\text{target_inventory} = ((421.6 + 10.1) + \text{forecast_demand})$

$\text{order_needed} = (\text{target_inventory} - \text{inventory_position})$

$\text{smoothed_order} = ((0.2 \cdot \text{order_needed}) + ((1 - 0.2) \cdot \text{avg_pipeline}))$

$\text{order_amount} = \max(20.0, \min(88.5, \text{smoothed_order}))$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 41-45

```
41.  
42.     # Apply bounds  
43.     order_amount = max(min_order, min(max_order, smoothed_order))  
44.  
45.     return order_amount
```

Declarative mathematical formula rows 1-11

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\text{non_zero_pipeline} = [q \mid q \in \mathbf{q}, q > 0]$

$$\text{avg_pipeline} = \begin{cases} \frac{\text{sum}(\text{non_zero_pipeline})}{\text{len}(\text{non_zero_pipeline})}, & \text{non_zero_pipeline} \\ 100.0, & \text{otherwise} \end{cases}$$

$\text{forecast_demand} = (\text{avg_pipeline} \cdot 0.5)$

$$\text{effective_pipeline} = \left(\sum_{i=0}^{L-1} q_i \cdot 1.0 \right)$$

$\text{inventory_position} = (h + \text{effective_pipeline})$

$\text{target_inventory} = ((421.6 + 10.1) + \text{forecast_demand})$

$\text{order_needed} = (\text{target_inventory} - \text{inventory_position})$

$\text{smoothed_order} = ((0.2 \cdot \text{order_needed}) + ((1 - 0.2) \cdot \text{avg_pipeline}))$

$\text{order_amount} = \max(20.0, \min(88.5, \text{smoothed_order}))$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-43

```
1. # sample_id: 086
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r3
3. # distribution: poisson_L6_c1_2
4. # generation: 8
5. # rank_in_population_file: 4
6. # objective: 1163.24
7. # test_objective: 1163.609
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 920.0 # OPT_PARAM: {"initial": 920.0, "min": 850, "max": 1050, "type": "float"}
14.     safety_stock = 95.199999999999763 # OPT_PARAM: {"initial": 95.199999999999763, "min": 50, "max":
120, "type": "float"}
15.     demand_estimate = 110.0 # OPT_PARAM: {"initial": 110.0, "min": 90, "max": 110, "type": "float"}
16.     smoothing_factor = 0.25 # OPT_PARAM: {"initial": 0.25, "min": 0.1, "max": 0.4, "type": "float"}
17.     pipeline_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.7, "max": 1.0, "type": "float"}
18.     lost_sales_weight = 2.2 # OPT_PARAM: {"initial": 2.2, "min": 1.5, "max": 2.2, "type": "float"}
19.     min_order = 0 # OPT_PARAM: {"initial": 0, "min": 0, "max": 10, "type": "int"}
20.     max_order = 110 # OPT_PARAM: {"initial": 110, "min": 90, "max": 130, "type": "int"}
21.
22.     # Calculate current inventory position
23.     inventory_position = on_hand_inventory + sum(pipeline_orders)
24.
25.     # Calculate expected demand during lead time
26.     expected_lead_time_demand = demand_estimate * len(pipeline_orders)
27.
28.     # Adjust safety stock based on cost ratio
29.     adjusted_safety_stock = safety_stock * (lost_sales_weight / 2.0)
30.
31.     # Calculate target inventory position
32.     target_position = expected_lead_time_demand + adjusted_safety_stock
33.
34.     # Apply base stock cap
35.     target_position = min(target_position, base_stock)
36.
37.     # Calculate raw order amount
38.     raw_order = max(0, target_position - inventory_position)
39.
40.     # Apply smoothing to reduce order volatility
41.     smoothed_order = raw_order * smoothing_factor + (target_position - inventory_position) * (1 -
smoothing_factor)
42.
43.     # Apply pipeline weight to account for existing orders
```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$
$$\text{expected_lead_time_demand} = (110.0 \cdot L)$$
$$\text{adjusted_safety_stock} = \left(95.2 \cdot \frac{2.2}{2.0} \right)$$
$$\text{target_position} = (\text{expected_lead_time_demand} + \text{adjusted_safety_stock})$$
$$\text{target_position} = \min(\text{target_position}, 920.0)$$
$$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$$
$$\text{smoothed_order} = ((\text{raw_order} \cdot 0.2) + ((\text{target_position} - \text{inventory_position}) \cdot (1 - 0.2)))$$
$$\text{weighted_order} = (\text{smoothed_order} \cdot 1.0)$$
$$\text{weighted_order} = \begin{cases} \max(0, \min(\text{weighted_order}, 110)), & \text{weighted_order} > 0 \\ \text{weighted_order}, & \text{otherwise} \end{cases}$$
$$\text{order_amount} = \text{round}_{\text{even}}(\text{weighted_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 44-53

```
44.     weighted_order = smoothed_order * pipeline_weight
45.
46.     # Apply order limits
47.     if weighted_order > 0:
48.         weighted_order = max(min_order, min(weighted_order, max_order))
49.
50.     # Round to nearest integer
51.     order_amount = int(round(weighted_order))
52.
53.     return order_amount
```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (110.0 \cdot L)$$

$$\text{adjusted_safety_stock} = \left(95.2 \cdot \frac{2.2}{2.0} \right)$$

$$\text{target_position} = (\text{expected_lead_time_demand} + \text{adjusted_safety_stock})$$

$$\text{target_position} = \min(\text{target_position}, 920.0)$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{smoothed_order} = ((\text{raw_order} \cdot 0.2) + ((\text{target_position} - \text{inventory_position}) \cdot (1 - 0.2)))$$

$$\text{weighted_order} = (\text{smoothed_order} \cdot 1.0)$$

$$\text{weighted_order} = \begin{cases} \max(0, \min(\text{weighted_order}, 110)), & \text{weighted_order} > 0 \\ \text{weighted_order}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{weighted_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-26

```
1. # sample_id: 087
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_no_15_default_m2_10_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 3
5. # rank_in_population_file: 7
6. # objective: 5738.58
7. # test_objective: 5747.539
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 350 # OPT_PARAM: {'initial': 350, 'min': 10, 'max': 1000, 'type': 'float'}
14.     safety_stock = 50 # OPT_PARAM: {'initial': 50, 'min': 0, 'max': 200, 'type': 'float'}
15.
16.     # Calculate net inventory position
17.     net_inventory = on_hand_inventory + sum(pipeline_orders)
18.
19.     # Order up to base_stock, but ensure at least safety_stock coverage for lead time
20.     order_amount = max(0, base_stock - net_inventory)
21.
22.     # Adjust order to maintain minimum safety stock
23.     if net_inventory + order_amount < safety_stock:
24.         order_amount = max(0, safety_stock - net_inventory)
25.
26.     return order_amount
```

Declarative mathematical formula rows 1-5

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{order_amount} = \max(0, (350 - \text{net_inventory}))$$

$$\text{order_amount} = \begin{cases} \max(0, (50 - \text{net_inventory})) & (\text{net_inventory} + \text{order_amount}) < 50 \\ \text{order_amount} & \text{otherwise} \end{cases}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-33

```

1. # sample_id: 088
2. # folder: deepseek-chat_poisson_L2_c1_2_50_plain_processed_scipy_15_default_m2_2_r6
3. # distribution: poisson_L2_c1_2
4. # generation: 3
5. # rank_in_population_file: 2
6. # objective: 782.0
7. # test_objective: 790.83
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment;order_smoothing
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 259.96374132943953 # OPT_PARAM: {"initial": 259.96374132943953, "min": 100, "max":
14.     400, "type": "float"}
15.     safety_stock = 34.96374132943884 # OPT_PARAM: {"initial": 34.96374132943884, "min": 0, "max":
16.     100, "type": "float"}
17.     smoothing_factor = 0.2206775028960104 # OPT_PARAM: {"initial": 0.2206775028960104, "min": 0.1,
18.     "max": 0.8, "type": "float"}
19.
20.     # Calculate net inventory position
21.     net_inventory = on_hand_inventory + sum(pipeline_orders)
22.
23.     # Simple base-stock policy with smoothing
24.     raw_order = max(0, base_stock + safety_stock - net_inventory)
25.
26.     # Apply smoothing using recent order if available
27.     if len(pipeline_orders) > 0:
28.         recent_order = pipeline_orders[-1]
29.         smoothed_order = smoothing_factor * raw_order + (1 - smoothing_factor) * recent_order
30.     else:
31.         smoothed_order = raw_order
32.
33.     # Round to nearest integer
34.     order_amount = int(round(smoothed_order))
35.
36.     return order_amount

```

Declarative mathematical formula rows 1-7

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{raw_order} = \max(0, ((260.0 + 35.0) - \text{net_inventory}))$$

$$\left. \begin{aligned} \text{recent_order} &= q_{-(1)} \\ \text{smoothed_order} &= ((0.2 \cdot \text{raw_order}) + ((1 - 0.2) \cdot \text{recent_order})) \end{aligned} \right\} \quad (L > 0)$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg (L > 0))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{smoothed_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-44

```

1. # sample_id: 089
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r2
3. # distribution: poisson_L6_c1_2
4. # generation: 4
5. # rank_in_population_file: 5
6. # objective: 2229.92
7. # test_objective: 2232.607
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 520.0 # OPT_PARAM: {"initial": 520.0, "min": 300, "max": 800, "type": "float"}
14.     safety_stock = 60.0 # OPT_PARAM: {"initial": 60.0, "min": 20, "max": 150, "type": "float"}
15.     demand_estimate = 100.0 # OPT_PARAM: {"initial": 100.0, "min": 80, "max": 120, "type": "float"}
16.
17.     # Calculate inventory position
18.     inventory_position = on_hand_inventory + sum(pipeline_orders)
19.
20.     # Calculate expected demand during lead time plus one period
21.     expected_lead_time_demand = demand_estimate * (len(pipeline_orders) + 1)
22.
23.     # Calculate target inventory position
24.     target_position = expected_lead_time_demand + safety_stock
25.
26.     # Calculate order amount
27.     order_amount = max(0, target_position - inventory_position)
28.
29.     # Apply smoothing to avoid large order fluctuations
30.     smoothing_factor = 0.9 # OPT_PARAM: {"initial": 0.9, "min": 0.5, "max": 1.0, "type": "float"}
31.     if order_amount > 0:
32.         order_amount = smoothing_factor * order_amount
33.
34.     # Cap order amount based on demand forecast
35.     max_order = demand_estimate * 2.5 # OPT_PARAM: {"initial": 2.5, "min": 2.0, "max": 5.0, "type":
"float"}
36.     order_amount = min(order_amount, max_order)
37.
38.     # Ensure minimum order quantity for efficiency
39.     min_order = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 50, "type": "float"}
40.     if order_amount > 0 and order_amount < min_order:
41.         order_amount = min_order
42.
43.     # Round to nearest integer
44.     order_amount = int(round(order_amount))

```

Declarative mathematical formula rows 1-11

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (100.0 \cdot (L + 1))$$

$$\text{target_position} = (\text{expected_lead_time_demand} + 60.0)$$

$$\text{order_amount} = \max(0, (\text{target_position} - \text{inventory_position}))$$

$$\text{order_amount} = \begin{cases} (0.9 \cdot \text{order_amount}), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{max_order} = (100.0 \cdot 2.5)$$

$$\text{order_amount} = \min(\text{order_amount}, \text{max_order})$$

$$\text{order_amount} = \begin{cases} 10.0, & (\text{order_amount} > 0 \wedge \text{order_amount} < 10.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 45-46

```
45.  
46.     return order_amount
```

Declarative mathematical formula rows 1-11

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$
$$\text{expected_lead_time_demand} = (100.0 \cdot (L + 1))$$
$$\text{target_position} = (\text{expected_lead_time_demand} + 60.0)$$
$$\text{order_amount} = \max(0, (\text{target_position} - \text{inventory_position}))$$
$$\text{order_amount} = \begin{cases} (0.9 \cdot \text{order_amount}), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$
$$\text{max_order} = (100.0 \cdot 2.5)$$
$$\text{order_amount} = \min(\text{order_amount}, \text{max_order})$$
$$\text{order_amount} = \begin{cases} 10.0, & (\text{order_amount} > 0 \wedge \text{order_amount} < 10.0) \\ \text{order_amount}, & \text{otherwise} \end{cases}$$
$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-20

```
1. # sample_id: 090
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_no_15_default_m2_10_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 6
5. # rank_in_population_file: 7
6. # objective: 3317.3
7. # test_objective: 3309.643
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;order_up_to
11. # extra_motifs: safety_stock_buffer
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 550 # OPT_PARAM: {'initial': 550, 'min': 10, 'max': 1000, 'type': 'float'}
14.     safety_stock = 30 # OPT_PARAM: {'initial': 30, 'min': 0, 'max': 200, 'type': 'float'}
15.
16.     inventory_position = on_hand_inventory + sum(pipeline_orders)
17.     target_inventory = base_stock + safety_stock
18.
19.     order_amount = max(0, target_inventory - inventory_position)
20.     return order_amount
```

Declarative mathematical formula rows 1-2

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$ = pipeline orders.

$$Q(h, \mathbf{q}) = \max \left(0, \left((550 + 30) - \left(h + \sum_{i=0}^{L-1} q_i \right) \right) \right).$$

Python source lines 1-36

```

1. # sample_id: 091
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e2-e2-e2_4_r3
3. # distribution: normal_std30_L6_c1_2
4. # generation: 6
5. # rank_in_population_file: 3
6. # objective: 3639.88
7. # test_objective: 3641.658
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;nonlinear_pipeline_composition
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;integer_rounding;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 595.3256931543754 # OPT_PARAM: {"initial": 595.3256931543754, "min": 10, "max": 1000, "type": "float"}
14.     safety_stock = 50.79935343897438 # OPT_PARAM: {"initial": 50.79935343897438, "min": 0, "max": 200, "type": "float"}
15.     order_floor = 0 # OPT_PARAM: {"initial": 0, "min": 0, "max": 100, "type": "int"}
16.     order_ceiling = 300 # OPT_PARAM: {"initial": 300, "min": 50, "max": 500, "type": "int"}
17.     demand_forecast_factor = 0.6218666867485925 # OPT_PARAM: {"initial": 0.6218666867485925, "min": 0.1, "max": 2.0, "type": "float"}
18.     pipeline_weight_factor = 0.36734933385589097 # OPT_PARAM: {"initial": 0.36734933385589097, "min": 0.0, "max": 1.0, "type": "float"}
19.     demand_smoothing_factor = 0.6473587686473684 # OPT_PARAM: {"initial": 0.6473587686473684, "min": 0.0, "max": 1.0, "type": "float"}
20.     safety_stock_multiplier = 1.2216292069395431 # OPT_PARAM: {"initial": 1.2216292069395431, "min": 0.5, "max": 3.0, "type": "float"}
21.     demand_variability_factor = 0.23499331893006076 # OPT_PARAM: {"initial": 0.23499331893006076, "min": 0.0, "max": 2.0, "type": "float"}
22.     pipeline_decay_factor = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.1, "max": 1.0, "type": "float"}
23.     on_hand_sensitivity = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 0.5, "type": "float"}
24.     pipeline_std_factor = 0.0036084261964755057 # OPT_PARAM: {"initial": 0.0036084261964755057, "min": 0.0, "max": 0.5, "type": "float"}
25.
26.     inventory_position = on_hand_inventory + sum(pipeline_orders)
27.
28.     # Calculate expected demand using exponential smoothing of recent pipeline arrivals
29.     if len(pipeline_orders) >= 3:
30.         recent_arrivals = pipeline_orders[:3]
31.         # Apply exponential smoothing weights
32.         weights = [demand_smoothing_factor ** i for i in range(len(recent_arrivals))]
33.         weights = [w / sum(weights) for w in weights]
34.         smoothed_demand = sum(w * d for w, d in zip(weights, recent_arrivals))
35.
36.     # Calculate variability in recent arrivals

```

Declarative mathematical formula rows 1-14

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{recent_arrivals} = q_{:3}$$

$$\text{weights} = \left[(0.6)^i \mid i \in \text{range}(\text{len}(\text{recent_arrivals})) \right]$$

$$\text{weights} = \left[\frac{w}{\text{sum}(\text{weights})} \mid w \in \text{weights} \right]$$

$$\text{smoothed_demand} = \sum_{(w,d) \in \text{zip}(\text{weights}, \text{recent_arrivals})} (w \cdot d)$$

$$\text{avg_recent} = \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})}$$

$$\text{squared_diffs} = \left[((x - \text{avg_recent}))^2 \mid x \in \text{recent_arrivals} \right]$$

$$\text{variability} = \left(\frac{\text{sum}(\text{squared_diffs})}{\text{len}(\text{recent_arrivals})} \right)^{0.5}$$

$$\text{expected_demand} = ((\text{smoothed_demand} \cdot 0.6) + (\text{variability} \cdot 0.2))$$

$$\text{expected_demand} = (100.0 \cdot 0.6) \quad (\neg (L \geq 3))$$

$$\text{pipeline_adjustment} = \sum_{i=0}^{L-1} \left(((q_i \cdot (1.0)^i) \cdot 0.4) \right)$$

$$\text{pipeline_mean} = \frac{\sum_{i=0}^{L-1} q_i}{L}$$

$$\text{pipeline_variance} = \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean}))^2}{L}$$

$$\text{pipeline_std} = \max(0, (\text{pipeline_variance})^{0.5})$$

$$\text{variability_adjustment} = (\text{pipeline_std} \cdot 3.6e - 03)$$

$$\text{variability_adjustment} = 0.0 \quad (\neg (L \geq 2))$$

$$\text{dynamic_safety} = (50.8 \cdot 1.2)$$

$$\text{on_hand_adjustment} = (h \cdot 0.0)$$

$$\text{order_up_to} = (((((595.3 + \text{dynamic_safety}) + \text{expected_demand}) - \text{pipeline_adjustment}) + \text{variability_adjustment}) - \text{on_hand_adjustment})$$

$$\text{raw_order} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$

$$\text{constrained_order} = \max(0, \min(300, \text{raw_order}))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{constrained_order})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 37-79

```

37.         avg_recent = sum(recent_arrivals) / len(recent_arrivals)
38.         squared_diffs = [(x - avg_recent) ** 2 for x in recent_arrivals]
39.         variability = (sum(squared_diffs) / len(recent_arrivals)) ** 0.5
40.
41.         # Combine smoothed demand with variability adjustment
42.         expected_demand = smoothed_demand * demand_forecast_factor + variability *
demand_variability_factor
43.     else:
44.         expected_demand = 100.0 * demand_forecast_factor
45.
46.     # Calculate pipeline-weighted adjustment with exponential decay
47.     pipeline_adjustment = 0
48.     for i, order in enumerate(pipeline_orders):
49.         decay = pipeline_decay_factor ** i
50.         pipeline_adjustment += order * decay * pipeline_weight_factor
51.
52.     # Calculate pipeline variability adjustment using all pipeline orders
53.     if len(pipeline_orders) >= 2:
54.         pipeline_mean = sum(pipeline_orders) / len(pipeline_orders)
55.         pipeline_variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) /
len(pipeline_orders)
56.         pipeline_std = max(0, pipeline_variance ** 0.5)
57.         variability_adjustment = pipeline_std * pipeline_std_factor
58.     else:
59.         variability_adjustment = 0.0
60.
61.     # Dynamic safety stock adjustment
62.     dynamic_safety = safety_stock * safety_stock_multiplier
63.
64.     # On-hand inventory adjustment (reduces order when on-hand is high)
65.     on_hand_adjustment = on_hand_inventory * on_hand_sensitivity
66.
67.     # Calculate order-up-to level with all adjustments
68.     order_up_to = base_stock + dynamic_safety + expected_demand - pipeline_adjustment +
variability_adjustment - on_hand_adjustment
69.
70.     # Base order amount
71.     raw_order = max(0, order_up_to - inventory_position)
72.
73.     # Apply floor and ceiling constraints
74.     constrained_order = max(order_floor, min(order_ceiling, raw_order))
75.
76.     # Round to nearest integer
77.     order_amount = int(round(constrained_order))
78.
79.     return order_amount

```

Declarative mathematical formula rows 1-14

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\begin{aligned}
 \text{inventory_position} &= \left(h + \sum_{i=0}^{L-1} q_i \right) \\
 \text{recent_arrivals} &= q_3 \\
 \text{weights} &= \left[(0.6)^i \mid i \in \text{range}(\text{len}(\text{recent_arrivals})) \right] \\
 \text{weights} &= \left[\frac{w}{\sum(\text{weights})} \mid w \in \text{weights} \right] \\
 \text{smoothed_demand} &= \sum_{(w,d) \in \text{zip}(\text{weights}, \text{recent_arrivals})} (w \cdot d) \\
 \text{avg_recent} &= \frac{\sum(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})} \\
 \text{squared_diffs} &= \left[((x - \text{avg_recent}))^2 \mid x \in \text{recent_arrivals} \right] \\
 \text{variability} &= \left(\frac{\sum(\text{squared_diffs})}{\text{len}(\text{recent_arrivals})} \right)^{0.5} \\
 \text{expected_demand} &= ((\text{smoothed_demand} \cdot 0.6) + (\text{variability} \cdot 0.2)) \\
 \text{expected_demand} &= (100.0 \cdot 0.6) \quad (\neg(L \geq 3)) \\
 \text{pipeline_adjustment} &= \sum_{i=0}^{L-1} (((q_i \cdot (1.0)^i) \cdot 0.4)) \\
 \text{pipeline_mean} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\
 \text{pipeline_variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{pipeline_mean})^2)}{L} \\
 \text{pipeline_std} &= \max(0, (\text{pipeline_variance})^{0.5}) \\
 \text{variability_adjustment} &= (\text{pipeline_std} \cdot 3.6e - 03) \\
 \text{variability_adjustment} &= 0.0 \quad (\neg(L \geq 2)) \\
 \text{dynamic_safety} &= (50.8 \cdot 1.2) \\
 \text{on_hand_adjustment} &= (h \cdot 0.0) \\
 \text{order_up_to} &= (((((595.3 + \text{dynamic_safety}) + \text{expected_demand}) - \text{pipeline_adjustment}) + \text{variability_adjustment}) - \text{on_hand_adjustment}) \\
 \text{raw_order} &= \max(0, (\text{order_up_to} - \text{inventory_position})) \\
 \text{constrained_order} &= \max(0, \min(300, \text{raw_order})) \\
 \text{order_amount} &= \text{round}_{\text{even}}(\text{constrained_order}) \\
 Q(h, \mathbf{q}) &= \text{order_amount}
 \end{aligned}$$

(L ≥ 3)

(L ≥ 2)

Python source lines 1-38

```

1. # sample_id: 092
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_4_r8
3. # distribution: poisson_L6_c1_2
4. # generation: 2
5. # rank_in_population_file: 4
6. # objective: 837.76535
7. # test_objective: 831.8376
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs: pipeline_demand_proxy
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 600.0 # OPT_PARAM: {"initial": 600.0, "min": 300, "max": 600, "type": "float"}
14.     demand_estimate = 116.45698343699016 # OPT_PARAM: {"initial": 116.45698343699016, "min": 80,
15.     "max": 120, "type": "float"}
16.     smoothing_factor = 0.2476408897165149 # OPT_PARAM: {"initial": 0.2476408897165149, "min": 0.1,
17.     "max": 0.9, "type": "float"}
18.     min_order = 50.0 # OPT_PARAM: {"initial": 50.0, "min": 0, "max": 50, "type": "float"}
19.     max_order = 100.0 # OPT_PARAM: {"initial": 100.0, "min": 100, "max": 200, "type": "float"}
20.
21.     # Calculate net inventory position
22.     net_inventory = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Calculate expected demand during lead time
25.     expected_lead_time_demand = demand_estimate * len(pipeline_orders)
26.
27.     # Calculate target inventory position with smoothing
28.     target_position = base_stock
29.
30.     # Calculate order amount with smoothing
31.     order_amount = max(0, target_position - net_inventory)
32.
33.     # Apply smoothing to reduce order volatility
34.     if order_amount > 0:
35.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * demand_estimate
36.
37.     # Apply order limits
38.     order_amount = max(min_order, min(order_amount, max_order))
39.
40.     return order_amount

```

Declarative mathematical formula rows 1-8

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{net_inventory} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = (116.5 \cdot L)$$

$$\text{target_position} = 600.0$$

$$\text{order_amount} = \max(0, (\text{target_position} - \text{net_inventory}))$$

$$\text{order_amount} = \begin{cases} ((0.2 \cdot \text{order_amount}) + ((1 - 0.2) \cdot 116.5)), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \max(50.0, \min(\text{order_amount}, 100.0))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-43

```

1. # sample_id: 093
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: poisson_L6_c1_2
4. # generation: 7
5. # rank_in_population_file: 9
6. # objective: 781.99954
7. # test_objective: 757.87781
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_clipping
11. # extra_motifs: safety_stock_buffer;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Fixed dataset constants
14.     d_hat = 100.0 # mean demand
15.     L = len(pipeline_orders)
16.
17.     # Optimizable parameters
18.     base_order = 89.31822339686998 # OPT_PARAM: {"initial": 89.31822339686998, "min": 80.0, "max":
110.0, "type": "float"}
19.     inv_feedback = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 0.3, "type": "float"}
20.     pipeline_feedback = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 0.2, "type": "float"}
21.     shortfall_gain = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 0.5, "type": "float"}
22.     pipeline_shape_weight = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 0.3, "type":
"float"}
23.     safety_buffer = 5.687000958575839 # OPT_PARAM: {"initial": 5.687000958575839, "min": 0.0, "max":
20.0, "type": "float"}
24.     target_coverage = 3 # OPT_PARAM: {"initial": 3, "min": 1, "max": 6, "type": "int"}
25.
26.     # Core features
27.     available_now = on_hand_inventory + pipeline_orders[0]
28.     inventory_position = on_hand_inventory + sum(pipeline_orders)
29.
30.     # Pipeline shape features - focus on near-term vs far-term balance
31.     if L > 0:
32.         # Next target_coverage periods vs rest of pipeline
33.         near_term = sum(pipeline_orders[:min(target_coverage, L)])
34.         far_term = sum(pipeline_orders[target_coverage:]) if L > target_coverage else 0
35.
36.         # Normalize for fair comparison
37.         near_term_norm = near_term / float(target_coverage) if L >= target_coverage else near_term /
L
38.         far_term_norm = far_term / max(1, L-target_coverage) if L > target_coverage else 0
39.
40.         # Shape indicator: positive if near-term is heavier than far-term
41.         pipeline_shape = near_term_norm - far_term_norm
42.     else:
43.         pipeline_shape = 0.0

```

Declarative mathematical formula rows 1-15

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$d_{\text{hat}} = 100.0$

$\text{available_now} = (h + q_0)$

$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$

$\text{near_term} = \sum_{i=0}^{(\min(3,L))-1} q_i$

$\text{far_term} = \begin{cases} \sum_{i=3}^{L-1} q_i, & L > 3 \\ 0, & \text{otherwise} \end{cases}$

$\text{near_term_norm} = \begin{cases} \frac{\text{near_term}}{\text{float}(3)}, & L \geq 3 \\ \frac{\text{near_term}}{L}, & \text{otherwise} \end{cases}$

$\text{far_term_norm} = \begin{cases} \frac{\text{far_term}}{\max(1, (L-3))}, & L > 3 \\ 0, & \text{otherwise} \end{cases}$

$\text{pipeline_shape} = (\text{near_term_norm} - \text{far_term_norm})$

$\text{pipeline_shape} = 0.0 \quad (\neg (L > 0))$

$\text{projected_shortfall} = \sum_{m=1}^{\min((3+1), (L+1))-1} \left(\max \left(0, - \left(\left(h + \sum_{i=0}^{(m)-1} q_i \right) - (m \cdot d_{\text{hat}}) \right) \right) \right)$

$\text{base} = (89.3 + 5.7)$

$\text{inv_adjust} = -(0.0) \cdot h$

$\text{pipe_adjust} = \left(-(0.0) \cdot \sum_{i=0}^{L-1} q_i \right)$

$\text{shortfall_adjust} = \frac{(0.0 \cdot \text{projected_shortfall})}{\text{float}(3)}$

$\text{shape_adjust} = -(0.0) \cdot \text{pipeline_shape}$

$\text{raw_order} = (((\text{base} + \text{inv_adjust}) + \text{pipe_adjust}) + \text{shortfall_adjust}) + \text{shape_adjust}$

$\text{order_amount} = \max(0.0, \min(\text{raw_order}, (2.0 \cdot 89.3)))$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 44-74

```

44.
45. # Projected shortfall over next target_coverage periods
46. projected_shortfall = 0.0
47. for m in range(1, min(target_coverage+1, L+1)):
48.     projected_inv = on_hand_inventory + sum(pipeline_orders[:m]) - m * d_hat
49.     projected_shortfall += max(0, -projected_inv)
50.
51. # Core order calculation
52. # 1. Base constant order with safety buffer
53. base = base_order + safety_buffer
54.
55. # 2. Inventory feedback (negative feedback on high inventory)
56. inv_adjust = -inv_feedback * on_hand_inventory
57.
58. # 3. Pipeline feedback (negative feedback on total pipeline)
59. pipe_adjust = -pipeline_feedback * sum(pipeline_orders)
60.
61. # 4. Shortfall compensation
62. shortfall_adjust = shortfall_gain * projected_shortfall / float(target_coverage)
63.
64. # 5. Pipeline shape adjustment
65. shape_adjust = -pipeline_shape_weight * pipeline_shape
66.
67. # Combine adjustments
68. raw_order = base + inv_adjust + pipe_adjust + shortfall_adjust + shape_adjust
69.
70. # Ensure non-negative and reasonable magnitude
71. order_amount = max(0.0, min(raw_order, 2.0 * base_order))
72.
73. # Round to nearest integer
74. return order_amount

```

Declarative mathematical formula rows 1-15

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$\hat{d} = 100.0$

$\text{available_now} = (h + q_0)$

$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$

$$\left. \begin{aligned} \text{near_term} &= \sum_{i=0}^{(\min(3,L))-1} q_i \\ \text{far_term} &= \begin{cases} \sum_{i=3}^{L-1} q_i, & L > 3 \\ 0, & \text{otherwise} \end{cases} \\ \text{near_term_norm} &= \begin{cases} \frac{\text{near_term}}{\text{float}(3)}, & L \geq 3 \\ \frac{\text{near_term}}{L}, & \text{otherwise} \end{cases} \\ \text{far_term_norm} &= \begin{cases} \frac{\text{far_term}}{\max(1, (L-3))}, & L > 3 \\ 0, & \text{otherwise} \end{cases} \end{aligned} \right\} (L > 0)$$

$\text{pipeline_shape} = (\text{near_term_norm} - \text{far_term_norm})$

$\text{pipeline_shape} = 0.0 \} \quad (\neg (L > 0))$

$\text{projected_shortfall} = \sum_{m=1}^{\min((3+1), (L+1))-1} \left(\max \left(0, - \left(\left(h + \sum_{i=0}^{(m)-1} q_i \right) - (m \cdot \hat{d}) \right) \right) \right)$

$\text{base} = (89.3 + 5.7)$

$\text{inv_adjust} = -(0.0) \cdot h$

$\text{pipe_adjust} = \left(-(0.0) \cdot \sum_{i=0}^{L-1} q_i \right)$

$\text{shortfall_adjust} = \frac{(0.0 \cdot \text{projected_shortfall})}{\text{float}(3)}$

$\text{shape_adjust} = -(0.0) \cdot \text{pipeline_shape}$

$\text{raw_order} = (((\text{base} + \text{inv_adjust}) + \text{pipe_adjust}) + \text{shortfall_adjust}) + \text{shape_adjust}$

$\text{order_amount} = \max(0.0, \min(\text{raw_order}, (2.0 \cdot 89.3)))$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-44

```

1. # sample_id: 094
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_6_r6
3. # distribution: normal_std30_L6_c1_2
4. # generation: 9
5. # rank_in_population_file: 10
6. # objective: 4095.46
7. # test_objective: 4122.034
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    pipeline_weighting;order_up_to;state_dependent_target;partial_adjustment;order_smoothing
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 530.0 # OPT_PARAM: {"initial": 530.0, "min": 400, "max": 700, "type": "float"}
14.     safety_stock = 50.0 # OPT_PARAM: {"initial": 50.0, "min": 20, "max": 100, "type": "float"}
15.     demand_estimate = 118.0 # OPT_PARAM: {"initial": 118.0, "min": 90, "max": 130, "type": "float"}
16.     pipeline_weight = 0.97 # OPT_PARAM: {"initial": 0.97, "min": 0.8, "max": 1.0, "type": "float"}
17.     smoothing_factor = 0.95 # OPT_PARAM: {"initial": 0.95, "min": 0.5, "max": 1.0, "type": "float"}
18.     cost_ratio_adjust = 0.667 # OPT_PARAM: {"initial": 0.667, "min": 0.5, "max": 0.8, "type":
"float"}
19.     pipeline_discount = 0.95 # OPT_PARAM: {"initial": 0.95, "min": 0.8, "max": 1.0, "type": "float"}
20.
21.     # Calculate weighted pipeline inventory with stronger discount for older orders
22.     weighted_pipeline = sum(p * (pipeline_weight ** i) * (pipeline_discount ** i)
23.                             for i, p in enumerate(reversed(pipeline_orders)))
24.
25.     # Calculate net inventory position with weighted pipeline
26.     net_inventory = on_hand_inventory + weighted_pipeline
27.
28.     # Adjust safety stock based on cost ratio (p=2, h=1) -> optimal critical ratio = p/(p+h) = 2/3
29.     adjusted_safety_stock = safety_stock * cost_ratio_adjust
30.
31.     # Calculate expected demand coverage for lead time
32.     lead_time_demand = demand_estimate * len(pipeline_orders)
33.
34.     # Calculate target inventory position
35.     target_position = lead_time_demand + adjusted_safety_stock
36.
37.     # Calculate raw order amount
38.     raw_order = max(0, target_position - net_inventory)
39.
40.     # Apply base stock as upper bound
41.     base_stock_order = max(0, base_stock - net_inventory)
42.     order_amount = min(raw_order, base_stock_order)
43.
44.     # Smooth ordering using previous order if available

```

Declarative mathematical formula rows 1-12

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_{L-1-i} \cdot (1.0)^i) \cdot (0.9)^i)$$

$$\text{net_inventory} = (h + \text{weighted_pipeline})$$

$$\text{adjusted_safety_stock} = (50.0 \cdot 0.7)$$

$$\text{lead_time_demand} = (118.0 \cdot L)$$

$$\text{target_position} = (\text{lead_time_demand} + \text{adjusted_safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{net_inventory}))$$

$$\text{base_stock_order} = \max(0, (530.0 - \text{net_inventory}))$$

$$\text{order_amount} = \min(\text{raw_order}, \text{base_stock_order})$$

$$\text{order_amount} = ((0.9 \cdot \text{order_amount}) + ((1 - 0.9) \cdot \text{previous_order})) \left. \vphantom{\text{order_amount}} \right\} \quad (L > 0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 45-52

```

45.     if len(pipeline_orders) > 0:
46.         previous_order = pipeline_orders[-1]
47.         order_amount = smoothing_factor * order_amount + (1 - smoothing_factor) * previous_order
48.
49.     # Ensure non-negative integer order
50.     order_amount = max(0, int(round(order_amount)))
51.
52.     return order_amount

```

Declarative mathematical formula rows 1-12

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{weighted_pipeline} = \sum_{i=0}^{L-1} ((q_{L-1-i} \cdot (1.0)^i) \cdot (0.9)^i)$$

$$\text{net_inventory} = (h + \text{weighted_pipeline})$$

$$\text{adjusted_safety_stock} = (50.0 \cdot 0.7)$$

$$\text{lead_time_demand} = (118.0 \cdot L)$$

$$\text{target_position} = (\text{lead_time_demand} + \text{adjusted_safety_stock})$$

$$\text{raw_order} = \max(0, (\text{target_position} - \text{net_inventory}))$$

$$\text{base_stock_order} = \max(0, (530.0 - \text{net_inventory}))$$

$$\text{order_amount} = \min(\text{raw_order}, \text{base_stock_order})$$

$$\left. \begin{array}{l} \text{previous_order} = q_{-(1)} \\ \text{order_amount} = ((0.9 \cdot \text{order_amount}) + ((1 - 0.9) \cdot \text{previous_order})) \end{array} \right\} \quad (L > 0)$$

$$\text{order_amount} = \max(0, \text{round}_{\text{even}}(\text{order_amount}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-44

```

1. # sample_id: 095
2. # folder: deepseek-chat_exponential_L6_c1_5_50_plain_processed_scipy_15_default_m2_10_r1
3. # distribution: exponential_L6_c1_5
4. # generation: 8
5. # rank_in_population_file: 7
6. # objective: 11091.44975
7. # test_objective: 11183.46189
8. # is_top10_by_distribution: True
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;nonlinear_pipeline_composition
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 376.9532527591698 # OPT_PARAM: {"initial": 376.9532527591698, "min": 300, "max":
14.     450, "type": "float"}
15.     safety_stock = 92.84179923380147 # OPT_PARAM: {"initial": 92.84179923380147, "min": 50, "max":
16.     150, "type": "float"}
17.     demand_smoothing = 0.3 # OPT_PARAM: {"initial": 0.3, "min": 0.05, "max": 0.3, "type": "float"}
18.     pipeline_weight = 1.0 # OPT_PARAM: {"initial": 1.0, "min": 0.6, "max": 1.0, "type": "float"}
19.     order_smoothing = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.5, "type": "float"}
20.
21.     # Calculate inventory position
22.     inventory_position = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Estimate demand from recent pipeline arrivals (last 4 periods)
25.     recent_arrivals = pipeline_orders[:4] if len(pipeline_orders) >= 4 else pipeline_orders
26.     if recent_arrivals:
27.         avg_recent_demand = sum(recent_arrivals) / len(recent_arrivals)
28.     else:
29.         avg_recent_demand = base_stock / 8
30.
31.     # Smooth demand estimate with stronger smoothing
32.     smoothed_demand = demand_smoothing * avg_recent_demand + (1 - demand_smoothing) * (base_stock /
33.     8)
34.
35.     # Calculate expected demand during lead time with adjusted weight
36.     expected_lead_time_demand = smoothed_demand * pipeline_weight * len(pipeline_orders)
37.
38.     # Determine order-up-to level with reduced safety stock
39.     order_up_to = base_stock + safety_stock + expected_lead_time_demand
40.
41.     # Calculate order amount
42.     order_amount = max(0, order_up_to - inventory_position)
43.
44.     # Apply stronger smoothing to order amount to reduce volatility
45.     if order_amount > 0:
46.         order_amount = order_smoothing * order_amount + (1 - order_smoothing) * smoothed_demand

```

Declarative mathematical formula rows 1-10

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{recent_arrivals} = \begin{cases} q_4, & L \geq 4 \\ \mathbf{q}, & \text{otherwise} \end{cases}$$

$$\text{avg_recent_demand} = \begin{cases} \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})}, & \text{recent_arrivals} \\ \frac{377.0}{8}, & \text{otherwise} \end{cases}$$

$$\text{smoothed_demand} = \left((0.3 \cdot \text{avg_recent_demand}) + \left((1 - 0.3) \cdot \frac{377.0}{8} \right) \right)$$

$$\text{expected_lead_time_demand} = ((\text{smoothed_demand} \cdot 1.0) \cdot L)$$

$$\text{order_up_to} = ((377.0 + 92.8) + \text{expected_lead_time_demand})$$

$$\text{order_amount} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$

$$\text{order_amount} = \begin{cases} ((0.1 \cdot \text{order_amount}) + ((1 - 0.1) \cdot \text{smoothed_demand})), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 45-46

```
45.         # Round to nearest integer (as required by output type)
46.         return order_amount
```

Declarative mathematical formula rows 1-10

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$
$$\text{recent_arrivals} = \begin{cases} q_{:4}, & L \geq 4 \\ \mathbf{q}, & \text{otherwise} \end{cases}$$
$$\text{avg_recent_demand} = \begin{cases} \frac{\text{sum}(\text{recent_arrivals})}{\text{len}(\text{recent_arrivals})}, & \text{recent_arrivals} \\ \frac{377.0}{8}, & \text{otherwise} \end{cases}$$
$$\text{smoothed_demand} = \left((0.3 \cdot \text{avg_recent_demand}) + \left((1 - 0.3) \cdot \frac{377.0}{8} \right) \right)$$
$$\text{expected_lead_time_demand} = ((\text{smoothed_demand} \cdot 1.0) \cdot L)$$
$$\text{order_up_to} = (377.0 + 92.8) + \text{expected_lead_time_demand}$$
$$\text{order_amount} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$
$$\text{order_amount} = \begin{cases} ((0.1 \cdot \text{order_amount}) + ((1 - 0.1) \cdot \text{smoothed_demand})), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$Q(h, \mathbf{q}) = \text{order_amount}$

Python source lines 1-40

```

1. # sample_id: 096
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_m2plural_4_r6
3. # distribution: normal_std30_L6_c1_2
4. # generation: 6
5. # rank_in_population_file: 10
6. # objective: 2308.9
7. # test_objective: 2277.102
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    safety_stock_buffer;pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 545.9771711201012 # OPT_PARAM: {"initial": 545.9771711201012, "min": 400, "max": 700, "type": "float"}
14.     safety_stock = 13.79731078150345 # OPT_PARAM: {"initial": 13.79731078150345, "min": 0, "max": 50, "type": "float"}
15.     demand_forecast_factor = 0.6003046695113123 # OPT_PARAM: {"initial": 0.6003046695113123, "min": 0.5, "max": 1.5, "type": "float"}
16.     smoothing_factor = 0.14594944369598148 # OPT_PARAM: {"initial": 0.14594944369598148, "min": 0.1, "max": 0.8, "type": "float"}
17.     min_order_qty = 10.0 # OPT_PARAM: {"initial": 10.0, "min": 0, "max": 30, "type": "float"}
18.     max_order_qty = 120.0 # OPT_PARAM: {"initial": 120.0, "min": 80, "max": 200, "type": "float"}
19.
20.     # Calculate inventory position
21.     inventory_position = on_hand_inventory + sum(pipeline_orders)
22.
23.     # Calculate expected demand using simple average of recent pipeline orders
24.     if len(pipeline_orders) > 0:
25.         # Use last 3 pipeline orders as demand estimate
26.         recent_window = min(3, len(pipeline_orders))
27.         avg_recent_demand = sum(pipeline_orders[:recent_window]) / recent_window
28.     else:
29.         avg_recent_demand = 0
30.
31.     # Adjust base stock based on demand forecast
32.     adjusted_base_stock = base_stock + demand_forecast_factor * avg_recent_demand
33.
34.     # Calculate order amount with safety stock adjustment
35.     order_amount = max(0, adjusted_base_stock + safety_stock - inventory_position)
36.
37.     # Apply smoothing to avoid extreme order fluctuations
38.     if len(pipeline_orders) > 0:
39.         avg_pipeline = sum(pipeline_orders) / len(pipeline_orders)
40.         smoothed_order = smoothing_factor * order_amount + (1 - smoothing_factor) * avg_pipeline

```

Declarative mathematical formula rows 1-11

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{recent_window} &= \min(3, L) \\ \text{avg_recent_demand} &= \frac{\sum_{i=0}^{(\text{recent_window})-1} q_i}{\text{recent_window}} \end{aligned} \right\} \quad (L > 0)$$

$$\text{avg_recent_demand} = 0 \quad (\neg (L > 0))$$

$$\text{adjusted_base_stock} = (546.0 + (0.6 \cdot \text{avg_recent_demand}))$$

$$\text{order_amount} = \max(0, ((\text{adjusted_base_stock} + 13.8) - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{smoothed_order} &= ((0.1 \cdot \text{order_amount}) + ((1 - 0.1) \cdot \text{avg_pipeline})) \end{aligned} \right\} \quad (L > 0)$$

$$\text{order_amount} = \begin{cases} 0, & \text{order_amount} < 10.0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \min(\text{order_amount}, 120.0)$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 41-53

```

41.         order_amount = max(0, smoothed_order)
42.
43.     # Apply ordering threshold to reduce small orders
44.     if order_amount < min_order_qty:
45.         order_amount = 0
46.
47.     # Cap large orders to prevent overordering
48.     order_amount = min(order_amount, max_order_qty)
49.
50.     # Round to nearest integer
51.     order_amount = int(round(order_amount))
52.
53.     return order_amount

```

Declarative mathematical formula rows 1-11

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\left. \begin{aligned} \text{recent_window} &= \min(3, L) \\ \text{avg_recent_demand} &= \frac{\sum_{i=0}^{(\text{recent_window})-1} q_i}{\text{recent_window}} \end{aligned} \right\} \quad (L > 0)$$

$$\text{avg_recent_demand} = 0 \quad (\neg (L > 0))$$

$$\text{adjusted_base_stock} = (546.0 + (0.6 \cdot \text{avg_recent_demand}))$$

$$\text{order_amount} = \max(0, ((\text{adjusted_base_stock} + 13.8) - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{avg_pipeline} &= \frac{\sum_{i=0}^{L-1} q_i}{L} \\ \text{smoothed_order} &= ((0.1 \cdot \text{order_amount}) + ((1 - 0.1) \cdot \text{avg_pipeline})) \end{aligned} \right\} \quad (L > 0)$$

$$\text{order_amount} = \max(0, \text{smoothed_order})$$

$$\text{order_amount} = \begin{cases} 0, & \text{order_amount} < 10.0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \min(\text{order_amount}, 120.0)$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$\boxed{Q(h, \mathbf{q}) = \text{order_amount}}$$

Python source lines 1-39

```

1. # sample_id: 097
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-e1-e1_4_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 4
5. # rank_in_population_file: 3
6. # objective: 2418.7
7. # test_objective: 2373.232
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;order_clipping
11. # extra_motifs:
    pipeline_demand_proxy;near_term_pipeline_focus;threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 548.2459833585183 # OPT_PARAM: {"initial": 548.2459833585183, "min": 300, "max": 800, "type": "float"}
15.
16.     # Newsvendor critical ratio adjustment factor
17.     newsvendor_factor = 0.49066732315215933 # OPT_PARAM: {"initial": 0.49066732315215933, "min": 0.1, "max": 2.0, "type": "float"}
18.
19.     # Pipeline imbalance penalty factor
20.     imbalance_penalty = 0.29801229754657366 # OPT_PARAM: {"initial": 0.29801229754657366, "min": 0.0, "max": 1.0, "type": "float"}
21.
22.     # Lead time urgency factor
23.     urgency_factor = 0.22256734330247824 # OPT_PARAM: {"initial": 0.22256734330247824, "min": 0.0, "max": 1.0, "type": "float"}
24.
25.     # Order smoothing factor
26.     smoothing = 0.1374992000765368 # OPT_PARAM: {"initial": 0.1374992000765368, "min": 0.0, "max": 1.0, "type": "float"}
27.
28.     # NOVEL FEATURE: Newsvendor-inspired critical ratio adjustment
29.     # Calculate optimal service level based on cost ratio p/(h+p) = 2/3
30.     critical_ratio = 2.0 / (1.0 + 2.0) # p/(h+p) = 2/3
31.     # Adjust base stock using newsvendor logic scaled by factor
32.     newsvendor_adjusted_base = base_stock * (1.0 + newsvendor_factor * (critical_ratio - 0.5))
33.
34.     L = len(pipeline_orders)
35.     if L == 0:
36.         return int(round(newsvendor_adjusted_base - on_hand_inventory))
37.
38.     total_pipeline = sum(pipeline_orders)
39.

```

Declarative mathematical formula rows 1-18

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $q = (q_0, \dots, q_{L-1})$

$$\text{critical_ratio} = \frac{2.0}{(1.0 + 2.0)}$$

$$\text{newsvendor_adjusted_base} = (548.2 \cdot (1.0 + (0.5 \cdot (\text{critical_ratio} - 0.5))))$$

$$Q(h, q) = \text{round}_{\text{even}}((\text{newsvendor_adjusted_base} - h)) \quad (L = 0)$$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\text{coverage_deficit} = 0.0$$

$$\text{cumulative_inventory} = h$$

$$\left. \begin{aligned} \text{available} &= \begin{cases} (\text{cumulative_inventory} + q_0), & k = 0 \\ (\text{cumulative_inventory} + q_k), & \text{otherwise} \end{cases} \\ \text{expected_coverage_needed} &= \frac{(\text{newsvendor_adjusted_base} \cdot (k + 1))}{L} \\ \text{urgency_weight} &= \frac{1.0}{(k + 1)} \\ \text{deficit} &= ((\text{expected_coverage_needed} - \text{available}) \cdot \text{urgency_weight}) \\ \text{coverage_deficit} &= (\text{coverage_deficit} + \text{deficit}) \end{aligned} \right\} \quad \begin{aligned} & (k = 0, \dots, L-1; \text{recurrence}) \\ & (\text{available} < \text{expected_coverage_needed}) \end{aligned}$$

$$\text{cumulative_inventory} = \begin{cases} \max(0, (\text{available} - \frac{\text{newsvendor_adjusted_base}}{L})), & k = 0 \\ \text{available}, & \text{otherwise} \end{cases}$$

$$\left. \begin{aligned} \text{mean_pipeline} &= \frac{\text{total_pipeline}}{L} \\ \text{variance} &= \frac{\sum_{q \in q} ((q - \text{mean_pipeline})^2)}{L} \\ \text{cv} &= \frac{(\text{variance})^{0.5}}{\text{mean_pipeline}} \end{aligned} \right\} \quad \begin{aligned} & (\text{mean_pipeline} > 0) \\ & ((L > 1 \wedge \text{total_pipeline} > 0)) \end{aligned}$$

$$\text{imbalance_adjustment} = (1.0 - (0.3 \cdot \min(\text{cv}, 2.0)))$$

$$\text{imbalance_adjustment} = 1.0 \quad (\neg(\text{mean_pipeline} > 0))$$

$$\text{imbalance_adjustment} = 1.0 \quad (\neg((L > 1 \wedge \text{total_pipeline} > 0)))$$

$$\text{urgency_adjustment} = \begin{cases} (1.0 + (0.2 \cdot \frac{\text{coverage_deficit}}{\text{newsvendor_adjusted_base}})), & \text{newsvendor_adjusted_base} > 0 \\ 1.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_target} = ((\text{newsvendor_adjusted_base} \cdot \text{imbalance_adjustment}) \cdot \text{urgency_adjustment})$$

$$\text{current_position} = (h + \text{total_pipeline})$$

$$\text{raw_order} = \max(0, (\text{adjusted_target} - \text{current_position}))$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\sum_{i=L-(\min(3, L))}^{L-1} q_i}{\min(3, L)} \end{aligned} \right\} \quad (L \geq 2)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{recent_avg}))$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L \geq 2))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))$$

$$Q(h, q) = \text{order_amount}$$

Python source lines 40-85

```

40. # NOVEL FEATURE: Lead-time-weighted pipeline coverage analysis
41. # Calculate coverage for each lead time period
42. coverage_deficit = 0.0
43. cumulative_inventory = on_hand_inventory
44.
45. for k in range(L):
46.     # Inventory available by period t+k
47.     if k == 0:
48.         # Current period: on-hand + arriving order
49.         available = cumulative_inventory + pipeline_orders[0]
50.     else:
51.         # Future periods: add pipeline orders arriving at that period
52.         available = cumulative_inventory + pipeline_orders[k]
53.
54.     # Expected demand coverage needed (using base stock as proxy for average demand)
55.     expected_coverage_needed = newsvendor_adjusted_base * (k + 1) / L
56.
57.     # Deficit if available inventory is less than needed coverage
58.     if available < expected_coverage_needed:
59.         # Weight deficit by urgency (earlier periods more urgent)
60.         urgency_weight = 1.0 / (k + 1)
61.         deficit = (expected_coverage_needed - available) * urgency_weight
62.         coverage_deficit += deficit
63.
64.     # Update cumulative inventory for next period
65.     if k == 0:
66.         cumulative_inventory = max(0, available - newsvendor_adjusted_base / L)
67.     else:
68.         cumulative_inventory = available
69.
70. # NOVEL FEATURE: Pipeline imbalance penalty
71. # Penalize if pipeline is concentrated in certain periods
72. if L > 1 and total_pipeline > 0:
73.     # Calculate coefficient of variation of pipeline orders
74.     mean_pipeline = total_pipeline / L
75.     if mean_pipeline > 0:
76.         variance = sum((q - mean_pipeline) ** 2 for q in pipeline_orders) / L
77.         cv = (variance ** 0.5) / mean_pipeline
78.         # Higher CV -> more imbalance -> apply penalty
79.         imbalance_adjustment = 1.0 - imbalance_penalty * min(cv, 2.0)
80.     else:
81.         imbalance_adjustment = 1.0
82. else:
83.     imbalance_adjustment = 1.0
84.
85. # Calculate urgency adjustment based on coverage deficit

```

Declarative mathematical formula rows 1-18

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{critical_ratio} = \frac{2.0}{(1.0 + 2.0)}$$

$$\text{newsvendor_adjusted_base} = (548.2 \cdot (1.0 + (0.5 \cdot (\text{critical_ratio} - 0.5))))$$

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}}((\text{newsvendor_adjusted_base} - h)) \quad (L = 0)$$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\text{coverage_deficit} = 0.0$$

$$\text{cumulative_inventory} = h$$

$$\left. \begin{aligned} \text{available} &= \begin{cases} (\text{cumulative_inventory} + q_0), & k = 0 \\ (\text{cumulative_inventory} + q_k), & \text{otherwise} \end{cases} \\ \text{expected_coverage_needed} &= \frac{(\text{newsvendor_adjusted_base} \cdot (k + 1))}{L} \\ \text{urgency_weight} &= \frac{1.0}{(k + 1)} \\ \text{deficit} &= ((\text{expected_coverage_needed} - \text{available}) \cdot \text{urgency_weight}) \\ \text{coverage_deficit} &= (\text{coverage_deficit} + \text{deficit}) \end{aligned} \right\} \quad \begin{aligned} &(\text{available} < \text{expected_coverage_needed}) \\ & \\ & \\ & \\ & \end{aligned} \quad (k = 0, \dots, L - 1; \text{recurrence})$$

$$\left. \begin{aligned} \text{cumulative_inventory} &= \begin{cases} \max(0, (\text{available} - \frac{\text{newsvendor_adjusted_base}}{L})), & k = 0 \\ \text{available}, & \text{otherwise} \end{cases} \\ \text{mean_pipeline} &= \frac{\text{total_pipeline}}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{mean_pipeline})^2)}{L} \\ \text{cv} &= \frac{(\text{variance})^{0.5}}{\text{mean_pipeline}} \\ \text{imbalance_adjustment} &= (1.0 - (0.3 \cdot \min(\text{cv}, 2.0))) \end{aligned} \right\} \quad \begin{aligned} & \\ & \\ & \\ & \\ & \end{aligned} \quad \begin{aligned} & \\ & \\ & \\ & \\ & \end{aligned} \quad ((L > 1 \wedge \text{total_pipeline} > 0))$$

$$\text{imbalance_adjustment} = 1.0 \quad (\neg(\text{mean_pipeline} > 0))$$

$$\text{imbalance_adjustment} = 1.0 \quad (\neg((L > 1 \wedge \text{total_pipeline} > 0)))$$

$$\text{urgency_adjustment} = \begin{cases} (1.0 + (0.2 \cdot \frac{\text{coverage_deficit}}{\text{newsvendor_adjusted_base}})), & \text{newsvendor_adjusted_base} > 0 \\ 1.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_target} = ((\text{newsvendor_adjusted_base} \cdot \text{imbalance_adjustment}) \cdot \text{urgency_adjustment})$$

$$\text{current_position} = (h + \text{total_pipeline})$$

$$\text{raw_order} = \max(0, (\text{adjusted_target} - \text{current_position}))$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\sum_{i=L-(\min(3, L))}^{L-1} q_i}{\min(3, L)} \end{aligned} \right\} \quad (L \geq 2)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{recent_avg}))$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L \geq 2))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 86-110

```

86.     if newsvendor_adjusted_base > 0:
87.         urgency_adjustment = 1.0 + urgency_factor * (coverage_deficit / newsvendor_adjusted_base)
88.     else:
89.         urgency_adjustment = 1.0
90.
91.     # Final adjusted target
92.     adjusted_target = newsvendor_adjusted_base * imbalance_adjustment * urgency_adjustment
93.
94.     # Current inventory position
95.     current_position = on_hand_inventory + total_pipeline
96.
97.     # Raw order amount
98.     raw_order = max(0, adjusted_target - current_position)
99.
100.    # Apply smoothing with recent pipeline average
101.    if L >= 2:
102.        recent_avg = sum(pipeline_orders[-min(3, L):]) / min(3, L)
103.        smoothed_order = smoothing * raw_order + (1 - smoothing) * recent_avg
104.    else:
105.        smoothed_order = raw_order
106.
107.    # Ensure non-negative integer
108.    order_amount = int(round(max(0, smoothed_order)))
109.
110.    return order_amount

```

Declarative mathematical formula rows 1-18

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{critical_ratio} = \frac{2.0}{(1.0 + 2.0)}$$

$$\text{newsvendor_adjusted_base} = (548.2 \cdot (1.0 + (0.5 \cdot (\text{critical_ratio} - 0.5))))$$

$$Q(h, \mathbf{q}) = \text{round}_{\text{even}}((\text{newsvendor_adjusted_base} - h)) \quad (L = 0)$$

$$\text{total_pipeline} = \sum_{i=0}^{L-1} q_i$$

$$\text{coverage_deficit} = 0.0$$

$$\text{cumulative_inventory} = h$$

$$\left. \begin{aligned} \text{available} &= \begin{cases} (\text{cumulative_inventory} + q_0), & k = 0 \\ (\text{cumulative_inventory} + q_k), & \text{otherwise} \end{cases} \\ \text{expected_coverage_needed} &= \frac{(\text{newsvendor_adjusted_base} \cdot (k + 1))}{L} \\ \text{urgency_weight} &= \frac{1.0}{(k + 1)} \end{aligned} \right\} \begin{aligned} & \text{deficit} = ((\text{expected_coverage_needed} - \text{available}) \cdot \text{urgency_weight}) \\ & \text{coverage_deficit} = (\text{coverage_deficit} + \text{deficit}) \end{aligned} \quad (k = 0, \dots, L - 1; \text{recurrence})$$

$$\left. \begin{aligned} \text{cumulative_inventory} &= \begin{cases} \max(0, (\text{available} - \frac{\text{newsvendor_adjusted_base}}{L})), & k = 0 \\ \text{available}, & \text{otherwise} \end{cases} \\ \text{mean_pipeline} &= \frac{\text{total_pipeline}}{L} \\ \text{variance} &= \frac{\sum_{q \in \mathbf{q}} ((q - \text{mean_pipeline})^2)}{L} \\ \text{cv} &= \frac{(\text{variance})^{0.5}}{\text{mean_pipeline}} \end{aligned} \right\} \begin{aligned} & \text{imbalance_adjustment} = (1.0 - (0.3 \cdot \min(\text{cv}, 2.0))) \\ & \text{imbalance_adjustment} = 1.0 \quad (\neg(\text{mean_pipeline} > 0)) \\ & \text{imbalance_adjustment} = 1.0 \quad (\neg((L > 1 \wedge \text{total_pipeline} > 0))) \end{aligned} \quad ((L > 1 \wedge \text{total_pipeline} > 0))$$

$$\text{urgency_adjustment} = \begin{cases} (1.0 + (0.2 \cdot \frac{\text{coverage_deficit}}{\text{newsvendor_adjusted_base}})), & \text{newsvendor_adjusted_base} > 0 \\ 1.0, & \text{otherwise} \end{cases}$$

$$\text{adjusted_target} = ((\text{newsvendor_adjusted_base} \cdot \text{imbalance_adjustment}) \cdot \text{urgency_adjustment})$$

$$\text{current_position} = (h + \text{total_pipeline})$$

$$\text{raw_order} = \max(0, (\text{adjusted_target} - \text{current_position}))$$

$$\left. \begin{aligned} \text{recent_avg} &= \frac{\sum_{i=L-\min(3, L)}^{L-1} q_i}{\min(3, L)} \end{aligned} \right\} (L \geq 2)$$

$$\text{smoothed_order} = ((0.1 \cdot \text{raw_order}) + ((1 - 0.1) \cdot \text{recent_avg}))$$

$$\text{smoothed_order} = \text{raw_order} \quad (\neg(L \geq 2))$$

$$\text{order_amount} = \text{round}_{\text{even}}(\max(0, \text{smoothed_order}))$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-38

```

1. # sample_id: 098
2. # folder: deepseek-chat_poisson_L6_c1_5_50_plain_processed_scipy_15_default_m2_2_r7
3. # distribution: poisson_L6_c1_5
4. # generation: 6
5. # rank_in_population_file: 3
6. # objective: 3082.4
7. # test_objective: 3096.685
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs: inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment
11. # extra_motifs: safety_stock_buffer;pipeline_demand_proxy;integer_rounding
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 650.0 # OPT_PARAM: {"initial": 650.0, "min": 400, "max": 900, "type": "float"}
14.     safety_stock = 155.24241310755608 # OPT_PARAM: {"initial": 155.24241310755608, "min": 100,
15.     "max": 250, "type": "float"}
16.     demand_forecast = 105.69008621250516 # OPT_PARAM: {"initial": 105.69008621250516, "min": 80,
17.     "max": 120, "type": "float"}
18.     smoothing_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.0, "type": "float"}
19.     lead_time = len(pipeline_orders)
20.
21.     # Calculate inventory position
22.     inventory_position = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Calculate expected demand during lead time plus one period
25.     expected_lead_time_demand = demand_forecast * (lead_time + 1)
26.
27.     # Calculate target inventory level
28.     target_inventory = expected_lead_time_demand + safety_stock
29.
30.     # Use the maximum of base_stock and target_inventory as order-up-to level
31.     order_up_to = max(base_stock, target_inventory)
32.
33.     # Calculate order amount
34.     order_amount = max(0, order_up_to - inventory_position)
35.
36.     # Apply smoothing only when order amount is positive
37.     if order_amount > 0:
38.         order_amount = int(order_amount * smoothing_factor + 0.5)
39.
40.     return order_amount

```

Declarative mathematical formula rows 1-9

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

lead_time = L

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

expected_lead_time_demand = $(105.7 \cdot (\text{lead_time} + 1))$

target_inventory = $(\text{expected_lead_time_demand} + 155.2)$

order_up_to = $\max(650.0, \text{target_inventory})$

order_amount = $\max(0, (\text{order_up_to} - \text{inventory_position}))$

order_amount = $\begin{cases} \text{int}(((\text{order_amount} \cdot 0.5) + 0.5)), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-40

```

1. # sample_id: 099
2. # folder: deepseek-chat_poisson_L6_c1_2_50_plain_processed_scipy_15_default_m2_10_r4
3. # distribution: poisson_L6_c1_2
4. # generation: 9
5. # rank_in_population_file: 10
6. # objective: 1045.24
7. # test_objective: 1018.631
8. # is_top10_by_distribution: False
9. # is_final_generation: False
10. # table_motifs:
    inventory_position;pipeline_weighting;state_dependent_target;partial_adjustment;order_clipping
11. # extra_motifs:
    pipeline_demand_proxy;threshold_order_activation;integer_rounding;emergency_or_shortage_boost
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     base_stock = 664.104497870118 # OPT_PARAM: {"initial": 664.104497870118, "min": 650, "max": 850,
    "type": "float"}
14.     demand_estimate = 91.62096643311519 # OPT_PARAM: {"initial": 91.62096643311519, "min": 90,
    "max": 110, "type": "float"}
15.     safety_factor = 1.0669489657821407 # OPT_PARAM: {"initial": 1.0669489657821407, "min": 1.05,
    "max": 1.25, "type": "float"}
16.     pipeline_weight = 0.1 # OPT_PARAM: {"initial": 0.1, "min": 0.1, "max": 0.5, "type": "float"}
17.     smoothing_factor = 0.5 # OPT_PARAM: {"initial": 0.5, "min": 0.5, "max": 1.0, "type": "float"}
18.     critical_ratio = 0.65 # OPT_PARAM: {"initial": 0.65, "min": 0.65, "max": 0.75, "type": "float"}
19.     min_order_threshold = 0.28315518642390647 # OPT_PARAM: {"initial": 0.28315518642390647, "min":
    0.05, "max": 0.3, "type": "float"}
20.
21.     # Calculate inventory position
22.     inventory_position = on_hand_inventory + sum(pipeline_orders)
23.
24.     # Calculate expected demand during lead time with safety factor
25.     expected_lead_time_demand = demand_estimate * len(pipeline_orders) * safety_factor
26.
27.     # Adjust base stock based on critical ratio (p/(p+h) = 2/3)
28.     # Higher critical ratio means we should carry more inventory
29.     adjusted_base_stock = base_stock * (1 + (critical_ratio - 0.6667) * 0.5)
30.
31.     # Calculate order-up-to level
32.     order_up_to = max(adjusted_base_stock, expected_lead_time_demand)
33.
34.     # Calculate base order amount
35.     base_order = max(0, order_up_to - inventory_position)
36.
37.     # Apply smoothing only when order is significant
38.     if base_order > demand_estimate * min_order_threshold:
39.         # Blend between demand-based ordering and gap-filling
40.         demand_based = demand_estimate * (1 + safety_factor * 0.1)

```

Declarative mathematical formula rows 1-11

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = ((91.6 \cdot L) \cdot 1.1)$$

$$\text{adjusted_base_stock} = (664.1 \cdot (1 + ((0.7 - 0.7) \cdot 0.5)))$$

$$\text{order_up_to} = \max(\text{adjusted_base_stock}, \text{expected_lead_time_demand})$$

$$\text{base_order} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{demand_based} &= (91.6 \cdot (1 + (1.1 \cdot 0.1))) \\ \text{order_amount} &= ((0.5 \cdot \text{base_order}) + ((1 - 0.5) \cdot \text{demand_based})) \end{aligned} \right\} \quad (\text{base_order} > (91.6 \cdot 0.3))$$

$$\text{order_amount} = \text{base_order} \quad (\neg(\text{base_order} > (91.6 \cdot 0.3)))$$

$$\text{order_amount} = \begin{cases} ((\text{order_amount} \cdot 0.1) + (91.6 \cdot (1 - 0.1))), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 41-53

```

41.         order_amount = smoothing_factor * base_order + (1 - smoothing_factor) * demand_based
42.     else:
43.         order_amount = base_order
44.
45.     # Incorporate pipeline information to smooth ordering
46.     if order_amount > 0:
47.         # Weight between current order and demand forecast
48.         order_amount = order_amount * pipeline_weight + demand_estimate * (1 - pipeline_weight)
49.
50.     # Round to nearest integer
51.     order_amount = int(round(order_amount))
52.
53.     return order_amount

```

Declarative mathematical formula rows 1-11

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

Inputs: h = on-hand inventory, $\mathbf{q} = (q_0, \dots, q_{L-1})$

$$\text{inventory_position} = \left(h + \sum_{i=0}^{L-1} q_i \right)$$

$$\text{expected_lead_time_demand} = ((91.6 \cdot L) \cdot 1.1)$$

$$\text{adjusted_base_stock} = (664.1 \cdot (1 + ((0.7 - 0.7) \cdot 0.5)))$$

$$\text{order_up_to} = \max(\text{adjusted_base_stock}, \text{expected_lead_time_demand})$$

$$\text{base_order} = \max(0, (\text{order_up_to} - \text{inventory_position}))$$

$$\left. \begin{aligned} \text{demand_based} &= (91.6 \cdot (1 + (1.1 \cdot 0.1))) \\ \text{order_amount} &= ((0.5 \cdot \text{base_order}) + ((1 - 0.5) \cdot \text{demand_based})) \end{aligned} \right\} \quad (\text{base_order} > (91.6 \cdot 0.3))$$

$$\text{order_amount} = \text{base_order} \quad (\neg (\text{base_order} > (91.6 \cdot 0.3)))$$

$$\text{order_amount} = \begin{cases} ((\text{order_amount} \cdot 0.1) + (91.6 \cdot (1 - 0.1))), & \text{order_amount} > 0 \\ \text{order_amount}, & \text{otherwise} \end{cases}$$

$$\text{order_amount} = \text{round}_{\text{even}}(\text{order_amount})$$

$$Q(h, \mathbf{q}) = \text{order_amount}$$

Python source lines 1-38

```
1. # sample_id: 100
2. # folder: deepseek-chat_normal_std30_L6_c1_2_50_plain_processed_scipy_15_default_e1-e1-e1_4_r1
3. # distribution: normal_std30_L6_c1_2
4. # generation: 10
5. # rank_in_population_file: 2
6. # objective: 2337.34
7. # test_objective: 2316.588
8. # is_top10_by_distribution: False
9. # is_final_generation: True
10. # table_motifs:
    inventory_position;pipeline_weighting;nonlinear_pipeline_composition;order_up_to;partial_adjustment;o
rder_clipping
11. # extra_motifs:
    threshold_order_activation;integer_rounding;emergency_or_shortage_boost;nonlinear_gap_transform
12. def compute_order_amount(on_hand_inventory, pipeline_orders):
13.     # Base stock level
14.     base_stock = 549.5480724239817 # OPT_PARAM: {"initial": 549.5480724239817, "min": 300, "max":
800, "type": "float"}
15.
16.     # Pipeline volatility factor
17.     volatility_factor = 0.0 # OPT_PARAM: {"initial": 0.0, "min": 0.0, "max": 1.0, "type": "float"}
18.
19.     # Order smoothing factor
20.     smoothing = 0.14596725097252355 # OPT_PARAM: {"initial": 0.14596725097252355, "min": 0.0, "max":
1.0, "type": "float"}
21.
22.     # Critical ratio adjustment
23.     cr_adjust = 0.4711446868515846 # OPT_PARAM: {"initial": 0.4711446868515846, "min": -0.5, "max":
0.5, "type": "float"}
24.
25.     # Pipeline pattern matching weight
26.     pattern_weight = 0.43107405757054845 # OPT_PARAM: {"initial": 0.43107405757054845, "min": 0.0,
"max": 1.0, "type": "float"}
27.
28.     # Pattern decay factor
29.     pattern_decay = 0.7159501518667241 # OPT_PARAM: {"initial": 0.7159501518667241, "min": 0.5,
"max": 0.95, "type": "float"}
30.
31.     # Pattern threshold
32.     pattern_threshold = 0.2395822333385727 # OPT_PARAM: {"initial": 0.2395822333385727, "min": 0.05,
"max": 0.5, "type": "float"}
33.
34.     L = len(pipeline_orders)
35.     if L == 0:
36.         return int(round(base_stock - on_hand_inventory))
37.
38.     total_pipeline = sum(pipeline_orders)
```

Declarative mathematical formula rows 1-16

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

MathJax internal buffer size exceeded; is there a recursive macro call?

Python source lines 39-84

```
39.
40. # NOVEL FEATURE 1: Pipeline volatility measurement
41. # Measure how much the pipeline fluctuates (standard deviation relative to mean)
42. if L >= 2 and total_pipeline > 0:
43.     pipeline_mean = total_pipeline / L
44.     if pipeline_mean > 0.001:
45.         variance = sum((q - pipeline_mean) ** 2 for q in pipeline_orders) / L
46.         std_dev = variance ** 0.5
47.         volatility = std_dev / pipeline_mean
48.         # High volatility suggests unstable ordering - smooth more aggressively
49.         volatility_adjustment = 1.0 - volatility_factor * min(volatility, 2.0)
50.     else:
51.         volatility_adjustment = 1.0
52. else:
53.     volatility_adjustment = 1.0
54.
55. # NOVEL FEATURE 2: Pattern-based pipeline analysis
56. # Detect recurring patterns in pipeline distribution (e.g., increasing, decreasing, U-shaped)
57. if L >= 3:
58.     # Calculate pattern scores for different archetypes
59.     patterns = {}
60.
61.     # Increasing pattern (orders getting larger over time)
62.     inc_score = 0
63.     for i in range(1, L):
64.         if pipeline_orders[i-1] > 0.001:
65.             inc_score += (pipeline_orders[i] - pipeline_orders[i-1]) / pipeline_orders[i-1]
66.     inc_score /= max(1, L-1)
67.     patterns['increasing'] = max(0, inc_score) # Only positive changes
68.
69.     # Decreasing pattern (orders getting smaller over time)
70.     dec_score = 0
71.     for i in range(1, L):
72.         if pipeline_orders[i-1] > 0.001:
73.             dec_score += (pipeline_orders[i-1] - pipeline_orders[i]) / pipeline_orders[i-1]
74.     dec_score /= max(1, L-1)
75.     patterns['decreasing'] = max(0, dec_score)
76.
77.     # U-shaped pattern (small in middle, large at ends)
78.     u_score = 0
79.     if L >= 3:
80.         mid = L // 2
81.         early_avg = sum(pipeline_orders[:mid]) / mid
82.         middle_avg = sum(pipeline_orders[mid:L-mid]) / max(1, L-2*mid)
83.         late_avg = sum(pipeline_orders[L-mid:]) / mid
84.
```

Declarative mathematical formula rows 1-16

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

MathJax internal buffer size exceeded; is there a recursive macro call?

Python source lines 85-130

```
85.         if middle_avg > 0.001:
86.             u_score = (early_avg + late_avg) / (2 * middle_avg) - 1.0
87.             u_score = max(0, u_score)
88.         patterns['u_shaped'] = u_score
89.
90.         # Find strongest pattern
91.         strongest_pattern = max(patterns.items(), key=lambda x: x[1])
92.
93.         # Apply pattern-based adjustment
94.         if strongest_pattern[1] > pattern_threshold:
95.             if strongest_pattern[0] == 'increasing':
96.                 # If pipeline is increasing, we might be over-ordering - reduce slightly
97.                 pattern_adjustment = 1.0 - pattern_weight * strongest_pattern[1] * pattern_decay
98.             elif strongest_pattern[0] == 'decreasing':
99.                 # If pipeline is decreasing, we might be under-ordering - increase slightly
100.                 pattern_adjustment = 1.0 + pattern_weight * strongest_pattern[1] * pattern_decay
101.             else: # u_shaped
102.                 # U-shaped suggests both recent and distant orders are high - maintain
103.                 pattern_adjustment = 1.0
104.         else:
105.             pattern_adjustment = 1.0
106.     else:
107.         pattern_adjustment = 1.0
108.
109.     # Cost-ratio adjustment (newsvendor logic)
110.     critical_ratio = 2.0 / (1.0 + 2.0) # p/(h+p) = 2/3
111.     cost_adjusted_base = base_stock * (1.0 + cr_adjust * (critical_ratio - 0.5))
112.
113.     # Combine adjustments
114.     adjusted_target = cost_adjusted_base * volatility_adjustment * pattern_adjustment
115.
116.     # Current inventory position
117.     current_position = on_hand_inventory + total_pipeline
118.
119.     # Raw order amount
120.     raw_order = max(0, adjusted_target - current_position)
121.
122.     # Apply smoothing with pipeline average
123.     if L > 0:
124.         pipeline_avg = total_pipeline / L
125.         smoothed_order = smoothing * raw_order + (1 - smoothing) * pipeline_avg
126.     else:
127.         smoothed_order = raw_order
128.
129.     # Ensure non-negative integer
130.     order_amount = int(round(max(0, smoothed_order)))
```

Declarative mathematical formula rows 1-16

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

MathJax internal buffer size exceeded; is there a recursive macro call?

Python source lines 131-132

```
131.  
132.     return order_amount
```

Declarative mathematical formula rows 1-16

The final formula segment repeated on this continuation page.

This policy is rendered as declarative equations. Reductions use summation notation and stateful updates use recurrences.

MathJax internal buffer size exceeded; is there a recursive macro call?